



**Informação e  
Comunicação**

# BANCO DE DADOS: UMA ABORDAGEM PRÁTICA

---

**SENAI-SP** editora

The background features a minimalist design with several overlapping rounded rectangular shapes in a light gray color. A vertical dashed line runs through the upper portion of the page, intersecting the shapes. The title is positioned to the right of these shapes.

# BANCO DE DADOS: UMA ABORDAGEM PRÁTICA

## DADOS INTERNACIONAIS DE CATALOGAÇÃO NA PUBLICAÇÃO (CIP)

Cruz, André Felipe Savedra

Banco de dados: uma abordagem prática / André Felipe Savedra Cruz;  
ilustrações de Palimpsestos. -- 1. ed. -- São Paulo: SENAI-SP editora, 2025.  
422 p.

ISBN 978-85-534-0192-5

1. Ciência da computação 2. Manipulação de banco de dados 3. Gestão de  
banco de dados I. Título.

CDD: 004

---

Índice para o catálogo sistemático:

1. Ciência da computação – 004

Bibliotecária responsável: Fabiana Alves de Moraes Oliveira CRB-8 7330

Papel, cartão e  
papelo: uma ótima  
história ambiental  
para contar



**Informação e  
Comunicação**

# BANCO DE DADOS: UMA ABORDAGEM PRÁTICA





DEPARTAMENTO REGIONAL DE SÃO PAULO

Paulo Skaf

**Presidente**

Ricardo Figueiredo Terra

**Diretor Regional**

Wilton Ruas da Silva

**Diretoria Corporativa Sesi-SP e SENAI-SP**

Marcello Luiz de Souza Junior

**Gerência de Relações com o Mercado**

Daniel da Silva Motta

**Gerência de Inovação e Tecnologia**

Cássia Regina Souza da Cruz

**Gerência de Educação**

André Felipe Savedra Cruz

**Elaboração de conteúdo**

Lucas Tadeu Monteiro Guedes Fernandes Salomão

**Revisão técnica**

**SENAI-SP Editora**

Av. Paulista, 1.313, 6º andar, 01311-923, São Paulo – SP

editora@sesisenaisp.org.br

www.senaispeditora.com.br

**SENAI-SP** editora

Adilson Castro de Souza Rocha

**Gerência editorial**

Breno Beneducci

**Edição**

Mariane Cristina de Oliveira

**Análise editorial**

Breno Beneducci

**Preparação**

Agnes Diana

**Revisão**

Caio Marques Rodrigues

Camila Ciarini Dias

Marisa Daniela Pereira Araújo

**Projeto gráfico de capa e miolo**

Rafael Zemantauskas

**Coordenação de produção gráfica**

Aline Navarro

José Augusto Moura do Amaral

Valquíria Chagas

**Produção gráfica**

Agnes Diana

**Diagramação**

Palimpsestos

**Ilustração**

Edilza Alves Leite

Viviane Medeiros de Souza Guedes

**Direitos autorais**

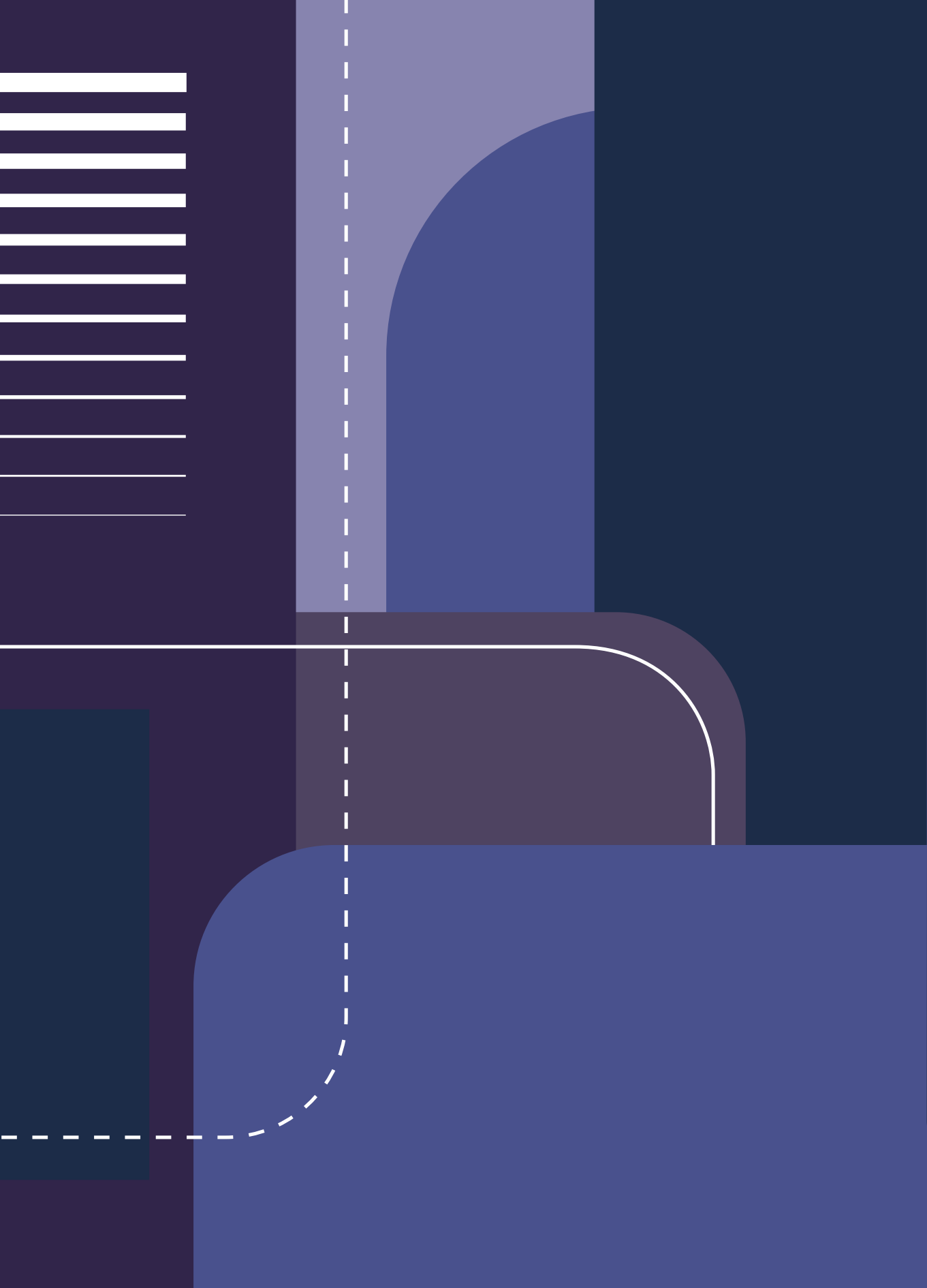
© SENAI-SP Editora, 2025

A SENAI-SP Editora empenhou-se em identificar e contatar todos os responsáveis pelos direitos autorais deste livro. Se porventura for constatada omissão na identificação de algum material, dispomo-nos a efetuar, futuramente, os possíveis acertos.

Material didático utilizado nos cursos do SENAI-SP.

# APRESENTAÇÃO

**E**ste livro foi desenvolvido a partir da experiência de um profissional da área de desenvolvimento de bancos de dados, com uma abordagem pedagógica voltada para o ensino. A obra busca integrar as vivências práticas adquiridas no mercado de trabalho com as metodologias de ensino, proporcionando aos leitores uma compreensão abrangente do desenvolvimento de bancos de dados. O objetivo é não apenas apresentar os conceitos técnicos, mas também facilitar a aplicação desses conhecimentos no dia a dia dos profissionais e estudantes que se aventuram nesse campo. Ao longo do livro, serão compartilhados tanto a teoria fundamental quanto exemplos práticos, oferecendo uma abordagem equilibrada para aqueles que desejam aprofundar seus conhecimentos em bancos de dados.



# SUMÁRIO

|                                                       |           |
|-------------------------------------------------------|-----------|
| <b>INTRODUÇÃO</b>                                     | <b>11</b> |
| <b>CAPÍTULO 1</b>                                     |           |
| <b>INTRODUÇÃO AO BANCO DE DADOS</b>                   | <b>13</b> |
| Definição e importância                               | 14        |
| Tipos de bancos de dados: relacional e não relacional | 17        |
| Sistema de Gerenciamento de Banco de Dados (SGBD)     | 39        |
| Principais opções do mercado                          | 41        |
| Desafio: <i>Startup</i>                               | 44        |
| Resumo                                                | 46        |

## CAPÍTULO 2

|                                                            |           |
|------------------------------------------------------------|-----------|
| <b>MODELAGEM DE DADOS</b>                                  | <b>47</b> |
| Definição e importância                                    | 48        |
| Modelo entidade relacionamento (MER)                       | 48        |
| Diagrama entidade relacionamento (DER)                     | 50        |
| Aplicando a modelagem em diagramas entidade relacionamento | 63        |
| Normalização dos dados                                     | 76        |
| Exemplos e casos de uso                                    | 84        |
| Resumo                                                     | 98        |

## CAPÍTULO 3

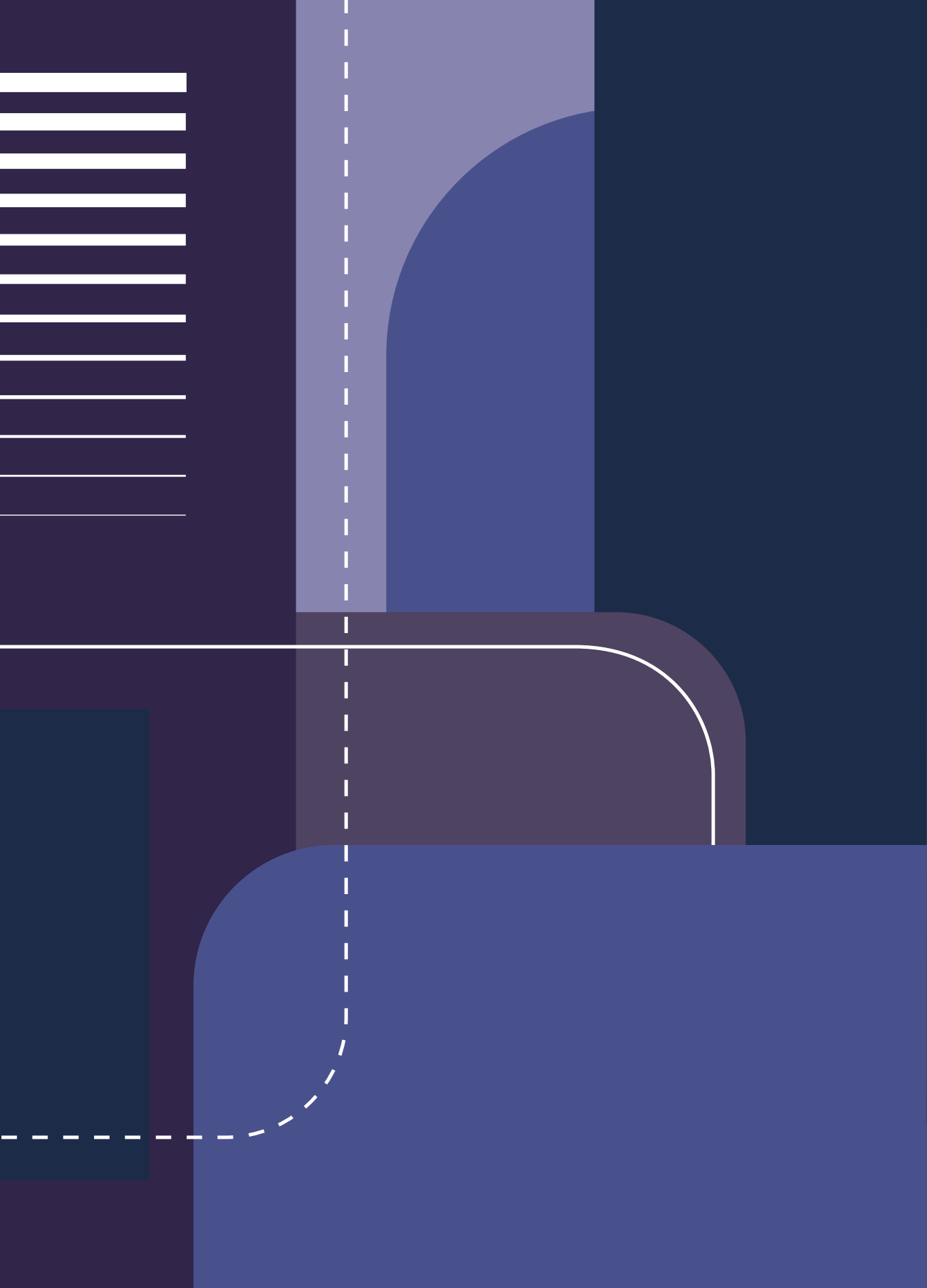
|                                                   |            |
|---------------------------------------------------|------------|
| <b>MANIPULAÇÃO DE DADOS I – MODELO RELACIONAL</b> | <b>101</b> |
| Conectando-se ao servidor                         | 102        |
| SQL – Linguagem de Consulta Estruturada           | 112        |
| Criando e utilizando uma base de dados            | 119        |
| CRUD – Criar, Ler, Atualizar e Deletar Dados      | 124        |
| Resumo                                            | 219        |

## CAPÍTULO 4

|                                                    |            |
|----------------------------------------------------|------------|
| <b>MANIPULAÇÃO DE DADOS II – MODELO RELACIONAL</b> | <b>221</b> |
| Funções                                            | 222        |
| Associação de tabelas                              | 236        |
| Agrupamentos e agregações                          | 255        |
| Subconsultas                                       | 278        |
| Resumo                                             | 288        |

## CAPÍTULO 5

|                                                     |                |
|-----------------------------------------------------|----------------|
| <b>GESTÃO DO BANCO DE DADOS – MODELO RELACIONAL</b> | <b>289</b>     |
| <i>Triggers</i>                                     | 291            |
| <i>Procedures</i>                                   | 305            |
| <i>Views</i>                                        | 356            |
| Gerenciamento de usuários                           | 364            |
| DCL – Linguagem de Controle de Dados                | 372            |
| TCL – Linguagem de Controle de Transações           | 381            |
| Migração de dados                                   | 411            |
| Resumo                                              | 420            |
| <br><b>SOBRE O ELABORADOR</b>                       | <br><b>421</b> |



# INTRODUÇÃO

**O**lá, aluno(a)! Neste livro trabalharemos de forma objetiva, mas aprofundada, tudo o que você precisa saber para atuar em modelagem, criação e manutenção de bancos de dados, parte fundamental do processo de desenvolvimento de qualquer aplicação minimamente complexa.

Inicialmente abordaremos os paradigmas relacionais e não relacionais, bem como a principal diferença entre eles. Embora o enfoque deste livro seja bases relacionais usando MySQL, faremos uma breve abordagem sobre o conceito não relacional.



Ao longo dos capítulos, traremos estudos de caso, exemplos e até mesmo exercícios, de forma a estimular o(a) aluno(a) a tentar reproduzir o conteúdo teórico abordado, sendo esse processo de fundamental importância para o aprendizado de qualquer linguagem de programação.

Por fim, esperamos que este conteúdo possa contribuir para seu amadurecimento como um(a) desenvolvedor(a), levando você ao entendimento de uma das áreas mais cruciais no processo de desenvolvimento, isto é, o armazenamento e a disponibilização de dados.

## CAPÍTULO 1

# INTRODUÇÃO AO BANCO DE DADOS

Imagine um *e-commerce* popular, com milhares de produtos, clientes cadastrados e pedidos a serem processados a cada minuto. Como seria possível gerenciar estoques, acompanhar entregas, processar pagamentos e garantir que cada cliente receba o produto certo, no tempo certo, sem um sistema robusto e organizado para armazenar e recuperar todas essas informações? Sem dúvida, não se trata de uma tarefa simples, correto? É aqui que entram os bancos de dados!

Neste primeiro capítulo, exploraremos o conceito fundamental de banco de dados, sua estrutura básica, os principais tipos existentes e o passo

a passo da instalação. Mas, além da técnica, entenderemos a importância estratégica dos bancos de dados no desenvolvimento de *software*, compreendendo por que a modelagem da base de dados se tornou uma etapa crucial, frequentemente precursora de todas as demais no processo de criação de um aplicativo, seja ele um gigante da internet ou um pequeno sistema interno.

De fato, essa é uma longa, mas importante jornada para você aluno(a), portanto, recomendamos o estudo aprofundado deste livro, o qual, associado à execução de exercícios práticos, certamente ajudará a desbravar esse novo mundo!

## Definição e importância

Talvez este seja seu primeiro contato com banco de dados, mas com toda certeza não é a primeira vez que um banco de dados interage com você.

Tenha você notado ou não, praticamente todas as aplicações que usamos no cotidiano precisam usar ou exibir dados previamente salvos, como informações de *login*, endereço, CPF ou CNPJ, e registrar novos dados gerados durante o processo de uso de tais aplicações. Por exemplo, quando você faz uma nova postagem em uma rede social ou realiza uma transação bancária, essas ações geram dados, que claramente precisam ser salvos de maneira permanente. Além disso, certamente você, como usuário dessas aplicações, espera poder voltar a ter acesso a essas informações mesmo tendo encerrado o aplicativo ou navegador utilizado para gerá-las, correto?

Desse modo, como acaba de perceber, quando se acessa uma conta bancária, esta contém seu saldo e seu extrato, sempre disponíveis para sua consulta, sendo tais informações as mesmas, não importando quantas vezes tenha reinstalado seu aplicativo ou se o acesso é por *internet banking* ou caixa eletrônico, fazendo-nos notar que tais dados não estão sendo salvos

no dispositivo de onde se acessa a aplicação e sim no que denominamos banco de dados.

Sem dúvida, exemplos não faltam, sejam aplicações financeiras, redes sociais, jogos, ferramentas de gestão, sistemas industriais ou corporativos, a **persistência** de dados antes, durante e depois do uso dessas aplicações é de fato realizada e gerenciada pelos conhecidos bancos de dados, os quais podem diferir em inúmeros aspectos e características. Contudo, todos possuem o mesmo objetivo: armazenar de forma segura e eficiente grande quantidade de registros e disponibilizá-los para serem usados pelas aplicações por meio de consultas simples ou avançadas, utilizando uma linguagem própria para cada tipo de banco.

Essa persistência de dados é fundamental, pois assegura que as informações não sejam perdidas quando a aplicação é fechada ou o dispositivo é desligado. Ao contrário da memória volátil de um computador, que perde todos os dados ao ser desligado, os bancos de dados garantem que as informações armazenadas estejam sempre acessíveis, independentemente de interrupções no funcionamento da aplicação ou do sistema.

Desse modo, simplificando, o banco de dados consiste em um sistema cujo objetivo é centralizar o armazenamento de dados, garantindo consistência, segurança e integridade dos registros, auxiliando inclusive na consulta, no agrupamento e na manipulação dos dados salvos por meio de recursos disponíveis em sua linguagem própria.

Agora que você notou o quanto os bancos de dados estão presentes nos mais variados sistemas, bem como sua importância fundamental, talvez tenha se questionado que gerenciá-los não deve ser tarefa fácil. Contudo, para auxiliar nesse contexto, foram criados os Sistemas de Gerenciamento de Banco de Dados (SGBD), isto é, *softwares* especializados com o intuito de auxiliar no uso das bases de dados, permitindo sua melhor visualização e organização, além de controle de acesso, *backup* e recuperação.

Além dos benefícios citados, os SGBDs oferecem uma ampla gama de recursos que os tornam quase que indispensáveis em aplicações de médio e grande porte, podendo ser destacadas algumas de suas **vantagens** principais, como:

- **Segurança:** o SGBD oferece mecanismos robustos de controle de acesso, garantindo que apenas usuários autorizados possam visualizar ou modificar os dados;
- **Integridade:** ele assegura que os dados armazenados sejam precisos e consistentes, através de restrições e regras de validação, e garantem a restauração de dados em casos de falha;
- **Concorrência:** um bom SGBD permite que múltiplos usuários acessem e modifiquem os dados simultaneamente sem causar conflitos, mantendo a integridade do sistema graças ao isolamento das transações;
- **Abstração da complexidade:** ao usar um SGBD, o programador não precisa se preocupar com os detalhes de baixo nível do armazenamento de dados, podendo se concentrar nas operações de consulta e manipulação de dados de forma mais intuitiva e eficiente.

É justamente nesse contexto que usaremos SGBD neste livro, possibilitando o gerenciamento de bancos de dados e o aprofundamento do conteúdo com exemplos e casos de uso compatíveis com as realidades do mercado.

#### CURIOSIDADE

Os **primeiros SGBDs** nasceram entre as décadas de 1960 e 1970, sendo um dos mais renomados nessa época o IBM Information Management System (IMS) ou Sistema de Gerenciamento de Informação, possuindo importante papel de auxílio ao governo norte-americano em sua empreitada para chegar à Lua com a National Aeronautics and Space Administration (Nasa). Outro importante fato histórico envolvendo os SGBDs ocorreu em seguida, na década de 1970, com a criação do IBM System R e do Oracle, os primeiros a utilizarem o que se conhece atualmente como modelo relacional e SQL.

## Tipos de bancos de dados: relacional e não relacional

Embora existam inúmeros tipos de banco de dados, tais como hierárquicos, orientados a objetos, em memória, entre outros, a maior parte das aplicações se divide atualmente em dois principais tipos: bancos de dados relacionais e não relacionais, sendo importante o entendimento de que cada tipo atende a diferentes necessidades e contextos de uso.

Os **bancos de dados relacionais** são os mais tradicionais e os que possuem maior representatividade de uso no mercado, principalmente em razão de terem sido criados primeiro. Como principais características, os bancos de dados relacionais são mais consistentes e organizados, pois distribuem todos os registros de uma tabela da mesma forma, como uma planilha. Esses bancos são amplamente utilizados em setores como finanças, saúde e gestão empresarial, controle de estoque e de pedidos, ou seja, em contextos em que a consistência e a integridade dos dados são fundamentais e os formatos desses dados são facilmente previsíveis e padronizados.

Por outro lado, os **bancos de dados não relacionais**, relativamente mais recentes, ganharam espaço em aplicações que demandam maior flexibilidade, como redes sociais, análise de dados em tempo real e sistemas com grandes volumes de dados não estruturados. Eles podem usar diferentes formatos, como documentos, colunas, ou grafos, sendo exemplos populares MongoDB, Cassandra e Neo4j. Setores como mídia e rede social, internet das coisas (IoT, *internet of things*), *games* e aplicativos descentralizados frequentemente utilizam esses bancos para gerenciar dados dinâmicos, como perfis de usuários, recomendações de produtos e fluxos de interações.

Além disso, existem **outros tipos de bancos de dados** que, embora menos utilizados em geral, têm aplicações específicas. Bancos de dados orientados a grafos, por exemplo, são ideais para representar conexões

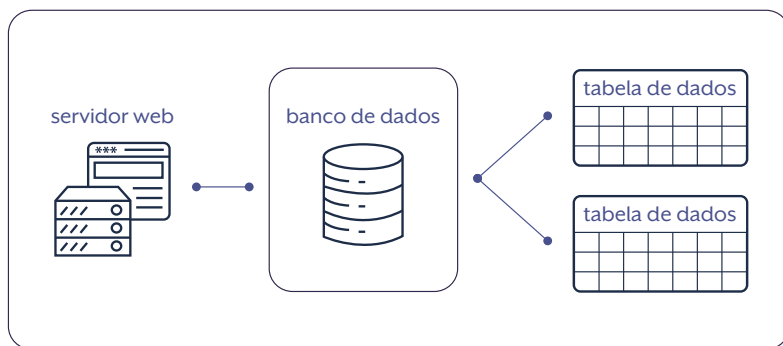
complexas, como em redes sociais ou sistemas de recomendação. Já os bancos de dados em memória (*in-memory databases*) priorizam velocidade, sendo amplamente utilizados em sistemas que exigem respostas rápidas, como *cache* e processamento de eventos em tempo real.

Compreender essa diversidade é crucial para aproveitar o potencial dos diferentes modelos e escolher o mais adequado às demandas de cada projeto.

## Banco relacional

Os bancos relacionais são os mais populares e amplamente utilizados, sendo uma das formas mais comuns de gerenciar e organizar dados, utilizando um modelo de fácil entendimento formado por **tabelas**, **linhas** e **colunas**.

Esses bancos usam uma linguagem chamada *Structured Query Language* (SQL) ou Linguagem de Consulta Estruturada, variando ligeiramente a sintaxe e alguns comandos, dependendo do modelo de banco selecionado.



**Figura 1.1** – Banco de dados e relação com tabelas.

Crédito: Palimpsestos.

### Estrutura

Para entendermos claramente o modo como esse tipo de banco organiza seus dados, basta fazermos uma analogia com o processo de criação de planilhas.

Nesse caso, para efeito de exemplificação, imagine que você deverá realizar um cadastramento de alunos em uma planilha, a qual deverá conter nome, registro do aluno (RA), data de nascimento e *e-mail*. Provavelmente,

após saber dessa necessidade, você estruturaria a planilha mencionada como mostrado na **Tabela 1.1**, não é mesmo?

**Tabela 1.1** – Registro de alunos em uma tabela

| Alunos                |           |                     |                    |
|-----------------------|-----------|---------------------|--------------------|
| Nome                  | RA        | e-mail              | Data de nascimento |
| João Silva            | 458445758 | joaosilva@senai.com | 01/01/1999         |
| Mário Luízio          | 458436645 |                     | 04/08/1961         |
| Camila Maria Oliveira | 177852344 | camila@senai.com    | 08/01/1986         |
| Ana Clara Cruz        | 546388882 | ana.clara@senai.com | 12/06/2000         |
| Maurício Santos       | 777445555 | mauricio@senai.com  | 04/04/2004         |
| Maria Medeiros        | 112529977 | medeiros@senai.com  | 08/09/2002         |

Crédito: do elaborador, 2025.

Agora que já possuímos nossa planilha de referência, podemos associar mais facilmente as terminologias existentes nos bancos relacionais:

- **Tabela ou entidade:** os dados são organizados em tabelas, que consistem em linhas e colunas, e cada tabela representa o que chamamos de “Entidades”. No exemplo, a entidade “Alunos”;
- **Colunas:** cada coluna representa um atributo da entidade, ou seja, um atributo do aluno, possuindo sempre um tipo, podendo ser texto, numérico, datas, entre outros que veremos mais adiante;
- **Linhas:** a linha é o registro de uma entidade, ou seja, no exemplo é o registro de um aluno, contendo todas as informações prescritas em cada coluna, isto é, em cada linha haverá o cadastro de um aluno contendo nome, RA, *e-mail* e data de nascimento.

#### IMPORTANTE

Uma importante característica do banco relacional – que você já deve perceber no exemplo – é que em cada coluna, no momento de construção da tabela, é determinado pelo programador o **tipo de informação** que deve estar contida nela, ou seja, na coluna “Data de nascimento”



**somente será possível colocar datas. Isso proporciona ao banco maior organização e padronização dos dados.**

#### SAIBA MAIS

##### Tabela e relação

É importante saber que, embora o termo “tabela” seja amplamente utilizado em contextos práticos, o termo formal da teoria relacional é “relação”. Tecnicamente, segundo o conceito da “teoria dos conjuntos”, uma relação é um conjunto de tuplas únicas (linhas) que segue princípios matemáticos, como ausência de duplicidade, ordem irrelevante das tuplas e atributos. Contudo, para melhorar a compreensão, iremos utilizar neste livro a conceituação prática do termo, uma vez que a associação com tabelas se torna mais concreta e visual.

#### Valores nulos

Entender os bancos relacionais como tabelas ou planilhas, embora seja uma analogia simplificada, torna-se muito útil, pois podemos notar características importantes que estão sempre presentes nesse tipo de banco, fazendo-nos compreender também sua aplicação.

Um fato importante que devemos observar em nosso exemplo é que cada linha/registro possui a obrigatoriedade de preenchimento de todas as informações exigidas pelas colunas, ou seja, se coletarmos qualquer linha, podemos obter sempre a **mesma quantidade de informações**, que nesse caso passa a ser: nome, RA, *e-mail* e data de nascimento.

Mas nesse momento você deve se perguntar: **e se a informação de uma coluna for opcional?** E a resposta é simples: preenchamos essa coluna com dados nulos, representados por ***null***. É como deveremos proceder no caso da **Tabela 1.2**, em que a informação de *e-mail* do aluno destacado é inexistente.

**Tabela 1.2** – Tabela de alunos com valor nulo

| Alunos                |                  |                     |                    |
|-----------------------|------------------|---------------------|--------------------|
| Nome                  | RA               | e-mail              | Data de nascimento |
| João Silva            | 458445758        | joaosilva@senai.com | 01/01/1999         |
| <b>Mário Luízio</b>   | <b>458436645</b> | <b>null</b>         | <b>04/08/1961</b>  |
| Camila Maria Oliveira | 177852344        | camila@senai.com    | 08/01/1986         |
| Ana Clara Cruz        | 546388882        | ana.clara@senai.com | 12/06/2000         |
| Maurício Santos       | 777445555        | mauricio@senai.com  | 04/04/2004         |
| Maria Medeiros        | 112529977        | medeiros@senai.com  | 08/09/2002         |

Crédito: do elaborador, 2025.

Contudo, é importante entender as implicações desse procedimento, tanto na modelagem do banco de dados quanto na lógica da aplicação.

O uso de valores nulos precisa ser cuidadosamente planejado durante a modelagem de dados, pois ele pode afetar a integridade e a consistência do banco. Em muitos casos, a presença de valores nulos pode ser interpretada de maneiras diferentes. Por exemplo, um campo nulo pode indicar:

- Que o dado ainda não foi preenchido;
- Que o dado não se aplica àquele registro específico;
- Que o dado foi deliberadamente omitido ou desconhecido.

Além disso, o uso excessivo de valores nulos pode levar a complicações nas consultas (*queries*) e na performance do banco de dados, já que operações de comparação com valores nulos geralmente exigem condições específicas (como **IS NULL** ou **IS NOT NULL**) que podem complicar a lógica da aplicação.

Na lógica da aplicação, o tratamento de valores nulos deve ser feito com atenção. Quando os dados nulos são lidos, a aplicação precisa saber como lidar com essa ausência de informação. Isso pode envolver a exibição de um valor padrão (como um texto “Não informado”) ou a implementação de verificações adicionais para garantir que o comportamento do sistema não seja comprometido.

Em resumo, os valores nulos desempenham um papel importante na modelagem de bancos de dados relacionais e na lógica de aplicações, mas seu uso requer cuidados para garantir que a integridade dos dados seja mantida e que o sistema funcione conforme o esperado.

### Chaves primárias

Dando continuidade ao nosso exemplo da tabela de alunos, vamos supor que você necessite saber qual a quantidade de alunos existentes em sua planilha, bem como a posição de cada um em relação ao total. O que você faria?

Muito provavelmente você deve ter pensado em colocar um índice numérico em cada linha, pois isso facilitaria a identificação tanto do total de alunos (observando o último da lista) quanto da posição individual de cada aluno.

**Tabela 1.3** – Tabela de alunos com índice

| Alunos |                       |           |                     |                    |
|--------|-----------------------|-----------|---------------------|--------------------|
| Índice | Nome                  | RA        | e-mail              | Data de nascimento |
| 1      | João Silva            | 458445758 | joaosilva@senai.com | 01/01/1999         |
| 2      | Mário Luízio          | 458436645 |                     | 04/08/1961         |
| 3      | Camila Maria Oliveira | 177852344 | camila@senai.com    | 08/01/1986         |
| 4      | Ana Clara Cruz        | 546388882 | ana.clara@senai.com | 12/06/2000         |
| 5      | Maurício Santos       | 777445555 | mauricio@senai.com  | 04/04/2004         |
| 6      | Maria Medeiros        | 112529977 | medeiros@senai.com  | 08/09/2002         |

Crédito: do elaborador, 2025.

De forma análoga, os bancos de dados relacionais também possuem obrigatoriamente esse índice em todos os registros de uma tabela.

Também conhecidos pelo nome de **id**, esses índices são extremamente importantes, pois servem também como um identificador único de cada

linha/entidade, isto é, mesmo que se repitam todas as informações, cada linha ainda assim passa a ser única para a base de dados, pois **os índices nunca se repetem**.

Dada sua importância para um banco relacional, esses ids são também denominados tecnicamente como **chaves primárias** de uma tabela, podendo ser representados por números sequenciais ou não, ou até mesmo textos, desde que se obedeça à regra principal: cada id deve ser único, exclusivo em cada tabela, imutável e, conseqüentemente, não nulo.

#### IMPORTANTE

**Em alguns casos, uma chave primária composta pode ser utilizada. Isso significa que, em vez de um único campo, a chave primária é formada por uma combinação de dois ou mais campos. Por exemplo, se uma tabela de matrículas de alunos envolver não apenas o identificador do aluno, mas também o identificador do curso, a combinação desses dois campos poderia formar a chave primária composta, garantindo que cada registro seja único, com base nessa dupla identificação. Mas fique tranquilo, pois, nas seções de exemplos e casos de uso deste livro, detalharemos como esse processo é formado.**

#### Chaves estrangeiras

Já evoluímos bastante no entendimento básico de um banco relacional com o exemplo de cadastramento de alunos em uma planilha, e aproveitaremos esse mesmo contexto para que o(a) aluno(a) consiga entender o que são chaves estrangeiras, outra peça fundamental nesse quebra-cabeça.

Vamos supor que agora você tenha que identificar qual o curso em que cada aluno está matriculado. Para efeito de simplificação, nesse primeiro momento consideraremos que cada estudante está matriculado em um único curso.

**Tabela 1.4** – Tabela de alunos contendo o curso

| Alunos |                       |           |                     |                    |                                           |
|--------|-----------------------|-----------|---------------------|--------------------|-------------------------------------------|
| Id     | Nome                  | RA        | e-mail              | Data de nascimento | Curso                                     |
| 1      | João Silva            | 458445758 | joaosilva@senai.com | 01/01/1999         | <b>Análise e Desenvolvimento Sistemas</b> |
| 2      | Mário Luízio          | 458436645 |                     | 04/08/1961         | <b>Medicina</b>                           |
| 3      | Camila Maria Oliveira | 177852344 | camila@senai.com    | 08/01/1986         | <b>Engenharia Mecatrônica</b>             |
| 4      | Ana Clara Cruz        | 546388882 | ana.clara@senai.com | 12/06/2000         | <b>Medicina</b>                           |
| 5      | Maurício Santos       | 777445555 | mauricio@senai.com  | 04/04/2004         | <b>Análise Des. Sistemas</b>              |
| 6      | Maria Medeiros        | 112529977 | medeiros@senai.com  | 08/09/2002         | <b>análise de sistemas</b>                |

Crédito: do elaborador, 2025.

Embora o objetivo tenha sido alcançado, note o que acaba de acontecer em nosso exemplo: há uma falta de padronização no preenchimento do nome de alguns cursos, mais especificamente, o curso de Análise de Desenvolvimento de Sistemas foi escrito de três maneiras diferentes. Esse fato torna-se muito comum em situações em que há campos livres que não possuem nenhum tipo de padronização ou validação.

Outro problema pode ocorrer caso necessitemos adicionar mais informações que são vinculadas diretamente ao curso, por exemplo, seu valor de mensalidade.

**Tabela 1.5** – Tabela de alunos com valores de mensalidade

| Alunos |                       |           |                     |                    |                                    |              |
|--------|-----------------------|-----------|---------------------|--------------------|------------------------------------|--------------|
| Id     | Nome                  | RA        | e-mail              | Data de nascimento | Curso                              | Valor        |
| 1      | João Silva            | 458445758 | joaosilva@senai.com | 01/01/1999         | Análise e Desenvolvimento Sistemas | R\$ 1 200,00 |
| 2      | Mário Luízio          | 458436645 | null                | 04/08/1961         | Medicina                           | R\$ 9 800,00 |
| 3      | Camila Maria Oliveira | 177852344 | camila@senai.com    | 08/01/1986         | Engenharia Mecatrônica             | R\$ 1 500,00 |
| 4      | Ana Clara Cruz        | 546388882 | ana.clara@senai.com | 12/06/2000         | Medicina                           | R\$ 9 000,00 |
| 5      | Maurício Santos       | 777445555 | mauricio@senai.com  | 04/04/2004         | Análise Des. Sistemas              | R\$ 1 200,00 |
| 6      | Maria Medeiros        | 112529977 | medeiros@senai.com  | 08/09/2002         | análise de sistemas                | R\$ 1 200,00 |

Crédito: do elaborador, 2025.

Como adicionamos o curso diretamente em nossa tabela de alunos, se quisermos saber o valor de sua mensalidade, devemos adicionar mais um campo e repeti-lo, o que acaba de gerar mais dois problemas ao nosso cenário fictício.

O primeiro problema é que não há nenhuma garantia de que o valor adicionado na tabela seja correspondente ao respectivo curso, podendo inclusive gerar situações de inconsistência no preenchimento, o que claramente ocorreu em nosso exemplo, pois existem dois valores de mensalidade diferentes para o mesmo curso de Medicina. Além disso, o segundo problema seria gerado caso houvesse a necessidade de atualização do valor de mensalidade de algum dos cursos, surgindo a necessidade de se alterar individualmente cada valor na tabela de alunos.

Se analisarmos com mais atenção, todos esses problemas ocorreram porque estamos misturando duas entidades que deveriam ser de fato independentes, isto é, alunos e cursos.

Para resolução de todos esses percalços, podemos proceder com a separação das tabelas, relacionando-as pelo uso de suas chaves primárias. Para isso, criamos primeiro nossa tabela de cursos, podendo definir o nome a ser padronizado, bem como o valor correto da mensalidade.

**Tabela 1.6** – Tabela de cursos

| Cursos |                                    |              |
|--------|------------------------------------|--------------|
| Id     | Nome                               | Valor        |
| 1      | Análise e Desenvolvimento Sistemas | R\$ 1 200,00 |
| 2      | Medicina                           | R\$ 9 800,00 |
| 3      | Engenharia Mecatrônica             | R\$ 1 500,00 |

Crédito: do elaborador, 2025.

Em seguida, ajustamos a tabela de alunos para que passe a usar como referência ao curso apenas o id identificador deste, ou seja, a chave primária da tabela de cursos.

**Tabela 1.7** – Tabela de cursos

| Alunos |                       |           |                     |                    |          |
|--------|-----------------------|-----------|---------------------|--------------------|----------|
| Id     | Nome                  | RA        | e-mail              | Data de nascimento | Curso Id |
| 1      | João Silva            | 458445758 | joaosilva@senai.com | 01/01/1999         | 1        |
| 2      | Mário Luízio          | 458436645 |                     | 04/08/1961         | 2        |
| 3      | Camila Maria Oliveira | 177852344 | camila@senai.com    | 08/01/1986         | 3        |
| 4      | Ana Clara Cruz        | 546388882 | ana.clara@senai.com | 12/06/2000         | 2        |
| 5      | Maurício Santos       | 777445555 | mauricio@senai.com  | 04/04/2004         | 1        |
| 6      | Maria Medeiros        | 112529977 | medeiros@senai.com  | 08/09/2002         | 1        |

Crédito: do elaborador, 2025.

Agora você talvez possa se questionar: a tabela de alunos passa a possuir então duas chaves primárias (Id e Curso Id)?

Quase isso! Para a tabela de alunos, ela continua a considerar como sua chave primária a coluna “Id”, porém, também passa a entender que a coluna “Curso Id” é uma chave primária de origem externa, ou seja, uma **chave estrangeira** que a vinculará à outra tabela, que no caso passa a ser a de cursos.

Uma chave estrangeira é um campo em uma tabela que faz referência à chave primária de outra tabela, garantindo desse modo que as informações entre elas estejam relacionadas corretamente, com foco em permitir a integridade dos dados. Essa relação assegura que, por exemplo, todo aluno seja vinculado a um curso válido.

Isso nos leva a um conceito muito importante nos bancos de dados relacionais: a **integridade referencial**, isto é, a premissa que garante que a chave estrangeira de um aluno sempre faça referência a uma chave primária válida na tabela de cursos. Isso impede que haja, por exemplo, um aluno matriculado em um curso que não exista, ou que uma matrícula de aluno seja removida sem que o registro do curso correspondente seja tratado adequadamente, aumentando, portanto, a consistência dos dados da aplicação como um todo.

Dessa maneira, de modo simplificado, com o uso de chaves estrangeiras conseguimos resolver todos os problemas anteriores:

- **Padronização dos dados e diminuição de inconsistências**, com o uso de chaves estrangeiras. Como a única parte a se repetir é o id do curso, temos uma certeza de que todos os alunos que foram endereçados ao mesmo curso passem a ter as mesmas informações de nome e valor de mensalidade;
- **Diminuição da complexidade de atualização**, pois, caso necessitemos alterar o valor da mensalidade de algum curso, bastar alterarmos diretamente na tabela de cursos.



Em resumo, a chave estrangeira é uma ferramenta essencial para garantir que os dados sejam inseridos e mantidos de forma coerente, e a integridade referencial assegura que as relações entre os dados sejam válidas e não causem erros no sistema.

## Aplicação

Neste momento, por meio da apresentação gradual de diversos cenários e analogias, esperamos que tenha ficado clara a identificação de algumas características fundamentais pertinentes ao uso de bancos de dados relacionais, as quais iremos apresentar no box **Na prática**, a seguir.

### NA PRÁTICA

#### Banco de dados relacional: características fundamentais

- É formado por tabelas que representam entidades;
- Cada entidade/linha de uma tabela possui a mesma quantidade de informações e, nos casos em que são opcionais, passam a ser preenchidas com valor **null**;
- Toda tabela possui um identificador único em cada linha, denominado chave primária;
- Tabelas podem relacionar-se entre si para garantir padronização e consistência e para evitar repetições, usando como referência chaves primárias externas, denominadas chaves estrangeiras.

Podemos considerar que os bancos relacionais ainda dominam o mercado tanto pelo fato de terem sido os precursores como também por apresentarem simplicidade de uso e serem facilmente comparados a planilhas.

De modo resumido, podemos inferir que, em aplicações em que as entidades devem apresentar sempre a mesma estrutura de informações, o uso de bancos de dados relacionais contribui eficientemente para a normalização, a padronização e a validação dos registros, garantindo por consequência a qualidade do *software* no qual é empregado. Por exemplo, em um sistema de gerenciamento hospitalar, tabelas relacionais podem ser utilizadas para

organizar dados de pacientes, em que cada registro segue a mesma estrutura, contendo campos como nome, data de nascimento, número do prontuário e histórico médico. Essa consistência permite consultas eficientes e evita erros no armazenamento e no cruzamento de informações críticas.

Por outro lado, em cenários mais dinâmicos em que há muitas informações opcionais ou diferentes entre si, o uso de uma estrutura relacional pode não ser indicado em razão da necessidade de preenchimento com dados nulos para cada coluna faltante, ou seja, nesses casos, bancos de dados NoSQL, como o MongoDB, podem ser mais adequados. Por exemplo, em um sistema de catálogo de produtos para um *e-commerce*, em que itens como roupas, eletrônicos e móveis possuem atributos distintos (uma roupa pode ter tamanho e cor, enquanto um eletrônico pode ter tensão de alimentação e garantia), um banco NoSQL permite armazenar cada produto com sua própria estrutura de dados, sem a necessidade de preencher campos irrelevantes com valores nulos.

## Banco não relacional

Em sentido oposto às restrições e organizações presentes em bancos relacionais, os bancos não estruturados, também chamados de não relacionais, têm se destacado bastante em cenários dinâmicos em que a escalabilidade se torna um requisito. Eles oferecem flexibilidade para lidar com grandes volumes de dados que mudam constantemente e podem ter **estruturas variáveis**, como informações provenientes de dispositivos IoT, redes sociais, ou catálogos de produtos com atributos diversos.

Em resumo, são sistemas de gerenciamento de dados que não seguem o modelo relacional tradicional com suas tabelas e chaves, sendo também conhecidos como bancos NoSQL (sem SQL). Esses bancos permitem armazenar e recuperar dados em formatos variados, como documentos JSON, pares de chave-valor, grafos e colunas amplas, dependendo da arquitetura escolhida.

Por exemplo, bancos baseados em documentos, como o MongoDB, são amplamente utilizados em aplicações que demandam alta flexibilidade de estrutura, como redes sociais. Nesses sistemas, cada documento pode conter informações personalizadas, como nome de usuário, postagens e preferências, sem que seja necessário seguir um esquema fixo.

Além disso, bancos de dados baseados em grafos, como o Neo4j, são ideais para modelar relações complexas, como as presentes em sistemas de recomendação (exemplo: sugerir amigos em plataformas sociais ou produtos em *marketplaces*). Por sua vez, bancos orientados a colunas, como Cassandra, são muito usados em soluções que demandam grande volume de dados e alta taxa de escrita, como sistemas de *log* em tempo real.

Essas características tornam os bancos NoSQL especialmente adequados para cenários nos quais a necessidade de escalabilidade horizontal (adição de novos servidores) e a diversidade de estruturas de dados são requisitos críticos. Por outro lado, essa flexibilidade pode sacrificar algumas das garantias de consistência oferecidas por bancos relacionais, o que deve ser considerado ao escolher a solução ideal para uma aplicação.

### JavaScript Object Notation (JSON)

Existem inúmeras formas e linguagens de representação usadas em bancos NoSQL, cada uma seguindo o padrão de sua respectiva orientação, ou seja, bancos não relacionais orientados a documentos obedecem a um formato diverso em relação aos orientados a grafos, que diferem também, por sua vez, dos orientados a chave-valor.

Toda essa diversidade deu-se ao fato de que tais bancos NoSQL tiveram que encontrar uma outra maneira para representar suas entidades em vez do uso de tabelas, uma vez que o formato destas representaria uma dificuldade para se adequar à extensa variabilidade das informações presentes ou ausentes em cada entidade.

Contudo, dentro do universo dos bancos não relacionais, um dos modelos atualmente em maior ascensão e uso é o dos **bancos orientados a documentos**. Estes usam uma forma representativa de dados muito conhecida e largamente aplicada em integrações com *application programming interfaces* (APIs): o JavaScript Object Notation (JSON) ou anotação de objeto JavaScript.

Resumidamente, JSON é um formato leve de intercâmbio de dados, projetado para ser fácil de ler e escrever por humanos, além de facilmente ser interpretado e gerado por sistemas, ganhando tanta popularidade que desvinculou seu uso do local onde foi criado, isto é, em programações usando JavaScript. Dessa maneira, podemos afirmar que a utilização do JSON vai muito além do JavaScript, possibilitando seu uso inclusive no contexto de bancos de dados não relacionais.

A estrutura padrão de um JSON é bem fácil de compreender, consistindo em uma relação de chave e valor em que o valor pode ser um texto, um número, uma expressão booleana ou até mesmo uma outra estrutura de chaves/valores, permitindo, portanto, a transcrição de qualquer informação de uma maneira detalhada, hierárquica e ao mesmo tempo simplificada.

Vejamos a seguir um **exemplo de JSON** retirado de um estoque de produtos de um *e-commerce* fictício.

```
[
  {
    "id": "28",
    "nome": "Ar condicionado inverter 12000 BTU",
    "valor": 2699.99,
    "imagem": "/api/images/ar-inverter-12.png",
    "categoria": {
      "id": 45,
      "nome": "Ar condicionados",
      "escaninho": 55447
    }
  },
]
```

```

    "codigo_barras": 789514422117475513
  },
  {
    "id": "29",
    "nome": "Cooktop 5 bocas acendimento automático",
    "valor": 559.00,
    "imagem": "/api/images/cooktop-5.png",
    "categoria": {
      "id": 64,
      "nome": "Fornos e Cooktops",
      "escaninho": 78004
    },
    "codigo_barras": 789993500140158754
  }
]

```

Após a análise da estrutura desse bloco de código, mesmo que seja seu primeiro contato com um JSON, é muito provável que você tenha facilmente compreendido o modo como a informação é organizada dentro dele, conseguindo diferenciar nesse exemplo os atributos de cada produto (id, nome, valor, imagem, código de barras e categoria), as subestruturas (categoria: id, nome e escaninho) e, finalmente, percebendo também que este JSON trafega dados de dois produtos diferentes dentro de um conjunto/array (produtos 28 e 29).

Conseguir entender e extrair os dados de estruturas JSON similares é fundamental para compreender o funcionamento básico dos bancos não relacionais, pois estes realizam o armazenamento e a gestão de seus registros usando esse padrão, o que veremos a seguir.

## Estrutura

Diferente de uma planilha ou tabela, em que cada linha e coluna precisa seguir uma estrutura bem-definida, os bancos de dados não relacionais permitem que cada conjunto de dados seja único, sem que todos precisem ter

exatamente o mesmo formato. Eles são projetados para lidar com grandes volumes de dados variados – abrangendo dados estruturados, numéricos, textos ou imagens, que estão em constante crescimento e transformação.

Vale ressaltar que toda essa flexibilidade pode ser vantajosa em cenários nos quais é preciso lidar com dados de diferentes tipos e origens, como textos, imagens e dados numéricos, além de permitir o armazenamento de informações que podem mudar rapidamente. Entretanto, essa mesma flexibilidade pode representar um desafio em termos de consistência e validação, já que não há uma estrutura rígida que garanta a integridade das informações de maneira automática, como nos bancos relacionais.

Desse modo, não é correto afirmar que os bancos relacionais possuem tabelas, colunas ou linhas como nos bancos tradicionais, havendo **novas terminologias** para representar o mesmo conceito.

Para explicarmos essas terminologias – e usaremos como base os bancos NoSQL orientados a documentos –, observe o **exemplo de JSON** a seguir, com as informações de alguns alunos.

```
[
  {
    "id": "1",
    "nome": "João Silva",
    "RA": 458445758
  },
  {
    "id": "2",
    "nome": "Mário Luízio",
    "RA": 458436645
  },
  {
    "id": "3",
    "nome": "Camila Maria Oliveira",
    "RA": 177852344
  }
]
```

### Terminologia:

- **Coleções:** em vez de dizer que os bancos NoSQL possuem tabelas, usamos o termo “coleções”, ou seja, o JSON do exemplo representa uma coleção de alunos;
- **Campos ou atributos:** as informações id, nome e RA de cada aluno são nomeadas como campos em vez do uso de colunas;
- **Documento:** cada aluno é considerado uma entidade, e cada entidade é nomeada de documento, algo que nos bancos relacionais seria representado por linhas.

### Flexibilidade

Para auxiliar o entendimento, vamos ao voltar ao cenário escolar utilizado no começo do capítulo ao abordarmos modelo relacional.

Você deve criar uma planilha para coletar dados de alunos, porém, nesse caso, vamos observar alguns pontos que podem sustentar e justificar o uso de um modelo não relacional nesse cenário:

- Em uma escola, alunos podem estar matriculados em um ou mais cursos;
- Os alunos podem apresentar informações diferentes de cadastro, por exemplo, não possuir telefone fixo ou ter mais de um telefone móvel. Podem também diferir na quantidade de endereços;
- Os cursos podem diferir na quantidade de informações se forem de modalidades presenciais ou à distância.

Diante desses critérios, observe o **exemplo de JSON** de alunos a seguir, retirado dessa exemplificação de um banco não relacional.

```
[  
  {  
    "id": "1",  
    "nome": "João Silva",
```

```

    "RA": 458445758,
    "email": "joaosilva@senai.com",
    "data_nascimento": "01/01/1999",
    "cursos": ["1","4"],
    "celular": ["19981818181"]
  },
  {
    "id": "2",
    "nome": "Mário Luízio",
    "RA": 458436645,
    "data_nascimento": "04/08/1961",
    "cursos": ["2"],
    "celular": ["11901010100","11922222233"]
  },
  {
    "id": "3",
    "nome": "Camila Maria Oliveira",
    "RA": 177852344,
    "email": "camila@senai.com",
    "data_nascimento": "08/01/1986",
    "cursos": ["3"],
    "celular": ["15985858585"],
    "bolsa": {
      "id": "333",
      "tipo": "DESCONTO",
      "desconto": 750.00
    }
  }
]

```

Perceba que, embora todas as estruturas se refiram aos alunos da mesma escola, existem certos atributos que são distintos entre eles, o que não seria uma tarefa fácil de se lidar em um relacionamento tradicional do banco.



Vamos observar alguns desses atributos:

- Note que o primeiro aluno, de nome “João Silva”, possui matrícula em dois cursos nessa escola, sendo essa uma vantagem interessante do banco não relacional: podemos utilizar estruturas como conjuntos/*arrays* em um atributo de forma simplificada, algo um pouco mais complexo para se fazer em um banco tradicional, pois exigiria a criação de uma tabela adicional para se obter o mesmo resultado;
- Já o segundo aluno, de nome “Mário Luízio”, não possui o atributo *e-mail*, não sendo necessário preenchê-lo com *null* como nos bancos relacionais. Além disso, ele tem em seu cadastro dois números de telefone para contato;
- Por fim, outra observação interessante de salientar é que a terceira aluna, de nome “Camila Maria Oliveira”, possui o benefício de bolsa de estudos, apresentando consequentemente uma estrutura completamente nova e inexistente em todos os outros alunos destacados, algo que também exigiria uma abordagem um pouco mais complexa em um banco relacional.

Para finalizar essa análise e reafirmar o conceito, vamos também observar como ficaria a coleção de cursos.

```
[
  {
    "id": "1",
    "nome": "Análise e Desenvolvimento Sistemas",
    "valor": 1200.00,
    "plataforma": "moodle"
  },
  {
    "id": "2",
    "nome": "Medicina",
    "valor": 9800.00,
    "polos": [
```

```

        "Campus Santa Bárbara"
    ]
},
{
    "id": "3",
    "nome": "Engenharia Mecatrônica",
    "valor": 1500.00,
    "polos": [
        "Campus Juscelino Kubitschek",
        "Campus Santa Bárbara",
        "Campus São Benedito"
    ]
}
]

```

Observe que, da mesma forma como ocorre com a coleção de alunos, alguns cursos possuem atributos distintos, como é o caso do curso de id 1, que por ser modalidade à distância não apresenta locais físicos (polos) e possui a propriedade de indicação do tipo de plataforma.

#### SAIBA MAIS

### MERN Stack

Você já ouviu falar da MERN Stack? Trata-se de uma abordagem tecnológica que propõe o uso de *frameworks* e soluções que dão o nome da sigla: **M**ongo DB, **E**xpress, **R**eact e **N**ode, prevendo a utilização de um dos mais famosos bancos NoSQL, isto é, o Mongo DB.

A MERN Stack é frequentemente utilizada em aplicações *web* que exigem alta escalabilidade e desenvolvimento ágil, como plataformas de *e-commerce* com grandes volumes de dados de produtos e usuários, aplicações de *streaming* de vídeo com necessidade de lidar com grandes quantidades de informações em tempo real ou, ainda, plataformas de redes sociais para gerenciar interações e conteúdo gerado pelos usuários. A flexibilidade do MongoDB, combinada com a velocidade do Node.js e a capacidade de renderização do React, torna a MERN Stack uma escolha popular para projetos modernos e complexos.

## Aplicação

Sem dúvida, o paradigma utilizado pelos bancos NoSQL pode fascinar os desenvolvedores cansados da estrutura relacional, chamando a atenção principalmente durante os momentos em que se torna necessário fazer mudanças bruscas na base de dados em certas aplicações.

No boxe **Na prática**, veremos algumas das **características** mais marcantes e que valem ser evidenciadas sobre a estrutura não relacional orientada a documentos.

### NA PRÁTICA

#### **Banco de dados não relacional: características fundamentais**

- Possui atributos chamados campos e, quando opcionais, dispensa o uso de valores nulos como o *null*;
- Não apresenta tabelas nem linhas, substituídas respectivamente por coleções e documentos, sendo que estes últimos são utilizados para representar as entidades;
- Tem grande flexibilidade devido ao fato de poder apresentar atributos distintos para cada documento, muito útil em cenários dinâmicos;
- Os dados são apresentados em formato que pode variar segundo a orientação do banco, porém, nos bancos orientados a documentos, o formato JSON acaba acrescentando simplicidade e flexibilidade, podendo utilizar, além da tipagem tradicional como textos, números e expressões booleanas, até mesmo subestruturas ou conjuntos como *arrays*;
- Diminui a complexidade do *backend*, ou seja, do lado do servidor da aplicação, uma vez que a lógica do código se alinha melhor ao formato dos dados no banco, reduzindo a necessidade de transformações complexas e otimizando o desempenho das operações.

**IMPORTANTE****Vantagens e desvantagens**

Podemos concluir então que a solução do “momento” é sempre optar pelo uso de bancos NoSQL? Eles apresentam apenas vantagens? E a resposta é claramente não, pois devemos sempre entender que cada solução apresenta um contexto de vantagens e desvantagens. Sem dúvida, os bancos NoSQL diminuem a complexidade de criação e gestão da base de dados e em partes do *backend*, contudo, pensando em um contexto mais amplo, exigem cuidados maiores quanto à possível ausência de propriedades ou atributos, podendo gerar erros inesperados tanto no *backend* quanto no *frontend* das aplicações em que é usado, além de poderem deixar a codificação um tanto quanto bagunçada se não usados da forma correta.

Resumidamente, as soluções não relacionais agregam flexibilidade e escalabilidade se usadas corretamente em aplicações que exigem esse dinamismo, caso contrário, em situações com pouca ou nenhuma variabilidade e até mesmo em condições em que se faz necessária a normalização dos dados, a melhor solução continua sendo os bancos tradicionais que fazem uso do esquema relacional SQL.

## Sistema de Gerenciamento de Banco de Dados (SGBD)

SGBD, sigla para Sistema de Gerenciamento de Banco de Dados, consiste em um *software* criado para servir efetivamente de intermediário entre os usuários e a base de dados.

Não é a única e exclusiva maneira de se trabalhar com banco de dados, visto que praticamente todos os tipos de bancos possibilitam que se utilize o terminal de comando do respectivo sistema operacional em que está sendo executado. Contudo, é de se convir que utilizar tais terminais de comando acaba por ser uma forma mais trabalhosa quando não se trata de modificações pontuais, por exemplo, durante o desenvolvimento de uma nova aplicação.

## SAIBA MAIS

## DBMS

SGBD é a terminologia em português, porém, caso você consulte alguma documentação em inglês, deve procurar pelo termo **Database Management System** (DBMS), que possui o mesmo significado.

Dessa maneira, os SGBDs foram desenvolvidos para agilizar as operações com os mais variados bancos de dados, evitando o uso de *prompts* de comandos tradicionais, embora essa opção ainda continue disponível caso o desenvolvedor deseje utilizá-la.

Em geral, cada tipo e modelo de banco de dados possui um ou mais SGBDs dedicados, não existindo um SGBD com abrangência para 100% das opções de banco existentes, devendo o desenvolvedor instalá-los para cada modelo desejado.

Elencamos a seguir alguns dos **principais SGBDs** de que o(a) aluno(a) deve ter conhecimento:

- **Bancos de dados relacionais:** entre os SGBDs mais populares estão o MySQL, amplamente usado em sistemas *web*; o PostgreSQL, reconhecido por sua robustez e conformidade com padrões; o Oracle Database, muito empregado em grandes corporações; e o Microsoft SQL Server, famoso por sua integração com o ecossistema Windows;
- **Bancos de dados NoSQL:** para necessidades específicas, destacam-se o MongoDB, utilizado em aplicações que demandam flexibilidade de estrutura; o Cassandra, ideal para grandes volumes de dados distribuídos; o Redis, amplamente usado para armazenamento em *cache* e dados chave-valor; e o Neo4j, especializado em grafos, com aplicações em redes sociais e sistemas de recomendação.

## Principais opções do mercado

Embora o contexto tecnológico seja extremamente dinâmico e volátil, o que acaba repercutindo na criação de novas soluções em detrimento das antigas, listaremos nos **Quadros 1.1** e **1.2** as principais opções de mercado relevantes para o uso de bancos de dados, relacionais ou não, no momento da elaboração deste livro.

**Quadro 1.1** – Opções de bancos relacionais

| Nome       | Open source | Licença            | Site oficial   | SGBDs                       | Plataformas             |
|------------|-------------|--------------------|----------------|-----------------------------|-------------------------|
| MySQL      | Sim         | GPL                | mysql.com      | MySQL Workbench             | Windows, Linux, macOS   |
| PostgreSQL | Sim         | PostgreSQL License | postgresql.org | pgAdmin                     | Windows, Linux, macOS   |
| MariaDB    | Sim         | GPL                | mariadb.org    | Dbeaver, Database Workbench | Windows, Linux, macOS   |
| Oracle DB  | Não         | Comercial          | oracle.com     | Oracle SQL Developer        | Windows, Linux, Solaris |
| SQL Server | Não         | Comercial          | microsoft.com  | SQL Server Studio           | Windows, Linux          |

Crédito: do elaborador, 2025.

**Quadro 1.2** – Opções de bancos não relacionais

| Nome      | Open Source | Licença            | Site oficial         | SGBDs                        | Plataformas           |
|-----------|-------------|--------------------|----------------------|------------------------------|-----------------------|
| MongoDB   | Sim         | SSPL               | mongodb.com          | Atlas, MongoDB Compass       | Windows, Linux, macOS |
| Cassandra | Sim         | Apache License 2.0 | cassandra.apache.org | DataStax Studio              | Windows, Linux, macOS |
| DynamoDB  | Não         | Comercial          | aws.amazon.com       | NoSQL Workbench for DynamoDB | AWS Cloud             |
| Firebase  | Não         | Comercial          | firebase.google.com  | –                            | Web, Android, iOS     |

Crédito: do elaborador, 2025.

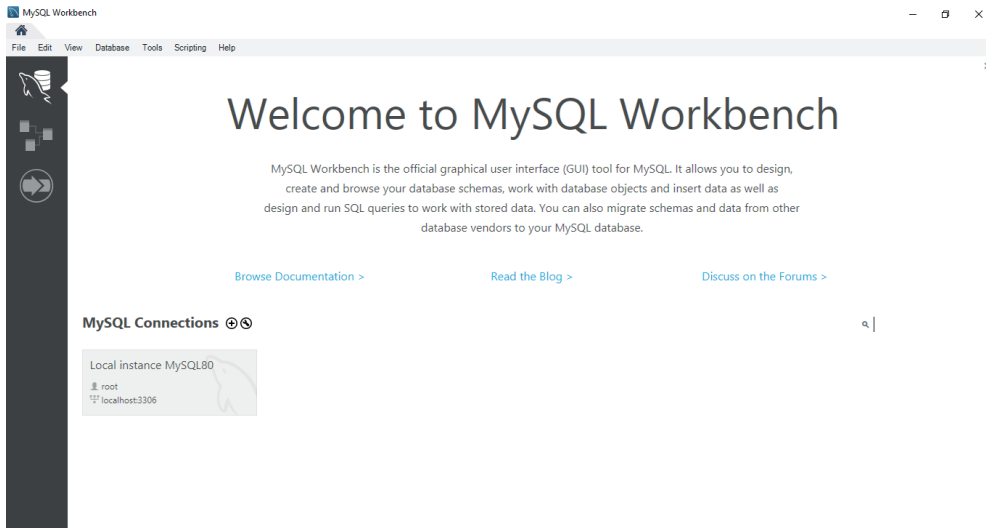
## Instalação SQL – MySQL Workbench

Existem inúmeras opções disponíveis de bancos de dados e SGBDs para se utilizar. Contudo, como necessitamos escolher uma delas para dar enfoque neste livro, usaremos o MySQL como banco de dados e para auxiliar na interação com o servidor escolheremos o SGBD MySQL Workbench, sendo essa combinação uma das mais utilizadas no uso de banco relacional na atualidade.

Justamente devido à rápida variação das versões de *softwares* e instaladores, optamos por não detalhar passo a passo cada etapa do processo de instalação, para que este livro não fique defasado ou confunda alunos(as) ainda inexperientes.

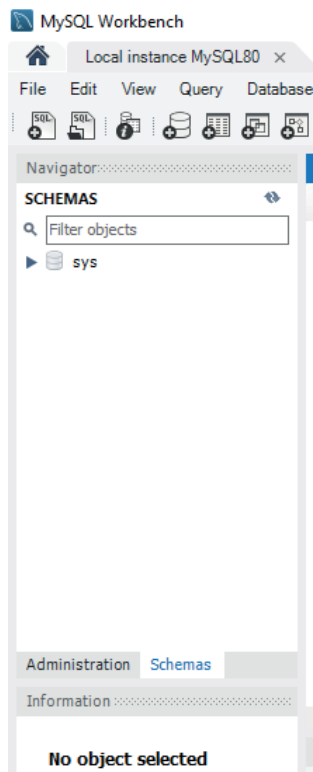
Dessa maneira, elencamos a seguir as principais dicas e etapas de instalação para referência, mas aconselhamos que o(a) aluno(a) faça uma breve pesquisa no *site* oficial da MySQL (disponível em: <https://www.mysql.com>; acesso em: 13 maio 2025) ou procure vídeos na internet para melhor acompanhamento:

1. Faça o *download* do instalador do MySQL no *site* oficial, atentando-se para utilizar o instalador que provê o MySQL Server e o MySQL Workbench, caso contrário seu ambiente estará incompleto;
2. Durante a instalação, defina o usuário e a senha de acesso ao servidor, geralmente sendo utilizado para ambos o termo **root**;
3. Após a instalação, abra o MySQL Workbench e teste se com o usuário e a senha configurados é possível estabelecer conexão com o MySQL Server.



**Figura 1.2** – Tela do SGBD MySQL Workbench, com conexão local de servidor.

Crédito: do elaborador, 2025.



**Figura 1.3** – Conexão ativa com MySQL Workbench.

Crédito: do elaborador, 2025.



**DESAFIO: STARTUP****Contexto**

Você foi contratado como consultor de TI por uma *startup* que está desenvolvendo uma plataforma digital com dois módulos principais:

1. Módulo de gerenciamento de clientes:
  - O **objetivo** é registrar os dados de clientes, incluindo nome, *e-mail*, CPF, endereço e histórico de compras;
  - É essencial garantir que os dados sejam **consistentes**, sem duplicidade, e que possam ser consultados com facilidade para geração de relatórios financeiros;
  - O módulo também deverá permitir a criação de diferentes **níveis de acesso** para os usuários da equipe, como administradores e analistas.
2. Módulo de recomendações personalizadas:
  - Este módulo utiliza os dados de **navegação e compras** dos clientes para sugerir produtos que podem interessá-los;
  - Cada cliente pode ter preferências diferentes, e a **estrutura de dados** para capturar essas interações varia amplamente (por exemplo, tempo gasto em páginas, categorias visualizadas ou cliques em promoções);
  - O sistema precisa ser **altamente escalável** para suportar o crescimento de usuários e interações em tempo real.

A *startup* quer que você recomende o tipo de banco de dados ideal para cada módulo, devendo você apresentar as razões que sustentam sua escolha.

## Resolução

Neste exercício, analisaremos cada módulo do projeto individualmente, considerando que existem arquiteturas de *software* que permitem a utilização, em uma mesma aplicação, de bancos de dados distintos, detalhando as principais razões para cada escolha.

### 1. Módulo de gerenciamento de clientes:

Banco de dados recomendado: relacional (SQL), como MySQL ou PostgreSQL.

Razões:

- Os dados dos clientes têm uma estrutura fixa e previsível (nome, CPF, endereço);
- A necessidade de garantir a integridade dos dados e evitar duplicatas é bem atendida pelas restrições e chaves primárias oferecidas por bancos relacionais;
- Relatórios financeiros e análises podem ser facilmente gerados com consultas SQL;
- Os bancos relacionais oferecem controle de acesso robusto e suporte a transações, essenciais para garantir a consistência dos dados.

### 2. Módulo de recomendações personalizadas

Banco de dados recomendado: não relacional (NoSQL), como MongoDB ou Cassandra.

Razões:

- A estrutura de dados é flexível e pode variar para cada cliente ou interação, algo que é facilmente suportado por bancos orientados a documentos (MongoDB) ou colunas amplas (Cassandra);
- A necessidade de escalabilidade horizontal para lidar com um grande volume de interações e crescimento de usuários é um ponto forte dos bancos NoSQL;
- A alta performance para leitura e escrita em tempo real é bem atendida, garantindo que as recomendações sejam geradas rapidamente.

## Resumo

Ufa! Fizemos importantes avanços, não é mesmo?

Este é o primeiro dos inúmeros passos que percorreremos neste livro, porém esperamos que o(a) aluno(a) tenha conseguido compreender os fundamentos iniciais para se trabalhar com bancos de dados com autonomia.

Neste capítulo, tentamos fazer o(a) aluno(a) entender que o contexto relacional é demasiadamente parecido com qualquer cadastramento de dados em planilhas, possuindo suas respectivas tabelas, colunas e linhas, sendo atualmente o contexto mais utilizado no mercado, especialmente para situações previsíveis, menos dinâmicas e bem-delineadas.

Por outro lado, conseguimos abordar outro contexto, o dos bancos não relacionais, em que a flexibilidade e a simplicidade são as maiores vantagens ofertadas, conceito em amplo crescimento, mas que deve ser utilizado com cautela para evitar projetos mal embasados ou confusos.

Por fim, pudemos conhecer um pouco das principais opções disponíveis no mercado, além dos *softwares* chamados SGBD, que possuem papel importante para auxiliar o desenvolvedor a atuar com mais produtividade na criação e gestão de bases de dados em cenários reais.

No próximo capítulo, desbravaremos um importante conceito chamado modelagem de dados, que é de crucial importância para o planejamento e a organização das bases de dados. A modelagem de dados é um passo essencial que permite estruturar informações de forma eficiente e será fundamental para dar início a diversas etapas do desenvolvimento de *software* das mais variadas aplicações. Até lá!



## CAPÍTULO 2

# MODELAGEM DE DADOS

**A**gora que o(a) aluno(a) já possui uma clara definição da importância dos bancos de dados, bem como seus tipos estruturais, podemos avançar para o conceito de modelagem de dados.

Como desenvolvedor, assuntos envolvendo código podem chamar mais sua atenção, contudo, é valioso ressaltar que a modelagem de dados é uma das etapas mais importantes de qualquer aplicação que utilize bancos de dados, pois afeta positiva ou negativamente todos os processos de desenvolvimento que virão posteriormente.

Uma modelagem de dados deficiente pode gerar consequências graves para o projeto. Por exemplo: a falta de planejamento adequado pode resultar em um banco de dados difícil de manter, exigindo horas extras de trabalho para correções e adaptações, além de gerar frequentemente problemas de

desempenho, como consultas lentas e processamento ineficiente, impactando diretamente a experiência do usuário e a escalabilidade da aplicação.

Em resumo, investir tempo e esforço na modelagem de dados de um projeto é fundamental para garantir a robustez, o desempenho e a confiabilidade da aplicação como um todo, sendo esse o assunto a ser trabalhado neste capítulo. Vamos lá!

## Definição e importância

De uma forma básica, os modelos de dados são uma forma de representar as tabelas e seus respectivos relacionamentos, sendo, portanto, mais importantes em projetos com bancos relacionais, embora também tenham sua utilidade em usos de bases NoSQL para se obter uma visão geral da estrutura de dados.

Em uma analogia simples, a modelagem de dados assemelha-se à fundação de um edifício e, logicamente, a boa ou má modelagem impacta grandemente em todo o processo de desenvolvimento que ocorrerá posteriormente, seja ele *frontend* ou *backend*, e até mesmo na etapa de implantação da aplicação.

O processo de modelagem está intrinsecamente relacionado com a interpretação das regras de negócio da aplicação a ser desenvolvida, portanto, é extremamente importante se ter a definição dessas regras e fluxos de funcionamento antes de se iniciar o trabalho de modelagem.

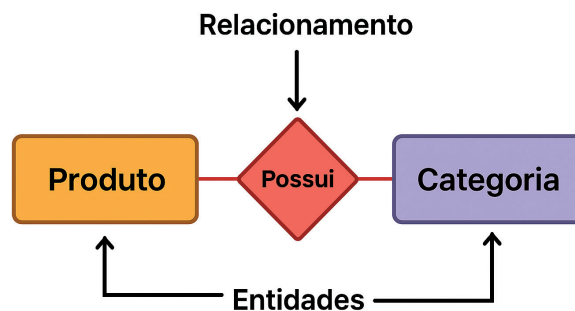
## Modelo entidade relacionamento (MER)

O MER é um modelo conceitual de alto nível, o que significa que ele tem foco na **estrutura lógica dos dados**, independentemente de como eles

serão fisicamente armazenados ou qual sistema de gerenciamento de banco de dados (SGBD) será utilizado.

Retomando a analogia sobre a fundação de um edifício, o MER poderia ser comparado com esboços e croquis de um projeto estrutural, em que seu objetivo principal é demonstrar de forma mais genérica e abstrata o que deve ser feito.

Embora tenha sua importância, sobretudo para auxiliar o entendimento de como aplicações complexas estão estruturadas, geralmente o uso do MER não é o suficiente para que o projeto seja desenvolvido por completo, uma vez que não apresenta detalhes técnicos operacionais. É por esse motivo que não adotaremos esse tipo de representação ao longo deste livro, pois optaremos pelo uso de modelos e diagramas que ofereçam um nível maior de detalhe. Contudo, separamos um exemplo de MER em que há a representação de duas entidades – produto e categoria –, sendo possível identificar o tipo de relação existente entre elas, conforme demonstrado na **Figura 2.1**.



**Figura 2.1** – Modelo entidade relacionamento de produto e categoria.

Crédito: do elaborador, 2025.

Note que nesse modelo conseguimos entender a estrutura dos relacionamentos entre as entidades, o que passa a ser extremamente útil em aplicações complexas, não tendo como foco o detalhamento de como cada uma dessas ligações será executada na prática.

## Diagrama entidade relacionamento (DER)

O DER é um padrão visual para se representar a estrutura e a organização dos dados, bem como o relacionamento entre eles, e, embora existam inúmeras outras formas e padrões para se realizar, a modelagem de dados torna-se um dos modelos frequentemente utilizados por permitir uma definição mais detalhada do modo de construção das tabelas e coleções utilizadas no projeto.

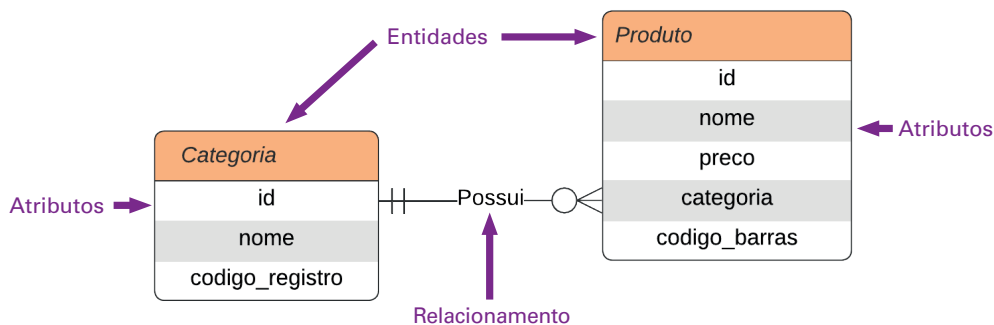
Nesse tipo de modelo, a representação da aplicação passa a ser mais detalhada em comparação ao MER, sendo muitas vezes elaborado após os modelos de entidade relacionamento.

O uso do DER facilita consideravelmente a compreensão até mesmo das arquiteturas de *software* mais complexas, auxiliando todos os profissionais que trabalharão na construção e manutenção da aplicação, independentemente do nível de conhecimento específico de cada um sobre bancos de dados, uma vez que disponibiliza uma forma simples e clara de funcionamento.

O DER apresenta uma estrutura de fácil entendimento, composta por três **partes fundamentais**:

- **Entidades:** representam uma abstração das tabelas ou coleções, geralmente apresentadas visualmente como retângulos;
- **Atributos:** são os campos ou colunas da tabela;
- **Relacionamentos:** detalham o relacionamento entre as entidades, apresentando a cardinalidade entre elas.

Para exemplificar as partes mencionadas, observe a **Figura 2.2**.

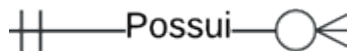


**Figura 2.2** – Diagrama entidade relacionamento de produtos e categorias.

Crédito: do elaborador, 2025.

Note que na **Figura 2.2** aparecem claramente as principais informações presentes em um DER, tais como atributos, entidades e seu respectivo relacionamento, sendo possível identificar que, nessa aplicação, há o cadastramento de produtos pertencentes a certas categorias, e que em um produto existem informações básicas, como nome, preço, categoria, código de barras e o número identificador do produto (**id**).

Contudo, caso não esteja familiarizado com a simbologia do que chamamos **cardinalidade**, é bem provável que tenha encontrado alguma dificuldade em entender alguns detalhes importantes no relacionamento entre as entidades Produto e Categoria de nosso exemplo, e que estão implícitas na simbologia apresentada na **Figura 2.3**.



**Figura 2.3** – Simbologia de relacionamento.

Crédito: do elaborador, 2025.

Abordaremos a cardinalidade e seus respectivos símbolos em um tópico específico. No entanto, para sanar a sua curiosidade, vamos interpretar esse símbolo específico agora.

Em primeiro lugar, vamos tentar entender o contexto: em uma situação típica de cadastramento de produtos, seria muito lógico e até mesmo comum pensarmos em dividi-los em categorias, tais como “Mercearia”, “Higiene”, “Laticínios” e assim por diante, não é mesmo?

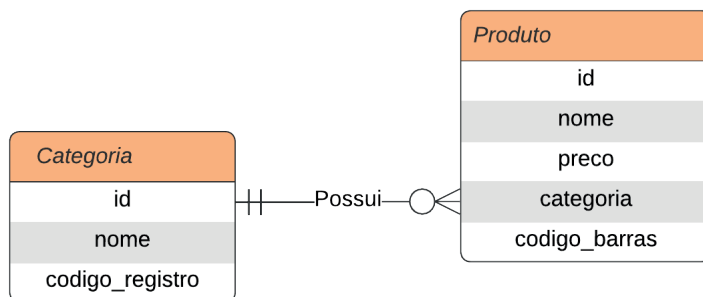


Seguindo essa linha de raciocínio, é fácil presumir também que um produto pode pertencer apenas a uma categoria por vez, correto?

E, por fim, para conduzir o(a) aluno(a) à explicação final, é compreensível que possam existir categorias cadastradas, mas que ainda não foram associadas a nenhum produto, como o registro de uma categoria nova na aplicação.

Finalmente, você deve estar neste momento se perguntando se todas essas regras estão sendo representadas com o uso de um único símbolo apresentado no relacionamento das **Figuras 2.2 e 2.3**: a resposta é **sim**!

De forma objetiva, os dois riscos na vertical, próximos à entidade Categoria, indicam que a entidade Produto **possui** obrigatoriamente associação em apenas uma única Categoria. E, por outro lado, os símbolos próximos à entidade de Produto informam que uma Categoria pode estar sendo utilizada por nenhum ou vários produtos ao mesmo tempo.



**Figura 2.4** – Diagrama entidade relacionamento de produtos e categorias.

Crédito: do elaborador, 2025.

## Ferramentas de modelagem

Como o(a) aluno(a) pode observar, as representações gráficas utilizadas em modelagens não apresentam nenhum nível de complexidade acentuado que exija um *software* ou ferramenta muito específica.

Contudo, existem no mercado algumas ferramentas disponíveis que visam agilizar o processo de criação desses diagramas, oferecendo inclusive diversos padrões e formatos de simbologia a serem adotados pelo usuário, muito úteis em casos em que é necessário seguir a padronização de uma empresa e/ou complementar a documentação de uma aplicação.

Algumas opções populares e de fácil acesso incluem o **Draw.io**, **Miro** e o **Lucidchart**, todas ferramentas *on-line* que oferecem recursos que simplificam a modelagem, permitindo a criação de diagramas profissionais e bem-estruturados em poucos segundos de uso, contando inclusive com a opção de colaboração e compartilhamento em equipe.

Para orientação, informamos que usaremos o LucidChart como ferramenta para a criação de todos os DER abordados neste livro, embora fique a cargo do(a) aluno(a) a escolha da opção de sua preferência.

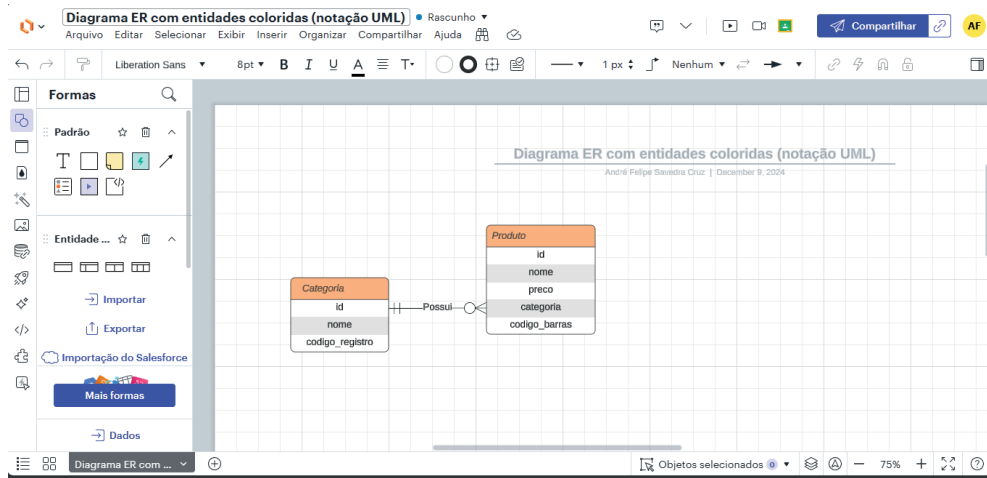
#### SAIBA MAIS

##### LucidChart

Para utilizar o LucidChart, basta acessar o *site* oficial e criar uma conta gratuitamente, havendo a possibilidade de uso do plano gratuito ou aquisição de planos pagos, caso seu interesse seja um uso mais ostensivo do *software* para fins corporativos. Contudo, para utilização da ferramenta com o objetivo pessoal ou acadêmico, muito provavelmente a opção gratuita atenderá perfeitamente o usuário.



Disponível em: <https://www.lucidchart.com/>. Acesso em: 15 maio 2025.



**Figura 2.5** – Diagrama entidade relacionamento criado no *software* LucidChart.

Crédito: do elaborador, 2025.

Vale ressaltar que dentro do LucidChart existem inúmeras opções de criação de diagramas que vão além das entidades de relacionamento, havendo a possibilidade também de geração de diagramas UML, fluxogramas, mapas mentais, entre outros.

## Cardinalidade

A cardinalidade é um termo que indica quantas vezes e de que forma uma entidade pode se relacionar com outra, sendo, sobretudo, de grande importância para os casos de uso de bancos relacionais, em que esse tipo de ligação é frequentemente mais presente.

### IMPORTANTE

#### Regras de negócio

**Existem inúmeras técnicas para definir qual cardinalidade será utilizada em cada caso, ficando a critério de cada desenvolvedor escolher a que julgar mais pertinente. Contudo, a dica crucial e infalível para auxílio nessa tarefa é sempre entender e interpretar corretamente as regras de negócio da aplicação, isto é, interpretar os requisitos, os critérios e a lógica de funcionamento da solução a ser desenvolvida.**

Para os desenvolvedores inexperientes, definir qual o tipo de cardinalidade ou relacionamento pode consistir em uma das tarefas mais desafiadoras ao se trabalhar com modelagem de banco de dados. Essa dificuldade, no entanto, tende a se reduzir com o treino, como a prática em diferentes cenários de modelagem e análise de casos reais (por esse motivo, recomendamos ao aluno a realização das atividades ao final deste capítulo).

Vale ressaltar também a importância da correta definição do tipo de relacionamento ou cardinalidade que será empregado, pois uma interpretação não condizente com a necessidade irá impactar diretamente a forma como o banco de dados será estruturado e, em consequência, o comportamento da aplicação.

Contudo, se você é iniciante nesses conceitos, não se preocupe! Abordaremos vários exemplos e casos de uso para que você domine esse tópico.

Uma informação essencial é compreender que todo relacionamento envolvendo o conceito de cardinalidade possui o que se chama de **cardinalidade mínima** e **cardinalidade máxima**, consistindo nos números mínimo e máximo de elementos envolvidos em um relacionamento entre entidades.

Existem inúmeros tipos de símbolos de cardinalidade utilizados em DER, e, para auxiliar o(a) aluno(a) nessa tarefa, detalharemos os principais a seguir:

- **Cardinalidade para um ou nenhum (0,1):** indicada por um traço na vertical e um círculo, representando a cardinalidade máxima de um elemento e mínima de zero, muito utilizada para relacionamentos opcionais;



**Figura 2.6** – Cardinalidade um ou nenhum.

Crédito: do elaborador, 2025.

- **Cardinalidade para um e apenas um (1,1):** representada por dois traços na vertical, evidenciando a cardinalidade de um e apenas um

elemento, usada em situações em que o relacionamento é único e obrigatório;



**Figura 2.7** – Cardinalidade um e apenas um.

Crédito: do elaborador, 2025.

- **Cardinalidade para nenhum ou muitos (0, N):** seu símbolo é composto por um círculo que representa a possibilidade de relacionamento com nenhum elemento juntamente com outro símbolo chamado “pé de galinha”, indicando, por sua vez, a cardinalidade máxima com “N” elementos, isto é, múltiplos relacionamentos. Esse símbolo é aplicado em condições em que podem existir várias atribuições de entidades, não sendo estas obrigatórias;



**Figura 2.8** – Cardinalidade nenhum ou muitos.

Crédito: do elaborador, 2025.

- **Cardinalidade para um ou muitos (1, N):** possui as mesmas simbologias usadas em outras cardinalidades, sendo formada por um traço na vertical para indicar cardinalidade mínima de um elemento e o “pé de galinha” para representar múltiplos relacionamentos, aplicada em condições em que se torna obrigatório um relacionamento juntamente com a possibilidade de múltiplos.



**Figura 2.9** – Cardinalidade um ou muitos.

Crédito: do elaborador, 2025.

## FIQUE ALERTA!

### Duas perspectivas

Outro fator importante a ser esclarecido é que, em termos práticos, todo relacionamento entre entidades apresenta dois lados, isto é, duas perspectivas, cada uma utilizando seu respectivo símbolo de cardinalidade, bem como apresentando seus valores de cardinalidade mínima e máxima.

## SAIBA MAIS

### Estilo Martin

Você sabia que os símbolos de cardinalidades apresentados neste capítulo podem ser representados de maneiras ligeiramente diferentes, embora se mantenha o conceito? Quando o assunto é entidades relacionamento, existem diversos estilos visuais de representação, tais como “Estilo Chen”, “Estilo Martin”, “Estilo Bachman”, “Estilo Barker”, entre outros. O estilo utilizado neste livro consiste no “Estilo Martin”, escolhido por opção pessoal do autor.

Para deixarmos o assunto mais concreto e menos subjetivo, apresentaremos exemplos práticos com os principais tipos de relacionamento no próximo tópico.

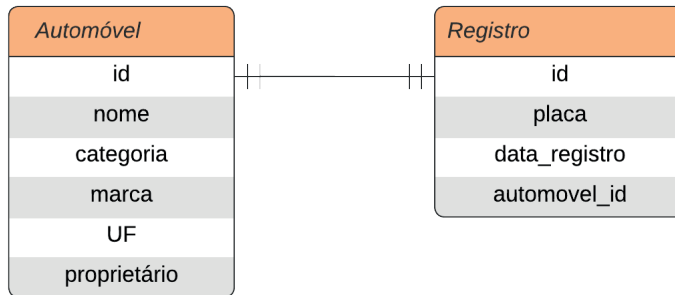
Vale ressaltar que, para efeito prático, os DER que serão apresentados nos próximos exemplos não consistem em toda a modelagem do que seria cada aplicação, sendo omitidas algumas tabelas e entidades para dar enfoque em cada tipo de cardinalidade.

Relacionamento um para um

Um relacionamento um para um tem como principais características a unicidade e a exclusividade entre as entidades, ou seja, em cada lado do relacionamento existe a interligação com um elemento único.

Como exemplo, podemos pensar em uma aplicação envolvendo Automóvel e Registro. Nesse caso, é correto afirmar que todo automóvel possui um único e exclusivo registro e, conseqüentemente, uma placa que lhe é única, não podendo coexistir dois automóveis diferentes compartilhando o mesmo registro.

Por outro lado, todo e qualquer registro deve estar associado a um único e exclusivo automóvel, sendo assim, usamos na **Figura 2.10** a cardinalidade (1,1) em ambos os relacionamentos, visto que eles são obrigatórios e exclusivos.



**Figura 2.10** – DER Automóvel e Registro.

Crédito: do elaborador, 2025.

## IMPORTANTE!

### UNIQUE

É interessante ressaltar para o(a) aluno(a) já familiarizado com bases relacionais que, para se construir esse tipo de relacionamento, em termos práticos, torna-se necessário utilizar um importante modificador de atributo/coluna: o **UNIQUE**. Com ele, é possível vincular que tanto o **automovel\_id** quanto a **placa** sejam elementos únicos e exclusivos, uma vez que esse modificador tem a capacidade de garantir que todos os valores em uma coluna sejam únicos, impedindo duplicatas. Mais adiante, em outro tópico, abordaremos esse tema com mais informações.

### Relacionamento um para muitos

Sendo uma das situações mais comuns de se encontrar em bancos relacionais, esse tipo de relacionamento envolve o cadastramento de uma das entidades, e o reuso de seu **id** múltiplas vezes como associação em outra entidade.

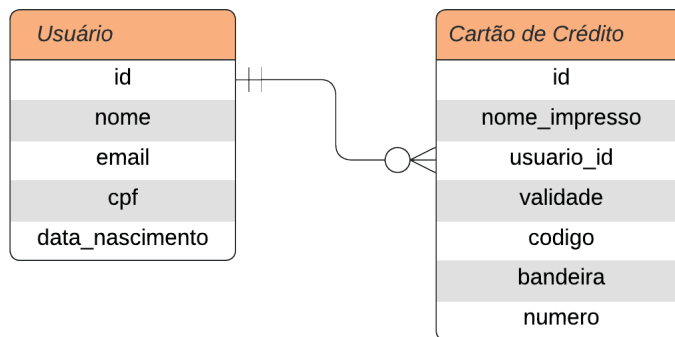
Usaremos, para exemplificar esse relacionamento, uma aplicação qualquer envolvendo um *e-commerce* e a consequente necessidade de efetuação de cadastro de usuários e métodos de pagamento.

Dessa maneira, fica lógico entender que um usuário pode ter adicionado em seu cadastro vários cartões de crédito para compor o método de pagamento desejado.

Além disso, é correto afirmar que na maioria das aplicações o cartão de crédito não é a única opção disponível para pagamento, sendo assim, é possível também considerar que um usuário pode não ter nenhum cartão de crédito associado à sua conta, caso ele opte por realizar o pagamento via boleto bancário, correto?

Por outro lado, um cartão de crédito cadastrado está associado de forma única a apenas um usuário, sendo obrigatório o vínculo com ele.

Considerando todos os aspectos mencionados, chegamos à conclusão de que a cardinalidade mais indicada para representar respectivamente usuário e cartão de crédito seria “um para vários”, sendo necessário também evidenciar a obrigatoriedade de se associar um cartão para um usuário (1,1), mas não existindo a mesma obrigatoriedade para um usuário ter múltiplos cartões (0, N).



**Figura 2.11** – DER Usuário e Cartões de Crédito.

Crédito: do elaborador, 2025.

## NA PRÁTICA

### Chave estrangeira

Para se construir esse tipo de relacionamento, em termos práticos, basta utilizar a chave primária do lado dominante (Usuário) como uma chave



estrangeira dentro da entidade que possui relacionamento múltiplo (Cartão de Crédito).

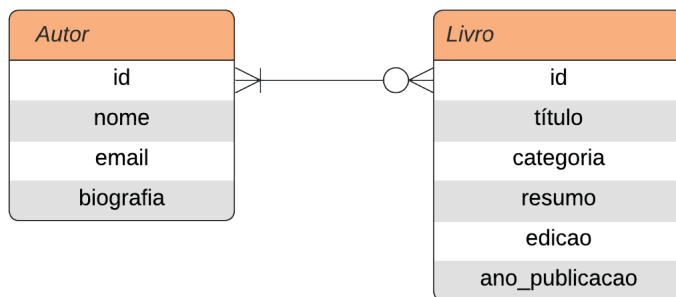
Relacionamento muitos para muitos

Relacionamentos múltiplos entre duas entidades, também denominados **N para N**, possuem inúmeras aplicações, evidenciando um relacionamento bidirecional entre as entidades.

Como exemplo, pense em uma aplicação de *e-commerce* de livros ou apenas em um sistema de uma biblioteca: é muito comum que os autores tenham publicado vários livros e, por outro lado, também há a possibilidade de que um livro tenha vários autores (coautoria), representando, portanto, uma relação de **muitos para muitos**.

No entanto, analisando a obrigatoriedade das relações, podemos afirmar que todo livro obrigatoriamente deve estar associado a um ou mais autores, embora o mesmo não ocorra com o autor, podendo ele estar associado a vários livros ou a nenhum (caso ainda não existam livros na biblioteca ou *e-commerce* para um autor em específico).

Portanto, usamos nesse exemplo a simbologia de nenhum ou muitos (0,N) livros por autor, e na outra extremidade a simbologia de um ou muitos (1,N) autores por livro.



**Figura 2.12** – DER Autor e Livro.

Crédito: do elaborador, 2025.

## IMPORTANTE

### Terceira tabela

**Geralmente, para se criar esse tipo de relacionamento em bancos relacionais, torna-se necessária a criação de uma terceira tabela para ligar as chaves primárias de ambas as entidades, uma vez que não é possível usar várias chaves estrangeiras dentro da mesma coluna ou atributo em bancos relacionais (o mesmo não ocorre em bancos NoSQL, existindo a opção de campos com *arrays*/listas).**

## Entidades fortes e fracas

No contexto de modelagem de dados, as entidades podem ser classificadas como “fortes”, também denominadas “dominantes”, ou “fracas”, dependendo de sua capacidade de existir de forma independente no modelo do banco de dados.

Esse conceito é importante para determinar como essas entidades se relacionam e o quão fortemente estão associadas entre si, consistindo também em informação muito útil para orientar o desenvolvedor a seguir uma ordem de criação das tabelas, optando-se primeiro pela criação das entidades fortes (já que não dependem de nenhuma outra para serem criadas) e, em seguida, pela criação das entidades fracas.

Para exemplificar, vamos recorrer ao nosso último exemplo sobre livros e autores (**Figura 2.12**), levando ao(a) aluno(a) os seguintes questionamentos:

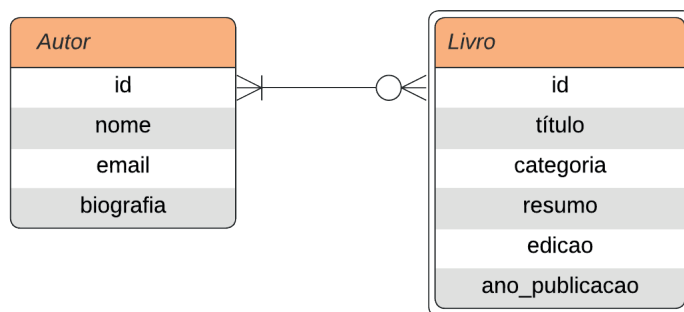
- Qual das duas entidades depende fortemente da outra?
- Um livro poderia existir no banco de dados sem nenhum autor?
- Um autor poderia existir no banco de dados sem nenhum livro?

É bem provável que você tenha chegado na conclusão de que um livro não poderia existir sem estar associado a alguém, contudo, um autor pode estar cadastrado em uma aplicação sem ter nenhum livro associado a ele ainda. Dessa maneira, podemos chegar à conclusão de que, nesse exemplo, a entidade forte é o autor e a entidade fraca é o livro.

Em termos práticos, as entidades fracas de um relacionamento usarão chaves estrangeiras para se associar com as entidades mais dominantes, e é por esse motivo que tais entidades são criadas por último, uma vez que suas chaves estrangeiras irão se ligar às chaves primárias das entidades fortes já existentes.

Embora não seja obrigatório, é possível sinalizar as entidades fortes e fracas nos DERs. Essa sinalização pode variar de acordo com o padrão de simbologia utilizado no diagrama.

Geralmente, as entidades fortes não possuem nenhuma sinalização em especial. Já as entidades fracas podem ser representadas por um retângulo de bordas duplas, conforme o exemplo da **Figura 2.13**.



**Figura 2.13** – DER Autores e Livros.

Crédito: do elaborador, 2025.

Na **Figura 2.13**, é possível notar que a entidade **Livro** está representada com bordas duplas, sinalizando ao desenvolvedor que essa é a entidade fraca do relacionamento de um para muitos que envolve o contexto de Livros e Autores dessa aplicação.

É importante ressaltar que, na construção de DERs, a sinalização das entidades fracas é opcional, uma vez que é possível identificá-las por meio do relacionamento e do contexto da associação.

# Aplicando a modelagem em diagramas entidade relacionamento

Você provavelmente percebeu que o uso do DER facilita a modelagem de conceitos abstratos, tais como requisitos e regras de negócio, em estruturas mais claras e consequentemente mais próximas do contexto de banco de dados.

É importante notar que, nos exemplos anteriores, incluímos notas informativas para esclarecer detalhes sobre a construção de cada diagrama, já que o DER não precisa ser totalmente fiel ao modelo real.

Apresentaremos, a seguir, variantes de diagramas que permitem a inclusão de mais informações, tornando a modelagem mais concreta e útil para a implementação do desenvolvedor.

## UML – Linguagem de Modelagem Unificada

### Definição

A Linguagem de Modelagem Unificada (UML, *Unified Modeling Language*) é um padrão visual para representar a estrutura e a organização dos dados, bem como o relacionamento entre eles, e, embora existam inúmeros outros modos e padrões para se realizar a modelagem de dados, torna-se um dos modelos frequentemente utilizados por permitir uma definição mais detalhada do modo de construção das tabelas e coleções utilizadas no projeto.

O uso da UML facilita consideravelmente a compreensão até mesmo das arquiteturas de *software* mais complexas, auxiliando todos os profissionais que trabalharão na construção e na manutenção da aplicação, independentemente do nível de conhecimento específico de cada um em bancos de dados, uma vez que disponibiliza uma forma simples e clara de funcionamento.

Existem dois tipos básicos de diagramas UML: **estruturais** e **comportamentais**. Os estruturais têm foco em representar de forma mais detalhada

a interação e as particularidades entre as partes do sistema; já os comportamentais possibilitam um melhor entendimento sobre o funcionamento da aplicação, pois envolvem o comportamento do sistema em relação aos usuários, em que estes últimos são geralmente representados pelo bom e velho “homenzinho-palito”.

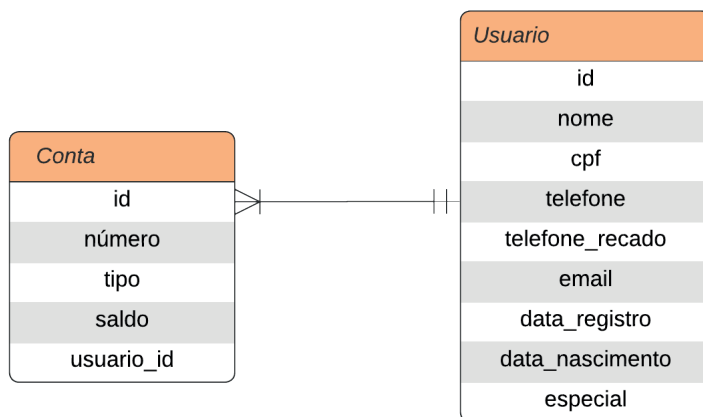
Contudo, se você em algum momento pensou que os tipos de diagramas UML se resumem a apenas essas duas categorias, enganou-se profundamente, uma vez que existem inúmeras subcategorias, cada uma criada para atender a uma necessidade específica.

### Diagramas de classe

Os diagramas de classe são frequentemente utilizados por desenvolvedores, fazendo analogia ao paradigma de **Orientação a Objetos**, auxiliando sobretudo no processo de codificação do *backend*, por conter informações de atributos, métodos e encapsulamento de cada classe.

Contudo, esse tipo de diagrama também pode ser muito útil para o processo de desenvolvimento e modelagem em bancos de dados, por apresentar informações importantes como tipo de dados e certos modificadores.

De forma a deixar o assunto mais claro, observe na **Figura 2.14** o DER tradicional do que seria uma aplicação bancária envolvendo Usuários e Contas.



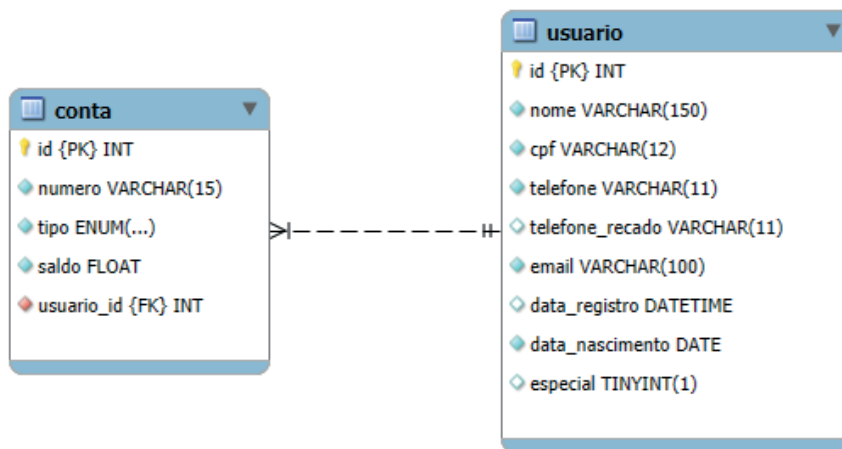
**Figura 2.14** – DER Usuário e Conta.

Crédito: do elaborador, 2025.

Embora seja possível retirar informações importantes da **Figura 2.14**, tais como os campos existentes em cada entidade e principalmente o nível de cardinalidade desse relacionamento (“um usuário para uma ou muitas contas”), ainda assim esse diagrama não apresenta o nível de detalhamento necessário para se criar tais entidades, faltando, por exemplo, as informações a seguir:

- Qual a tipagem de cada campo/atributo?
- Qual campo/atributo é a chave primária?
- Quais campos/atributos são chaves estrangeiras?

Desse modo, para sanar tais ausências, usaremos uma forma de representação UML, contando com informações mais detalhadas, conforme podemos ver na **Figura 2.15**.



**Figura 2.15** – UML Usuário e Conta.

Crédito: do elaborador, 2025.

Conforme pode-se observar, nesse modelo de representação temos informações do tipo de cada atributo das entidades (DATE, VARCHAR, INT etc.) e podemos identificar quais são as chaves primárias (PK, *primary key*) e as chaves estrangeiras (FK, *foreign key*) de cada tabela, auxiliando consequentemente no seu processo de criação.

**IMPORTANTE****Diferenciação de atributos**

Você sabe diferenciar em que momentos seria melhor o uso de um atributo como ENUM ou como FK (chave estrangeira)? No exemplo da **Figura 2.15**, utilizamos o atributo “tipo” da entidade “conta” como um ENUM, embora também fosse possível desmembrá-lo em uma outra tabela individual e associá-lo via FK. Contudo, é sempre aconselhável o uso de ENUM para situações estáticas e bem conhecidas, sendo o caso de nosso exemplo sobre tipos de contas (consistindo geralmente em “Contas Poupança” e “Contas Corrente”). Já para casos variáveis e que exigem flexibilidade, a escolha mais assertiva é utilizar uma tabela individual e associá-la via FK.

## Tipos de dados

Antes de avançarmos para a aplicação da modelagem em um nível maior de detalhamento, é importante que o(a) aluno(a) tenha conhecimento sobre os principais tipos de dados disponíveis para serem usados como atributos ou colunas de cada tabela ou coleção.

Outra importante informação a se pontuar é que os tipos de dados disponíveis para uso podem variar ligeiramente, dependendo de qual conceito de banco será utilizado (relacional ou não relacional) e até mesmo de qual modelo utilizado (MySQL, SQLServer, Oracle etc.).

Portanto, para informações mais detalhadas, orientamos o(a) aluno(a) a consultar as documentações do banco a ser empregado, garantindo assertividade em seu uso.

Demonstraremos no **Quadro 2.1** os tipos de dados mais utilizados do banco relacional MySQL, utilizado como modelo principal de referência neste livro.

Quadro 2.1 – Principais tipos de dados MySQL

| Numéricos |                                                                                                                            |
|-----------|----------------------------------------------------------------------------------------------------------------------------|
| INT       | Valores numéricos inteiros, usados na maioria dos casos.                                                                   |
| BIGINT    | Valores numéricos inteiros 64 bits, usados para armazenamento de valores muito grandes.                                    |
| DECIMAL   | Números decimais com precisão definida, usados na maioria dos casos que envolvem valores após a vírgula.                   |
| FLOAT     | Números decimais com aproximação simples (4 bytes), usados quando não se necessita de alta precisão.                       |
| DOUBLE    | Números decimais com aproximação dupla (8 bytes), usados para alta precisão.                                               |
| Texto     |                                                                                                                            |
| CHAR      | Texto com tamanho fixo, quando o tamanho do texto é sempre o mesmo.                                                        |
| VARCHAR   | Texto com tamanho variável, fazendo com que o campo se adapte ao tamanho do dado, limitado por um tamanho máximo definido. |
| TEXT      | Textos com tamanhos grandes.                                                                                               |
| LONGTEXT  | Textos com tamanhos muito grandes.                                                                                         |
| Data      |                                                                                                                            |
| DATE      | Data em formato AAAA-MM-DD.                                                                                                |
| TIME      | Hora em formato HH:MM:SS.                                                                                                  |
| DATETIME  | Data e hora combinadas.                                                                                                    |
| TIMESTAMP | Data e hora com fuso horário.                                                                                              |
| YEAR      | Ano com 2 ou 4 dígitos.                                                                                                    |
| Booleanos |                                                                                                                            |
| BOOL      | Valores booleanos, ou seja, true ou false.                                                                                 |
| Conjuntos |                                                                                                                            |
| ENUM      | Conjunto de valores, permitindo a seleção de apenas um.                                                                    |
| SET       | Conjunto de valores, permitindo a seleção de vários.                                                                       |
| Especiais |                                                                                                                            |
| BLOB      | Armazena grandes quantidade de dados binários, usado quando se deseja salvar arquivos.                                     |
| JSON      | Armazena dados em formato JSON.                                                                                            |

Crédito: do elaborador, 2025.



## SAIBA MAIS

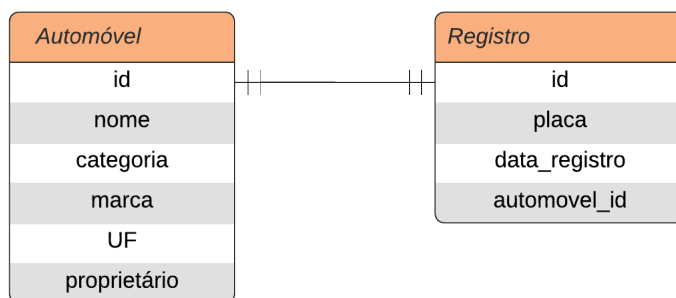
Você sabe como seria possível armazenar imagens em um banco de dados tradicional? Existem inúmeras maneiras, uma delas é com o uso de campos do tipo BLOB. Outra opção muito utilizada é pelo uso do Base64, um método de transformar dados binários em texto, ou seja, é possível transformar sua imagem em uma representação textual e assim armazená-la em um campo do tipo TEXT. Finalmente, você também pode optar por não armazenar a imagem ou arquivo diretamente no banco de dados, caso sua preocupação seja proteger a performance do banco, e sim apenas salvar o *link* de acesso à imagem/arquivo usando um campo TEXT ou VARCHAR (neste caso, o *link* deve se direcionar à imagem dentro do servidor *backend* ou a serviços próprios de armazenamento em nuvem conhecidos como *buckets*).

## Aplicando o conceito

Usaremos os mesmos exemplos abordados no tópico de cardinalidade dos DER, para que o(a) aluno(a) possa compreender a diferença de detalhamento em cada tipo de cardinalidade com o uso de UML.

### Automóveis e registros

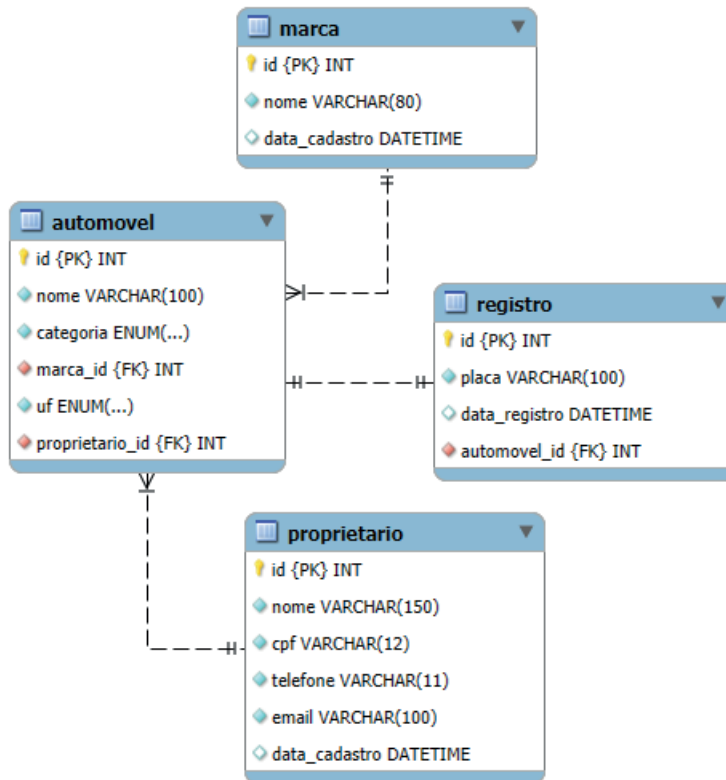
Na **Figura 2.16** apresentamos a mesma necessidade de registro/emplacamento de automóveis, envolvendo a cardinalidade “um para um”, ou seja, cada automóvel é único e deve ser associado a um único e exclusivo registro/placa.



**Figura 2.16** – DER Automóvel e Registro.

Crédito: do elaborador, 2025.

Com base nesse contexto, aplicaremos agora o detalhamento da modelagem em um nível mais concreto e fiel ao que deve ser implementado, gerando a representação da **Figura 2.17**.



**Figura 2.17** – UML Automóvel e Marca.

Crédito: do elaborador, 2025.

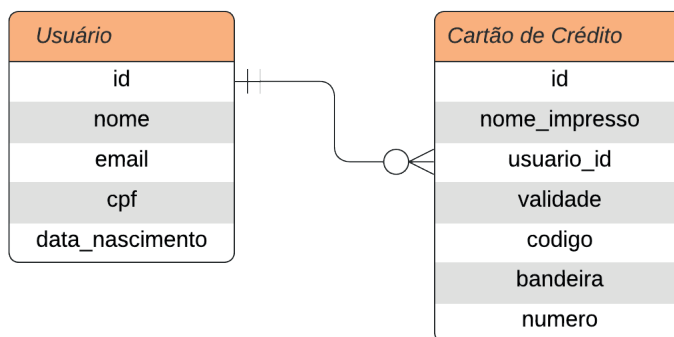
Como você pode notar, para convertermos o DER em uma modelagem mais fiel, foram adicionadas várias informações no novo diagrama, das quais destacamos as seguintes:

- Adicionamos os tipos de dados usados nos atributos de cada entidade, colocando inclusive alguns parâmetros que se faziam necessários. Como exemplo, quando adicionamos `VARCHAR(100)` no atributo “nome” da tabela “automóvel”, isso indica que estamos criando um campo de texto variável, limitado a 100 caracteres;

- Adicionamos os campos de marca e proprietário como chaves estrangeiras (FK) de tabelas individuais. Esse processo é fundamental, pois, através do cadastro único de uma marca e de um proprietário, é possível reaproveitá-los para serem usados em inúmeros automóveis;
- Incluímos o campo “uf” da tabela “automóvel” usando o tipo de dado ENUM, sendo desnecessária a criação de uma entidade apenas para armazenamento dos estados (unidades federativas), uma vez que seus valores são conhecidos e não serão mudados;
- Embora não esteja visível, é importante ressaltar que, para assegurar o relacionamento “um para um” entre “automóvel” e “registro”, os campos de “placa” e “automovel\_id”, foram definidos como únicos graças ao uso do modificador de atributo chamado UNIQUE, fazendo com que o banco de dados não permita a inclusão de dados já existente (abordaremos em detalhes no próximo capítulo).

### Usuários e cartões de crédito

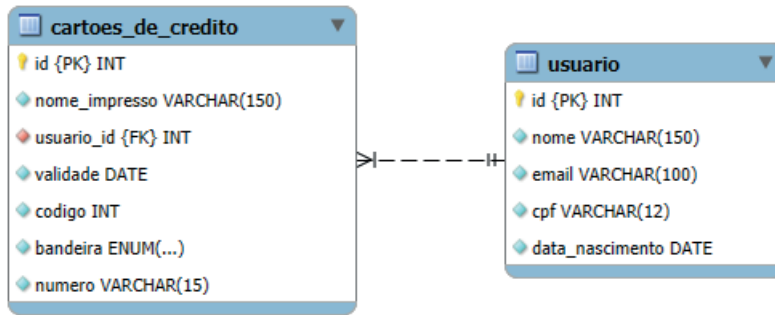
Agora, usaremos em nosso segundo exemplo um DER com relacionamento “um para muitos”, reutilizando o contexto abordado anteriormente entre “Usuários” e “Cartões de Crédito”, conforme mostrado na **Figura 2.18**.



**Figura 2.18** – DER Usuário e Cartão de Crédito.

Crédito: do elaborador, 2025.

Aplicaremos agora o detalhamento desse DER, como pode ser visto na **Figura 2.19**.



**Figura 2.19** – UML Usuário e Cartão de Crédito.

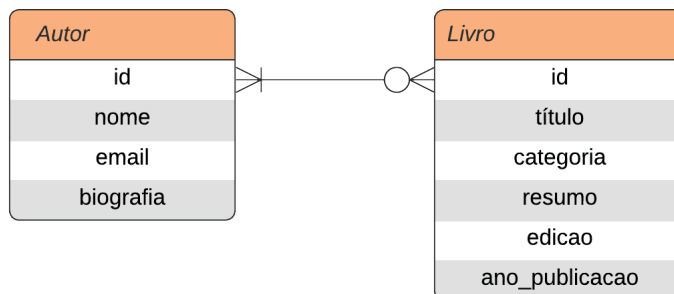
Crédito: do elaborador, 2025.

Alguns apontamentos sobre esse processo:

- É lógico afirmar que cada cartão de crédito possui obrigatoriamente apenas um usuário, dessa forma, adicionamos o “id” desse respectivo “usuário” dentro da tabela “cartoes\_de\_credito”;
- Todos os atributos de cada entidade foram colocados como obrigatórios, graças ao uso do modificador de atributo NOT NULL (abordaremos em detalhes no próximo capítulo).

Livros e autores

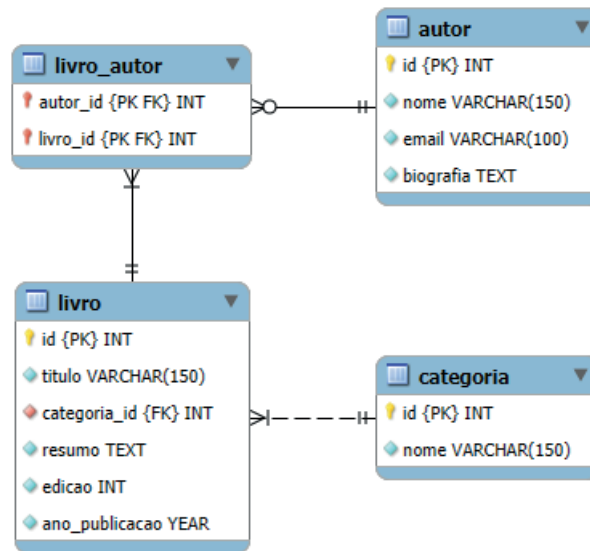
Finalmente, exploraremos a abordagem de “muitos para muitos”, reaproveitando também o exemplo trabalhado no tópico de cardinalidade, envolvendo Autores e Livros, conforme apresentado na **Figura 2.20**.



**Figura 2.20** – DER Autores e Livros.

Crédito: do elaborador, 2025.

Em seguida, aplicamos o detalhamento necessário para a implementação concreta desse relacionamento, como apresentado na **Figura 2.21**.



**Figura 2.21** – UML Autores e Livros.

Crédito: do elaborador, 2025.

Entre todos os exemplos abordados, é bem provável que esse diagrama seja o mais desafiador para os(as) alunos(as), ainda iniciantes no conteúdo deste livro, portanto, teremos o cuidado em separar as informações mais importantes:

- Adicionamos a “categoria” como uma nova tabela a ser associada com “livro”, possibilitando o reuso de seus elementos cadastrados em diversos livros, bem como a flexibilidade na adição de novos (flexibilidade não disponível com a utilização de ENUM, por exemplo);
- Os relacionamentos “muitos para muitos”, em bancos relacionais, somente são possíveis de serem implementados concretamente com o uso de tabelas adicionais. Desse modo, adicionamos uma nova entidade denominada “livro\_autor”, utilizada para relacionar individualmente um respectivo livro (“livro\_id”) a um respectivo autor (“autor\_id”), construindo dessa maneira o relacionamento desejado;
- Utilizamos na entidade “livro\_autor” duas chaves estrangeiras (FK) que ao mesmo tempo consistem em duas chaves primárias (PK). Na realidade, essa tabela continua apresentando em termos práticos uma

única chave primária, porém, esta última é formada pela concatenação das duas chaves estrangeiras (autor\_id + livro\_id), garantindo a unicidade do relacionamento (um autor não poderá ser atribuído ao mesmo livro duas vezes).

Para possibilitar um melhor entendimento, construímos na **Figura 2.22** o que seria um possível uso da estrutura de dados abordada nesse exemplo.

| livro |             |              |                 |        |                |
|-------|-------------|--------------|-----------------|--------|----------------|
| id    | titulo      | categoria_id | resumo          | edicao | ano_publicacao |
| 1     | Java Básico | 3            | "Neste livro... | 2      | 2024           |
| 2     | Big Data    | 5            | "Big data é...  | 1      | 2022           |

| autor |                  |                    |                        |
|-------|------------------|--------------------|------------------------|
| id    | nome             | email              | biografia              |
| 1     | Rosana Maria     | rosana@senai.br    | Autora iniciou seu ... |
| 2     | Jaqueline Sirius | jaqueline@senai.br | Sua biografia é ...    |
| 3     | Mário Órion      | mario@senai.br     | Autor renomado de ...  |

| livro_autor |          |
|-------------|----------|
| autor_id    | livro_id |
| 3           | 1        |
| 1           | 2        |
| 2           | 2        |

**Figura 2.22** – Possível utilização da estrutura de dados para representar o uso da associação.

Crédito: do elaborador, 2025.

Observe que pela tabela “livro\_autor” podemos identificar, por exemplo, que o livro 2, de título “Big Data”, é o resultado da coautoria de duas pessoas, uma vez que este id aparece múltiplas vezes nessa tabela, sendo as autoras “1 – Rosana” e “2 – Jaqueline”.

## Dicionário de dados

Você já deve ter percebido a importância dos diagramas e modelos para representar as aplicações que fazem uso de bancos de dados, como excelente forma para compreender seu funcionamento e sua organização.

Apresentamos uma outra forma útil e bem objetiva para representar a estrutura de um banco de dados: o dicionário de dados. Ele funciona como uma espécie de manual técnico que descreve cada elemento de dado: seus nomes, tipos, tamanhos, formatos, restrições, relacionamentos, origem, destino e até quem pode acessá-los.

Em outras palavras, o dicionário de dados documenta de forma detalhada tudo o que está por trás das tabelas, colunas, relacionamentos e regras de um banco de dados.

Embora os dicionários de dados possam variar de acordo com a necessidade de cada aplicação e de acordo com a formalidade adotada por cada empresa/desenvolvedor, existem algumas informações cujas presenças são mais comuns, conforme destacamos a seguir:

- Nome da tabela;
- Nome da coluna;
- Tipo de dado;
- Tamanho máximo;
- Permite valores nulos;
- Chave primária;
- Chave estrangeira;
- Descrição do campo;
- Valor padrão;
- Regra de validação.

Para exemplificar, demonstraremos o uso do dicionário de dados no diagrama UML utilizado na aplicação de “Livros e Autores”, conforme podemos ver na **Figura 2.23**.

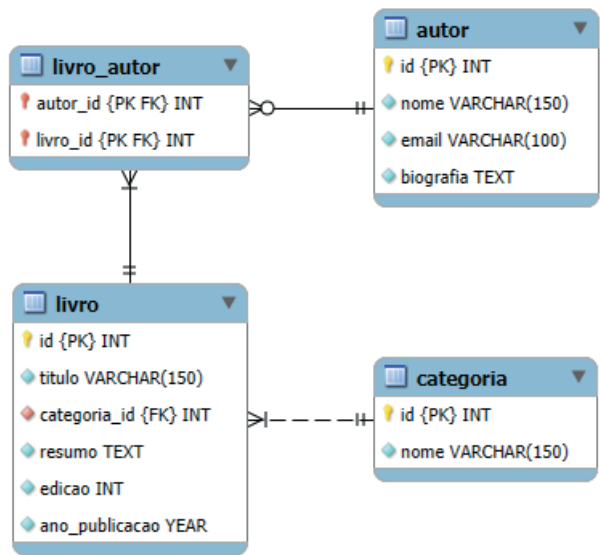


Figura 2.23 – UML Livros e Autores.

Crédito: do elaborador, 2025.

Primeiro, realizaremos o dicionário envolvendo as entidades “Autor” e “Categoria”, conforme a **Figura 2.24**.

| Nome da Tabela: Autor |              |         |               |                |                   |                    |              |                    |
|-----------------------|--------------|---------|---------------|----------------|-------------------|--------------------|--------------|--------------------|
| Coluna                | Tipo de Dado | Tamanho | Valores Nulos | Chave Primária | Chave Estrangeira | Descrição          | Valor Padrão | Regra de Validação |
| id                    | int          | -       | Não           | Sim            | Não               | ID do Autor        | -            | -                  |
| nome                  | varchar      | 150     | Não           | Não            | Não               | Nome do Autor      | -            | -                  |
| email                 | varchar      | 100     | Não           | Não            | Não               | Email do Autor     | -            | -                  |
| biografia             | text         | -       | Não           | Não            | Não               | Biografia do Autor | -            | -                  |

| Nome da Tabela: Categoria |              |         |               |                |                   |                   |              |                    |
|---------------------------|--------------|---------|---------------|----------------|-------------------|-------------------|--------------|--------------------|
| Coluna                    | Tipo de Dado | Tamanho | Valores Nulos | Chave Primária | Chave Estrangeira | Descrição         | Valor Padrão | Regra de Validação |
| id                        | int          | -       | Não           | Sim            | Não               | ID da Categoria   | -            | -                  |
| nome                      | varchar      | 150     | Não           | Não            | Não               | Nome da Categoria | -            | -                  |

Figura 2.24 – Dicionário de dados de Autores e Categorias.

Crédito: do elaborador, 2025.

Observe que a estrutura dos dados em um dicionário é relativamente simples, assemelhando-se a uma planilha, já que seu intuito principal é justamente transmitir informações de forma clara e objetiva.

Na **Figura 2.25**, mostraremos o dicionário de dados das duas outras entidades faltantes (Livro e Autores por Livro).



| Nome da Tabela: Livro |              |         |               |                |                   |                   |              |                    |
|-----------------------|--------------|---------|---------------|----------------|-------------------|-------------------|--------------|--------------------|
| Coluna                | Tipo de Dado | Tamanho | Valores Nulos | Chave Primária | Chave Estrangeira | Descrição         | Valor Padrão | Regra de Validação |
| id                    | int          | -       | Não           | Sim            | Não               | ID do Livro       | -            | -                  |
| titulo                | varchar      | 150     | Não           | Não            | Não               | Título do Livro   | -            | -                  |
| categoria_id          | int          | -       | Não           | Não            | Sim               | ID da Categoria   | -            | -                  |
| resumo                | text         | -       | Não           | Não            | Não               | Resumo do Livro   | -            | -                  |
| edicao                | int          | -       | Não           | Não            | Não               | Edição do Livro   | 1            | maior que 0        |
| ano_publicacao        | year         | -       | Não           | Não            | Não               | Ano de Publicação | ano atual    | -                  |

| Nome da Tabela: Livro_Autor |              |         |               |                |                   |             |              |                    |
|-----------------------------|--------------|---------|---------------|----------------|-------------------|-------------|--------------|--------------------|
| Coluna                      | Tipo de Dado | Tamanho | Valores Nulos | Chave Primária | Chave Estrangeira | Descrição   | Valor Padrão | Regra de Validação |
| autor_id                    | int          | -       | Não           | Sim            | Sim               | ID do Autor | -            | -                  |
| livro_id                    | int          | -       | Não           | Sim            | Sim               | ID do Livro | -            | -                  |

**Figura 2.25** – Dicionário de dados de Livros e Autores por Livro.

Crédito: do elaborador, 2025.

Dessa forma, graças ao uso dos dicionários de dados, os desenvolvedores podem obter as informações essenciais para construir o projeto que envolve bancos de dados, sendo mais uma opção disponível para que o(a) aluno(a) utilize em seus desenvolvimentos.

## Normalização dos dados

Ao projetar um banco de dados, é fundamental não apenas modelar as entidades e seus relacionamentos, mas também garantir que a estrutura resultante seja eficiente, livre de redundâncias e consistente.

É nesse contexto que a normalização se torna uma técnica essencial, consistindo em um **processo de organização dos dados**, com o objetivo de minimizar a duplicação de informações e evitar problemas como anomalias de inserção, atualização e exclusão.

Em resumo, a aplicação da normalização dos dados visa trazer boas práticas ao desenvolvimento de projetos que envolvem bancos de dados, proporcionando como **principais vantagens**:

- Redução de redundância: evita dados duplicados ou desnecessários, contribuindo inclusive na otimização do espaço ocupado;
- Melhoria da consistência: garante que as informações atualizadas sejam realizadas de maneira uniforme;

- Manutenibilidade aprimorada: reduz a complexidade da estrutura do banco e proporciona intervenções mais simplificadas em situações em que o projeto necessita receber modificações.

## IMPORTANTE

### Regras e premissas

**Em muitas situações, o processo de normalização dos dados é implementado quase que instintivamente pelo desenvolvedor, sem ele se dar conta de que está seguindo tais regras de normalização, comportamento adquirido graças a uma longa prática de exercícios e projetos de modelagem com diferentes cenários. Contudo, é sempre valioso conhecer quais são as regras e premissas desse processo para assegurar as boas práticas e o sucesso da aplicação em situações mais desafiadoras.**

## Formas normais

As formas normais (FNs) são uma sequência de passos, etapas ou regras sucessivas que visam atingir um nível de organização dos dados de uma modelagem. Em outras palavras, cada FN se baseia em princípios específicos, e o objetivo é progredir de formas menos rigorosas para formas mais rigorosas, sempre visando aprimorar a qualidade do banco de dados.

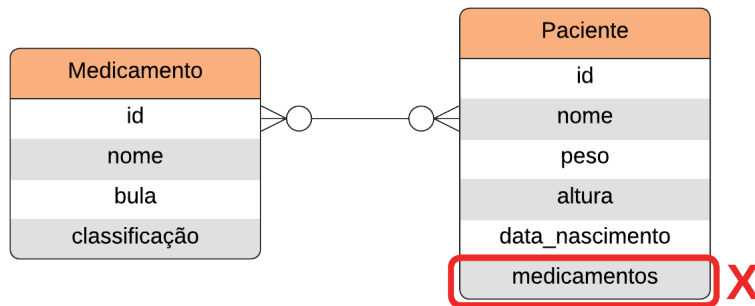
Cada FN é detentora de uma regra que visa atingir um objetivo, sendo normalmente divididas em três, detalhadas a seguir.

### Primeira forma normal (1FN)

A 1FN exige que cada atributo de uma tabela contenha apenas valores atômicos, ou seja, indivisíveis. Além disso, não permite grupos repetitivos de atributos em uma mesma tabela.

Para exemplificar, imagine um banco de dados hospitalar, havendo a necessidade de se registrar pacientes e seus respectivos medicamentos. Nesse contexto, um paciente pode fazer uso de uma quantidade indefinida de medicamentos.

Para atender aos fins didáticos, iremos modelar primeiro o DER desse exemplo **fora do padrão** da 1FN, conforme **Figura 2.26**.

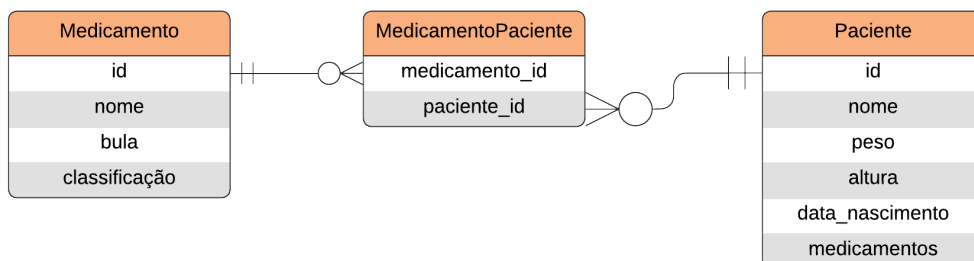


**Figura 2.26** – DER Medicamento e Paciente.

Crédito: do elaborador, 2025.

Como se observa na **Figura 2.26**, temos a presença de um atributo múltiplo e não atômico na entidade Paciente, com nome de “medicamentos”, o que consiste em um grave erro, uma vez que em bancos relacionais não é possível conter uma lista de chaves estrangeiras que compartilham um mesmo campo. Dizemos também que essa situação se trata de uma **coluna multivalorada**, isto é, armazena mais de um valor em uma única célula ou atributo, rompendo nesse caso a regra estabelecida pela 1FN.

Para solucionar esse problema e adequar a coluna multivalorada à 1FN, iremos propor na **Figura 2.27** a criação de uma tabela auxiliar que relacionará cada par Medicamento-Paciente individualmente.



**Figura 2.27** – DER Medicamento e Paciente.

Crédito: do elaborador, 2025.

Na **Figura 2.27**, percebemos que, com a inclusão de uma entidade auxiliar, conseguimos acomodar os múltiplos medicamentos de um mesmo paciente em diferentes linhas da tabela “MedicamentoPaciente”, obedecendo também à regra prescrita na 1FN.

Na **Figura 2.28** demonstraremos um exemplo de distribuição dos dados.

| Medicamento |                    |               |               |
|-------------|--------------------|---------------|---------------|
| id          | nome               | bula          | classificação |
| 1           | Analgésico XYZ     | Este medi...  | Livre         |
| 2           | tamina C Top Saúde | Usada para... | Livre         |
| 3           | Antibiótico Bio +  | Medicame...   | Controlado    |

| Paciente |             |      |        |                 |
|----------|-------------|------|--------|-----------------|
| id       | nome        | peso | altura | data_nascimento |
| 1        | Luís Suzano | 80   | 1.70   | 18/08/1996      |
| 2        | Edite Joana | 65   | 1,59   | 20/07/1977      |

| MedicamentoPaciente |             |
|---------------------|-------------|
| medicamento_id      | paciente_id |
| 2                   | 1           |
| 1                   | 1           |
| 2                   | 2           |

**Figura 2.28** – Tabelas Medicamentos e Pacientes.

Crédito: do elaborador, 2025.

Concluimos, então, a primeira adequação à forma normal, garantindo a atomicidade de cada atributo sem perda do relacionamento necessário para o projeto. Essa separação traz benefícios importantes principalmente em situações em que se deseja fazer consultas na base de dados e filtrar por valores individuais ou em operações de atualização (iremos abordar esse tema com detalhes mais técnicos no próximo capítulo).

## Segunda forma normal (2FN)

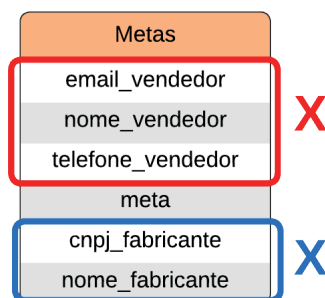
Para atender à 2FN, é necessário que a modelagem se enquadre nos critérios da 1FN e não apresente atributos que possuam dependência parcial de sua respectiva chave primária.

Em outras palavras, dizemos que os atributos não chave não podem se referir a apenas uma parte da chave primária (em casos de chaves compostas) ou conter valores que, toda vez que aparecem, tenham outro valor associado sempre de valor igual.

Embora pareça complexo, você conseguirá compreender mais claramente acompanhando mais um exemplo prático.

Imagine uma aplicação de gestão de representantes comerciais que recebem comissão com base no atingimento de metas compatíveis com as vendas estabelecidas por certos fabricantes.

Uma tabela não normalizada conforme a 2FN seria como a da **Figura 2.29**.



**Figura 2.29** – DER Metas.

Crédito: do elaborador, 2025.

Nessa modelagem da **Figura 2.29**, possuímos uma entidade cuja chave primária é a combinação entre “email do vendedor” e “cnpj do fabricante”, ou seja, uma chave composta.

Contudo, podemos notar que, embora essa configuração seja funcional, acaba por descumprir a 2FN, uma vez que existem atributos que possuem dependência apenas de uma das partes da chave primária composta.

Por exemplo, por um lado, os nomes e telefones do vendedor não são influenciados pelo “cnpj do fabricante” e, por outro lado, o nome do fabricante não é influenciado pelo “email do vendedor”.

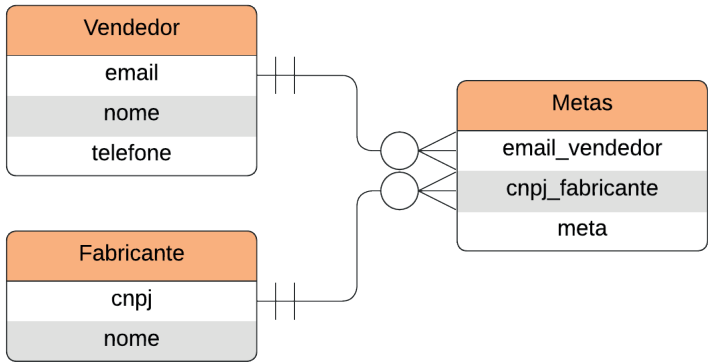
Além desses fatores, podemos observar que nesse cenário também ocorre a repetição desnecessária de dados, conforme é possível visualizar na **Figura 2.30**.

| Metas          |               |                   |               |                   |                 |
|----------------|---------------|-------------------|---------------|-------------------|-----------------|
| email_vendedor | nome_vendedor | telefone_vendedor | meta          | nome_fabricante   | cnpj_fabricante |
| luis@senai.br  | Luís Suzano   | 1955555555        | R\$ 19.000,00 | MX Cimentos S.A.  | 7777777777      |
| edite@senai.br | Edite Joana   | 1188888888        | R\$ 17.500,00 | Tomadas Luzzy Top | 8888888888      |
| luis@senai.br  | Luís Suzano   | 1955555555        | R\$ 95.000,00 | Jezy S.A. Metais  | 1111111111      |
| edite@senai.br | Edite Joana   | 1188888888        | R\$ 77.000,00 | Jezy S.A. Metais  | 1111111111      |

**Figura 2.30** – Tabela não normalizada (Metas).

Crédito: do elaborador, 2025.

Na **Figura 2.31** faremos a correção para atendimento da 2FN, dividindo as entidades que não possuem correlação direta.



**Figura 2.31** – DER Metas, Vendedores e Fabricantes.

Crédito: do elaborador, 2025.

Nesse caso, conforme se vê na **Figura 2.32**, mantivemos o uso do “email” e “cnpj\_fabricante” como chaves compostas (embora também fosse possível o uso de ids) da tabela de Metas, deixando todo o tipo de informação não correspondente para cada entidade individual, Vendedores e Fabricantes, respectivamente.

| vendedor       |             |             |
|----------------|-------------|-------------|
| email          | nome        | telefone    |
| luis@senai.br  | Luís Suzano | 19555555555 |
| edite@senai.br | Edite Joana | 11888888888 |

| Metas          |                 |               |
|----------------|-----------------|---------------|
| email_vendedor | cnpj_fabricante | meta          |
| luis@senai.br  | 77777777777     | R\$ 19.000,00 |
| edite@senai.br | 8888888888888   | R\$ 17.500,00 |
| luis@senai.br  | 11111111111     | R\$ 95.000,00 |
| edite@senai.br | 11111111111     | R\$ 77.000,00 |

| Fabricante    |                   |
|---------------|-------------------|
| cnpj          | nome              |
| 77777777777   | MX Cimentos S.A.  |
| 8888888888888 | Tomadas Luzzy Top |
| 11111111111   | Jezy S.A. Metais  |

**Figura 2.32** – Tabelas de Metas, Vendedores e Fabricantes.

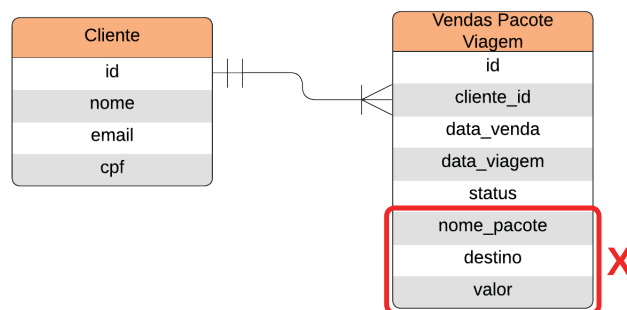
Crédito: do elaborador, 2025.

Terceira forma normal (3FN)

Finalmente, para se atender à 3FN, é necessário que a modelagem se enquadre nos critérios de 1FN + 2FN e não apresente atributos não chave que possuam dependência de outro atributo não chave.

Esse conceito é muito parecido com a regra 2FN, porém não envolve apenas ids de chaves compostas, por exemplo, e sim atributos comuns que possam ter algum tipo de relação entre si.

Novamente, nas **Figuras 2.33** e **2.34**, recorreremos ao bom e velho exemplo para detalhar esse conceito na prática.



**Figura 2.33** – DER Cliente e Vendas Pacote Viagem.

Crédito: do elaborador, 2025.

| Vendas Pacote Viagem |            |            |             |          |                  |                               |       |
|----------------------|------------|------------|-------------|----------|------------------|-------------------------------|-------|
| id                   | cliente_id | data_venda | data_viagem | status   | nome_pacote      | destino                       | valor |
| 1                    | 1          | 01/01/2024 | 07/07/2024  | aprovado | Premium Acapulco | Resort Habanero Acapulco - MX | 2600  |
| 2                    | 2          | 07/05/2024 | 25/12/2024  | pendente | Natal Orlando    | Disney Orlando - EUA          | 8300  |
| 2                    | 2          | 03/09/2024 | 11/11/2025  | pendente | Premium Acapulco | Resort Habanero Acapulco - MX | 2600  |

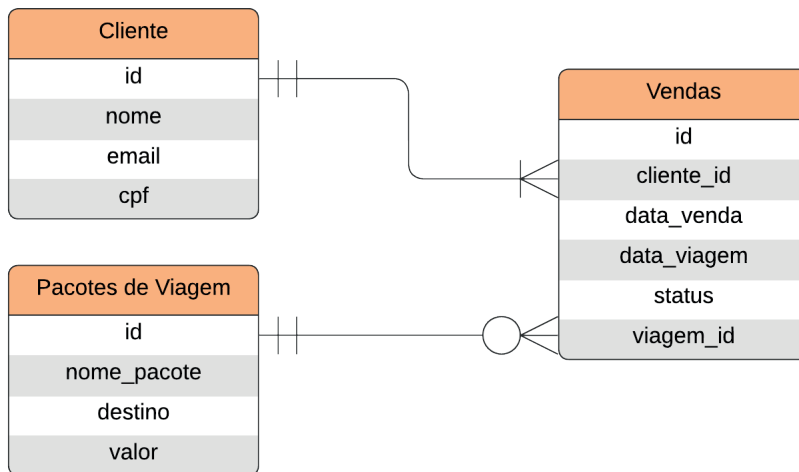
| Cliente |             |                |             |
|---------|-------------|----------------|-------------|
| id      | nome        | email          | cpf         |
| 1       | Luís Suzano | luis@senai.br  | 33377788888 |
| 2       | Edite Joana | edite@senai.br | 58858855555 |

**Figura 2.34** – Tabelas de Cliente e Vendas Pacote Viagem.

Crédito: do elaborador, 2025.

Note que nas **Figuras 2.33** e **2.34** podemos identificar que, embora seja compreensível que alguma informação do pacote de viagem esteja presente na tabela de vendas, existem algumas informações adicionais repetitivas que estão exclusivamente relacionadas com o pacote de viagem em si e não necessariamente com a venda, como é o caso do “destino” e “valor”.

Dessa maneira, para adequarmos à 3FN, deveremos separar tais entidades, conforme mostrado nas **Figuras 2.35** e **2.36**.



**Figura 2.35** – DER Cliente, Pacotes de Viagem e Vendas.

Crédito: do elaborador, 2025.



| Vendas |            |            |             |          |           |
|--------|------------|------------|-------------|----------|-----------|
| id     | cliente_id | data_venda | data_viagem | status   | viagem_id |
| 1      | 1          | 01/01/2024 | 07/07/2024  | aprovado | 1         |
| 2      | 2          | 07/05/2024 | 25/12/2024  | pendente | 2         |
| 2      | 2          | 03/09/2024 | 11/11/2025  | pendente | 1         |

| Vendas Pacote Viagem |                  |                               |       |
|----------------------|------------------|-------------------------------|-------|
| id                   | nome_pacote      | destino                       | valor |
| 1                    | Premium Acapulco | Resort Habanero Acapulco - MX | 2600  |
| 2                    | Natal Orlando    | Disney Orlando - EUA          | 8300  |

| Cliente |             |                |             |
|---------|-------------|----------------|-------------|
| id      | nome        | email          | cpf         |
| 1       | Luís Suzano | luis@senai.br  | 33377788888 |
| 2       | Edite Joana | edite@senai.br | 58858855555 |

**Figura 2.36** – Tabelas de Vendas, Vendas Pacote Viagem e Cliente.

Crédito: do elaborador, 2025.

Após a adequação, temos uma separação na entidade de Vendas, gerando consequentemente uma nova entidade para armazenar os dados que estão somente relacionados com Viagens, aumentando dessa maneira a consistência e a organização da aplicação.

Chegando ao fim das formas normais principais, podemos afirmar que, embora possa parecer inicialmente um assunto complexo ou muito teórico, seu objetivo é auxiliar efetivamente na implementação prática dos processos de modelagem para assegurar que eles sigam boas práticas e não gerem efeitos colaterais negativos, sejam de performance, dificuldade de manutenção ou complexidade desnecessária.

## Exemplos e casos de uso

Listaremos nos tópicos a seguir alguns exemplos importantes, selecionados com o intuito de direcionar o(a) aluno(a) para uma melhor compreensão do processo lógico de modelagem e para demonstrar o uso das simbologias e dos conceitos abordados sobre DER.

Usaremos também uma outra abordagem nesses exemplos, apresentando a lista de requisitos para cada aplicação, e poderemos aplicar a modelagem DER e UML para atender a cada necessidade quando esses requisitos forem interpretados.

Sugerimos que o(a) aluno(a) aproveite tais exemplos, uma vez que o processo de aprendizagem de modelagem está diretamente relacionado com a quantidade de exercícios realizados, pois a prática reforça a compreensão dos conceitos e ajuda a fixar os conhecimentos adquiridos.

## Exemplo 1: serviço de *e-mails*

### Contexto

É fato que o *e-mail* é uma ferramenta indispensável no mundo corporativo, alcançando tamanha importância que se tornou inclusive um documento aceito como comprovação legal.

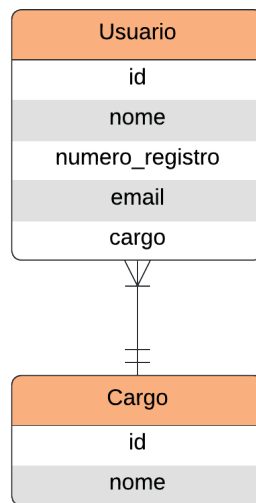
Desse modo, devido à preocupação com vazamento de dados sigilosos, uma empresa deseja iniciar o desenvolvimento de uma solução interna, criando seu próprio serviço de *e-mail*, apresentando os seguintes requisitos básicos:

- Todo *e-mail* possui texto, título, prioridade (normal ou alta), data de criação e pode conter ou não um ou mais anexos;
- Obviamente, um *e-mail* precisa possuir um remetente e um ou mais destinatários, sendo permitido apenas o tráfego de *e-mail* entre os usuários já cadastrados no sistema;
- Os usuários possuem *e-mail*, nome, número de registro e cargo, e este último também deve ser associado aos cargos cadastrados no sistema;
- Os anexos dos *e-mails* apresentam nome, arquivo, tamanho e data de criação;

- Um *e-mail* pode estar ou não relacionado com outro *e-mail* (para casos de respostas, por exemplo).

### Resolução

Como primeiro passo, identificaremos quais entidades são mais independentes e, portanto, não precisam da criação prévia de outras. Para isso, iniciaremos criando as entidades “Cargos” e “Usuários”, bem como todos os atributos mencionados para cada uma.

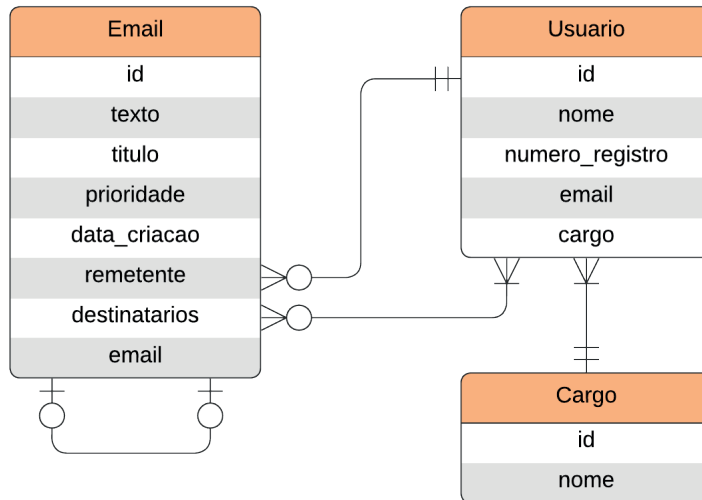


**Figura 2.37** – DER Usuário e Cargo.

Crédito: do elaborador, 2025.

Agora criaremos a entidade “Email”, que utilizará como dependência direta o “Usuário”, uma vez que o remetente do *e-mail* deve também pertencer ao cadastro de usuários do sistema, considerando a existência de apenas um único remetente para cada entidade de *e-mail*.

Aproveitaremos para identificar que os destinatários também consistem em um ou mais usuários, adicionando o respectivo relacionamento com a entidade “Email” como “muitos para muitos”.



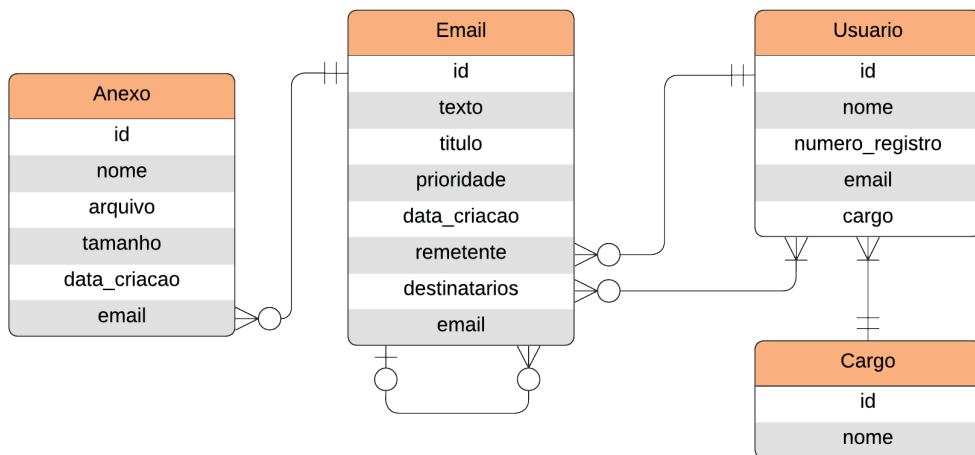
**Figura 2.38** – DER Email, Usuário e Cargo.

Crédito: do elaborador, 2025.

Um detalhe importante é o **relacionamento recursivo** existente na entidade “Email”, isto é, quando uma entidade se relaciona consigo mesma. Por exemplo, um *e-mail* pode estar vinculado a outro *e-mail* como uma resposta, formando desse modo uma associação com o mesmo tipo de entidade, situação muito comum em sistemas de comunicação, em que cada mensagem pode referenciar outra dentro do mesmo conjunto de dados. Embora esse caso possa parecer complexo, seu modo de implementação é semelhante ao realizado em associações com outras entidades, utilizando chaves estrangeiras para execução desse relacionamento.

Nesse caso, de acordo com a simbologia utilizada nesse relacionamento recursivo, um “Email” pode ou não estar associado a outro “Email”, situação utilizada quando a construção de um *e-mail* se torna a resposta de um outro já existente.

Por fim, adicionaremos a entidade restante, ou seja, a de “Anexo”.

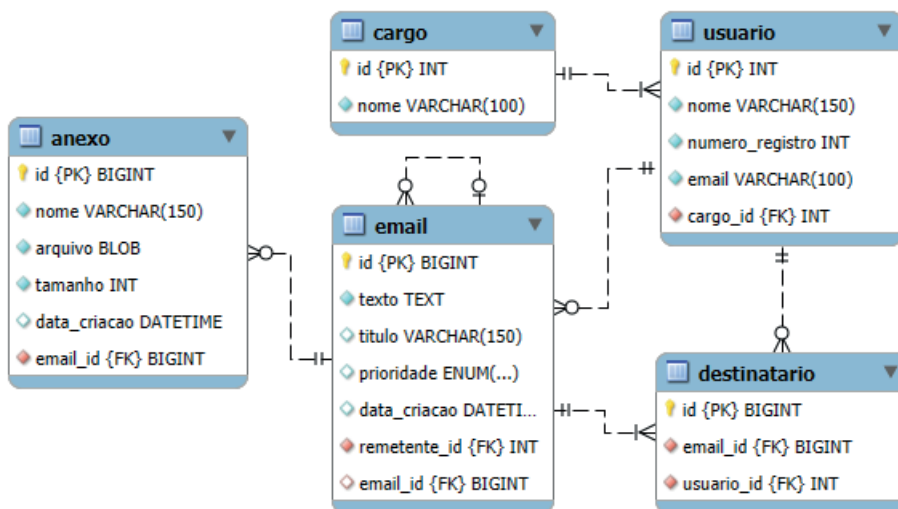


**Figura 2.39** – DER Serviço de Email finalizado.

Crédito: do elaborador, 2025.

Resultado final

Finalmente, definiremos a estrutura UML para construção do banco de dados dessa aplicação.



**Figura 2.40** – UML Serviço de Email finalizado.

Crédito: do elaborador, 2025.

Algumas observações:

- As prioridades da entidade “E-mail” são conhecidas e dificilmente mudarão, portanto, criamos esse campo como um ENUM;

- Para armazenamento de arquivos diretamente no banco de dados, utilizamos na entidade “Anexo” o tipo BLOB (outra maneira muito usada em algumas aplicações é salvar apenas o caminho do arquivo em vez do arquivo por completo, deixando para salvá-lo em um servidor de arquivos);
- Para atender à cardinalidade “muitos para muitos” entre *e-mail* e usuários destinatários, criamos uma tabela separada para poder relacionar cada id de *e-mail* com seu respectivo usuário.

## Exemplo 2: estoque de produtos

### Contexto

Uma empresa de estoque de produtos necessita digitalizar seu processo e, para isso, define vários requisitos funcionais para auxiliar no desenvolvimento de seu sistema:

- Todos os produtos a serem cadastrados possuem uma categoria, a qual tem nome e data de registro, devendo o sistema ser flexível para adição de novas categorias;
- As marcas/fabricantes dos produtos também devem ser cadastradas no sistema, de forma flexível, contendo nome, CNPJ e data de inclusão no sistema;
- Os produtos, por sua vez, apresentam nome, código de registro, código de barras numérico, categoria, marca, data de cadastro, custo, valor de venda, informações adicionais (opcional) e se está em promoção;
- O estoque é dividido em setores de armazenamento, identificados por letras, como setor A, setor B e assim por diante, devendo ser flexível seu cadastro;
- Cada setor possui vários escaninhos, isto é, um local físico de armazenamento de *pallets*, e cada escaninho pode apresentar ou não um

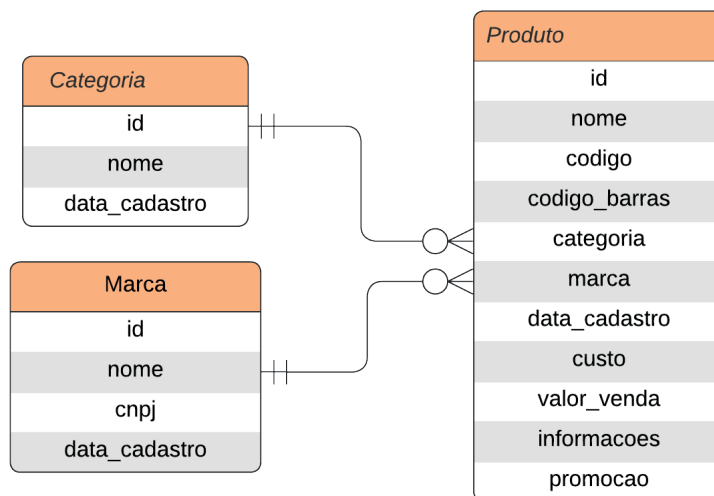
produto, a quantidade desse produto no local e o código do escaninho composto por números.

### Resolução

Inicialmente, construiremos o diagrama entidade relacionamento, com enfoque na identificação das entidades/tabelas presentes no escopo dos requisitos.

Analisando o produto, entidade principal deste projeto, conseguimos identificar facilmente duas outras entidades relacionadas, isto é, marca e categoria, sendo ambas um relacionamento “um para muitos”, ou seja, uma marca/categoria para muitos produtos.

Observe também que adicionamos a simbologia (0,N) ao lado de produto, uma vez que uma categoria pode estar cadastrada no sistema, mas ainda não ter nenhum produto que a utilize.



**Figura 2.41** – DER Produto.

Crédito: do elaborador, 2025.

Em seguida, observaremos as demais entidades, sendo possível identificar a existência de um relacionamento entre Setor-Escaninho-Produto, considerando que um setor apresenta vários escaninhos e que cada escaninho possui apenas um produto.

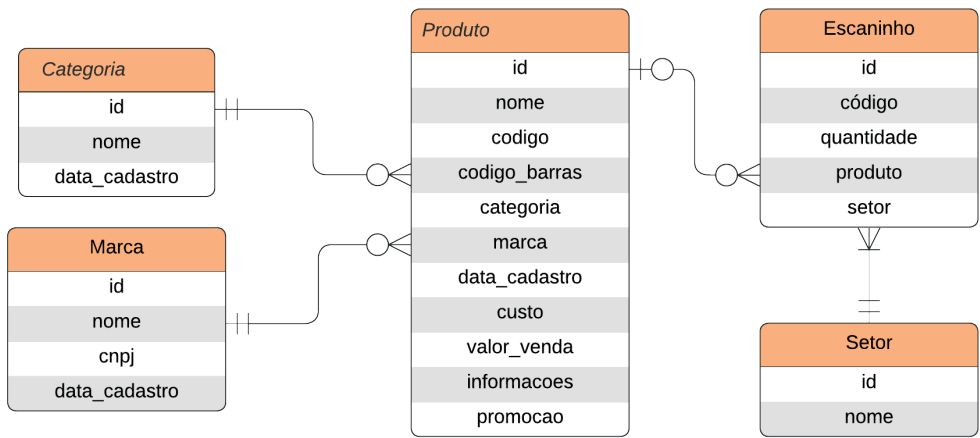


Figura 2.42 – DER Produto Finalizado.

Crédito: do elaborador, 2025.

Resultado final

Uma vez que possuímos o DER que engloba todos os critérios e requisitos exigidos pelo projeto, podemos iniciar o detalhamento UML, chegando ao resultado a seguir.

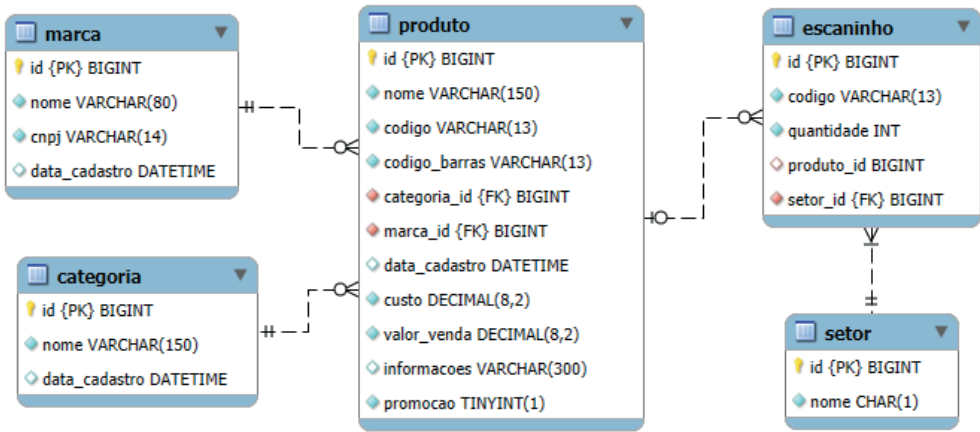


Figura 2.43 – UML Produto.

Crédito: do elaborador, 2025.



Algumas observações:

- Por se tratar de um sistema no qual se presume que haverá muitos produtos, optamos por utilizar as chaves primárias (PK) e, consequentemente, as chaves estrangeiras (FK), com o tipo de dado chamado BIGINT, uma vez que esse tipo possui o tamanho de 8 *bytes* de armazenamento;
- Para o atributo “promoção”, embora ele seja booleano, o MySQL interpreta tais valores com o uso do tipo TINYINT(1), tendo o mesmo conceito.

## Exemplo 3: linha de produção

### Contexto

Uma das etapas fundamentais para se aplicar os conceitos da Indústria 4.0 é a digitalização dos processos, sendo a base de implementação de praticamente todas as tecnologias habilitadoras dessa revolução industrial.

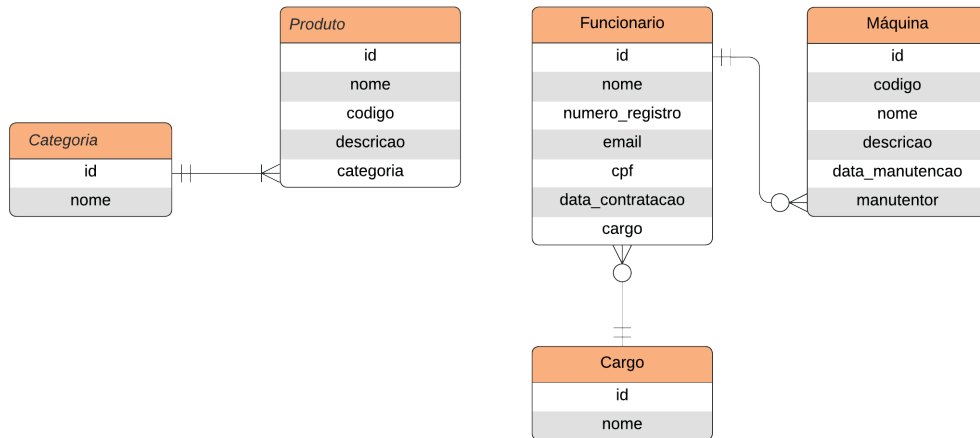
Desse modo, para atender à necessidade de digitalização da produção, uma empresa levantou os seguintes requisitos:

- Os produtos fabricados possuem nome, código, descrição, categoria;
- A produção é dividida em lotes, e cada lote possui código, data e hora de início, data e hora de finalização, data de inspeção, responsável pela inspeção (feita por um funcionário cadastrado), *status* da inspeção (“Aprovado” e “Reprovado”);
- Cada lote possui vários itens produzidos, bem como data e hora de cada item, e a identificação da máquina pela qual foi fabricado;
- Cada máquina possui código identificador, nome, descrição e última data de manutenção, além do funcionário responsável pela última manutenção;
- Cada funcionário possui nome, número de registro, *e-mail*, CPF, data de contratação e cargo.

## Resolução

Identificamos primeiro as entidades que já podemos criar sem a necessidade ou dependência de outras, ou seja, podemos criar “Cargos”, “Categorias” e “Máquinas”.

Em seguida, usaremos as entidades criadas para também relacionar com “Funcionários” e “Produtos”.



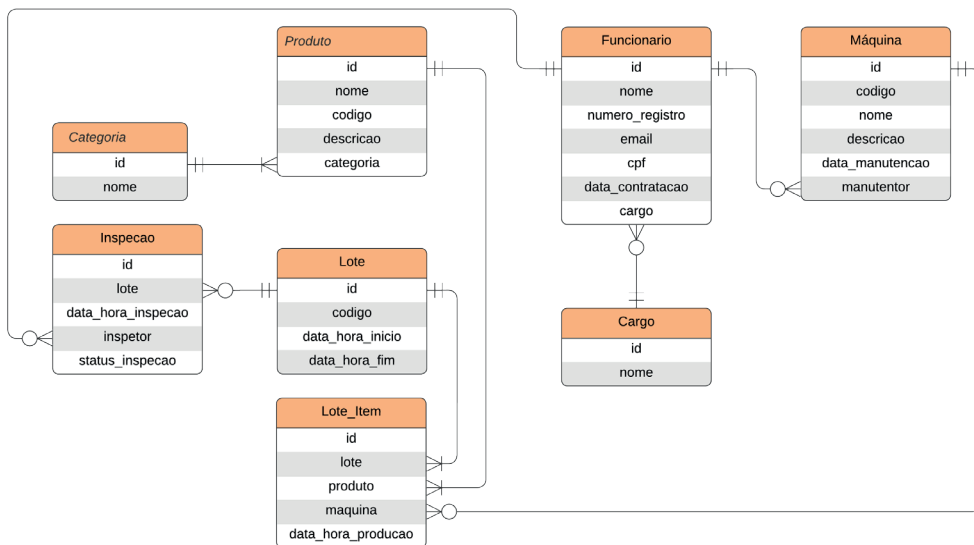
**Figura 2.44** – DER Produto e Funcionários.

Crédito: do elaborador, 2025.

Desse modo, já conseguimos atender a vários requisitos, tais como:

- Um funcionário apresenta obrigatoriamente um cargo, e este, em contrapartida, pode ser ocupado por vários funcionários;
- Toda máquina possui um funcionário atrelado à sua manutenção;
- Todo produto possui uma categoria.

Em seguida, faremos as associações faltantes para o processo de produção, envolvendo respectivamente “Produtos”, “Lotes” e “Itens do Lote”.

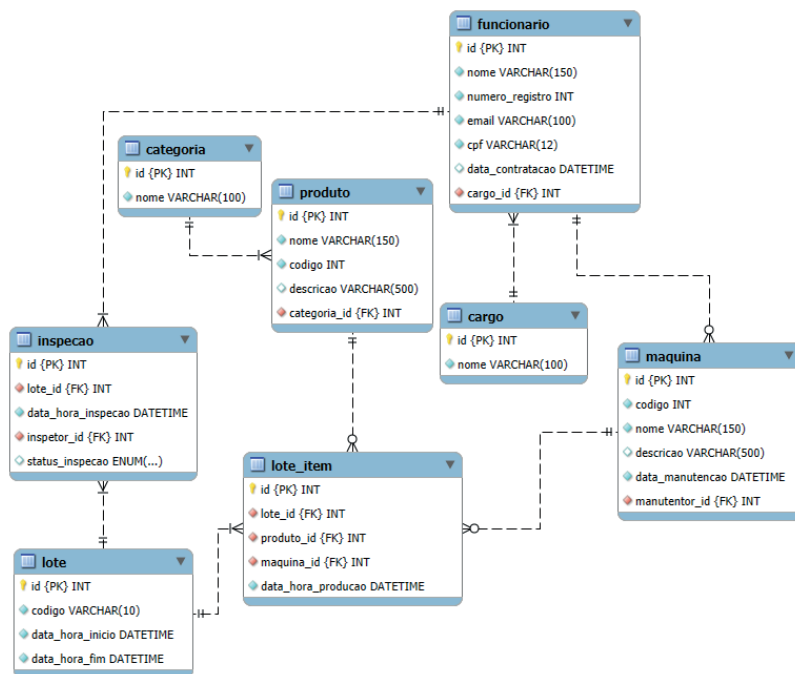


**Figura 2.45** – DER Linha de Produção finalizada.

Crédito: do elaborador, 2025.

Resultado final

Após a finalização do DER, conseguimos detalhar a diagramação UML.



**Figura 2.46** – DER Linha de Produção finalizada.

Crédito: do elaborador, 2025.

Alguns apontamentos sobre o resultado final:

- Criamos uma entidade denominada “lote\_item” para ser associada com “lote”, possibilitando que um lote apresente um conjunto de itens;
- Criamos uma entidade chamada “inspeção” para separar as informações pertinentes à inspeção do lote, também possibilitando que um lote tenha mais de uma inspeção, caso necessário;
- Reaproveitamos a entidade “funcionário” em diversos relacionamentos, uma vez que tanto o inspetor quanto o responsável pela manutenção das máquinas são funcionários da empresa.

## Exemplo 4: *streaming* de filmes

### Contexto

Uma *startup* deseja iniciar suas atividades no setor de *streaming* de filmes nacionais criados por produtores locais autônomos e independentes.

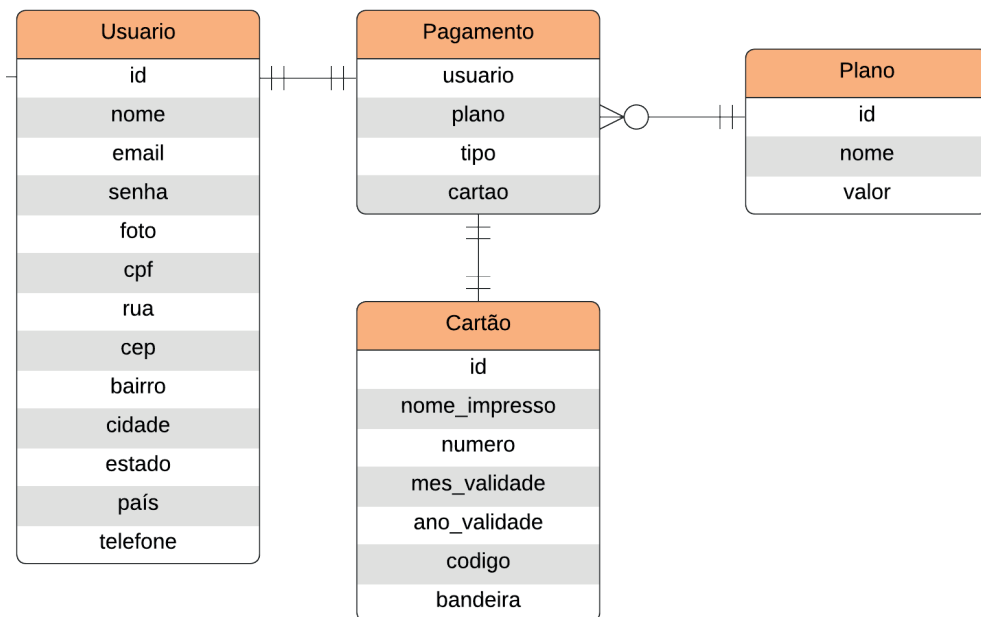
Dessa forma, foram levantados os requisitos básicos de funcionamento descritos a seguir:

- A aplicação deve ser acessada apenas por usuários clientes, mediante cadastro de nome, *e-mail*, senha, foto do usuário (opcional), CPF, endereço (rua, CEP, bairro, cidade, estado, país), telefone e plano escolhido;
- Cada plano (Bronze, Prata e Ouro) possui um valor de assinatura associado;
- Cada usuário terá um método de pagamento associado à sua conta (boleto, PIX ou cartão de crédito);
- Caso o usuário opte por cartão de crédito, devem ser armazenadas as principais informações, como nome impresso, bandeira, data de validade, número e código de segurança;
- Cada usuário pode favoritar em sua conta inúmeros filmes/séries;

- Os *streamings* possuem as informações de título, resumo, descrição, diretores, categoria (Ficção científica, Comédia, Terror etc.), ano de lançamento, tempo de duração, tipo (Filme ou Série) e arquivo de reprodução;
- As séries devem estar associadas às temporadas;
- As temporadas, por sua vez, possuem vários *streamings* associados, também possuindo a informação de número da temporada e ano de lançamento.

### Resolução

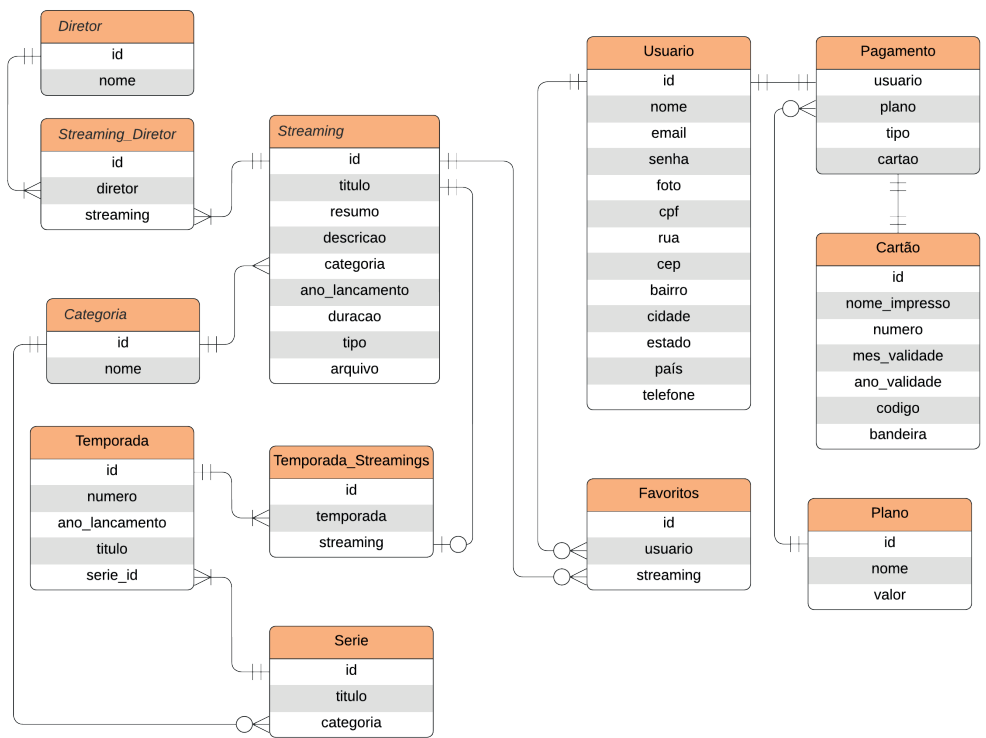
Primeiro, faremos todo o relacionamento que envolve usuários, conta, plano e outras entidades correlatas.



**Figura 2.47** – DER Streaming.

Crédito: do elaborador, 2025.

Em seguida, criaremos a estrutura do *streaming*, com atenção para interligar o usuário aos favoritos.

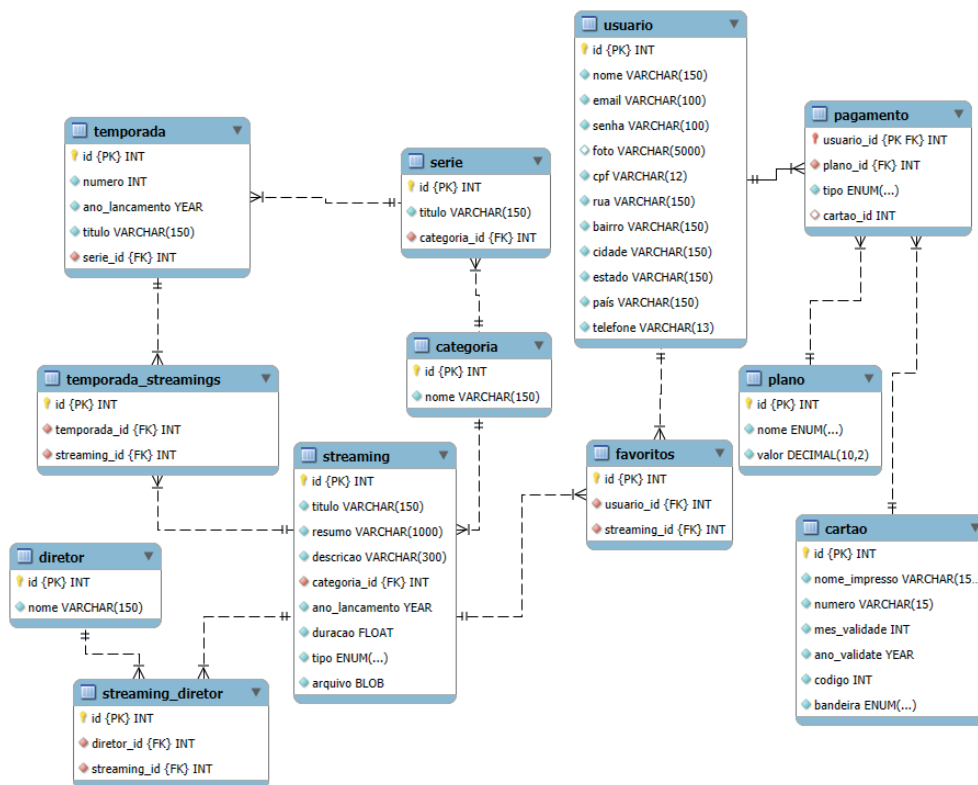


**Figura 2.48** – DER Streaming Finalizado.

Crédito: do elaborador, 2025.

### Resultado

Agora, finalizaremos com o diagrama UML, detalhando a estrutura DER já planejada.



**Figura 2.49** – UML Streaming Finalizado.

Crédito: do elaborador, 2025.

## Resumo

Chegamos ao fim de uma etapa crucial: a compreensão da modelagem de dados!

Após explorar os fundamentos essenciais dos bancos de dados, adentramos agora um dos pilares do desenvolvimento eficiente: a Modelagem de Dados.

Neste capítulo, discutimos a importância de estruturar corretamente os dados antes mesmo de criar as bases, garantindo clareza, organização e aderência às necessidades do sistema. Apresentamos o diagrama entidade

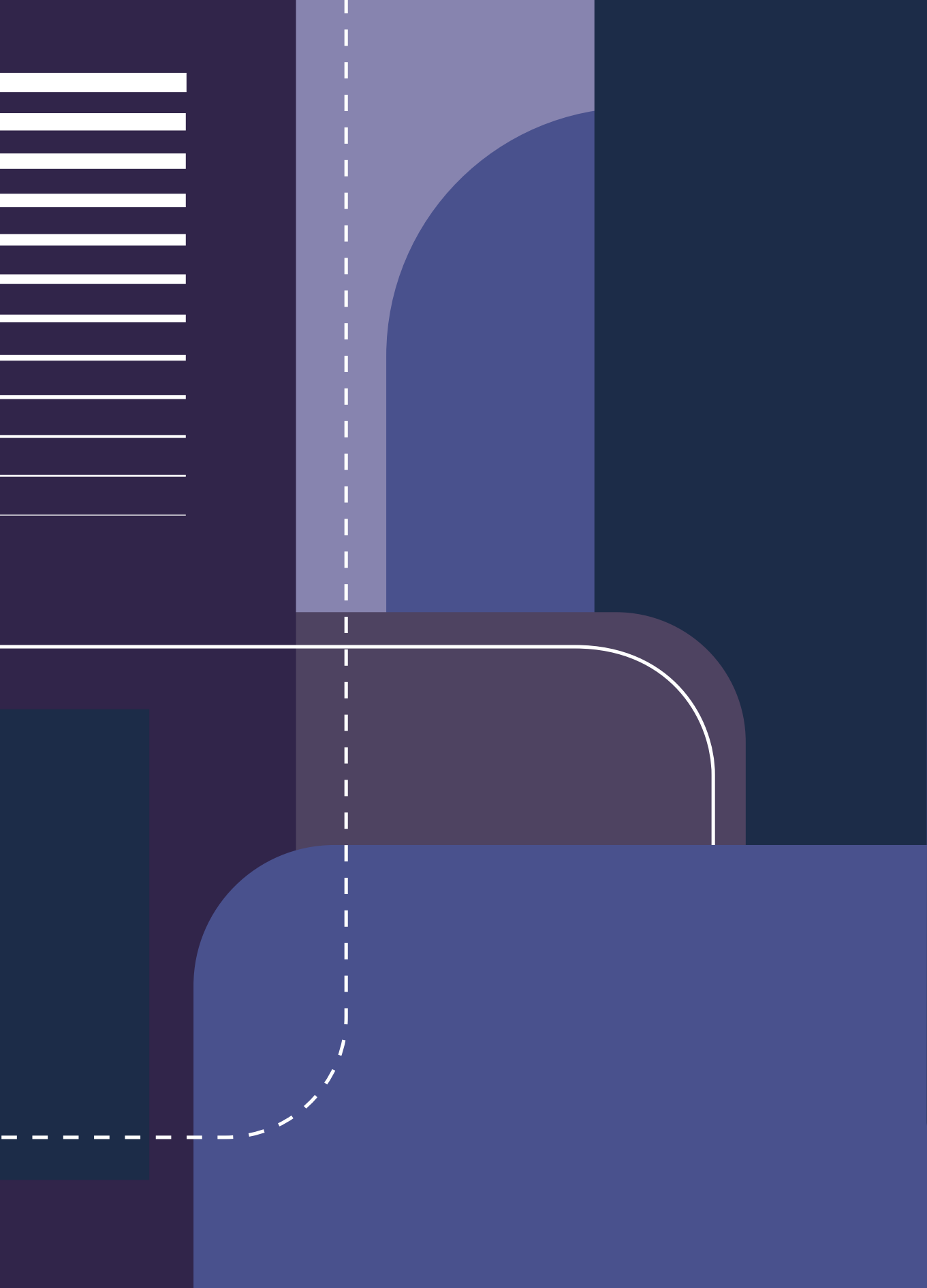
relacionamento (DER), que permite visualizar as entidades, seus atributos e os relacionamentos entre elas, além de explicar as cardinalidades e as simbologias utilizadas para expressar conexões de maneira clara e precisa.

Também exploramos como a Linguagem de Modelagem Unificada (UML) pode complementar o trabalho de modelagem, proporcionando uma visão abrangente e padronizada para documentar requisitos e interações entre os elementos do sistema.

Por fim, abordamos situações práticas de aprendizagem com casos de uso, simulando cenários reais em que a modelagem de dados é aplicada. Esses exemplos nos ajudam a entender como os conceitos se conectam e se traduzem em soluções efetivas no dia a dia do desenvolvimento.

No próximo capítulo, vamos continuar aprofundando os conhecimentos e avançar nas próximas etapas do trabalho com bancos de dados.







## CAPÍTULO 3

# MANIPULAÇÃO DE DADOS I – MODELO RELACIONAL

Já imaginou construir aplicações incríveis que interagem com bancos de dados? Neste capítulo, você dará um passo fundamental nessa direção, dominando a linguagem SQL.

Chegamos agora em um importante capítulo, pois apresentaremos a você, aluno(a), a tão esperada concretização de assuntos que foram abordados anteriormente.

Nos capítulos anteriores, você deve se recordar que exploramos os fundamentos dos bancos de dados, desde os conceitos iniciais sobre armazenamento e organização das informações até a modelagem relacional, o que

nos permite estruturar dados de maneira eficiente. Além disso, você também aprendeu sobre entidades, atributos, relacionamentos e suas cardinalidades e a normalização de tabelas para evitar redundâncias e inconsistências.

Agora você terá a oportunidade de integrar todos esses conceitos, já que estudaremos de forma prática a linguagem SQL (*Structured Query Language* ou Linguagem de Consulta Estruturada), indispensável para o desenvolvimento de aplicações que fazem uso de bancos relacionais.

Neste capítulo, abordaremos desde a criação de bancos e tabelas até a manipulação de dados presentes nessas tabelas, por meio da utilização de cláusulas que nos auxiliarão na seleção e obtenção das informações desejadas.

Por fim, destacamos a importância de explorar ao máximo o conteúdo apresentado neste capítulo, pois ele foi cuidadosamente elaborado para oferecer a você uma base sólida e prática. As habilidades que você desenvolverá aqui são essenciais para o uso eficiente de bancos de dados relacionais. Aproveite cada seção e desafio proposto, pois isso contribuirá significativamente para o seu desenvolvimento como um(a) profissional habilidoso(a) e confiante na utilização de SQL.

## Conectando-se ao servidor

Vimos nos capítulos anteriores que, para utilizar os benefícios de um banco de dados, é necessária a instalação do servidor. Neste momento, consideraremos que o(a) aluno(a) já possua o Sistema de Gerenciamento de Banco de Dados (SGBD) do servidor MySQL em sua máquina.

No entanto, para iniciarmos a interação com o servidor, é necessário estar conectado a ele. Existem inúmeras formas de realizarmos essa conexão, e apresentaremos mais adiante a possibilidade de uso do terminal de comando ou de uso da interface gráfica do MySQL Workbench.

## Terminal de comando

Apesar de apresentarmos uma alternativa mais intuitiva, é fundamental o conhecimento da conexão via terminal de comando. No cenário profissional, é comum que desenvolvedores se deparem com situações em que essa é a única forma de interagir com servidores, especialmente em ambientes remotos ou na nuvem. Além disso, o uso do terminal é essencial para automação de tarefas repetitivas ou complexas, permitindo a criação de *scripts* que agilizam processos e garantem a qualidade na execução dos comandos.

Siga os passos seguintes, podendo haver uma ligeira necessidade de mudança de acordo com seu sistema operacional:

1. Abra o terminal de comando de seu sistema operacional;
2. Antes de iniciarmos o comando final, vamos entender sua estrutura e parâmetros:

```
mysql -h ENDEREÇO -P PORTA -u USUÁRIO -p
```

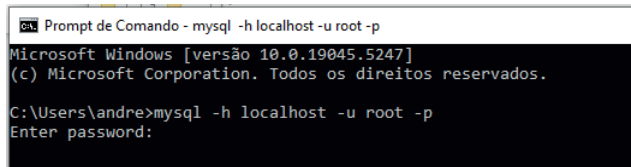
- O parâmetro “-h” tem a função de informar qual o endereço de domínio ou de IP em que o servidor está sendo executado. Em nosso caso, estamos utilizando o MySQL localmente, portanto podemos preencher com o valor de “localhost” ou “127.0.0.1”;
- O parâmetro “-P” serve para identificar a porta em que o servidor está em funcionamento, que por padrão para o MySQL é a 3306, podendo esse parâmetro ser omitido caso não haja alteração desse padrão (deve ser sempre informado em letra maiúscula para não ser confundido com outro parâmetro);
- O parâmetro “-u” informa qual o usuário a ser utilizado na conexão, devendo ser utilizado aquele criado durante o processo de instalação do servidor (comumente utilizado como “root”);
- O parâmetro “-p” indica que deve ser digitada uma senha para conexão após a execução do comando (é possível informar a senha juntamente com este parâmetro, mas por motivos de segurança essa prática não é indicada, uma vez que a senha ficará visível no histórico de comandos do terminal).

**FIQUE ALERTA****Linux**

Se estiver utilizando um sistema operacional Linux, em alguns casos, dependendo da configuração de permissões do sistema, será necessário elevar os privilégios para executar o comando. Isso geralmente é feito com o prefixo **sudo** antes do comando original. Se o comando não for executado corretamente sem o **sudo**, tente adicioná-lo e observe se a execução é bem-sucedida, conforme exemplo a seguir:

```
sudo mysql -h ENDEREÇO -P PORTA -u USUÁRIO -p
```

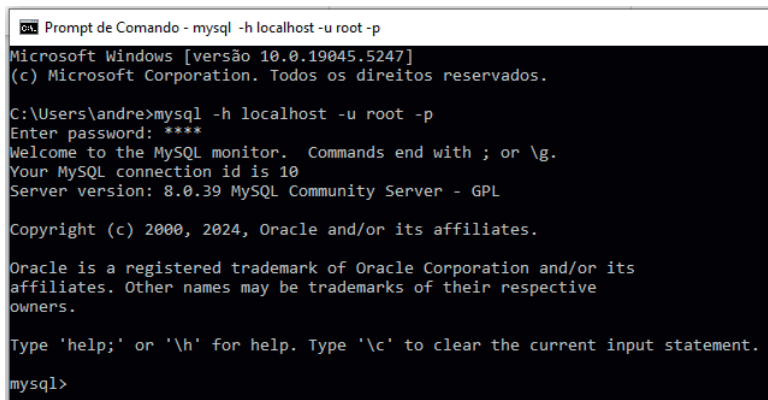
3. Agora podemos substituir os valores faltantes no comando original:



**Figura 3.1** – Imagem de conexão do MySQL via terminal de comando.

Crédito: do elaborador, 2025.

4. Digite em seguida a senha do seu usuário criado durante a instalação do MySQL (note que sua digitação não será visível no terminal por questões de segurança):



**Figura 3.2** – Imagem de conexão bem-sucedida do MySQL via terminal de comando.

Crédito: do elaborador, 2025.

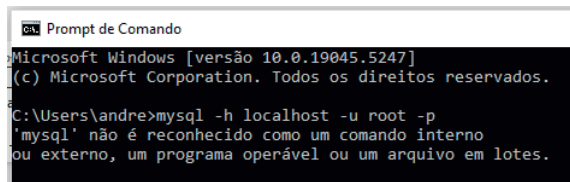
5. Finalmente, seu sistema operacional se conectará ao servidor, liberando seu terminal para o uso de comandos em SQL. Caso não consiga se conectar, reveja os comandos utilizados e principalmente se as credenciais de usuário e senha que você utilizou estão corretas.

### FIQUE ALERTA

#### MySQL e sistema operacional

Para que o comando de conexão funcione conforme o esperado, é importante que a instalação do MySQL esteja sendo corretamente reconhecida pelo sistema operacional.

Caso você esteja utilizando o sistema Windows, mas o comando **mysql** não esteja sendo reconhecido, muito provavelmente será necessário configurar as variáveis de ambiente de sua máquina primeiro:



```
CS: Prompt de Comando
Microsoft Windows [versão 10.0.19045.5247]
(c) Microsoft Corporation. Todos os direitos reservados.

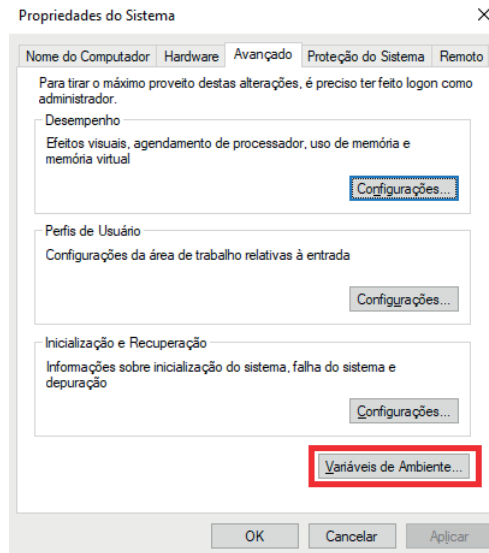
C:\Users\andre>mysql -h localhost -u root -p
'mysql' não é reconhecido como um comando interno
ou externo, um programa operável ou um arquivo em lotes.
```

**Figura 3.3** – Imagem do comando **mysql** não reconhecido via terminal de comando.

Crédito: do elaborador, 2025.

Neste caso, siga os procedimentos seguintes:

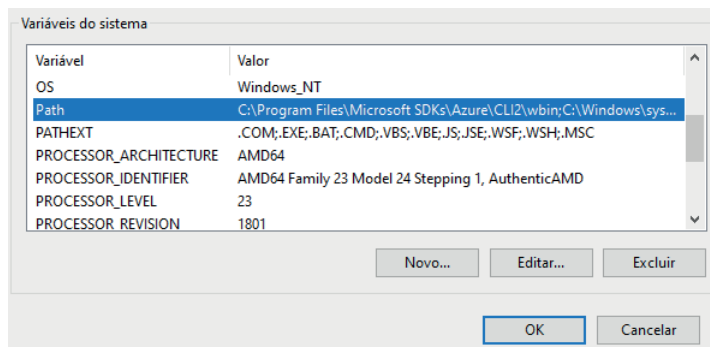
- Encontre o local no qual seu MySQL foi instalado e copie o caminho até a pasta “/bin”. Se você seguiu o processo de instalação padrão, essa localização normalmente se encontra em “C:\Program Files\MySQL\MySQL Server 8.0\bin”;
- Procure “Variáveis de Ambiente” em sua barra de pesquisa do Windows e selecione o botão contendo o mesmo nome:



**Figura 3.4** – Propriedades do sistema (Windows) com a opção de seleção de Variáveis de Ambiente.

Crédito: do elaborador, 2025.

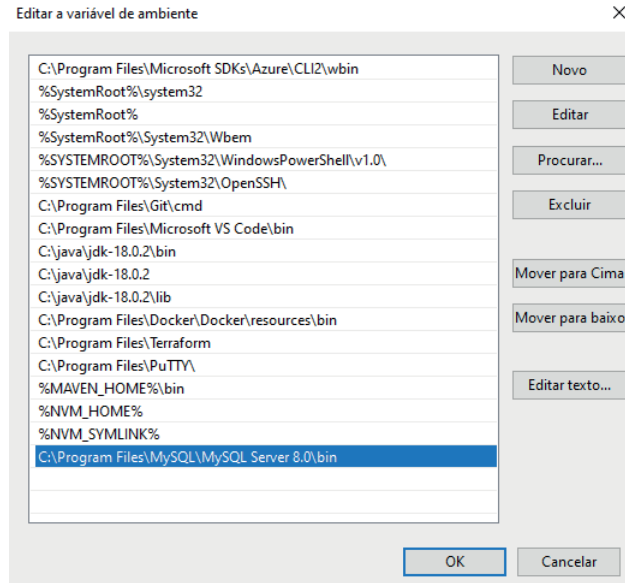
- Dentro da seção de “Variáveis de Sistema”, selecione a opção “Path” e clique em “Editar”:



**Figura 3.5** – Variáveis de Ambiente Windows.

Crédito: do elaborador, 2025.

- Cole o caminho copiado na última linha disponível e feche em seguida todas as janelas abertas, clicando em “OK”:



**Figura 3.6** – Variáveis de Ambiente Windows para adicionar o diretório de instalação do MySQL.

Crédito: do elaborador, 2025.

- Feche os terminais de comando existentes e tente novamente a execução do comando de conexão do MySQL, seguindo os passos descritos neste capítulo.

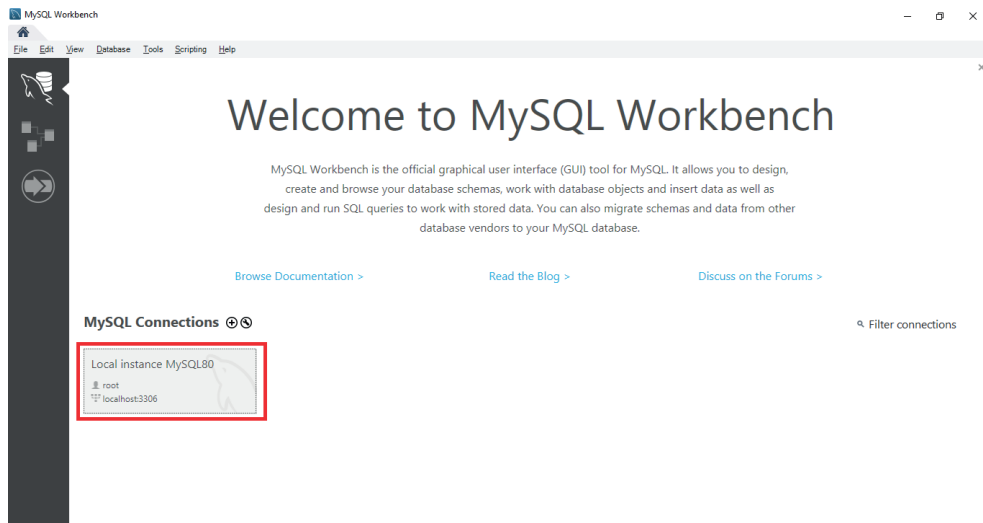
## MySQL Workbench

O MySQL Workbench é uma ótima opção para interagir com o servidor MySQL de forma mais visual e intuitiva, apresentando ferramentas de conexão que dispensam o uso do terminal de comando pelo usuário.

Para isso, siga os passos a seguir:

1. Abra o MySQL Workbench que foi instalado (como visto no **Capítulo 1**);
2. Em geral, após a instalação do Workbench, é gerada automaticamente uma conexão com o servidor local, conforme a **Figura 3.7**:





**Figura 3.7** – Conexão local gerada pelo MySQL Workbench.

Crédito: do elaborador, 2025.

3. Contudo, caso essa conexão tenha sido excluída ou não gerada em seu Workbench, você pode seguir os **passos 4 a 7**, ou, se tudo já estiver certo, siga apenas o **passo 8**;
4. Após abrir o MySQL Workbench e se certificar de que não existe nenhuma conexão, clique no botão adicionar representado pelo símbolo de “+”:

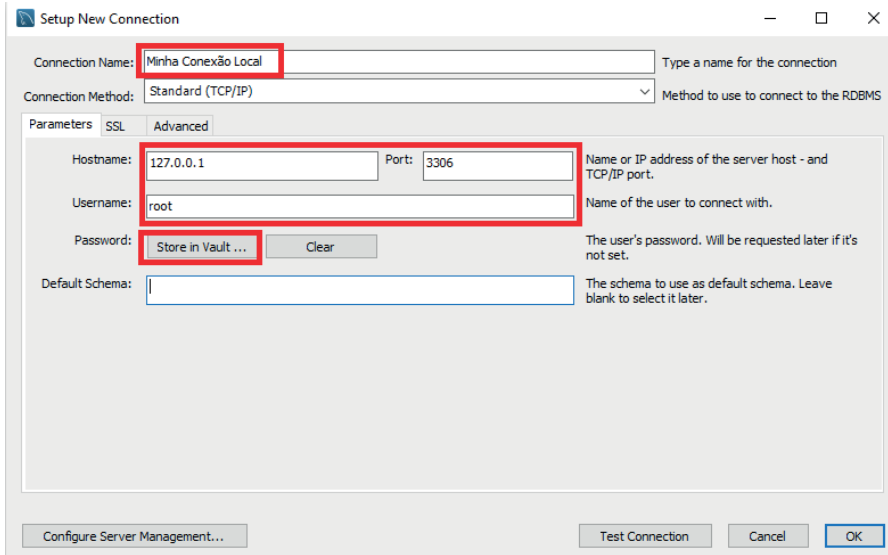


**Figura 3.8** – Indicação do botão de adicionar conexão do MySQL Workbench.

Crédito: do elaborador, 2025.

5. Preencha o nome da conexão (Connection Name) com qualquer texto desejado, deixe o endereço (Hostname) e a porta (Port) do servidor

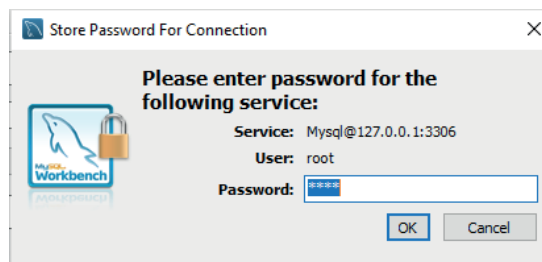
conforme a **Figura 3.9**. Em usuário (Username), preencha conforme o que você criou durante a instalação do MySQL e, em seguida, clique no botão de Store in Vault:



**Figura 3.9** – Adicionando nova conexão – MySQL Workbench.

Crédito: do elaborador, 2025.

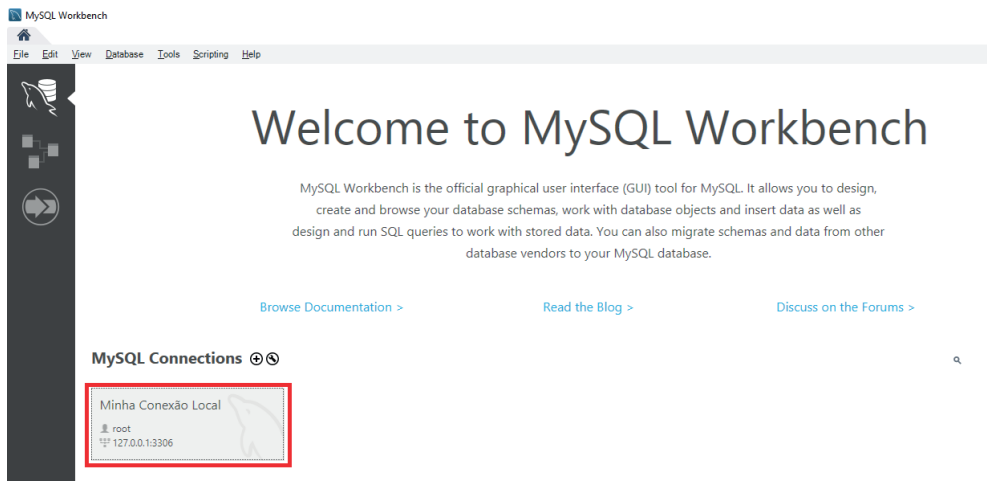
6. Esse procedimento não é obrigatório, mas para maior comodidade iremos optar por deixar a senha de conexão salva no MySQL Workbench, a fim de evitar que tenhamos que digitar a senha toda vez que a conexão for aberta (não recomendamos que seja habilitada essa opção em bancos de dados em cenários reais). Portanto, digite a senha correspondente ao usuário utilizado:



**Figura 3.10** – Armazenamento da senha de conexão – MySQL Workbench.

Crédito: do elaborador, 2025.

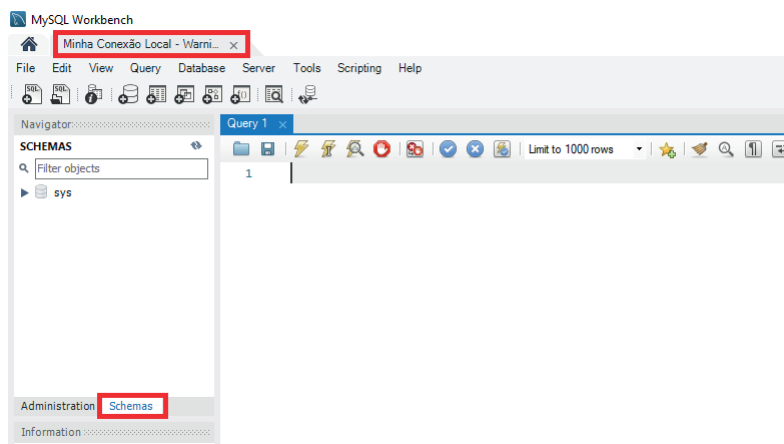
7. Clique em OK até voltar na tela principal do Workbench, na qual poderá visualizar sua conexão criada:



**Figura 3.11** – Acessando a conexão – MySQL Workbench.

Crédito: do elaborador, 2025.

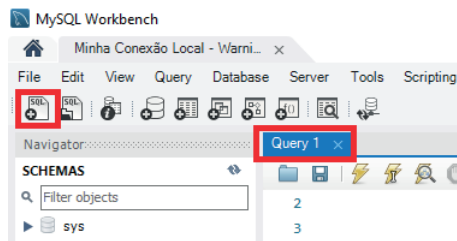
8. Clique na conexão desejada e você poderá visualizar todo o ambiente de trabalho de seu Workbench. Note que, como mostrado na **Figura 3.12**, o nome da aba aberta é exatamente o nome da conexão criada. Além disso, na seção SCHEMAS, é possível enxergar a existência do *schema* “sys”, que nada mais é do que um banco gerado automaticamente pelo MySQL com o objetivo de facilitar o monitoramento e a administração de banco de dados em geral:



**Figura 3.12** – Conexão realizada – MySQL Workbench.

Crédito: do elaborador, 2025.

9. O MySQL Workbench é bem intuitivo e não deve apresentar complexidade para iniciantes no uso do *software*. Existem diferentes formas de execução de comandos, e em todas é necessário que você esteja com algum documento SQL aberto. Normalmente, um novo documento é aberto assim que se estabelece uma conexão com o servidor; porém, caso o tenha fechado, você pode criar um novo utilizando o ícone disponível na barra de ferramentas:



**Figura 3.13** – Abrir novo documento – MySQL Workbench.

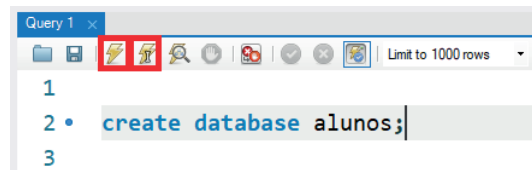
Crédito: do elaborador, 2025.

10. Para mostrar as diferentes formas de execução de comandos dentro do Workbench, utilizaremos um comando simples como exemplo (não se preocupe em tentar entendê-lo agora, pois o faremos adiante). Digite o comando a seguir (criação de banco de dados) dentro do novo documento:

```
create database escola;
```

11. Para execução de um comando, existem três formas:

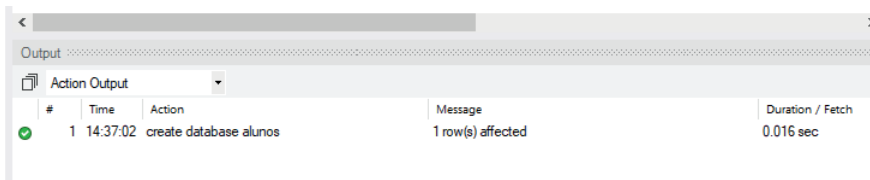
- Deixe o cursor do seu teclado na mesma linha do comando e pressione as teclas `ctrl + enter`;
- Selecione um ou mais comandos que se deseja executar com o *mouse*, envolvendo o comando por completo (como se desejasse copiá-lo), e em seguida clique no botão representado por um raio;
- Deixe o cursor do seu teclado na mesma linha do comando e clique no botão representado por um raio com o símbolo de cursor.



**Figura 3.14** – Comando SQL de exemplo com indicação dos botões de execução – MySQL Workbench.

Crédito: do elaborador, 2025.

12. Após a execução do comando, você deve poder visualizar a sua confirmação:



**Figura 3.15** – Confirmação de execução de comando – MySQL Workbench.

Crédito: do elaborador, 2025.

## SQL – Linguagem de Consulta Estruturada

Antes de nos aprofundarmos nos principais comandos do SQL, o que faremos ainda neste capítulo, é conveniente que você entenda primeiro algumas características fundamentais dessa linguagem, com enfoque no MySQL, banco de dados que escolhemos para este livro.

SQL (*Structured Query Language* ou Linguagem de Consulta Estruturada) é uma linguagem usada para interagir com bancos de dados relacionais e, no contexto do MySQL, é a ferramenta principal para executar tarefas como criação de bancos, inserção de dados, consultas, manipulações, entre outras operações.

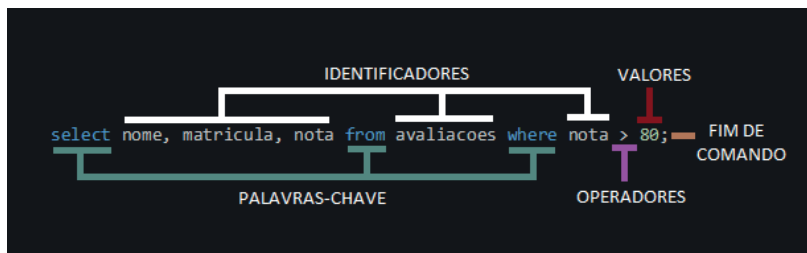
Embora o SQL seja padronizado, o MySQL oferece algumas particularidades e extensões que o tornam único em alguns comandos, o que significa

que alguns comandos que são funcionais em um banco MySQL podem não ser em outro tipo de banco relacional (por exemplo, no SQL Server).

## Estrutura básica

Um comando SQL é composto por uma combinação de **palavras-chave** e **identificadores** (como nomes de tabelas ou colunas), podendo conter também **valores** e **operadores** e possuir um indicador de fim de comando. Os comandos são lidos de cima para baixo e da esquerda para a direita, como uma linguagem humana.

Para exemplificar esse conceito, apresentaremos na **Figura 3.16** um comando SQL com os referidos elementos (não se preocupe em entender agora o que ele faz, apenas observe sua estrutura).



**Figura 3.16** – Exemplo de comando SQL e seus elementos.

Crédito: do elaborador, 2025.

É importante ressaltar que algumas palavras-chave, operadores e valores nem sempre são obrigatórios, podendo estar presentes ou não em certos comandos, dependendo da combinação de palavras-chave e do objetivo desejado.

### IMPORTANTE

#### Comandos SQL

Diferentemente de algumas linguagens de programação que frequentemente executam *loops* contínuos para manter aplicações em funcionamento, os comandos SQL (também chamados de *scripts*) têm como objetivo principal realizar operações pontuais e bem-definidas. Cada

**comando SQL é projetado para atingir um propósito específico, como consultar dados, inserir registros ou alterar estruturas no banco de dados.**

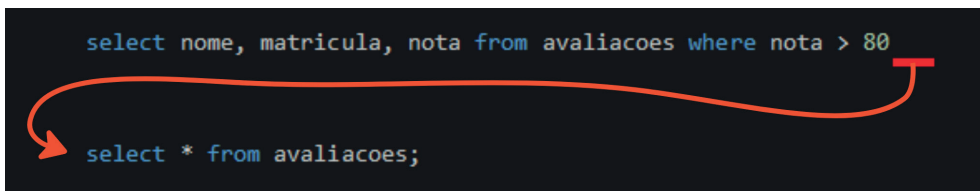
**Esse comportamento é intencional, já que o SQL é orientado a transações e não a processos contínuos.**

**Por exemplo, enquanto uma aplicação que usa a linguagem de programação Python pode monitorar constantemente um sensor e sempre reagir a mudanças, um *script* SQL, de modo contrário, tem a função de ser executado uma vez para, digamos, atualizar o preço de um produto com base em uma nova cotação, finalizando sua execução logo em seguida, pois o objetivo da transação já foi concluído.**

## Finalização de comando

Todo comando no MySQL deve ser terminado com o caractere finalizador de comando, isto é, um **ponto e vírgula (;)**, informando ao servidor que a instrução terminou e está pronta para ser executada.

Caso o comando que esteja sendo executado seja o último do *script*, é possível que ele seja executado sem apresentar ponto e vírgula no MySQL Workbench; caso contrário, o SGBD interpretará todo o *script* como um único comando até encontrar um caractere finalizador subsequente, mesmo estando em linhas separadas, conforme exemplo da **Figura 3.17**.



**Figura 3.17** – *Script* SQL com indicação de fim de comando faltante.

Crédito: do elaborador, 2025.

Por esse motivo, para evitar confusões ou execuções de comandos não intencionais, é sempre recomendado utilizar ponto e vírgula para finalizar cada instrução.

Conforme indicado na **Figura 3.17**, observe que, apesar dos dois códigos estarem separados por espaço e linhas diferentes, a ausência do ponto

e vírgula no comando de cima faz com que o servidor interprete como se houvesse uma continuação no comando subsequente, o que com certeza resultaria em uma falha de execução nesse *script*.

## Case sensitivity – Sensibilidade a maiúsculas e minúsculas

Uma das coisas que se deve verificar ao aprender uma nova linguagem de programação é a sensibilidade às letras maiúsculas e minúsculas, característica chamada *case sensitivity* e que difere muito em cada linguagem.

No caso do SQL, **as palavras-chave não são sensíveis** a maiúsculas ou minúsculas, o que quer dizer que os três comandos mostrados a seguir, para o servidor MySQL, serão interpretados da mesma forma:

```
select * from avaliacoes;
```

```
SELECT * FROM avaliacoes;
```

```
Select * From avaliacoes;
```

No entanto, de acordo com o sistema operacional em que o servidor MySQL estiver sendo executado, **identificadores** como nomes de bancos, tabelas ou colunas **podem ser sensíveis** às letras maiúsculas ou minúsculas.

Basicamente, em sistemas operacionais baseados em Unix (como o Linux e o macOS), o sistema de arquivos é *case sensitive*, o que faz com que o servidor MySQL, caso esteja em algum desses sistemas, se comporte de forma a fazer tais diferenciações de identificadores em maiúscula ou minúscula, situação um tanto quanto comum, uma vez que vários servidores em cenários reais possuem o Linux como seu sistema operacional.

Desse modo, é possível haver uma interpretação diferente entre tais comandos para o mesmo identificador, conforme mostrado a seguir:



```
select * from avaliacoes;  
  
select * from AVALIACOES;  
  
select * from Avaliacoes;
```

## Comentários

A possibilidade de incluir comentários em meio ao código de uma linguagem é um recurso muito útil para melhorar a legibilidade e servir como documentação de partes de um sistema, além do fato de servir também como uma ferramenta valiosa para estudantes de programação realizarem suas anotações.

De modo semelhante à maioria das linguagens, o SQL possibilita a inclusão de comentários de uma única linha ou de múltiplas linhas, conforme podemos ver a seguir:

```
-- este é um comentário de uma linha  
  
/* este é um comentário  
   de múltiplas linhas  
*/
```

- Para comentários em uma mesma linha, podemos utilizar a sequência de caracteres de dois traços (--);
- Para comentários envolvendo uma ou mais linhas, podemos utilizar a sequência de caracteres barra + duplo asteriscos + barra (/\*\*/), incluindo todo o texto desejado entre os dois asteriscos.

## Tipos de dados

Apresentaremos no **Quadro 3.1** os principais tipos de dados existentes no MySQL (não se preocupe em decorá-los, pois você poderá entender melhor o uso dos principais tipos com o decorrer das situações-problemas que apresentaremos neste capítulo).

**Quadro 3.1 – Tipos de dados no MySQL**

| Categoria            | Tipo de Dados     | Descrição                                                                                             |
|----------------------|-------------------|-------------------------------------------------------------------------------------------------------|
| Numéricos inteiros   | TINYINT           | Inteiro pequeno (1 byte); valores entre -128 e 127 ou 0 a 255 (sem sinal).                            |
|                      | SMALLINT          | Inteiro pequeno (2 bytes); valores entre -32 768 e 32 767 (sem sinal até 65 535).                     |
|                      | MEDIUMINT         | Inteiro médio (3 bytes); valores entre -8 388 608 e 8 388 607 (sem sinal até 16 777 215).             |
|                      | INT / INTEGER     | Inteiro padrão (4 bytes); valores entre -2 147 483 648 e 2 147 483 647 (sem sinal até 4 294 967 295). |
|                      | BIGINT            | Inteiro grande (8 bytes); valores enormes, ideal para armazenar IDs globais.                          |
| Numéricos decimais   | DECIMAL / NUMERIC | Para números com precisão definida, como valores monetários [DECIMAL(10,2)].                          |
|                      | FLOAT             | Ponto flutuante de precisão simples; valores aproximados.                                             |
|                      | DOUBLE / REAL     | Ponto flutuante de precisão dupla; mais preciso que FLOAT.                                            |
| Caracteres e strings | CHAR              | Texto de tamanho fixo (até 255 caracteres). Ideal para dados de comprimento constante.                |
|                      | VARCHAR           | Texto de tamanho variável (até 65 535 caracteres, dependendo do limite de linha).                     |
|                      | TEXT              | Texto de até 65 535 caracteres; ideal para grandes blocos de texto.                                   |
|                      | TINYTEXT          | Texto pequeno (até 255 caracteres).                                                                   |
|                      | MEDIUMTEXT        | Texto médio (até 16 777 215 caracteres).                                                              |
|                      | LONGTEXT          | Texto longo (até 4 GB de texto).                                                                      |
|                      | ENUM              | Lista predefinida de valores possíveis [exemplo: ENUM('MASCULINO', 'FEMININO')].                      |
| Datas e hora         | SET               | Lista de valores múltiplos predefinidos [exemplo: SET('A', 'B', 'C')].                                |
|                      | DATE              | Data no formato AAAA-MM-DD.                                                                           |
|                      | DATETIME          | Data e hora no formato AAAA-MM-DD HH:MM:SS.                                                           |
|                      | TIMESTAMP         | Data e hora com fuso horário, ideal para marcações temporais.                                         |
|                      | TIME              | Hora no formato HH:MM:SS.                                                                             |
| Binários             | YEAR              | Ano de 4 dígitos (1901 a 2155).                                                                       |
|                      | BINARY            | Dados binários de tamanho fixo.                                                                       |
|                      | VARBINARY         | Dados binários de tamanho variável.                                                                   |
|                      | TINYBLOB          | Dados binários pequenos (até 255 bytes).                                                              |
|                      | BLOB              | Dados binários maiores (até 65 535 bytes).                                                            |
|                      | MEDIUMBLOB        | Dados binários médios (até 16 777 215 bytes).                                                         |
|                      | LOB               | Dados binários grandes (até 4 GB).                                                                    |

Crédito: do elaborador, 2025.

## Constraints – Restrições

No MySQL, as *constraints* são restrições que modificam atributos com o objetivo de definir regras e propriedades específicas para as colunas em uma tabela. Esses modificadores ajudam a garantir a integridade e a validação dos dados, aumentando a confiabilidade da aplicação, uma vez que dados somente podem ser registrados após se enquadrarem nas regras definidas.

Veremos como melhor utilizá-los ao longo deste capítulo, pois, ao explorarmos os principais comandos SQL, seremos levados a utilizá-los em algum momento, principalmente durante a execução de situações-problemas e casos de uso.

Contudo, apresentaremos no **Quadro 3.2** uma relação das principais *constraints* existentes no MySQL e sua respectiva funcionalidade.

**Quadro 3.2** – Tipos de *constraints* no MySQL

| Restrição/Constraint  | Descrição                                                                                                   |
|-----------------------|-------------------------------------------------------------------------------------------------------------|
| <b>not null</b>       | Garante que a coluna não pode conter valores nulos.                                                         |
| <b>null</b>           | Permite que a coluna contenha valores nulos (este é o comportamento padrão).                                |
| <b>default</b>        | Define um valor padrão para a coluna, que será usado quando nenhum valor for especificado na inserção.      |
| <b>unique</b>         | Garante que todos os valores na coluna sejam únicos.                                                        |
| <b>primary key</b>    | Combina not null e unique, identificando exclusivamente cada linha na tabela.                               |
| <b>auto_increment</b> | Gera automaticamente um valor incremental único para a coluna, geralmente usado com chaves primárias.       |
| <b>check</b>          | Impõe uma condição que os valores na coluna devem satisfazer (disponível a partir do MySQL 8.0).            |
| <b>index</b>          | Cria um índice para a coluna, melhorando o desempenho das consultas que a utilizam.                         |
| <b>unsigned</b>       | Define que a coluna não pode armazenar valores negativos, aumentando o intervalo de valores positivos.      |
| <b>zerofill</b>       | Preenche automaticamente números com zeros à esquerda até o comprimento especificado (usado com unsigned).  |
| <b>foreign key</b>    | Define uma relação entre a coluna e uma chave primária em outra tabela, garantindo integridade referencial. |

Crédito: do elaborador, 2025.

**IMPORTANTE****Constraints**

Mais de uma *constraint* pode ser utilizada ao mesmo tempo em um mesmo atributo, dependendo do objetivo que se deseja alcançar. Por exemplo, em uma situação em que se deseja definir que um atributo não seja nulo e ao mesmo tempo seja único, podemos utilizar a *constraint not null* juntamente com a *unique*.

A *constraint null* pode ser informada ou não, uma vez que a omissão dela já faz com que o MySQL considere o atributo como nulo. Por exemplo, em uma situação em que se deseja definir que um atributo possa ser nulo, podemos utilizar o *null* ou simplesmente não utilizar nenhum modificador relacionado (*not null* e *default*).

## Criando e utilizando uma base de dados

### create database – Criando um *schema*

Damos início ao nosso aprofundamento no SQL com um dos comandos mais essenciais: a criação de bancos de dados. Esse comando serve como ponto de partida para a construção de qualquer projeto em um sistema de gerenciamento de banco de dados relacional, e é com ele que estabelecemos o espaço em que todas as tabelas, índices e demais estruturas de dados serão organizadas e mantidas.

Embora tenha uma grande importância, sua estrutura é muito simples, conforme podemos observar a seguir, sendo necessário informar apenas o nome desejado para o banco/*schema*:

```
-- ESTRUTURA DO COMANDO:  
create database NOME_DO_BANCO;  
  
-- EXEMPLOS:  
create database mercado;  
create database sistema_de_gestao_rh;
```

## use – Ativando um *schema*

Em uma conexão com o MySQL, podem existir vários bancos ou *schemas*, portanto é fundamental informar ao servidor qual banco será utilizado no momento como referência para aplicação dos comandos.

Desse modo, é importante conhecer o comando **use**, cuja função é ativar o uso de um banco em específico:

```
--ESTRUTURA DO COMANDO:  
use NOME_DO_BANCO;  
  
--EXEMPLOS:  
use mercado;  
use sistema_de_gestao_rh;
```

## drop database – Deletando um *schema*

Agora que vimos como criar e ativar um banco de dados, talvez você se questione como deletá-lo.

É nesse contexto que apresentamos o comando **drop database**, que tem o objetivo de remover permanentemente um *schema*/banco de dados, seguindo a estrutura de comando:

```
--ESTRUTURA DO COMANDO:
drop database NOME_DO_BANCO;

--EXEMPLOS:
drop database mercado;
drop database sistema_de_gestao_rh;
```

## Exemplos e casos de uso

### *Blog* de notícias

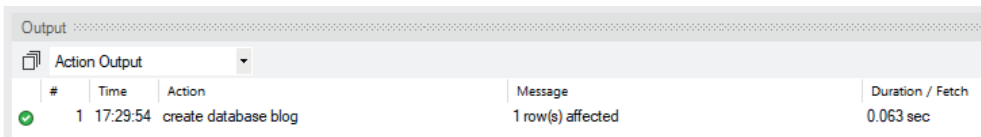
Um *blog* de notícias necessita de um banco de dados para armazenar notícias, categorias de leitura, autores de artigos etc.

Para atender a essa necessidade, seguiremos os passos seguintes:

1. Criaremos um banco de dados chamado **blog**, utilizando o comando **create database**, conforme mostrado a seguir:

```
create database blog;
```

2. Executaremos o comando e verificaremos se ele foi bem-sucedido:

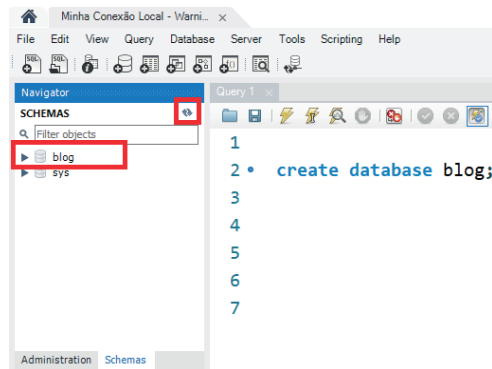


| Output        |          |                      |                   |                  |
|---------------|----------|----------------------|-------------------|------------------|
| Action Output |          |                      |                   |                  |
| #             | Time     | Action               | Message           | Duration / Fetch |
| ✓ 1           | 17:29:54 | create database blog | 1 row(s) affected | 0.063 sec        |

**Figura 3.18** – Confirmação de criação do banco – MySQL Workbench.

Crédito: do elaborador, 2025.

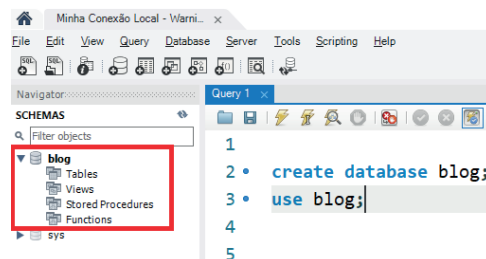
3. Dentro do ambiente de trabalho do MySQL Workbench, procuraremos o banco de dados criado na seção **SCHEMAS**. Para visualização, pode ser necessário clicar no botão de atualizar, conforme indicado na **Figura 3.19**:



**Figura 3.19** – Confirmação de criação do banco de dados – MySQL Workbench.

Crédito: do elaborador, 2025.

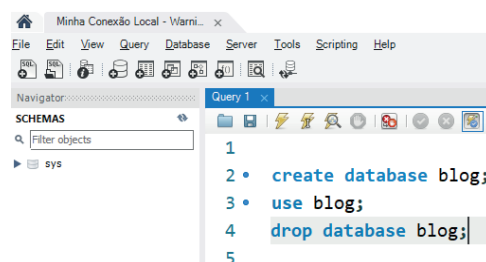
4. Agora faremos a digitação do comando de utilização do banco. Após a execução do comando, será possível visualizar que o banco ativo no momento é o mesmo criado (**blog**):



**Figura 3.20** – Comando SQL para utilização do banco de dados do *blog* de notícias.

Crédito: do elaborador, 2025.

5. Finalmente, para treinarmos o comando de deleção de um banco, usaremos o **drop database**:



**Figura 3.21** – Comando SQL para remoção do banco de dados do *blog* de notícias.

Crédito: do elaborador, 2025.

## Exercícios

### *E-commerce*

Um sistema de *e-commerce* possui inúmeros produtos para serem comercializados, cada um com sua característica, nome, marca, preço etc. Além disso, em sistemas com essa funcionalidade, é necessário o armazenamento de informações de venda, rastreamento de entregas etc.

Nada disso seria possível sem o uso de um banco de dados!

Portanto, crie e utilize um banco de dados chamado **e\_commerce** para atender a essa necessidade!

### Papelaria

Uma papelaria local precisa de um sistema para gerenciar seu estoque e suas vendas. A papelaria vende uma variedade de produtos, como canetas, cadernos, lápis, borrachas e diversos materiais de escritório. Cada produto tem um código, descrição, preço e quantidade em estoque. Além disso, é necessário registrar as vendas realizadas, incluindo o cliente, a data da venda e os produtos vendidos.

Para organizar todas essas informações de forma eficiente, a papelaria precisa de um banco de dados. Portanto, crie e utilize um banco de dados chamado **papelaria** para atender a essa necessidade!

### Biblioteca comunitária

Uma biblioteca comunitária local está modernizando seu sistema de gerenciamento de acervos e empréstimos. A biblioteca possui diversos livros, revistas e outros materiais que precisam ser catalogados.

Recentemente, durante uma tentativa de atualização no sistema, o banco de dados existente foi corrompido devido a inconsistências na estrutura das tabelas. Assim, o banco de dados atual precisa ser excluído e reconstruído.



Portanto, exclua o banco de dados antigo, de nome **biblioteca**, e recrie-o novamente, ativando-o em seguida.

## CRUD – Criar, Ler, Atualizar e Deletar Dados

### create – Criando e removendo tabelas

create table – Criando tabelas independentes

Aprendemos a criar bancos de dados, mas um banco vazio não atende às necessidades de nenhuma aplicação!

É nesse contexto que entram as tabelas, nas quais de fato os dados são armazenados, organizados e podem ser acessados facilmente.

Usaremos então o comando **create table**, conforme a estrutura apresentada a seguir, em que também é possível identificar um exemplo de criação de uma tabela para armazenar usuários de um jogo *on-line* (tabela **gamer**):

```
--ESTRUTURA DO COMANDO:
create table NOME_DA_TABELA (
  NOME_DA_COLUNA TIPO_DE_DADO CONSTRAINTS,
  NOME_DA_COLUNA TIPO_DE_DADO CONSTRAINTS,
  .
  .
  .
);

--EXEMPLO:
create table gamer (
  id bigint primary key auto_increment,
```

```

nome varchar(250) not null,
email varchar(250) not null,
apelido varchar(150),
pontuacao int default 0,
usuario_premium boolean default false,
data_cadastro datetime default now()
);

```

Conforme você pode notar, o comando de criação de tabelas é relativamente simples, composto pelo nome desejado para a tabela e as definições das colunas desejadas, isto é, nome, tipo de dado e as *constraints* da coluna.

#### IMPORTANTE

**Caso o(a) aluno(a) não se recorde dos termos “constraints” e “tipagem de dado”, recomendamos que reveja os tópicos **Tipos de dados e Constraints – Restrições**, que apresentam as definições, e os Quadros 3.1 e 3.2 com os principais tipos.**

Alguns pontos sobre nosso exemplo da tabela **gamer**, apresentada anteriormente, que vale a pena observarmos:

1. Para os nomes compostos, jamais utilizamos espaço, pois isso resultaria em uma interpretação errônea pelo servidor, sendo mais comum o uso de “\_” (*underlines*) para separá-los, como fizemos com as colunas de **usuario\_premium** e **data\_cadastro** (o uso de acentuação também não é recomendado);
2. Utilizamos um tipo de dado de acordo com cada funcionalidade da coluna:
  - **varchar** para colunas nas quais se armazenarão os textos, sendo os casos de **nome**, **email** e **apelido** – neste último, optamos por utilizar uma quantidade caracteres menor em relação a **nome** e **email**;

- **int** para a coluna de pontuação, uma vez que, geralmente em jogos, esse tipo de informação é representado por números inteiros;
  - **bigint** para a coluna de **id**, que, por se tratar da chave primária dessa tabela, deve estar adequada para o armazenamento de uma quantidade muito grande de *gamers*;
  - **boolean** para a coluna que identificará se o usuário é *premium* ou não, portanto, pode-se utilizar o conceito booleano de “verdadeiro” ou “falso”;
  - **datetime** para armazenamento de data e hora de cadastro do *gamer* dentro da aplicação.
3. Observe que optamos por deixar o uso de um apelido para o usuário *gamer* como opcional, já que não foi utilizada a *constraint not null* nessa coluna;
  4. Usou-se a chave primária **id** com a opção de **auto\_increment**, o que fará com que o próprio servidor MySQL gerencie o sequenciamento das chaves de cada usuário por ordem de cadastro, o que quer dizer que, se um último usuário tem **id** igual a 500, o próximo será consequentemente o 501 e assim por diante;
  5. Adicionamos um valor padrão de pontuação equivalente a 0 para usuários novos, através da utilização do **default**, o que também foi feito para a definição de usuário *premium*, com padrão **false**;
  6. Por fim, para a coluna de **data\_cadastro**, optamos por utilizar uma estratégia interessante: deixar que o servidor preencha automaticamente a data e a hora em que um usuário foi cadastrado, através da utilização do **default now()**.

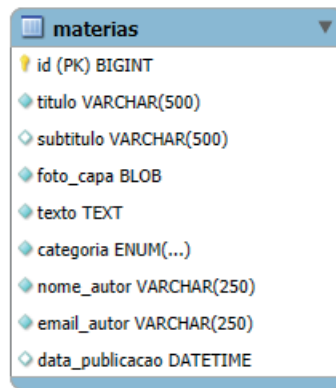
Podemos extrair muitas observações de apenas um único exemplo, não é mesmo? Por esse motivo, apresentaremos agora mais alguns exemplos e cenários para reforçar o conceito.

Exemplos e casos de uso

### Exemplo 1 – *Blog de notícias*

Vamos reaproveitar o exemplo tratado no tópico sobre criação de bancos de dados, ou seja, uma aplicação para atender a um *blog* de notícias:

1. Daremos corpo ao que seria a tabela principal para armazenamento das matérias, representado primeiro em UML para que o(a) aluno(a) possa ter maior clareza do objetivo:



**Figura 3.22** – Diagrama UML – tabela de matérias.

Crédito: do elaborador, 2025.

Algumas observações sobre a estrutura escolhida para a tabela:

- Para armazenamento da foto de capa da matéria do *blog*, escolhemos o tipo **BLOB**, ou seja, optamos pelo armazenamento da foto de forma binária diretamente no banco, embora também fosse possível o armazenamento no formato de texto por meio de conversão para **Base64** ou salvando apenas o caminho da imagem no servidor;
- Para armazenamento do texto da matéria, o que se espera ter uma quantidade de caracteres razoável, utilizamos o tipo de dado **TEXT**;
- Já para as categorias, foram definidos valores específicos como opções de preenchimento, utilizando, portanto, o tipo **ENUM**.

2. Criaremos e ativaremos o banco de dados para essa aplicação:

```
create database blog;  
use blog;
```

3. Finalmente, criaremos a tabela seguindo a estrutura do comando **create table**, obedecendo ao que foi definido anteriormente na modelagem UML:

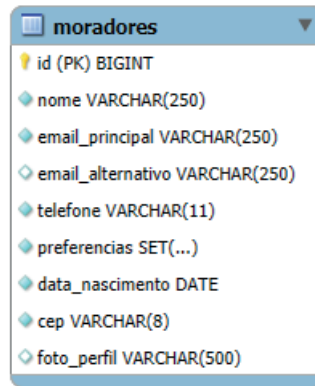
```
create table materias (  
  id bigint primary key auto_increment,  
  titulo varchar(500) not null,  
  subtitulo varchar(500),  
  foto_capa blob not null,  
  texto text not null,  
  categoria enum('POLÍTICA','POLICIAL','HUMOR','CINEMA',  
  'CELEBRIDADES')  
    not null,  
  nome_autor varchar(250) not null,  
  email_autor varchar(250) not null,  
  data_publicacao datetime default now()  
);
```

## Exemplo 2 – Central de alertas

A Defesa Civil da cidade de Campinas, localizada em uma região com histórico de chuvas intensas e deslizamentos, implementou o sistema “Alerta Cidadão” para garantir a segurança de seus moradores. Por meio de um aplicativo para *smartphones* e um *site*, os cidadãos podem se cadastrar e receber alertas personalizados sobre riscos de inundações, deslizamentos e outros eventos climáticos extremos.

Para isso, será criada uma tabela em que os moradores poderão se registrar nesse sistema, seguindo esses passos:

1. Representaremos a tabela de moradores em UML:



**Figura 3.23** – Diagrama UML – tabela de moradores.

Crédito: do elaborador, 2025.

Algumas observações sobre a estrutura escolhida para a tabela:

- O *e-mail* alternativo será opcional, portanto, não utilizará o **not null**;
- Ambos os *e-mails*, alternativo e principal, receberão a *constraint unique*, garantindo, portanto, que sejam únicos em todo o banco de dados, evitando cadastros em duplicidade;
- Utilizamos o tipo de dado **SET** para possibilitar que o morador selecione as opções de comunicação desejadas, permitindo múltiplas escolhas;
- Para apresentar outra possibilidade de armazenamento de imagens, utilizamos o **VARCHAR**, uma vez que a proposta é que a foto do morador seja arquivada externamente, ficando no banco de dados apenas o *link* de acesso a esse local.

2. Criaremos e ativaremos o banco de dados para essa aplicação:

```
create database central_alertas;
use central_alertas;
```

3. Finalmente, criaremos a tabela seguindo a estrutura do comando **create table**, obedecendo ao que foi definido anteriormente na modelagem UML:

```
CREATE TABLE moradores (
  id bigint auto_increment primary key,
  nome varchar(250) not null,
  email_principal varchar(250) unique not null,
  email_alternativo varchar(250) unique,
  telefone varchar(11) not null,
  preferencias set('EMAIL', 'SMS', 'WHATSAPP') not
    null,
  data_nascimento date not null,
  cep varchar(8) not null,
  foto_perfil varchar(500)
);
```

create table – Criando tabelas relacionadas

Abordamos até o momento como proceder com a criação de tabelas que não possuem nenhuma chave estrangeira em sua estrutura. Contudo, sabemos que em cenários reais tais tabelas não atendem a 100% dos casos, portanto veremos agora como criar tabelas e relacioná-las com outras tabelas já existentes.

Como podemos visualizar a seguir, a estrutura de comando é praticamente a mesma de uma tabela independente, com adição apenas da cláusula **foreign key**, dando mais detalhes sobre o relacionamento desejado:

```
--ESTRUTURA DO COMANDO:
create table NOME_DA_TABELA (
  NOME_DA_COLUNA TIPO_DE_DADO MODIFICADORES/CONSTRAINTS,
  NOME_DA_COLUNA TIPO_DE_DADO MODIFICADORES/CONSTRAINTS,
  .
  .
```

```

,,
foreign key(NOME_DA_COLUNA_ESTRANGEIRA) references
    NOME_DA_TABELA_RELACIONADA(CHAVE_PRIMÁRIA_TABELA_
RELACIONADA)
);

```

### FIQUE ALERTA

Para utilizar o comando de criação de uma tabela relacionada, é fundamental que a tabela que se deseja relacionar já exista na base de dados, caso contrário, o comando de criação acusará um erro! Por esse motivo, recomendamos que o(a) aluno(a) comece sempre com a criação das tabelas independentes (entidades fortes) e posteriormente avance para a criação das demais.

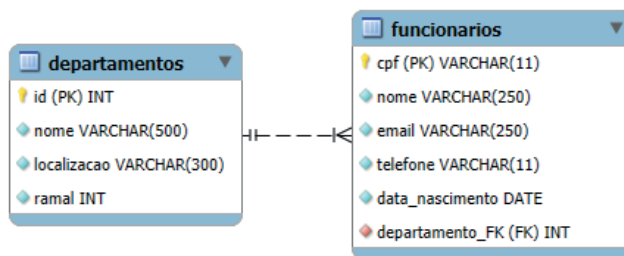
Veremos a seguir alguns exemplos para melhor clareza sobre o assunto.

#### Exemplos e casos de uso

##### Exemplo 1 – Funcionários e departamentos

Começaremos com um exemplo bastante simples e intuitivo que exige um relacionamento entre duas tabelas, como é o caso de funcionários de uma empresa e seus respectivos departamentos:

1. Detalharemos o relacionamento proposto em um diagrama UML:



**Figura 3.24** – Diagrama UML representando o relacionamento de “um para muitos” entre funcionários e departamentos.

Crédito: do elaborador, 2025.

Algumas observações sobre a modelagem apresentada:

- Na tabela de funcionários, optamos por nomear o que será a chave estrangeira da relação com o sufixo **FK** no nome do campo



(**departamento\_FK**), sendo uma prática comum para facilitar a identificação de relacionamentos, mas não obrigatória, podendo o(a) aluno(a) optar por não a utilizar;

- Ainda na tabela de funcionários, sua chave primária será o próprio CPF do funcionário, uma vez que conceitualmente essa informação não deve se repetir e nem ser nula, apresentando ao(a) aluno(a) a possibilidade de utilização de outros tipos de chaves primárias além do tradicional **bigint** + **auto\_increment**.

2. Em seguida, criaremos e ativaremos o banco de dados para essa aplicação:

```
create database empresa;
use empresa;
```

3. Agora podemos criar as tabelas, seguindo primeiro a criação da entidade de departamentos (entidade forte) e em seguida a de funcionários:

```
create table departamentos (
  id int primary key auto_increment,
  nome varchar(500) not null,
  localizacao varchar(300) not null,
  ramal int not null
);

create table funcionarios (
  cpf varchar(11) primary key not null,
  nome varchar(250) not null,
  email varchar(250) unique not null,
  telefone varchar(11) not null,
  data_nascimento date not null,
  departamento_FK int not null,
  foreign key(departamento_FK) references departamentos(id)
);
```

Note que, para adicionar esse relacionamento, indicamos que o campo com a chave estrangeira dentro da tabela de funcionários é **departamento\_FK**, referenciando-o com a chave primária **id** presente na tabela de departamentos.

### FIQUE ALERTA

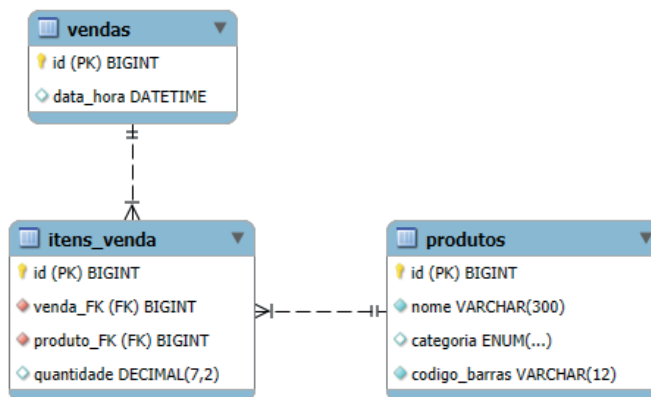
#### Chave estrangeira e chave primária

É fundamental que o tipo de dado do campo que receberá a chave estrangeira seja o mesmo da chave primária da tabela relacionada. Em outras palavras, como o tipo de dado do campo **id** da tabela **departamentos** é um **int**, então o campo **departamento\_FK** presente na tabela **funcionários** também deve ser do mesmo tipo, caso contrário, o servidor acusará um erro ao tentar executar o comando.

### Exemplo 2 – Quitanda

Uma pequena quitanda tem controlado seu fluxo de vendas manualmente em planilhas, no entanto, como esse número vem crescendo, esse processo manual tem gerado muito trabalho e expondo os resultados a erros de digitação. Desse modo, essa pequena empresa gostaria de digitalizar suas vendas pelo uso de um sistema para contabilizar os itens vendidos e a receita gerada:

1. Criaremos uma modelagem UML que atenda a essa necessidade, gerando uma tabela para armazenamento dos produtos com preços, e outras tabelas para gerenciamento das vendas e itens vendidos:



**Figura 3.25** – Diagrama UML representando o relacionamento entre produtos e vendas de uma quitanda.

Crédito: do elaborador, 2025.

Algumas observações sobre a modelagem apresentada:

- Optamos por dividir as vendas de itens vendidos para poder identificar o que foi vendido em cada operação efetuada;
- Utilizamos o tipo de dado **decimal(7,2)** para registro de quantidade, já que em quitandas é comum a venda de itens por quilo, havendo consequentemente dados fracionários (7 indica a quantidade total máxima de dígitos permitida já considerando casa decimal, e 2 indica a precisão da casa decimal, ou seja, um número máximo de 99999,99).

## 2. Criaremos a base de dados:

```
create database quitanda;  
use quitanda;
```

## 3. Faremos agora a utilização dos comandos de criação das tabelas de acordo com a modelagem projetada:

```
create table produtos (  
  id bigint primary key auto_increment,  
  nome varchar(300) not null,  
  categoria enum ('FRUTAS','LEGUMES','VERDURAS','LATICÍNIOS',  
  'GRÃOS'),  
  codigo_barras varchar(12) not null unique  
);  
  
create table vendas (  
  id bigint primary key auto_increment,  
  data_hora datetime default now()  
);  
  
create table itens_venda (  
  id bigint primary key auto_increment,  
  venda_FK bigint not null,
```

```

produto_FK bigint not null,
quantidade decimal(7,2) default 1.0,
foreign key (venda_FK) references vendas(id),
foreign key (produto_FK) references produtos(id)
);

```

create table – Criando tabelas com chaves primárias compostas  
Chaves primárias compostas são chaves formadas por duas ou mais colunas de uma tabela, ao contrário de uma chave primária simples, que utiliza apenas uma coluna para identificar cada registro de forma única.

Seu uso se dá mais comumente em situações em que apenas uma coluna não é suficiente para identificar cada registro de forma única e se deseja evitar a criação de identificadores com **auto\_increment**.

Esse cenário é mais frequentemente encontrado em relacionamentos “muitos para muitos” ou em tabelas que armazenam dados históricos ou de transações (preço de um produto em relação ao tempo, por exemplo).

A criação de tabelas com tais chaves compostas é relativamente simples, basta seguir a estrutura seguinte, reaproveitando a mesma cláusula **primary key**, colocando como parâmetros os nomes de quaisquer colunas a serem utilizadas para formação da chave:

```

--ESTRUTURA DO COMANDO
create table NOME_DA_TABELA (
    NOME_DA_COLUNA_1 TIPO_DE_DADO MODIFICADORES/CONSTRAINTS,
    NOME_DA_COLUNA_2 TIPO_DE_DADO MODIFICADORES/CONSTRAINTS,
    .
    .
    .,
    primary key (NOME_DA_COLUNA_1, NOME_DA_COLUNA_2)
);

```

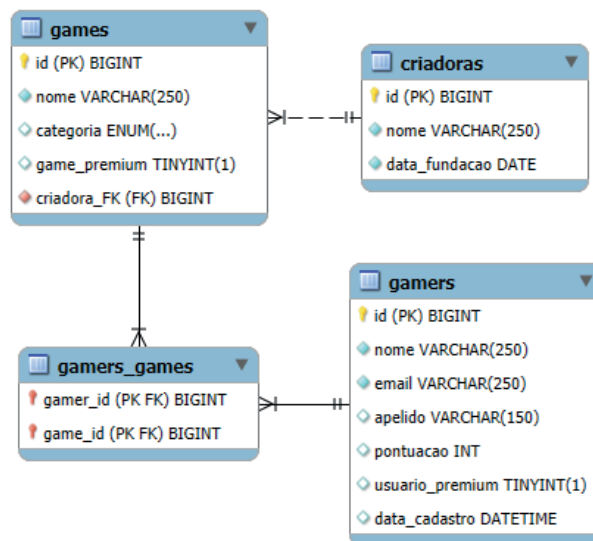
Veremos a seguir alguns exemplos.

## Exemplos e casos de uso

**Exemplo 1 – Plataforma de *games***

Retomando o exemplo já dado neste capítulo, ao construir uma tabela para cadastro de *gamers*, vamos agora ampliar a aplicação para também possibilitar o cadastro de jogos e atrelá-los aos respectivos usuários, fazendo uma analogia ao que seria uma plataforma *on-line* de *games*:

1. Adotaremos a abordagem da **Figura 3.26**, representada em UML, reaproveitando a mesma tabela de *gamers* e complementando-a com outras duas tabelas para reproduzir o relacionamento “muitos para muitos”, ou seja, muitos jogos para muitos usuários da plataforma. Adicionaremos também uma outra tabela para identificação das empresas criadoras dos jogos:



**Figura 3.26** – Diagrama UML representando o relacionamento entre entidades de uma plataforma de *games*.

Crédito: do elaborador, 2025.

2. Em seguida, criaremos o banco de dados para atender à aplicação:

```
create database plataforma_games;
use plataforma_games;
```

3. Agora executaremos os seguintes comandos para criação das tabelas, usando a chave primária composta na entidade **gamers\_games**:

```
create table gamers (
  id bigint primary key auto_increment,
  nome varchar(250) not null,
  email varchar(250) not null,
  apelido varchar(150),
  pontuacao int default 0,
  usuario_premium boolean default false,
  data_cadastro datetime default now()
);
create table criadoras (
  id bigint primary key auto_increment,
  nome varchar(250) not null,
  data_fundacao date not null
);
create table games (
  id bigint primary key auto_increment,
  nome varchar(250) not null,
  categoria enum ('ESPORTES','AVENTURA','RPG','TIRO'),
  game_premium boolean default false,
  criadora_FK bigint not null,
  foreign key (criadora_FK) references criadoras(id)
);
create table gamers_games (
  gamer_id bigint not null,
  game_id bigint not null,
  primary key(gamer_id,game_id),
  foreign key(gamer_id) references gamers(id),
  foreign key(game_id) references games(id)
);
```

Algumas observações mais relevantes sobre esses comandos, mais precisamente envolvendo a tabela **gamers\_games**, sendo a única a

apresentar uma diferenciação em relação aos exemplos que já vimos até o momento:

- Criamos essa tabela para possibilitar a cardinalidade “muitos *games* para muitos *gamers*”;
- Combinar a informação de identificação do *gamer* com a identificação de seu respectivo *game* resulta na criação de uma chave única;
- Note nesse exemplo que é possível que uma chave estrangeira possa ser ao mesmo tempo uma chave primária na mesma tabela.

#### IMPORTANTE

**Poderíamos optar por dispensar uma chave primária composta para utilizar uma chave não composta com o uso do `auto_increment`:**

```
create table gamers_games (
  id bigint primary key auto_increment,
  gamer_id bigint not null,
  game_id bigint not null,
  foreign key(gamer_id) references gamers(id),
  foreign key(game_id) references games(id)
);
```

**Contudo, isso poderia acarretar um cenário de aceitação da repetição de um mesmo *gamer* para um mesmo *game*, o que não faria sentido para essa aplicação.**

drop table – Removendo tabelas

Agora que você aprendeu um pouco mais sobre como criar tabelas, pode ter se questionado como removê-las. Para isso, utilizamos um comando muito simples, chamado **drop table**, obedecendo a estrutura seguinte:

```
--ESTRUTURA DO COMANDO:
drop table NOME_DA_TABELA;

--EXEMPLO:
```

```
drop table produtos;  
drop table pedidos;
```

### FIQUE ALERTA

#### drop table e especificação

Ao utilizar o comando **drop table**, é importante compreender que ele irá excluir permanentemente a tabela especificada no banco de dados que está em uso no momento. Por exemplo, se você executar o comando **drop table produtos** enquanto o banco ativo não contém uma tabela chamada **PRODUTOS**, ocorrerá um erro. Para evitar isso, é necessário certificar-se de que o banco correto está ativo, utilizando previamente o comando **use nome\_do\_banco**.

Além disso, há outra maneira de especificar diretamente o banco de dados e a tabela no comando **drop table**, independentemente do banco ativo. Basta utilizar a seguinte sintaxe:

```
--ESTRUTURA DO COMANDO:  
drop table NOME_DO_BANCO.NOME_DA_TABELA;  
  
--EXEMPLO:  
drop table mercado.produtos;  
drop table mercado.pedidos;
```

check – Criando tabelas com validações de dados

Em aplicações que trabalham com o salvamento de alguns tipos de dados, saber armazená-los não é a única preocupação, mas também validá-los. É por meio dessas validações que as aplicações garantem que o dado atenda de fato ao objetivo desejado.

Por exemplo, caso uma aplicação de cadastro tenha que coletar a data de nascimento de um usuário para poder oferecer a ele serviços ou acessos compatíveis com sua idade, mas não tenha um mecanismo de validação de dados, o que você imagina que poderia ocorrer?



Com toda a certeza você deve ter chegado à conclusão de que uma falta de validação poderia gerar inúmeros problemas que afetam diretamente o funcionamento dessa aplicação. Um usuário, por exemplo, poderia digitar um formato inexistente de data, adicionar letras inesperadas ou até mesmo utilizar datas futuras.

Mas os problemas vão além de erros de digitação, pois a ausência de validação abre portas para fraudes e ações de má-fé. Um usuário mal-intencionado poderia, por exemplo, inserir uma data de nascimento falsa para obter acesso a conteúdos restritos a adultos, manipular sorteios e promoções, ou até mesmo criar contas falsas para fins ilícitos. Em sistemas financeiros, por exemplo, informações inconsistentes podem levar a transações inválidas e prejuízos significativos. Portanto, a validação de dados é uma barreira fundamental para proteger a aplicação contra erros e fraudes e garantir a qualidade da informação armazenada.

Pensando em um contexto amplo de uma aplicação, é claro que existem inúmeras formas de executar a validação do que seria, nesse exemplo, uma data de nascimento.

Tratando-se de uma aplicação tradicional que possua minimamente *frontend*, *backend* e banco de dados, podemos adicionar validações em qualquer uma dessas partes do sistema ou até mesmo em todas elas de forma complementar, aumentando a integridade do projeto.

Contudo, como o objetivo deste livro é orientar a atuação do(a) aluno(a) em bancos de dados, daremos enfoque às validações que possam ocorrer nesse contexto.

O MySQL disponibiliza diversas ferramentas de validação de dados, abrangendo desde *constraints* que já exploramos, como **not null**, **unique** e **foreign key**, até novas possibilidades, como a *constraint check*, que será detalhada a seguir. Além disso, em outros capítulos, abordaremos recursos avançados, como os *triggers*, que ampliam ainda mais o controle e a consistência dos dados no banco.

## IMPORTANTE

### Validações complexas

**Vale ressaltar ao aluno e desenvolvedor que, embora validações relativamente complexas possam ser implementadas em bancos de dados, essa**

não é uma tendência utilizada atualmente no mercado, já que tais lógicas não ficam claramente visíveis, podem variar de acordo com o modelo do banco de dados e não são tão flexíveis para receber alterações.

Desse modo, para validações mais complexas ou regras de negócio da aplicação, o recomendado é realizá-las em outras camadas, por exemplo no *frontend*, no *backend* ou até mesmo em alguns serviços oferecidos por plataformas na nuvem.

O comando **check** é relativamente simples, obedecendo à estrutura apresentada a seguir:

```
--ESTRUTURA DO COMANDO:
create table NOME_DA_TABELA (
  NOME_DA_COLUNA TIPO_DE_DADO MODIFICADORES/CONSTRAINTS,
  NOME_DA_COLUNA TIPO_DE_DADO MODIFICADORES/CONSTRAINTS,
  .
  .
  .,
  check (CONDIÇÃO1),
  check (CONDIÇÃO2),
  .
  .
  .,
);

--EXEMPLO:
create table produtos (
  id bigint primary key auto_increment,
  nome varchar(100) not null,
  preco decimal(10, 2) not null,
  estoque int not null,
  CHECK (preco >= 0),
  CHECK (estoque >= 0)
);
```

Conforme você deve ter percebido, adicionamos também um exemplo de tabela de produtos, implementando validações para que tanto informações de preço quanto de estoque somente possam ser salvas se forem maiores ou iguais a zero.

Resumidamente, o comando **check** testa se uma condição é verdadeira e, caso seja, a inclusão dos dados será permitida; caso contrário, não será permitida a inclusão, acusando um erro na *constraint*, como é o caso da **Figura 3.27**, utilizando o mesmo exemplo da tabela de **produtos**.

Error Code: 3819. Check constraint 'produtos\_chk\_1' is violated.

**Figura 3.27** – Mensagem de erro na violação da *constraint* **check** – MySQL Workbench.

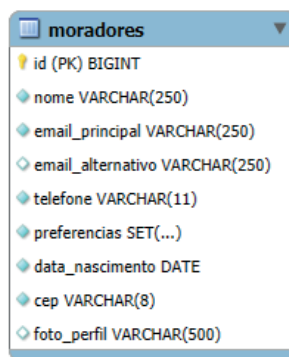
Crédito: do elaborador, 2025.

## Exemplos e casos de uso

### Exemplo 1 – Central de alertas

Vamos retomar o exemplo já trabalhado neste livro sobre um sistema desenvolvido para alertar moradores em áreas de risco pela defesa civil. Contudo, implementaremos na tabela de cadastramento desses moradores algumas validações para checar se o tamanho mínimo de caracteres foi preenchido e fazer também uma pequena verificação no formato do *e-mail* do usuário:

1. Trazemos novamente a modelagem UML desse exemplo – já realizada anteriormente:



**Figura 3.28** – Diagrama UML – tabela de moradores.

Crédito: do elaborador, 2025.

2. Usaremos o mesmo *schema* criado, portanto, caso você já o possua em seu servidor, apenas utilize o comando de ativação, caso contrário, proceda com a criação:

```
create database central_alertas;
use central_alertas;
```

3. Caso o(a) aluno(a) já possua a tabela **moradores** em seu banco, faça a deleção dela; se não a possui, basta seguir para o próximo passo:

```
drop table moradores;
```

4. Finalmente, criaremos a tabela seguindo a estrutura do comando **create table**, obedecendo ao que foi definido anteriormente na modelagem UML, incluindo nesse momento algumas validações a serem realizadas com a *constraint check*:

```
create table moradores (
  id bigint auto_increment primary key,
  nome varchar(250) not null,
  email_principal varchar(250) unique not null,
  email_alternativo varchar(250) unique,
  telefone varchar(11) not null,
  preferencias set('EMAIL', 'SMS', 'WHATSAPP') not
    null,
  data_nascimento date not null,
  cep char(8) not null,
  foto_perfil varchar(500),
  check (length(nome) >= 3),
  check (length(telefone) in (10,11)),
  check (length(cep) = 8),
  check (email_principal like '%@%.%'),
  check ((email_alternativo is null) or (email_alternativo
like
  '@%.%'))
);
```

Alguns comentários sobre as validações realizadas:

- O comando **length()** tem a função de informar a quantidade de caracteres existentes em atributos/colunas que usam o tipo de texto;
- O operador **in** tem a função de comparar se um determinado valor está presente em uma lista de valores possíveis;
- O operador **like** tem a função de comparar um valor com um padrão, por meio da inclusão de chaves “curingas”;
- Utilizamos o **check** no atributo **nome** para garantir que qualquer nome cadastrado tenha no mínimo três caracteres;
- Utilizamos o **check** no atributo **telefone** associado com o operador **in** para verificar se a quantidade de dígitos é compatível com telefone fixo + DDD (10 dígitos) ou telefone móvel + DDD (11 dígitos);
- Para o atributo **cep**, verificamos se a quantidade de dígitos é a válida, ou seja, oito dígitos;
- Para os atributos envolvendo *e-mail*, utilizamos o operador **like**, que associado ao padrão de curingas **%@%.%** nos permite validar se o dado possui o símbolo de “@” e uma pontuação;
- Note que para o atributo **email\_alternativo**, por se tratar de informação opcional, incluímos também como condição a aceitação de valores nulos.

index e fulltext – Otimizando tabelas

Para entendermos o que são índices e sua importância, imagine que você está manuseando um livro com muitas páginas, mas que não possui nenhum índice ou sumário. Para encontrar uma informação específica, você precisaria percorrer página por página, o que seria extremamente demorado.

Os índices em bancos de dados funcionam de forma similar: eles são estruturas de dados que otimizam a velocidade das consultas, permitindo

que o banco de dados encontre registros de forma muito mais rápida do que se tivesse que verificar todos os registros da tabela.

Para criarmos índices, basta utilizarmos os comandos **index** e **fulltext** nas colunas que servirão de referência para essas buscas otimizadas.

O comando **index** tem uso geral, já o **fulltext** é indicado em variáveis do tipo texto (**char**, **varchar**, **text**, **longtext** etc.), especialmente em textos longos, pois permite buscas otimizadas com palavras-chaves (veremos os métodos de consulta em outro capítulo).

A seguir, podemos visualizar a estrutura de comando e um exemplo de uso:

```
--ESTRUTURA DO COMANDO:
create table NOME_DA_TABELA (
  NOME_DA_COLUNA_1 TIPO_DE_DADO MODIFICADORES/CONSTRAINTS,
  NOME_DA_COLUNA_2 TIPO_DE_DADO MODIFICADORES/CONSTRAINTS,
  NOME_DA_COLUNA_3 TIPO_DE_DADO MODIFICADORES/CONSTRAINTS,
  NOME_DA_COLUNA_4 TIPO_DE_DADO MODIFICADORES/CONSTRAINTS,
  NOME_DA_COLUNA_5 TIPO_DE_DADO MODIFICADORES/CONSTRAINTS,
  NOME_DA_COLUNA_6 TIPO_DE_DADO MODIFICADORES/CONSTRAINTS,
  NOME_DA_COLUNA_7 TIPO_DE_DADO MODIFICADORES/CONSTRAINTS,
  .
  .
  .,
  index (NOME_DA_COLUNA_1), --ÍNDICE SIMPLES
  index (NOME_DA_COLUNA_2, NOME_DA_COLUNA_4), --ÍNDICE
COMPOSTO
  fulltext (NOME_DA_COLUNA_3) --ÍNDICE TEXTO COMPLETO
);

--EXEMPLO:
create table livros (
  id bigint primary key auto_increment,
```

```

titulo varchar(255) not null,
codigo int not null unique,
categoria enum ('FICÇÃO','TERROR','CIÊNCIA','OUTROS'),
sinopse text not null,
paginas int not null,
ano_publicacao char(4) not null,
data_cadastro datetime not null default now(),
check(codigo > 0),
check(paginas > 0),
check(length(ano_publicacao) = 4),
index(categoria),
fulltext(sinopse),
index(titulo,categoria)
);

```

Como você pode perceber, implementamos um índice simples que envolve unicamente o campo **categoria** e outro composto que envolve os campos **titulo** e **categoria** juntos (para uso nos casos em que se deseja encontrar livros que atendam a ambos os valores). Além disso, aplicamos o índice **fulltext** para a **sinopse**, por naturalmente se tratar de campo com textos maiores.

## IMPORTANTE

### Uso controlado dos índices

Neste momento, você pode estar tentado a pensar: “se o uso de **index** e **fulltext** otimiza buscas por coluna, vou sempre utilizá-los para cada coluna existente, melhorando assim a performance de minha tabela inteira!”. E, na realidade, pensando dessa forma, o resultado de melhoria esperado não será alcançado. Para entender melhor, imagine um livro com 500 páginas de conteúdo que tivesse mais outras 400 páginas apenas de sumário. Você conseguiria encontrar um assunto em específico rapidamente nesse enorme sumário? Será que o índice estaria atendendo seu objetivo principal? É o que ocorre quando utilizamos

**índices descontroladamente, devendo ser usados apenas para aqueles campos de referências para buscas. Por exemplo, se você tiver uma tabela de usuários com dez colunas, mas o método de busca mais usual de um usuário é pelo e-mail ou CPF, então coloque apenas esses dois atributos como índices da tabela.**

Quando o assunto é índices de tabela, dúvidas podem ser geradas sobre a possibilidade de aplicação desses recursos para otimizar buscas em atributos que são chaves estrangeiras. De fato, essa não é somente uma possibilidade, mas uma realidade, uma vez que, quando criada uma *foreign key*, o MySQL cria também um índice associado a ela.

#### IMPORTANTE

##### Criação automática de índices

**Assim como ocorre com chaves estrangeiras, quando se utiliza a *constraint unique*, o MySQL também faz a criação de índices automaticamente, fazendo com que se torne desnecessária a criação de um *index* para tais campos.**

Para que o desenvolvedor possa enxergar quais os índices presentes em cada tabela, o MySQL nos fornece duas possibilidades: utilizar o comando **show create table**, que já abordamos anteriormente, ou o comando seguinte:

```
--ESTRUTURA DO COMANDO:  
show index from NOME_DA_TABELA;  
  
--EXEMPLOS:  
show index from produtos;  
show index from agenda;
```

Mostraremos na **Figura 3.29** o que seria o resultado do comando **show index from** para a tabela **livros** exibida no exemplo mencionado anteriormente:



21 • `show index from livros;`

| Table  | Non_unique | Key_name  | Seq_in_index | Column_name | Collation | Cardinality | Sub_part | Packed | Null | Index_type | Comment | Index_comment | Visible | Expression |
|--------|------------|-----------|--------------|-------------|-----------|-------------|----------|--------|------|------------|---------|---------------|---------|------------|
| livros | 0          | PRIMARY   | 1            | id          | A         | 0           | NULL     | NULL   |      | BTREE      |         |               | YES     | NULL       |
| livros | 0          |           | 1            | codigo      | A         | 0           | NULL     | NULL   |      | BTREE      |         |               | YES     | NULL       |
| livros | 1          | categoria | 1            | categoria   | A         | 0           | NULL     | NULL   | YES  | BTREE      |         |               | YES     | NULL       |
| livros | 1          | titulo    | 1            | titulo      | A         | 0           | NULL     | NULL   |      | BTREE      |         |               | YES     | NULL       |
| livros | 1          | titulo    | 2            | categoria   | A         | 0           | NULL     | NULL   | YES  | BTREE      |         |               | YES     | NULL       |
| livros | 1          | sinopse   | 1            | sinopse     | NULL      | 0           | NULL     | NULL   |      | FULLTEXT   |         |               | YES     | NULL       |

**Figura 3.29** – Resultado do comando de exibição dos índices da tabela de livros – MySQL Workbench.

Crédito: do elaborador, 2025.

Com o comando **show index from**, é possível ter um nível de detalhamento maior dos índices e de seus comportamentos.

Perceba que, além dos índices gerados explicitamente em nosso código (envolvendo campos de título, categoria e sinopse), existem outros dois índices criados automaticamente pelo MySQL. Para a coluna **id**, foi criado um índice com nome **primary**, uma vez que essa é a chave primária da tabela, enquanto para a coluna **codigo**, gerou-se um outro índice devido ao fato de termos utilizado a *constraint unique* durante a criação da tabela de livros. Além disso, podemos identificar que, para o índice composto (título e categoria), foram criadas duas chaves com o mesmo nome de **titulo**, diferenciando-as pela sequência que ocupam no índice composto através da coluna **Seq\_in\_index**.

Apresentamos no **Quadro 3.3** algumas informações básicas de cada coluna apresentada pelo **show index from**.

**Quadro 3.3** – Descrição das colunas exibidas no show index

| Coluna       | Descrição                                                                                                                                                                                                               |
|--------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Table        | O nome da tabela na qual o índice foi criado.                                                                                                                                                                           |
| Non_unique   | Indica se o índice permite valores duplicados: <ul style="list-style-type: none"><li>• 0: índice único (valores devem ser exclusivos);</li><li>• 1: índice não exclusivo (valores duplicados são permitidos).</li></ul> |
| Key_name     | O nome do índice, para índices primários, será exibido como <b>primary</b> .                                                                                                                                            |
| Seq_in_index | A posição da coluna no índice (1 para a primeira; 2 para a segunda etc.). Isso é relevante para índices compostos.                                                                                                      |

(continua)

(continuação)

| Coluna               | Descrição                                                                                                                                                                                                 |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Column_name</b>   | O nome da coluna que faz parte do índice.                                                                                                                                                                 |
| <b>Collation</b>     | Como os valores da coluna são armazenados no índice: <ul style="list-style-type: none"> <li>• A: valores são armazenados em ordem crescente;</li> <li>• <b>null</b>: sem ordenação específica.</li> </ul> |
| <b>Cardinality</b>   | Uma estimativa do número de valores únicos na coluna indexada. É útil para avaliar a seletividade do índice.                                                                                              |
| <b>Sub_part</b>      | Se o índice é baseado em prefixo, indica o número de caracteres usados. Para índices completos, será <b>null</b> .                                                                                        |
| <b>Packed</b>        | Indica se o índice é compactado. Geralmente <b>null</b> para índices padrão.                                                                                                                              |
| <b>Null</b>          | Indica se a coluna pode conter valores <b>null</b> : <ul style="list-style-type: none"> <li>• YES: a coluna aceita valores <b>null</b>;</li> <li>• NO: a coluna não aceita <b>null</b>.</li> </ul>        |
| <b>Index_type</b>    | O tipo do índice, como <b>btree</b> , <b>fulltext</b> ou <b>hash</b> .                                                                                                                                    |
| <b>Comment</b>       | Comentários adicionais sobre o índice, como <b>fulltext</b> para índices de texto completo.                                                                                                               |
| <b>Index_comment</b> | Comentários definidos pelo usuário ao criar o índice.                                                                                                                                                     |

Crédito: do elaborador, 2025.

## Exemplos e casos de uso

**Exemplo 1 – Indústria 4.0 – *Industrial Internet of Things***

Em algum momento durante sua jornada como estudante, você já deve ter se deparado com o termo **Indústria 4.0**, marco de um importante processo de transformação industrial que estamos vivendo neste momento, também denominada Quarta Revolução Industrial.

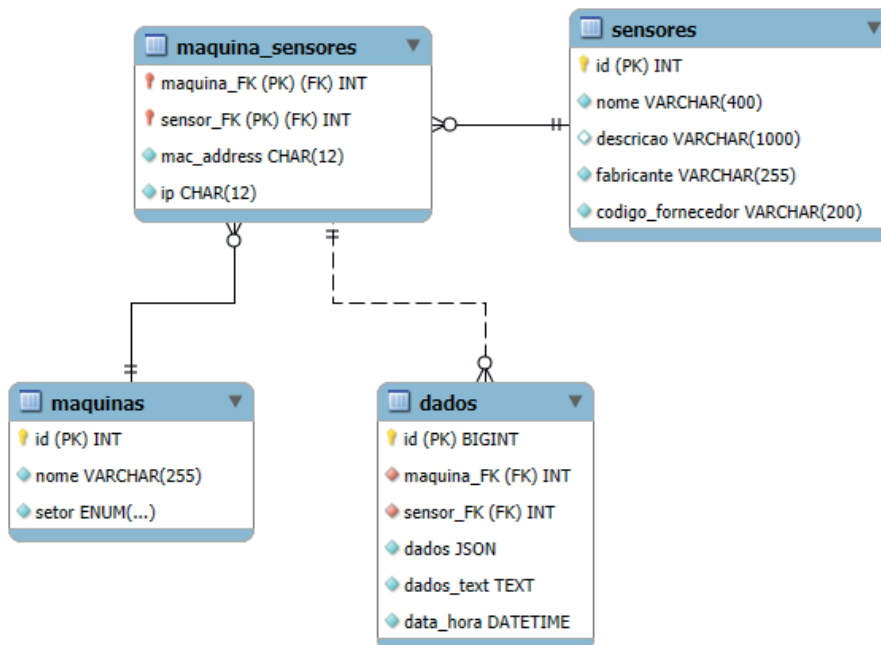
Nesse contexto, entre as inúmeras tecnologias que são trazidas com essa revolução, temos a ***Industrial Internet of Things* (IIoT ou Internet Industrial das Coisas ou Internet das Coisas Industrial)**, permitindo que maquinários, sensores e peças participem ativamente e de forma inteligente na contribuição para o processo produtivo em um ambiente industrial.

É por meio dos recursos da IIoT que se torna possível a implementação de conceitos mais avançados como inteligência artificial (IA), redes neurais e outros mecanismos de aprendizagem de máquina (*machine learning*), pois

todos necessitam fundamentalmente de uma grande quantidade de dados coletados para serem desenvolvidos de forma eficiente, dados que são gerados em sua maioria por sensores IIoT.

Todos esses dados necessitam ser armazenados em algum lugar, e é nesse contexto que temos um exemplo de aplicação que irá coletar inúmeras variáveis vindas de sensores industriais e salvá-las em um banco de dados:

1. Criaremos então uma modelagem UML para relacionar uma aplicação em que inúmeros sensores IIoT são instalados em determinados maquinários, enviando dados coletados diretamente para uma tabela do banco de dados:



**Figura 3.30** – Diagrama UML de uma aplicação envolvendo IIoT.

Crédito: do elaborador, 2025.

Algumas observações relevantes sobre esse contexto, que podem interessar ao(a) aluno(a):

- Criamos uma entidade de **maquina\_sensores** para possibilitar um relacionamento “muitos para muitos”, ou seja, muitas máquinas compartilham muitos sensores, utilizando uma chave primária composta para esse tipo de cardinalidade;

- Cada sensor instalado em uma máquina possui um endereço de IP (*Internet Protocol* ou Protocolo de Internet) e um MAC Address (*Media Access Control Address* ou Endereço de Controle de Acesso à Mídia), dois atributos muito comuns em sistemas que se conectam em redes;
- A entidade de **dados** será composta pelo envio de cada “pacote de dados” oriundo de uma “máquina + sensor”;
- Os dados são armazenados em dois atributos com tipagem diferente: **dados** (tipo **JSON**), utilizados para salvar a resposta enviada pelo sensor, e **dados\_text** (tipo **TEXT**), contendo o mesmo que foi salvo em **JSON**, porém convertido para tipo texto. A réplica dos dados **JSON** em **TEXT** foi realizada para possibilitar o uso de índices de busca avançados com o uso que já vimos, ou seja, o **fulltext**.

2. Em seguida, iremos transformar esta modelagem em código SQL:

```
CREATE TABLE maquinas (
  id int primary key auto_increment,
  nome varchar(255) not null,
  setor enum('CONVENCIONAL','CNC','CALDERARIA') not null,
  index(setor)
);

CREATE TABLE sensores (
  id int primary key auto_increment,
  nome varchar(400) not null,
  descricao varchar(1000),
  fabricante varchar(255) not null,
  codigo_fornecedor varchar(200) not null unique
);

CREATE TABLE maquina_sensores (
  maquina_FK int not null,
  sensor_FK int not null,
  mac_address char(12) not null unique,
  ip char(12) not null unique,
```

```

    check ((length(mac_address) = 12)),
    check ((length(ip) = 12)),
    primary key(maquina_FK,sensor_FK),
    foreign key(maquina_FK) references maquinas(id),
    foreign key(sensor_FK) references sensores(id)
);

CREATE TABLE dados (
    id bigint primary key auto_increment,
    maquina_FK int not null,
    sensor_FK int not null,
    dados json not null,
    dados_text text not null,
    data_hora datetime not null default now(),
    foreign key(maquina_FK, sensor_FK) references
maquina_sensores
    (maquina_FK, sensor_FK),
    fulltext(dados_text)
);

```

Alguns comentários relevantes sobre os comandos apresentados, mais precisamente sobre os índices utilizados:

- Na entidade **maquinas**, adicionamos o índice para busca otimizada por setor;
- Em **maquina\_sensores**, adicionamos índices únicos para os campos **ip** e **mac\_address**, uma vez que conceitualmente tais valores devem ser únicos dentro de uma mesma rede;
- Por fim, para viabilizar buscas avançadas em textos longos de maneira eficiente, foi utilizado o índice **fulltext** no campo **dados\_text**. Esse índice é essencial para otimizar a performance das consultas relacionadas às informações coletadas pelos sensores. Considerando o contexto da aplicação, em que a tabela de dados provavelmente armazenará uma quantidade massiva de registros, o uso do **fulltext** é uma solução estratégica para garantir a agilidade e a eficácia na recuperação de informações específicas.

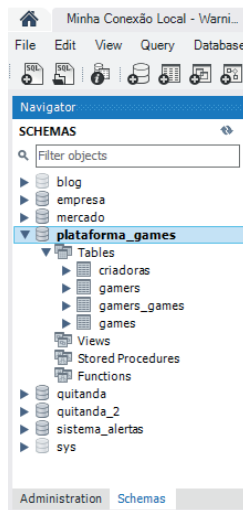
**IMPORTANTE****JSON e TEXT**

Nesse exemplo de aplicação, não adicionamos índices diretamente no atributo **dados** que faz uso do tipo **JSON** devido ao fato de o **MySQL** não conseguir indexar o conteúdo interno de colunas especiais, como o **JSON**. Contudo, optamos por deixar os dois tipos (**JSON** e **TEXT**), uma vez que colunas com essa anotação apresentam funcionalidades especiais de busca que veremos adiante, restritas apenas para tipos **JSON** (como **JSON\_EXTRACT**, **JSON\_UNQUOTE** etc.).

`show databases` e `show tables` – Visualizando bancos e tabelas criadas

Apresentamos agora dois comandos simples, mas bastante úteis: **show databases** e **show tables**, utilizados para mostrar os *schemas* ou as tabelas existentes no banco de dados em uso.

Se você estiver utilizando algum SGBD com interface visual, como é o caso do **MySQL Workbench**, talvez esses comandos possam não ser tão utilizados, uma vez que é possível visualizar os bancos e as tabelas criadas por meio da seção **SCHEMAS**, já apresentada anteriormente.



**Figura 3.31** – Árvore de *schemas* do **MySQL Workbench**, com a possibilidade de visualização de bancos e tabelas existentes.

Crédito: do elaborador, 2025.

Contudo, é muito comum em ambientes com cenários reais de uso (chamados de ambientes produtivos) haver somente a possibilidade de conexão com terminais de comando, em que não há interfaces visuais disponíveis para exibição dessas informações.

Desse modo, é valioso conhecer a existência desses comandos, representados a seguir:

```
--ESTRUTURA DO COMANDO:
show databases;
show tables;
```

Para melhor entendimento do(a) aluno(a), demonstramos na **Figura 3.32** o uso de ambos os comandos pela conexão com o servidor MySQL via terminal de comando.

### FIQUE ALERTA

#### Banco e tabelas

Note que, na **Figura 3.32**, utilizamos o comando **use plataforma\_games** antes de executar o comando **show tables**, uma vez que é necessário que algum banco esteja em uso para exibição das suas tabelas.

```
mysql> show databases;
+-----+
| Database |
+-----+
| blog     |
| empresa |
| information_schema |
| mercado |
| mysql    |
| performance_schema |
| plataforma_games |
| quitanda |
| quitanda_2 |
| sistema_alertas |
| sys      |
+-----+
11 rows in set (0.00 sec)

mysql> use plataforma_games;
Database changed
mysql> show tables;
+-----+
| Tables_in_plataforma_games |
+-----+
| criadoras |
| gamers    |
| gamers_games |
| games     |
+-----+
4 rows in set (0.02 sec)
```

**Figura 3.32** – Comandos em SQL para exibição de bancos e tabelas criadas pela conexão com o MySQL via terminal de comando.

Crédito: do elaborador, 2025.

describe e explain – Detalhando tabelas criadas

Outro comando bastante útil para obtenção de detalhes de tabelas é o **describe**, especialmente em conexões via terminal de comando, possibilitando a visualização de informações relevantes como nomes das colunas, tipagem de dado e suas *constraints* básicas.

Vale ressaltar que esse comando, no MySQL, pode ser utilizado de outras duas formas sinônimas, o **explain** e o **desc**, tendo todos a mesma função.

Apresentaremos a seguir a estrutura de todos:

```
--ESTRUTURA DO COMANDO:
describe NOME_DA_TABELA;
desc NOME_DA_TABELA;
explain NOME_DA_TABELA;

--EXEMPLOS:
describe produtos;
desc pedidos;
explain estoque;
```

Mostraremos nas **Figuras 3.33 e 3.34** o resultado reproduzido, respectivamente, pelo uso de qualquer um dos comandos via MySQL Workbench e por terminal de comando.

153 • **explain** gamers;

154

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [IA](#)

| Field           | Type         | Null | Key | Default           | Extra             |
|-----------------|--------------|------|-----|-------------------|-------------------|
| id              | bigint       | NO   | PRI | <b>HULL</b>       | auto_increment    |
| nome            | varchar(250) | NO   |     | <b>HULL</b>       |                   |
| email           | varchar(250) | NO   |     | <b>HULL</b>       |                   |
| apelido         | varchar(150) | YES  |     | <b>HULL</b>       |                   |
| pontuacao       | int          | YES  |     | 0                 |                   |
| usuario_premium | tinyint(1)   | YES  |     | 0                 |                   |
| data_cadastro   | datetime     | YES  |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |

**Figura 3.33** – Detalhamento de uma tabela via MySQL Workbench.

Crédito: do elaborador, 2025.



```
mysql> desc gamers;
```

| Field           | Type         | Null | Key | Default           | Extra             |
|-----------------|--------------|------|-----|-------------------|-------------------|
| id              | bigint       | NO   | PRI | NULL              | auto_increment    |
| nome            | varchar(250) | NO   |     | NULL              |                   |
| email           | varchar(250) | NO   |     | NULL              |                   |
| apelido         | varchar(150) | YES  |     | NULL              |                   |
| pontuacao       | int          | YES  |     | 0                 |                   |
| usuario_premium | tinyint(1)   | YES  |     | 0                 |                   |
| data_cadastro   | datetime     | YES  |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |

7 rows in set (0.01 sec)

**Figura 3.34** – Detalhamento de uma tabela via terminal de comando.

Crédito: do elaborador, 2025.

`show create table` – Reproduzindo o comando de criação de tabelas

Sem dúvida, ao longo de sua trajetória como desenvolvedor de bancos de dados, você, aluno(a), encontrará situações em que precisará compreender os detalhes de como o comando **create table** foi utilizado para criar uma tabela já existente no banco de dados. Seja para reproduzi-la em outro ambiente ou servidor, realizar ajustes em sua estrutura, ou até mesmo entender melhor um banco de dados projetado por outro desenvolvedor, conhecer a origem exata de sua definição será essencial.

É nesse cenário que o comando **show create table** se destaca, fornecendo exatamente o que você precisa: ele retorna o comando SQL completo utilizado para criar a tabela em questão, incluindo os detalhes de sua estrutura, como tipos de dados, *constraints* e chaves.

Veremos a seguir a estrutura do comando:

```
--ESTRUTURA DO COMANDO:
show create table NOME_DA_TABELA;

--EXEMPLOS:
show create table produtos;
show create table pedidos;
```

Na **Figura 3.35**, mostraremos o resultado recebido via terminal de comando (no MySQL Workbench, o resultado é muito parecido, portanto, não o exibiremos aqui).

```
mysql> show create table gamers;
+-----+-----+
| Table | Create Table
+-----+-----+
| gamers | CREATE TABLE `gamers` (
  `id` bigint NOT NULL AUTO_INCREMENT,
  `nome` varchar(250) NOT NULL,
  `email` varchar(250) NOT NULL,
  `apelido` varchar(150) DEFAULT NULL,
  `pontuacao` int DEFAULT '0',
  `usuario_premium` tinyint(1) DEFAULT '0',
  `data_cadastro` datetime DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci |
+-----+-----+
1 row in set (0.00 sec)
```

**Figura 3.35** – Comandos em SQL para exibição dos detalhes de criação de uma tabela.

Crédito: do elaborador, 2025.

Dessa maneira, se quisermos copiar o resultado do comando **show create table** e utilizá-lo em outro servidor, teremos uma cópia exata da mesma estrutura de tabela.

### IMPORTANTE

#### **Backup**

**Utilizar o resultado do comando `show create table` irá reproduzir a mesma estrutura da tabela de referência, porém não trará os dados guardados nela, ou seja, não é um comando de *backup*. Veremos em outro capítulo algumas soluções para atender à necessidade de *backup*, isto é, a reprodução das tabelas juntamente com os dados, por meio da utilização de comandos chamados *dump*.**

`alter table` – Alterando tabelas já criadas

Ao projetar e implementar um banco de dados, é comum que as necessidades de negócio mudem ou que novos requisitos sejam identificados ao longo do tempo, situações muito comuns durante o ciclo de vida de aplicações e sistemas.

Nessas situações, é necessário realizar alterações na estrutura das tabelas existentes, como adicionar novas colunas, modificar tipos de dados, criar índices, ou até mesmo redefinir restrições de integridade.

O comando **alter table** foi projetado exatamente para atender a tais demandas, permitindo que desenvolvedores e administradores de banco de dados façam ajustes estruturais em tabelas já criadas, sem a necessidade de excluí-las e recriá-las, preservando assim os dados existentes.

### FIQUE ALERTA

#### Alterações em tabelas

Em bancos de dados relacionais, qualquer alteração em tabelas já existentes, especialmente em ambientes produtivos (cenários reais), deve ser realizada com extrema cautela. Essas mudanças podem ser comparadas a ajustes estruturais em um edifício já construído: exigem planejamento cuidadoso e uma análise detalhada dos impactos que podem causar.

#### Adicionando novas colunas

Para adicionarmos novas colunas em tabelas já existentes, utilizamos o comando **alter table** com o **add column**, conforme mostrado a seguir:

```
--ESTRUTURA DO COMANDO:
alter table NOME_DA_TABELA add column NOME_DA_COLUNA
TIPO_DE_DADO
    CONSTRAINTS;
--ALTERNATIVA (COM AFTER):
alter table NOME_DA_TABELA add column NOME_DA_COLUNA
TIPO_DE_DADO
    CONSTRAINTS AFTER NOME_DE_OUTRA_COLUNA;

--EXEMPLOS:
alter table produtos add column promocao boolean not null
default
false;
```

```
alter table vendas add column desconto decimal(10,2) default
0.0 after
preco;
alter table vagas add column modalidade
enum('REMOTA','PRESENCIAL','HÍBRIDA') not null;
```

Como é possível notar, o comando **alter table** para adição de colunas é composto essencialmente de uma estrutura de indicação da tabela a ser modificada e de definição dos atributos novos, seguindo o mesmo conceito utilizado na criação de uma tabela nova, isto é, seguido por **nome\_da\_coluna + tipo\_de\_dado + modificadores/constraints**.

Além disso, é possível também definir a ordem em que a coluna nova deve ser adicionada. Utilizando a cláusula **after**, podemos informar após qual coluna ocorrerá a adição (se não for usado o **after** no comando, a nova coluna será adicionada após todas as outras).

### Renomeando colunas

Você já criou uma tabela e, possivelmente, já adicionou alguns dados, porém chegou à conclusão de que o nome que escolheu para alguma coluna não foi dos melhores. O que fazer então? Recriar a tabela inteira?

É nesse caso que o comando **alter table** juntamente com o **rename column** se tornam extremamente úteis, dando flexibilidade para que o desenvolvedor possa renomear as colunas conforme a necessidade.

Vejamos sua estrutura:

```
--ESTRUTURA DO COMANDO:
alter table NOME_DA_TABELA rename column NOME_DA_COLUNA to
NOVO_NOME_DA_COLUNA;

--EXEMPLOS:
alter table vendas rename column qtd to quantidade;
alter table livros rename column id to livro_id;
```

## Alterando atributos de colunas

Não é novidade que trabalhar com tecnologia exige um dinamismo maior que em outras áreas, fazendo com que, frequentemente, alguns sistemas tenham que se adaptar a tais mudanças.

Suponha que você criou uma tabela para armazenar números de telefone, abrangendo tanto telefones fixos quanto móveis. Nesse caso, você precisaria ter um campo com nove dígitos (sem considerar o código de área). Entretanto, suponha que daqui a alguns anos, com o crescimento exponencial da quantidade de números de telefones móveis, seja necessário adicionar mais um dígito. Nesse caso, necessitaremos de uma alteração na definição da coluna responsável por armazenar tais números, utilizando o comando **alter table** juntamente com **modify column**.

Com esse comando, é possível alterar tanto o tipo do dado como também suas *constraints*.

Vamos observar sua estrutura a seguir:

```
--ESTRUTURA DO COMANDO:
alter table NOME_DA_TABELA modify column NOME_DA_COLUNA
TIPO_DE_DADO
CONSTRAINTS;

--EXEMPLOS:
alter table automoveis modify column valor decimal(10,2) not
null;
alter table cadastro modify column complemento_endereco
varchar(300);
```

## Removendo colunas

Vimos até agora inúmeros comandos importantes envolvendo adição ou edição de colunas; contudo, em um cenário em que seja necessária a remoção de alguma coluna existente, precisaremos de outra solução ainda não apresentada: o **alter table** com o **drop column**.

Sua estrutura é bastante simples, sendo necessário apenas informar a tabela a ser alterada e a coluna a ser removida, conforme mostrado a seguir:

```
-- ESTRUTURA DO COMANDO:  
alter table NOME_DA_TABELA drop column NOME_DA_COLUNA;  
  
-- EXEMPLOS:  
alter table cadastro drop column fax;  
alter table empresas drop column informacoes_adicionais;
```

#### Adicionando *constraints* de validação

Considerando os mesmos cenários abordados anteriormente, em que se faz necessária a edição de uma tabela já existente, seja para execução de uma correção ou para implementação de uma nova funcionalidade, podem surgir condições em que seja necessária a modificação de uma tabela para adição de novas *constraints*.

Usamos então o comando **alter table add constraint**:

```
-- ESTRUTURA DO COMANDO:  
alter table NOME_DA_TABELA add constraint check (CONDIÇÃO);  
  
-- EXEMPLOS:  
alter table pagamento add constraint check (parcelas >= 1);  
alter table calendario add constraint check (hora >= 0 and  
hora <= 23);
```

#### Adicionando índices

Vimos anteriormente a importância do uso dos índices para otimização da performance de consultas em tabelas, em especial nas que possuem uma quantidade expressiva de dados.

Vamos entender agora como adicionar tais índices em uma tabela já formada:

```
-- ESTRUTURA DO COMANDO:
alter table NOME_DA_TABELA add TIPO_ÍNDICE (NOME_DA_COLUNA);

-- EXEMPLOS:
alter table entregas add index (cep); -- índice simples
alter table graficos add index (eixo_x , eixo_y); -- índice
composto
alter table postagens add fulltext (comentario); -- índice
fulltext
alter table usuarios add unique (telefone); -- índice único
```

## SAIBA MAIS

### Criar índices em tabelas existentes

Você já deve ter percebido a grande quantidade de comandos existentes no MySQL. Devido a isso, é muito comum existirem duas ou mais formas diferentes para se chegar ao mesmo objetivo. Outro modo interessante para se criar índices em tabelas já existentes é utilizando o comando **create index**.

Vejamos um pouco mais de sua estrutura:

```
-- ESTRUTURA DO COMANDO:
create [FULLTEXT / UNIQUE] index NOME_DO_ÍNDICE on
NOME_DA_TABELA
(NOME_DA_COLUNA);

-- EXEMPLOS:
create index cep_index on entregas (cep); -- índice simples
create index eixos_idx on graficos (eixo_x , eixo_y); --
índice composto
create fulltext index comentarios_indice on postagens
(comentario); -- índice fulltext
create unique index tel_idx on usuarios (telefone); -- índice
único
```

No exemplo deste box, podemos identificar que o comando **create index** apresenta as mesmas possibilidades de criação dos tipos que já vimos, ou seja, índices simples, compostos, fulltext, ou únicos, ficando a cargo do(a) aluno(a)/desenvolvedor(a) qual escolher.

### Removendo índices

Caso o uso de algum índice não faça mais sentido para sua aplicação, é sempre uma boa prática eliminá-lo, uma vez que já abordamos no tópico específico desse assunto que o uso de índices em demasia pode provocar efeitos ruins quanto à performance de operações de busca em tabelas.

Dessa maneira, para remover índices, basta utilizar o comando **alter table** juntamente com o **drop index**:

```
-- ESTRUTURA DO COMANDO:  
alter table NOME_DA_TABELA drop index NOME_DO_ÍNDICE;  
  
-- EXEMPLOS:  
alter table usuarios drop index telefone_idx;  
alter table hospital drop index prontuario;
```

### IMPORTANTE

#### Nome do índice a ser removido

Note que é necessário conhecer o nome de um índice para removê-lo, mesmo que não necessariamente seja o mesmo nome da coluna em que ele é atribuído. Um índice usado em uma coluna chamada **e\_mail** pode assumir nomes gerados automaticamente pelo MySQL, como **e\_mail\_idx**. Nesse caso, para maior assertividade na remoção do índice desejado, você pode utilizar o comando **show create table**, já que ele revela os nomes reais assumidos para cada índice ou *constraint* existente.

### Adicionando chaves estrangeiras

A variedade de comandos com **alter table** é gigantesca. Para finalizarmos a lista de seus principais comandos, vamos explorar a possibilidade de se



incluir até mesmo chaves estrangeiras após a definição e a formação de uma tabela.

Suponha que você criou uma tabela **carros**, que deve se relacionar com outra tabela de nome **fabricantes**. Porém, ao criar o campo **fabricante\_FK**, você se esqueceu de adicionar a cláusula **foreign key**, o que evidentemente não atende à necessidade de relacionamento.

Nesse caso, utilizamos o comando **alter table add constraint foreign key**, para possibilitar a definição do campo já existente como uma chave estrangeira e assegurar o relacionamento desejado:

```
-- ESTRUTURA DO COMANDO:
alter table NOME_DA_TABELA add constraint foreign
key(NOME_DA_COLUNA_ESTRANGEIRA) references
NOME_DA_TABELA_RELACIONADA(CHAVE_PRIMÁRIA_TABELA_
RELACIONADA);

-- EXEMPLOS:
alter table materias add constraint foreign key
(categoria_FK)
references categorias(id);
alter table jogos add constraint foreign key (licenca_FK)
references
licencas(licenca_id);
```

## IMPORTANTE

### Nova coluna e chave estrangeira

O comando apresentado apenas define que uma coluna existente será uma chave estrangeira relacionada com outra tabela, mas não cria a coluna caso ela não exista. Dessa maneira, se você deseja adicionar uma coluna nova que será uma chave estrangeira, você deve executar esse processo em dois passos separados, ou seja, utilizar o comando de adição de colunas e, em seguida, utilizar o comando de adição de *constraint* de chave estrangeira.

Vejamos um exemplo:

```
-- EXEMPLO DE ADIÇÃO DE UMA NOVA COLUNA ESTRANGEIRA:
alter table academia add column aparelho_FK bigint not null;

alter table academia add constraint foreign key (aparelho_FK)
references aparelhos(id);
```

### Removendo chaves estrangeiras

De forma muito semelhante aos comandos de eliminação de *constraints* e índices, a remoção de chaves estrangeiras também é muito simples, utilizando-se o comando **alter table** associado com o **drop foreign key**.

Vale ressaltar que estamos nos referindo ao processo de remoção da definição de chave estrangeira de uma coluna e não à eliminação da coluna em si. Para deleção de uma coluna por completo, você deve utilizar o comando já abordado (**alter table drop column**).

Vamos ver mais de perto o comando mencionado:

```
-- ESTRUTURA DO COMANDO:
alter table NOME_DA_TABELA drop foreign key
    NOME_DA_FOREIGN_KEY_CONSTRAINT;

-- EXEMPLOS:
alter table materias drop foreign key materias_ibfk_1;
alter table disciplinas drop foreign key professor_fk_1;
```

## IMPORTANTE

### Remover índice criado automaticamente

Como já mencionado, ao criar uma chave estrangeira, um índice associado a essa chave é gerado automaticamente. No entanto, remover a *constraint* da chave estrangeira **não elimina o índice** associado à coluna.

Por isso, se for necessário remover também esse índice criado automaticamente, é fundamental utilizar o comando apropriado (relembrando: use o comando **show create table** para checar os nomes das *constraints* corretamente):

```
-- EXEMPLO DE REMOÇÃO DE UMA CHAVE ESTRANGEIRA E SEU ÍNDICE:  
alter table materias drop foreign key categorias_ibfk_1;  
alter table materias drop index categoria_FK;
```

Exemplo e casos de uso

### Exemplo – *Startup* de banco digital

Uma *startup* de banco digital começou suas operações oferecendo aos clientes apenas uma conta básica, em que cada usuário tinha um único número de conta associado diretamente ao seu cadastro. Contudo, com o passar do tempo e a expansão dos serviços, o banco começou a oferecer diferentes tipos de contas, como conta corrente, poupança e conta de investimentos.

Agora, cada cliente pode possuir múltiplas contas associadas, e o modelo de dados existente não suporta essa nova estrutura. O atributo **numero\_conta**, que antes estava na tabela **usuarios**, precisa ser removido. Em vez disso, será criada uma tabela separada chamada **contas**, estabelecendo um relacionamento de “um para muitos” (um usuário pode ter muitas contas).

Além disso, o banco implementou um sistema para identificar o padrão de investidor dos clientes com base em um questionário. Os resultados desse questionário precisam ser armazenados na tabela de usuários, exigindo a adição de uma chave estrangeira que referencie a tabela **padroes\_investidor**.

Outra recomendação dirigida aos desenvolvedores responsáveis por esse projeto é incluir mais validações no banco de dados para aumentar a consistência dos dados, bem como redefinir o campo de foto do usuário para um tipo **TEXT** ao invés de **BLOB**, pois permitirá a inclusão de uma imagem no formato **Base64** com criptografia adicional para maior segurança.

E a última informação a se considerar, mas não menos importante, é que, com a crescente demanda da *startup*, há uma preocupação com o desempenho na obtenção de informações contidas no banco de dados, uma

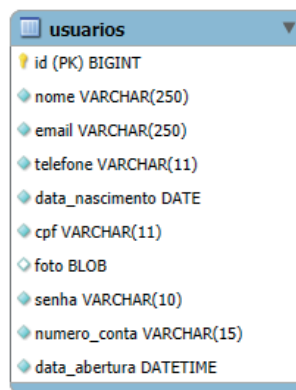
vez que a tendência é a quantidade de contas e usuários crescer exponencialmente. Portanto, analise a possibilidade de melhoria, considerando que uma das buscas mais importantes nesse sistema (e que deve apresentar boa performance) é a autenticação com *e-mail* e senha.

Dessa maneira, de forma resumida e pontual, devemos atender aos seguintes itens:

- Remover a informação de número da conta diretamente do usuário;
- Criar uma relação de um usuário para muitas contas;
- Criar uma tabela de armazenamento do perfil de investidor do usuário;
- Trocar o campo de foto de **BLOB** para **TEXT**;
- Adicionar mais *constraints* de validações;
- Otimizar performance em buscas por *e-mail* e senha.

Vamos atender a essa necessidade seguindo esses passos:

1. Analisaremos o diagrama UML que envolve as partes mencionadas no escopo deste projeto:



**Figura 3.36** – Imagem de um diagrama UML indicando o cenário de usuários de um banco digital.

Crédito: do elaborador, 2025.

Conforme é possível perceber, esse cenário não atende mais à demanda dessa nova fase do projeto, já que um mesmo usuário deve poder possuir diferentes contas, porém, dentro do atributo **numero\_conta** dessa tabela, somente é possível armazenar a identificação de apenas uma delas.

2. Usaremos o comando **show create table** para identificar todos os atributos e índices desse cenário atual:

```
-- RESULTADO DO COMANDO: show create table usuarios;
CREATE TABLE usuarios (
  id bigint NOT NULL AUTO_INCREMENT,
  nome varchar(250) NOT NULL,
  email varchar(250) NOT NULL,
  telefone varchar(11) NOT NULL,
  data_nascimento date NOT NULL,
  cpf varchar(11) NOT NULL,
  foto blob,
  senha varchar(10) NOT NULL,
  numero_conta varchar(15) NOT NULL,
  data_abertura datetime NOT NULL,
  PRIMARY KEY (id),
  UNIQUE KEY email (email),
  UNIQUE KEY cpf (cpf),
  UNIQUE KEY numero_conta (numero_conta)
);
```

3. Faremos primeiro a criação da nova tabela de contas, relacionando-a com a tabela de usuários via chave estrangeira, de forma a possibilitar que os mesmos usuários possam ter múltiplas contas registradas:

```
create table contas (
  numero_conta varchar(15) primary key,
  tipo enum('CORRENTE','POUPANÇA','INVESTIMENTOS') not null
default
  'CORRENTE',
  data_abertura datetime not null default now(),
  usuario_FK bigint not null,
  foreign key (usuario_FK) references usuarios(id),
  check (length(numero_conta) = 15)
);
```

4. Em seguida, adicionaremos a tabela de padrões de investidores:

```
create table perfis_investidores (  
  id int primary key auto_increment,  
  nome_perfil varchar(100) not null,  
  nota_tolerancia_risco int not null default 0,  
  nota_conhecimento int not null default 0,  
  nota_imediatismo int not null default 0,  
  nota_planejamento int not null default 0,  
  nota_diversificacao int not null default 0,  
  check (nota_tolerancia_risco >= 0 and nota_tolerancia_risco  
<= 10),  
  check (nota_conhecimento >= 0 and nota_conhecimento <= 10),  
  check (nota_imediatismo >= 0 and nota_imediatismo <= 10),  
  check (nota_planejamento >= 0 and nota_planejamento <= 10),  
  check (nota_diversificacao >= 0 and nota_diversificacao <= 10)  
);
```

Embora não tenha sido definida qual será a estrutura dessa tabela, optamos por adicionar alguns parâmetros condizentes com o contexto, realizando respectivamente algumas validações nas notas de cada perfil de investidor.

5. Agora, removeremos a coluna indesejada **numero\_conta** de dentro da tabela de usuários, graças à utilização do comando **alter table drop column**:

```
alter table usuarios drop column numero_conta;
```

6. Dando continuidade aos comandos de alteração da tabela de usuários, faremos a adição da coluna de perfil de investidor, definindo-a também como chave estrangeira para atender ao relacionamento desejado para o contexto dessa aplicação:

```
alter table usuarios add column perfil_investidor_FK int not
null;

alter table usuarios add constraint foreign key
(perfil_investidor_FK)
references perfs_investidores(id);
```

7. Em seguida, para atender ao requisito de segurança no armazenamento das fotos dos usuários, modificaremos o tipo do campo de **BLOB** para **TEXT**, possibilitando a adição de valores criptografados mais facilmente:

```
alter table usuarios modify column foto text;
```

8. Ainda na tabela de usuários, adicionaremos algumas validações que estão faltando, aumentando assim a confiabilidade e a consistência da base de dados:

```
-- validação do mínimo de caracteres para nome:
alter table usuarios add constraint check (length(nome) >=
3);

-- validação dos caracteres obrigatórios em e-mails:
alter table usuarios add constraint check (email like
'%@%.%');

-- validação dos caracteres de telefonia fixa e móvel com
código de área:
alter table usuarios add constraint check (length(telefone)
in
(10,11));
-- validação da quantidade de caracteres obrigatórios em
cpfs:
```

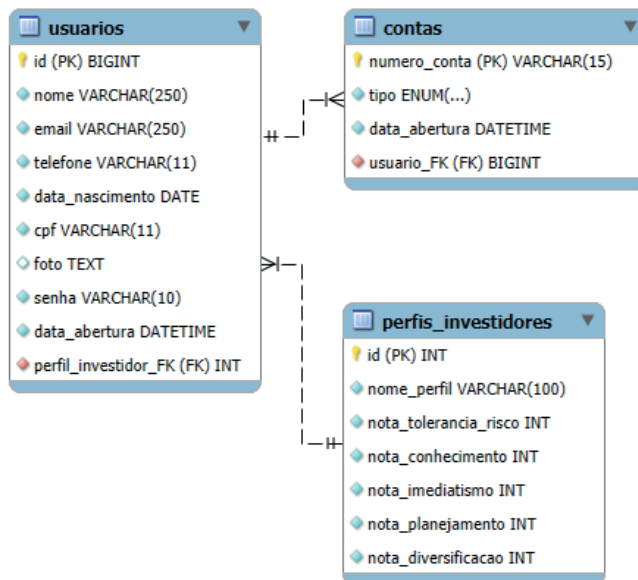
```
alter table usuarios add constraint check (length(cpf) = 11);

-- validação da quantidade de caracteres mínima e máxima
para senhas:
alter table usuarios add constraint check (length(senha) >= 8
and
length(senha) <= 10);
```

9. Nesse próximo passo, adicionaremos um índice composto para otimizar o processo de autenticação via *e-mail* e senha, também na tabela de usuários:

```
alter table usuarios add index (email,senha);
```

10. Por fim, mostraremos na **Figura 3.37** o diagrama UML após as alterações feitas, para possibilitar uma melhor compreensão do(a) aluno(a):



**Figura 3.37** – Imagem de um diagrama UML indicando o novo cenário de usuários, contas e perfis de investimento de um banco digital.

Crédito: do elaborador, 2025.

Após as modificações realizadas, o diagrama UML revela uma estrutura significativamente diferente da anterior, refletindo os benefícios



proporcionados pela modelagem relacional. Essa nova abordagem promove maior consistência, flexibilidade e integridade nos dados, resultando em uma aplicação mais robusta e alinhada às boas práticas de *design* de bancos de dados que já vimos até aqui.

11. Por fim, como algumas informações (por exemplo, *constraints* e índices) não ficam visíveis em diagramas UML, apresentaremos a seguir o resultado do comando **show create table** executado para a tabela de usuários (mostraremos apenas essa tabela, já que as demais entidades foram criadas do zero durante a execução deste exercício):

```
-- RESULTADO DO COMANDO: show create table usuarios;
CREATE TABLE usuarios (
  id bigint NOT NULL AUTO_INCREMENT,
  nome varchar(250) NOT NULL,
  email varchar(250) NOT NULL,
  telefone varchar(11) NOT NULL,
  data_nascimento date NOT NULL,
  cpf varchar(11) NOT NULL,
  foto text,
  senha varchar(10) NOT NULL,
  data_abertura datetime NOT NULL,
  perfil_investidor_FK int NOT NULL,
  PRIMARY KEY (id),
  UNIQUE KEY email (email),
  UNIQUE KEY cpf (cpf),
  KEY perfil_investidor_FK (perfil_investidor_FK),
  KEY email_2 (email,senha),
  CONSTRAINT usuarios_ibfk_1 FOREIGN KEY (perfil_investidor_FK)
    REFERENCES perfis_investidores (id),
  CONSTRAINT usuarios_chk_1 CHECK ((length(nome) >= 3)),
  CONSTRAINT usuarios_chk_2 CHECK ((email like
_utf8mb4'%%@%.%'')),
  CONSTRAINT usuarios_chk_3 CHECK ((length(telefone) in
(10,11))),
```

```

CONSTRAINT usuarios_chk_4 CHECK ((length(cpf) = 11)),
CONSTRAINT usuarios_chk_5 CHECK (((length(senha) >= 8) and
(length(senha) <= 10)))
);

```

### Exercício – Controle de acesso – catraca digital

Uma escola quer implementar um controle de acesso com o uso de catracas, mas deseja fazer disso um projeto prático de seus alunos formandos, em uma parceria com turmas de Automação e turmas de Tecnologia da Informação.

Você ficará responsável por criar a modelagem dos dados, de forma a atender aos requisitos, enquanto outros alunos desenvolverão o *frontend*, o *backend* e os sensores e microcontroladores da parte de Automação.

Para o controle de acesso desse sistema, considere:

- Existem três categorias de pessoas que acessam a escola: alunos, funcionários e visitantes;
- No sistema, os alunos devem estar associados a turmas e cursos, para permitir o rastreamento;
- Os alunos possuem o cadastro básico de nome, *e-mail*, telefone, CPF, número de matrícula, data de nascimento etc. Os funcionários e os visitantes também, porém, no caso dos funcionários não há número de matrícula, mas sim um NIF (número de identificação funcional), e, no caso dos visitantes, não há nem matrícula nem NIF;
- Cada turma do curso tem um horário de início e de término, bem como datas de início e fim do semestre letivo daquela turma. Essas informações serão úteis para checar se o aluno pode estar na escola naquele horário/dia;
- Todos os acessos devem ser registrados, indicando data e hora do acesso, se é entrada ou saída e quem acessou;

- Crie nas tabelas as validações necessárias dos campos para evitar inconsistências;
- Adicione índices importantes em número de matrícula, NIF e CPF, para otimização de buscas;
- No final, faça um **show create table** de cada tabela e salve-as, para que o time de desenvolvimento restante possa replicá-las (enquanto não aprendemos sobre exportação de banco de dados).

## insert – Inserindo dados em tabelas

Sem dúvidas, estamos avançando muito em nossos estudos com bancos de dados. Afinal, já conseguimos entender a aplicabilidade dos bancos de dados, modelá-los de acordo com requisitos, criar as tabelas e os relacionamentos, obter detalhamentos da estrutura criada e até mesmo alterar tabelas já existentes, seja por meio da inclusão ou remoção de colunas, índices ou *constraints*.

Contudo, se você tem seguido rigorosamente nossa jornada, ao longo de exemplos, exercícios e, é claro, casos de uso, você já deve ter criado inúmeras tabelas e bancos de dados. Porém, tenha notado ou não, todas elas estão até o momento vazias. Em outras palavras, é como se criássemos um guarda-roupas extremamente grande, eficiente e organizado, mas sem nenhuma roupa dentro.

Sendo assim, exploraremos agora um dos principais comandos envolvendo bancos de dados: a inserção de registros com o comando **insert into**.

Como de costume, veremos primeiro a estrutura desse comando em detalhes, o qual é formado basicamente pela indicação da tabela que receberá os dados, a sequência das colunas informadas e, por fim, os valores a ser armazenados:

```

-- ESTRUTURA DO COMANDO:
insert into NOME_DA_TABELA (COLUNA1, COLUNA2, COLUNA3...)
values
    (VALOR1, VALOR2, VALOR3...), (VALOR1, VALOR2, VALOR3...), ...;

-- EXEMPLOS:
-- alternativa 1: INDICANDO AS COLUNAS
insert into roupas (nome, marca, cor, tamanho, quantidade,
preco)
values
    ('Calça Jeans Slim', 'Top Jeans', 'Preta', '38', 9, 109.90),
    ('Camiseta Regata', 'Free', 'Azul Royal', 'M', 4, 67.85);

-- alternativa 2: SEGUINDO A ORDEM NATURAL DAS COLUNAS
insert into roupas
values
    (1,'Calça Jeans Slim', 'Top Jeans', 'Preta', '38', 9, 109.90,
null),
    (2, 'Camiseta Regata', 'Free', 'Azul Royal', 'M', 4, 67.85,
null);

```

Observe que a estrutura de comando é relativamente simples, sendo possível executá-lo de duas maneiras distintas: escolhendo a sequência dos valores das colunas a ser informada (**alternativa 1**) ou obedecendo à ordem fixa de colunas definida na tabela (**alternativa 2**).

Mas, afinal, qual é essa “ordem fixa” de colunas? Como encontrá-la?

Para esclarecer isso, basta utilizarmos o comando SQL de descrição de uma tabela: o **describe**, já abordado anteriormente neste livro (se preferir, você pode usar também seus comandos sinônimos, como **desc** ou **explain**).

Assim, observe a **Figura 3.38**, que apresenta a sequência lógica da tabela **roupas**, mencionada no exemplo anterior:

251 • **describe** roupas;

|   | Field      | Type          | Null | Key | Default     | Extra          |
|---|------------|---------------|------|-----|-------------|----------------|
| ▶ | id         | int           | NO   | PRI | <b>NULL</b> | auto_increment |
|   | nome       | varchar(255)  | NO   |     | <b>NULL</b> |                |
|   | marca      | varchar(100)  | NO   |     | <b>NULL</b> |                |
|   | cor        | varchar(50)   | NO   |     | <b>NULL</b> |                |
|   | tamanho    | varchar(10)   | NO   |     | <b>NULL</b> |                |
|   | quantidade | int           | NO   |     | <b>NULL</b> |                |
|   | preco      | decimal(10,2) | NO   |     | <b>NULL</b> |                |
|   | observacao | varchar(800)  | YES  |     | <b>NULL</b> |                |

**Figura 3.38** – Sequência natural das colunas da tabela de roupas – MySQL Workbench.

Crédito: do elaborador, 2025.

## IMPORTANTE

### Indicação dos valores de todas as colunas

Embora o resultado de inclusão em ambas as situações seja praticamente o mesmo, é importante enfatizar que a escolha da **alternativa 2**, ou seja, seguindo a ordem natural da tabela, acaba por obrigar a indicação dos valores de todas as colunas, até mesmo as opcionais (do tipo **null**) e as chaves primárias, mesmo que sejam do tipo **auto\_complete**.

É por esse motivo que a quantidade de valores informados na **alternativa 2** é maior, uma vez que tivemos que atribuir um valor, mesmo que nulo, para a coluna **observacao**.

Veja novamente na figura a seguir:

```
-- alternativa 1: INDICANDO AS COLUNAS
insert into roupas (nome, marca, cor, tamanho, quantidade,
preco)
values
('Calça Jeans Slim', 'Top Jeans', 'Preta', '38', 9, 109.90),
('Camiseta Regata', 'Free', 'Azul Royal', 'M', 4, 67.85);

-- alternativa 2: SEGUINDO A ORDEM NATURAL DAS COLUNAS
insert into roupas
values
```

```
(1, 'Calça Jeans Slim', 'Top Jeans', 'Preta', '38', 9, 109.90,
null),
(2, 'Camiseta Regata', 'Free', 'Azul Royal', 'M', 4, 67.85,
null);
```

### Exemplos e casos de uso – Controle de estacionamento digital

Uma empresa desenvolveu uma plataforma de controle para estacionamento, buscando modernizar o processo de gestão e oferecer uma experiência otimizada para seus usuários. A plataforma precisa armazenar informações detalhadas sobre os veículos que utilizam o estacionamento, bem como avaliar a satisfação dos clientes após o uso do serviço. Para isso, é necessário implementar um banco de dados relacional com as seguintes entidades:

- **Tipos\_Veiculos:** responsável por armazenar os tipos de automóveis existentes no cotidiano (“Motocicletas”, “Carros de Pequeno e Médio Porte”, “Carros de Grande Porte”, “Ônibus e Caminhões” etc.), além de precificar o valor do minuto para cada tipo. Essa tabela também conterá o valor fixo de taxa de estacionamento, independentemente do tempo de permanência do automóvel;
- **Registros:** responsável por adicionar as entradas e saídas dos automóveis, relacionando-se com o tipo do veículo, além de conter informação do nome do veículo, placa e cor;
- **Avaliacao\_Satisfacao:** relacionada com a tabela de registro, essa entidade fica responsável por armazenar a avaliação do cliente quanto à sua satisfação, registrando a nota dada de 0 a 10, e um campo de comentário (que é opcional).

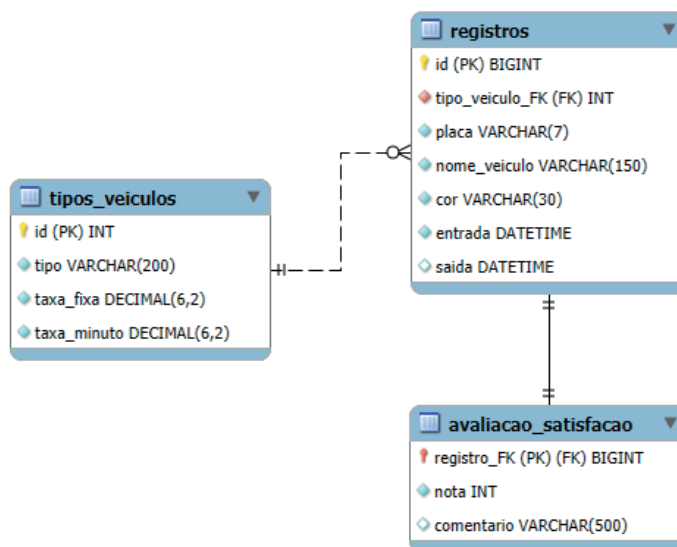
Nesta situação-problema, iremos criar a base de dados, a modelagem e as tabelas, considerando os requisitos. Também executaremos alguns testes, inserindo dados em cada entidade. Vamos lá!

1. Como primeiro passo, criaremos o banco de dados/*schema*, ativando-o em seguida:

```
create database estacionamento;

use estacionamento;
```

2. Agora, nos concentraremos em criar a modelagem dos dados, considerando que, de acordo com os requisitos, será necessário criar três tabelas, **tipos\_veiculos**, **registros** e **avaliacao\_satisfacao**, em que, respectivamente, a primeira se relaciona com a segunda, e a segunda se relaciona com a terceira:



**Figura 3.39** – Diagrama UML com as entidades de tipos de veículos, registros e pesquisa de satisfação.

Crédito: do elaborador, 2025.

Considerando a quantidade de situações-problemas que fizemos até aqui, provavelmente o(a) aluno(a) não encontrará nenhuma dificuldade de entendimento e interpretação da modelagem UML apresentada, uma vez que ela não tem grande complexidade.

Apenas como observação, evidenciamos que, para garantir que cada avaliação de satisfação esteja associada apenas a um único registro e vice-versa, incluímos o que seria a chave estrangeira da tabela **avaliacao\_satisfacao**, isto é, **registro\_FK**, para também atuar como chave primária da própria tabela de avaliação, já que chaves primárias conceitualmente nunca se repetirão, gerando então um relacionamento “um para um”.

3. Chegou o momento dos comandos de criação das tabelas em SQL, o que faremos primeiro para a entidade dominante de tipos de veículos, já que ela não depende da existência de nenhuma outra para ser criada:

```
create table tipos_veiculos (
  id int primary key auto_increment,
  tipo varchar(200) not null,
  taxa_fixa decimal(6,2) not null,
  taxa_minuto decimal(6,2) not null,
  check (taxa_fixa > 0.0),
  check (taxa_minuto > 0.0)
);
```

4. Em seguida, é necessário selecionar obrigatoriamente a tabela **registros**. Isso ocorre porque a criação da entidade de avaliação de satisfação depende diretamente da existência da entidade **registros**, devido à relação de dependência estabelecida entre elas. Portanto, a tabela **registros** deve ser criada primeiro para garantir a consistência do relacionamento:

```
create table registros (
  id bigint primary key auto_increment,
  tipo_veiculo_FK int not null,
  placa varchar(7) not null,
  nome_veiculo varchar(150) not null,
  cor varchar(30) not null,
  entrada datetime not null default now(),
  saida datetime,
```



```
foreign key (tipo_veiculo_FK) references tipos_veiculos(id),
check (length(placa) = 7)
);
```

5. Por último, mas não menos importante, criaremos a entidade de avaliação de satisfação:

```
create table avaliacao_satisfacao (
registro_FK bigint primary key,
nota int not null default 0,
comentario varchar(500),
foreign key (registro_FK) references registros(id),
check (nota >= 0 and nota <= 10)
);
```

Note que, para essa entidade, **registro\_FK** é tanto uma chave estrangeira quanto uma chave primária, de modo a assegurar o relacionamento “um para um”, mais coerente para o contexto apresentado.

Além disso, incluímos um delimitador das notas de avaliação, garantindo maior consistência à aplicação.

6. Chegou a hora de utilizarmos os novos comandos abordados para inserção de dados nas tabelas da aplicação, testando, assim, se todo o projeto está de acordo com as regras de negócio esperadas.

## IMPORTANTE

### Relacionamentos entre chaves estrangeiras

Em geral, não existem segredos para a tarefa de inserção, contudo, é importante ter uma atenção especial e se recordar do efeito gerado pelos relacionamentos entre as chaves estrangeiras das tabelas, que impactam diretamente a inclusão dos dados. Por exemplo, nossas três tabelas estão vazias e, se optarmos por começar a incluir dados na tabela **registros**, com certeza teremos erro na execução do comando, já que para cadastrar um registro é necessário que algum tipo de veículo esteja

cadastrado antes. Do mesmo modo, se tentarmos a inclusão em **avaliacao\_satisfacao** utilizando um **registro\_FK** que não existe, também não conseguiremos executar essa operação. Por esse motivo, deve-se fazer as inclusões dos registros seguindo a mesma ordem de criação das tabelas, ou seja, das entidades mais fortes para as mais fracas.

Começaremos então a cadastrar a entidade mais forte de nossa aplicação, ou seja, **tipo\_veiculos**, adicionando o nome de cada categoria de veículos e seus valores fixos e variáveis:

```
insert into tipos_veiculos (tipo, taxa_fixa, taxa_minuto) values
('Motocicletas', 2.50, 0.20),
('Automóveis Pequeno e Médio Porte', 3.5, 0.25),
('Automóveis Grande Porte', 3.5, 0.30),
('Caminhões e Ônibus', 10, 0.40);
```

## IMPORTANTE

### auto\_increment

Observe que optamos por utilizar o método **insert**, selecionando a ordem de colunas e valores a serem passados, o que nos permitiu não informar o **id** de cada elemento, já que aplicamos o **auto\_increment** em cada chave primária em todas as tabelas que criamos, o que significa que o próprio banco de dados irá criar esse sequenciamento. Dessa maneira, como nossa tabela estava vazia, inserindo quatro registros, teremos os **ids 1, 2, 3 e 4**, respectivamente.

7. Agora podemos inserir alguns registros, simulando clientes que utilizaram o estacionamento:

```
insert into registros (tipo_veiculo_FK, placa, nome_veiculo,
cor,
entrada, saida) values
(2, 'ABC8J8J', 'Honda Civic Sedan', 'Preto', '2024-12-17
14:01:33',
```

```

    '2024-12-17 14:55:02'),
  (1, 'CBC9047', 'Kawasaki Ninja 500', 'Verde', '2024-12-17
14:14:07',
    '2024-12-17 16:01:08'),
  (3, 'AYC8L19', 'Rampage Big Horn', 'Preto', '2024-12-17
15:00:59',
    '2024-12-17 16:23:07'),
  (2, 'RJC3K6B', 'Kia Sportage', 'Branco', '2024-12-17 15:03:00',
    '2024-12-17 17:03:03'),
  (2, 'RTF6M4N', 'Jeep Renegade', 'Branco', '2024-12-17
17:05:08',
    null);

```

Nessas inclusões, apenas para indicar ao(à) aluno(a), deixamos o último veículo adicionado sem informação do horário de saída do estacionamento, pelo uso do valor nulo (**null**), simulando que o veículo ainda esteja no local. É por esse motivo que não colocamos o **not null** para a coluna de saída, já que o comportamento natural desse sistema será a inclusão dos dados do veículo na hora de entrada e, após algum período, a atualização do horário de saída, com a devida cobrança proporcional às taxas cadastradas.

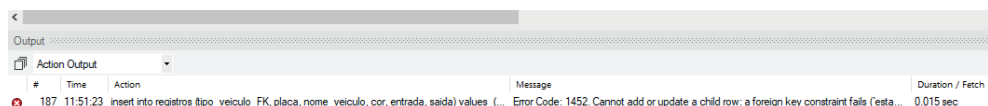
Outro ponto que você deve observar é que todos os valores aplicados para **tipo\_veiculo\_FK** estão dentro da faixa de **ids** existentes na tabela de tipos de veículos que cadastramos na etapa anterior.

Apenas para demonstração, veremos o erro gerado, caso se utilize um **id** de tipo de veículo inexistente, conforme **Figura 3.40**.

```

335 • insert into registros (tipo_veiculo_FK, placa, nome_veiculo, cor, entrada, saida) values
336   (800, 'ZXY5K5P', 'Teste Id inexistente', 'Preto', '2024-12-17 19:01:33', null);
337

```



**Figura 3.40** – Comando em SQL utilizando um id de tipo de veículo inexistente.

Crédito: do elaborador, 2025.

Observe que, nesse caso, utilizamos para **tipo\_veiculo\_FK** o valor de 800, o que de fato não existe, fazendo com que o MySQL gere no terminal o seguinte erro de *constraint*: **“Error Code: 1452. Cannot add or update a child row: a foreign key constraint fails (‘estacionamento’.‘registros’, CONSTRAINT ‘registros\_ibfk\_1’ FOREIGN KEY (‘tipo\_veiculo\_FK’) REFERENCES ‘tipos\_veiculos’ (‘id’))”**.

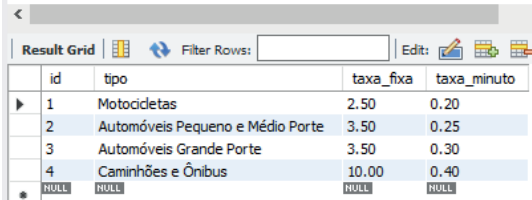
8. Por fim, adicionaremos algumas avaliações de satisfação, com atenção para utilizar como **registro\_FK** apenas valores existentes. Além disso, devemos usar apenas um registro por vez, já que essa tabela está estruturada com um relacionamento “um para um”.

Portanto, como nossa tabela de registros estava vazia e adicionamos cinco comandos de inserção, somente poderemos utilizar **ids** de 1 até 5 para a entidade de satisfação:

```
insert into avaliacao_satisfacao values
(1, 8, null), (2, 10, 'Atendimento rápido!'),
(3, 5, 'Estacionamento seguro, mas deveria ter área
coberta');
```

9. Como bônus especial pela sua dedicação em ter acompanhado este estudo de caso até aqui, vamos dar uma breve demonstração de como visualizar os dados armazenados nas tabelas. Agora que incluímos vários dados, vamos verificar se eles realmente foram salvos em cada entidade. Para isso, utilizaremos o comando visto na **Figura 3.41**:

343 • `select * from tipos_veiculos;`



|   | id   | tipo                             | taxa_fixa | taxa_minuto |
|---|------|----------------------------------|-----------|-------------|
| ▶ | 1    | Motocicletas                     | 2.50      | 0.20        |
|   | 2    | Automóveis Pequeno e Médio Porte | 3.50      | 0.25        |
|   | 3    | Automóveis Grande Porte          | 3.50      | 0.30        |
|   | 4    | Caminhões e Ônibus               | 10.00     | 0.40        |
| • | NULL | NULL                             | NULL      | NULL        |

**Figura 3.41** – Comando em SQL para visualização dos dados salvos na tabela de tipos de veículos.

Crédito: do elaborador, 2025.

Não se preocupe em entender agora esse comando, apenas observe que o resultado gerado é exatamente o que incluímos utilizando o **insert**.

10. Agora, realizaremos a consulta da tabela de registros, conforme visto na **Figura 3.42**:

344 • **select \*from** registros;

Result Grid

Filter Rows:

Edit:

Export/Import:

Wrap Cell Content:

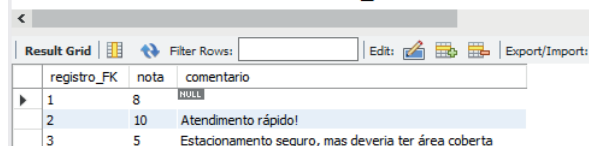
|   | id | tipo_veiculo_FK | placa   | nome_veiculo       | cor    | entrada             | saida               |
|---|----|-----------------|---------|--------------------|--------|---------------------|---------------------|
| ▶ | 1  | 2               | ABC8J8J | Honda Civic Sedan  | Preto  | 2024-12-17 14:01:33 | 2024-12-17 14:55:02 |
|   | 2  | 1               | CBC9O47 | Kawasaki Ninja 500 | Verde  | 2024-12-17 14:14:07 | 2024-12-17 16:01:08 |
|   | 3  | 3               | AYC8L19 | Rampage Big Horn   | Preto  | 2024-12-17 15:00:59 | 2024-12-17 16:23:07 |
|   | 4  | 2               | RJC3K6B | Kia Sportage       | Branco | 2024-12-17 15:03:00 | 2024-12-17 17:03:03 |
|   | 5  | 2               | RTF6M4N | Jeep Renegade      | Branco | 2024-12-17 17:05:08 | NULL                |

**Figura 3.42** – Comando em SQL para visualização dos dados salvos na tabela de registros.

Crédito: do elaborador, 2025.

11. Por fim, faremos a busca dos dados contidos na tabela de avaliação de satisfação dos clientes dessa plataforma de controle de estacionamentos, como se vê na **Figura 3.43**:

345 • **select \*from** avaliacao\_satisfacao;



|   | registro_FK | nota                                                | comentario |
|---|-------------|-----------------------------------------------------|------------|
| 1 | 8           | NULL                                                |            |
| 2 | 10          | Atendimento rápido!                                 |            |
| 3 | 5           | Estacionamento seguro, mas deveria ter área coberta |            |

**Figura 3.43** – Comando em SQL para visualização dos dados salvos na tabela de avaliação de satisfação.

Crédito: do elaborador, 2025.

Exercício – *Startup* corretora de criptomoedas

Como você já sabe, com o avanço da tecnologia, o mundo muda rapidamente, não sendo uma novidade que isso tenha impactado também o mercado financeiro. Dessa maneira, com o crescimento exponencial das *block-chains* e das criptomoedas, uma *startup* iniciou seu projeto para desenvolver uma nova corretora, com taxas mais acessíveis, recursos mais vantajosos e o melhor de tudo: ser nacional.

Você ficará responsável por criar a modelagem dos dados, o *schema* e as tabelas e inserir dados (os mais reais possíveis) para testar seu desenvolvimento.

Os requisitos serão:

- Para ter acesso à corretora, o usuário deverá fazer um cadastro com nome, *e-mail*, telefone, CPF, CEP, estado, cidade, rua, bairro, número. O usuário deve ter seu *e-mail* confirmado (somente assim ele estará com o *status* ativado);
- Deve haver o cadastro das criptomoedas disponíveis para venda, com nome, descrição do projeto, data de criação, data de lançamento à venda na corretora, endereço do *site* do projeto, valor máximo histórico e valor mínimo histórico;
- Deve existir uma tabela que gerencie o histórico contínuo dos preços de cada criptomoeda, com data e hora da medição, valor atual e qual criptomoeda. É mediante a última informação nessa tabela que o usuário poderá comprar o ativo em valor de mercado;
- Cada usuário possui uma conta na corretora, com um saldo em moeda fiduciária, ou seja, em dólar ou reais, que é utilizado para a compra de criptomoedas;
- Cada usuário pode possuir muitas criptomoedas – lembre-se de considerar a quantidade delas, sendo valor fracionário;
- Registre as transações de compra e venda, bem como a quantidade;
- Crie validações importantes em cada entidade, bem como índices para otimização de buscas;
- Crie a modelagem e as tabelas e insira os dados em cada uma delas, testando o relacionamento de cada uma;
- Faça uma busca simples para mostrar todos os resultados com o **select**.

## Operadores

Embora os operadores não possuam função sozinhos, sendo sempre associados a outros comandos, eles desempenham um papel fundamental na manipulação, consulta e análise das informações armazenadas em bancos de dados. São ferramentas que permitem aos desenvolvedores realizar comparações, executar operações matemáticas, filtrar dados com base em critérios, manipular textos (*strings*), trabalhar com *bits* e até mesmo controlar o fluxo de uma consulta ou *script*.

Os operadores são usados principalmente com os comandos **select**, **update**, **delete** e **insert**, ajudando a especificar condições, criar cálculos dinâmicos e realizar transformações nos dados. Seja para calcular a média de preços de produtos, comparar datas de entrada e saída de registros ou concatenar *strings* em um relatório, os operadores tornam essas tarefas possíveis e eficientes.

Neste tópico, apresentaremos os principais tipos de operadores disponíveis no MySQL. Faremos isso de uma forma mais direta, já que exploraremos vários deles na prática durante os exemplos e casos de uso mais adiante, por meio dos comandos **select**, **update** e **delete**.

Vamos lá!

### Lógicos

Os operadores lógicos são essenciais para a criação de condições complexas em consultas SQL, permitindo combinar ou modificar critérios para filtrar dados de maneira mais eficaz. Esses operadores retornam valores booleanos (**true**, **false** ou **null**) e são amplamente utilizados em instruções como **where**, **having** e **on**, além de estruturas condicionais em comandos **case** ou em funções armazenadas (não se preocupe, veremos tais instruções detalhadamente mais adiante).

No **Quadro 3.4**, temos os principais operadores lógicos.

### Quadro 3.4 – Operadores lógicos do MySQL

| Operador                  | Descrição                                                                                                                                                     | Comportamento                                                       |
|---------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| <b>and ( &amp;&amp; )</b> | Condição <b>E</b> existente em praticamente todas as linguagens de programação, podendo ser usada com a palavra <b>and</b> ou os símbolos <b>&amp;&amp;</b> . | Retorna verdadeiro se todas as condições lógicas forem verdadeiras. |
| <b>or (    )</b>          | Condição <b>OU</b> existente em praticamente todas as linguagens de programação, podendo ser usada com a palavra <b>or</b> ou os símbolos <b>  </b> .         | Retorna verdadeiro se qualquer uma das condições for verdadeira.    |
| <b>not ( ! )</b>          | Condição de negação existente em praticamente todas as linguagens de programação, podendo ser usada com a palavra <b>not</b> ou o símbolo <b>!</b> .          | Inverte o resultado da condição original.                           |
| <b>xor</b>                | Condição de <b>OU EXCLUSIVO</b> , muito comum em sistemas digitais.                                                                                           | Retorna verdadeiro se apenas uma das condições é verdadeira.        |

Crédito: do elaborador, 2025.

#### Aritméticos

Os operadores aritméticos são utilizados para realizar cálculos matemáticos básicos em consultas SQL. Eles permitem que você manipule valores numéricos diretamente nas consultas, tornando possível criar cálculos dinâmicos, gerar relatórios financeiros ou ajustar valores com base em condições específicas.

### Quadro 3.5 – Operadores aritméticos do MySQL

| Operador | Descrição                                |
|----------|------------------------------------------|
| +        | Operação matemática de adição.           |
| –        | Operação matemática de subtração.        |
| *        | Operação matemática de multiplicação.    |
| /        | Operação matemática de divisão.          |
| %        | Operação matemática de resto de divisão. |

Crédito: do elaborador, 2025.



## Comparação

Os operadores de comparação são utilizados em consultas SQL para comparar valores entre colunas, constantes ou expressões. Esses operadores retornam valores lógicos (**true**, **false** ou **null**) e são fundamentais para a criação de condições em cláusulas como **where**, **having**, **join** e outras (veremos essas cláusulas em outro momento).

### Quadro 3.6 – Operadores de comparação do MySQL

| Operador      | Descrição                           | Comportamento                                                                                                                    |
|---------------|-------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| =             | Igual a                             | Compara se um valor está igual.                                                                                                  |
| != ou <>      | Diferente de                        | Compara se um valor está diferente.                                                                                              |
| >             | Maior que                           | Compara se um valor é maior que outro.                                                                                           |
| <             | Menor que                           | Compara se um valor é menor que outro.                                                                                           |
| >=            | Maior ou igual a                    | Compara se um valor é maior ou igual a outro.                                                                                    |
| <=            | Menor ou igual a                    | Compara se um valor é menor ou igual a outro.                                                                                    |
| <=>           | Igual a (seguro para valores nulos) | Compara se um valor é igual a outro de forma segura para valores nulos; quando ambos os valores forem nulos, retorna verdadeiro. |
| (not) is null | (Não) É nulo                        | Compara se é nulo ou se não é nulo ao utilizar o <b>NOT</b> .                                                                    |

Crédito: do elaborador, 2025.

## Atribuição

Os operadores de atribuição no MySQL são usados para armazenar valores em variáveis ou colunas.

### Quadro 3.7 – Operador de atribuição do MySQL

| Operador | Descrição                                                                     |
|----------|-------------------------------------------------------------------------------|
| :=       | Atribui um valor. Utilizado principalmente para atribuir valores a variáveis. |

Crédito: do elaborador, 2025.

### Strings (textos)

Os operadores de *strings* no MySQL permitem manipular e trabalhar com valores de texto. Esses operadores são úteis para realizar operações como concatenar, comparar e modificar *strings*.

No **Quadro 3.8**, apresentamos os principais operadores e suas funcionalidades.

**Quadro 3.8** – Operadores de *strings* do MySQL

| Operador            | Comportamento                                                                                                                                            |
|---------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>concat</b>       | Concatena duas ou mais <i>strings</i> .                                                                                                                  |
| <b>(not) like</b>   | Compara se um valor está igual, podendo-se aplicar curingas para buscas mais avançadas. Quando utilizado com o <b>NOT</b> , tem comportamento contrário. |
| <b>(not) regexp</b> | Verifica se uma <i>string</i> corresponde a uma expressão regular. Quando utilizado com o <b>NOT</b> , tem comportamento contrário.                      |

Crédito: do elaborador, 2025.

### Conjunto

Os operadores de conjunto no MySQL permitem combinar os resultados de múltiplas consultas **select** em uma única tabela de resultados. Eles são úteis quando é necessário mesclar, comparar ou diferenciar dados provenientes de diferentes tabelas ou consultas.

**Quadro 3.9** – Operadores de conjunto do MySQL

| Operador             | Descrição                     | Comportamento                                                                                           |
|----------------------|-------------------------------|---------------------------------------------------------------------------------------------------------|
| <b>(not) between</b> | (Não) Entre um intervalo      | Compara se um valor está entre um intervalo, ou se não está em um intervalo ao utilizar o <b>NOT</b> .  |
| <b>(not) in</b>      | (Não) Em uma lista de valores | Compara se um valor está em uma lista de valores, ou se não está nesta lista ao utilizar o <b>NOT</b> . |
| <b>union</b>         | União de valores distintos    | Une os resultados, eliminando resultados repetidos.                                                     |
| <b>union all</b>     | União de valores              | Une todos os resultados.                                                                                |

Crédito: do elaborador, 2025.

Bitwise

Os operadores *bitwise* no MySQL são usados para realizar operações em nível de *bits* em valores inteiros. Eles permitem manipular e comparar *bits* individuais dentro de um número, sendo úteis em situações que envolvem compactação de dados, permissões, *flags* ou operações matemáticas complexas.

Quadro 3.10 – Operadores de *bitwise* do MySQL

| Operador | Descrição               | Comportamento                                                                                                                                        |
|----------|-------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| &        | E bit a bit             | Executa operação lógica <b>E</b> em cada par de <i>bits</i> .                                                                                        |
|          | OU bit a bit            | Executa operação lógica <b>OU</b> em cada par de <i>bits</i> .                                                                                       |
| ^        | OU EXCLUSIVO bit a bit  | Executa operação lógica <b>OU EXCLUSIVO</b> em cada par de <i>bits</i> .                                                                             |
| ~        | NOT bit a bit           | Executa operação lógica <b>NOT</b> em cada par de <i>bits</i> .                                                                                      |
| <<       | Deslocamento à esquerda | Desloca os <i>bits</i> de um valor para a esquerda, preenchendo com zeros à direita.                                                                 |
| >>       | Deslocamento à direita  | Desloca os <i>bits</i> de um valor para a direita, preenchendo com zeros à esquerda ou com o <i>bit</i> de sinal, dependendo do número e do sistema. |

Crédito: do elaborador, 2025.

Especiais

Esses operadores são denominados especiais porque possuem funcionalidade que não se enquadra em nenhuma das categorias apresentadas anteriormente. Seu comportamento é descrito no **Quadro 3.11**.

Quadro 3.11 – Operadores especiais do MySQL

| Operador         | Comportamento                                   |
|------------------|-------------------------------------------------|
| <b>coalesce</b>  | Retorna o primeiro valor não nulo em uma lista. |
| <b>case when</b> | Define uma lógica condicional.                  |
| <b>exists</b>    | Retorna verdadeiro se existir algum resultado.  |

Crédito: do elaborador, 2025.

## select – Extraindo dados de tabelas

Chegamos agora no que talvez seja o auge de nossa jornada com bancos de dados: a extração de dados. Afinal, qual seria o sentido de armazená-los se não pudéssemos lê-los novamente?

Aliás, essa é uma das premissas da Ciência de Dados, uma das áreas mais emergentes atualmente: coletar dados que, após lidos, organizados e separados de forma sistemática, podem ser transformados em **informações**, ou seja, apresentam uma utilidade clara e definida. Claramente, todo esse processo não seria possível ou minimamente viável sem a existência dos bancos e sua capacidade de extração de dados.

Nesse primeiro contato com a extração de dados, veremos consultas simples. Contudo, não se engane, uma das partes que podem se tornar mais complicadas, envolvendo bancos de dados relacionais, é justamente o processo de leitura dos dados, sobretudo em consultas mais complexas, como veremos adiante.

Nos tópicos seguintes, exploraremos cada parte do comando **select** com um determinado enfoque, de modo a abrir as possibilidades de uso pelo(a) aluno(a).

Contudo, para dar a primeira noção desse importante comando e servir como base para todos os tópicos que virão em breve, apresentamos a estrutura básica do **select** e seus complementos. Não se preocupe em entender todos os complementos agora, pois detalharemos cada um deles no momento pertinente.

```
-- ESTRUTURA GENÉRICA DO COMANDO SELECT
select [distinct] LISTA_DE_COLUNAS
  from NOME_DA_TABELA
 [join NOME_DA_TABELA_2 on CONDIÇÃO_DE_JUNÇÃO]
 ...
 [join NOME_DA_TABELA_2 on CONDIÇÃO_DE_JUNÇÃO]
 [where (CONDIÇÕES_FILTRO ...)]
```

```
[group by LISTA_DE_COLUNAS]
[having (CONDIÇÕES_FILTRO_CONSOLIDAÇÃO...)]
[order by LISTA_DE_COLUNAS (asc/desc) ]
[limit NÚMERO_DE_LINHAS [offset VALOR_INICIAL] ];
```

Nota-se que há uma verdadeira “sopa de letras” a ser desvendada, o que faremos cuidadosamente no momento apropriado, mas, para saciar brevemente a curiosidade do(a) aluno(a), apresentamos algumas informações:

- **select from:** é a estrutura básica e obrigatória da extração dos dados, ou seja, você precisa minimamente indicar que deseja realizar uma seleção (**select**) e de onde virão os dados (**from**);
- **distinct:** é opcional e indica que você deseja obter resultados não repetidos;
- **join on:** é opcional e faz a união de tabelas para obtenção de dados frequentemente interligados por chaves estrangeiras;
- **where:** é opcional, executa a busca de dados por meio de condições determinadas, ou seja, filtrando os resultados;
- **group by:** é opcional e faz o agrupamento de resultados, consolidando-os e podendo associá-los com funções estatísticas;
- **having:** é opcional e tem o comportamento semelhante ao **where**, mas voltado a filtrar resultados agrupados pelo **group by**;
- **order by:** é opcional e classifica a ordem de exibição dos dados;
- **asc/desc:** é opcional e, quando associado ao **order by**, altera a classificação dos dados em ordem ascendente ou descendente;
- **limit:** é opcional e limita a quantidade de resultados obtidos na consulta;
- **offset:** é opcional e especifica o ponto de partida das linhas para a extração dos dados.

Muita informação, não é mesmo? Mas fique calmo; como prometido, detalharemos o uso de cada situação. Portanto, prepare-se para explorar, praticar e desvendar o poder do comando **select**!

select – Consultas simples

Para extrairmos os dados de tabelas, utilizamos o comando **select** e, embora tenhamos afirmado anteriormente que ele pode se tornar bastante complexo em determinadas ocasiões, no entanto, em consultas simples, sua estrutura também se torna muito simplificada e bem compreensível.

Apresentaremos a seguir a estrutura do **select** com enfoque em consultas simples:

```
-- ESTRUTURA DO COMANDO (CONSULTAS SIMPLES):  
-- alternativa 1: indicando as colunas:  
select COLUNA1,COLUNA2,... from NOME_DA_TABELA;  
  
-- alternativa 2: selecionando todas as colunas:  
select * from NOME_DA_TABELA;  
  
-- EXEMPLOS:  
select id, nome, email from usuarios;  
select * from caixa_emails;
```

Ilustramos duas formas básicas de executar consultas **select**. A primeira envolve a especificação das colunas desejadas, uma abordagem útil quando não é necessário obter todas as colunas da tabela. A segunda opção utiliza o caractere \*, que instrui o MySQL a retornar todas as colunas da tabela no resultado da consulta.

Na **Figura 3.44**, veremos o resultado prático da execução desses comandos, iniciando com a seleção de todas as colunas.

12 • `select *from gamers;`

| Result Grid                                                                                                                                                                                                                                                 |                  |                            |             |           |                 |                     |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|----------------------------|-------------|-----------|-----------------|---------------------|
| Filter Rows: <input type="text"/>                                                                                                                                                                                                                           |                  |                            |             |           |                 |                     |
| Edit:    |                  |                            |             |           |                 |                     |
| Export/Import:                                                                            |                  |                            |             |           |                 |                     |
| Wrap Cell Content:                                                                                                                                                       |                  |                            |             |           |                 |                     |
| id                                                                                                                                                                                                                                                          | nome             | email                      | apelido     | pontuacao | usuario_premium | data_cadastro       |
| 1                                                                                                                                                                                                                                                           | Alice Santos     | alice.santos@senai.com     | GamerAlice  | 1200      | 1               | 2024-12-01 10:15:00 |
| 2                                                                                                                                                                                                                                                           | Bruno Oliveira   | bruno.oliveira@senai.com   | BrunoMaster | 850       | 0               | 2024-12-02 14:30:00 |
| 3                                                                                                                                                                                                                                                           | Carla Mendes     | carla.mendes@senai.com     | CarlaThePro | 1500      | 1               | 2024-12-03 16:45:00 |
| 4                                                                                                                                                                                                                                                           | Diego Costa      | diego.costa@senai.com      | NULL        | 400       | 0               | 2024-12-04 11:20:00 |
| 5                                                                                                                                                                                                                                                           | Elaine Freitas   | elaine.freitas@senai.com   | FreitasHero | 980       | 1               | 2024-12-05 19:00:00 |
| 6                                                                                                                                                                                                                                                           | Fernando Lima    | fernando.lima@senai.com    | F3rl 1m4    | 730       | 0               | 2024-12-06 08:25:00 |
| 7                                                                                                                                                                                                                                                           | Gabriela Souza   | gabriela.souza@senai.com   | NULL        | 0         | 0               | 2024-12-07 13:10:00 |
| 8                                                                                                                                                                                                                                                           | Henrique Barbosa | henrique.barbosa@senai.com | HB MVP      | 2200      | 1               | 2024-12-08 17:55:00 |

**Figura 3.44** – Comando `select` em SQL para extração de dados de todas as colunas.

Crédito: do elaborador, 2025.

Na **Figura 3.45**, observe que, quando for conveniente, você pode selecionar apenas as colunas que lhe interessem no momento:

13 • `select nome, email, pontuacao from gamers;`

| Result Grid                                                                                 |                            |           |
|---------------------------------------------------------------------------------------------|----------------------------|-----------|
| Filter Rows: <input type="text"/>                                                           |                            |           |
| Export:  W |                            |           |
| nome                                                                                        | email                      | pontuacao |
| Alice Santos                                                                                | alice.santos@senai.com     | 1200      |
| Bruno Oliveira                                                                              | bruno.oliveira@senai.com   | 850       |
| Carla Mendes                                                                                | carla.mendes@senai.com     | 1500      |
| Diego Costa                                                                                 | diego.costa@senai.com      | 400       |
| Elaine Freitas                                                                              | elaine.freitas@senai.com   | 980       |
| Fernando Lima                                                                               | fernando.lima@senai.com    | 730       |
| Gabriela Souza                                                                              | gabriela.souza@senai.com   | 0         |
| Henrique Barbosa                                                                            | henrique.barbosa@senai.com | 2200      |

**Figura 3.45** – Comando `select` em SQL para extração de dados das colunas selecionadas.

Crédito: do elaborador, 2025.

alias – Renomeando colunas

É fato que, quando nos deparamos com algumas tabelas, sobretudo em cenários reais, alguns nomes de colunas não são tão claros e intuitivos. Para sanar essa questão, o MySQL possibilita renomear temporariamente as colunas durante a exibição dos resultados de busca (vale ressaltar que esse processo não altera permanentemente o nome real da coluna).

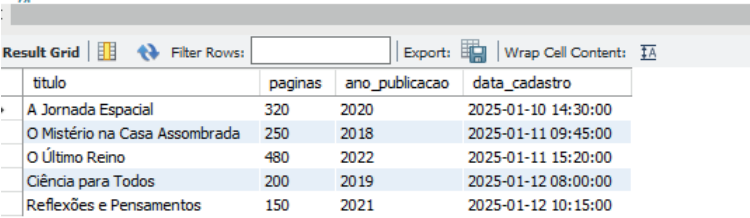
Dessa forma, o comando **alias** permite dar “apelidos” às colunas conforme a necessidade. Vejamos a estrutura de exemplo:

```
-- ESTRUTURA DO COMANDO (RENAMEANDO COLUNAS):
select COLUNA1 as APELIDO1, COLUNA2 as APELIDO2,... from
NOME_DA_TABELA;
```

Vale ressaltar que o **alias** é totalmente opcional e não precisa ser aplicado em todas as colunas.

Alguns exemplos podem facilitar o entendimento de sua importância e de sua funcionalidade. Na **Figura 3.46**, apresentaremos uma consulta sem o uso do **alias**; na **Figura 3.47**, com o seu uso.

```
23 • select titulo, paginas, ano_publicacao, data_cadastro from livros;
```



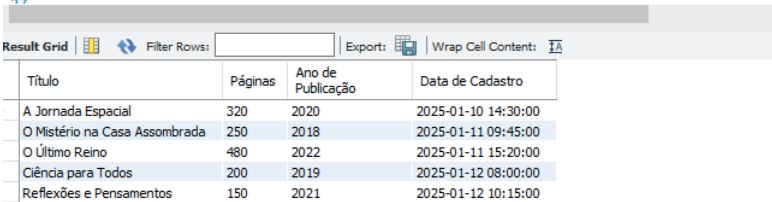
| titulo                        | paginas | ano_publicacao | data_cadastro       |
|-------------------------------|---------|----------------|---------------------|
| A Jornada Espacial            | 320     | 2020           | 2025-01-10 14:30:00 |
| O Mistério na Casa Assombrada | 250     | 2018           | 2025-01-11 09:45:00 |
| O Último Reino                | 480     | 2022           | 2025-01-11 15:20:00 |
| Ciência para Todos            | 200     | 2019           | 2025-01-12 08:00:00 |
| Reflexões e Pensamentos       | 150     | 2021           | 2025-01-12 10:15:00 |

**Figura 3.46** – Comando **select** em SQL para extração de dados das colunas selecionadas, sem o uso de **alias**.

Crédito: do elaborador, 2025.

Note que algumas colunas possuem nomes pouco sugestivos, tais como **ano\_publicacao** e **data\_cadastro**. Portanto, usaremos o **alias** para melhorá-los e para incluir acentuação nas demais colunas.

```
25 • select titulo as Título, paginas as Páginas, ano_publicacao as 'Ano de Publicação',
26 data_cadastro as 'Data de Cadastro' from livros;
```



| Título                        | Páginas | Ano de Publicação | Data de Cadastro    |
|-------------------------------|---------|-------------------|---------------------|
| A Jornada Espacial            | 320     | 2020              | 2025-01-10 14:30:00 |
| O Mistério na Casa Assombrada | 250     | 2018              | 2025-01-11 09:45:00 |
| O Último Reino                | 480     | 2022              | 2025-01-11 15:20:00 |
| Ciência para Todos            | 200     | 2019              | 2025-01-12 08:00:00 |
| Reflexões e Pensamentos       | 150     | 2021              | 2025-01-12 10:15:00 |

**Figura 3.47** – Comando **select** em SQL para extração de dados das colunas selecionadas, com o uso do **alias**.

Crédito: do elaborador, 2025.

## IMPORTANTE

### Aspas em expressões compostas

Para usar expressões compostas como **alias**, você precisa obrigatoriamente colocá-las entre aspas, caso contrário, o MySQL não interpretará de modo correto os espaços. Foi por esse motivo que utilizamos aspas em **Ano de Publicação** e **Data de Cadastro**, na **Figura 3.47**.



where – Filtrando os resultados

Em todas as consultas que fizemos até o momento, obtivemos o resultado de toda a tabela. Contudo, na grande maioria dos cenários, as extrações de dados obedecem a determinados filtros seletores. Por exemplo, imagine uma plataforma de vendas *on-line* (*e-commerce*) em que você está cadastrado: ao entrar em sua conta, o comportamento esperado é que você possa visualizar apenas os seus pedidos, mesmo existindo uma infinidade de outros pedidos feitos por outros usuários nessa plataforma e estando todos armazenados provavelmente na mesma base de dados, não é mesmo?

Vamos a outra situação: imagine que você está em uma plataforma de *streaming* de filmes, acessando a categoria “Documentários”. Com certeza, você espera poder visualizar os filmes da categoria selecionada, correto?

Enfim, todos esses exemplos somente podem ocorrer graças à possibilidade de filtrar os dados desejados, dentro de uma infinidade de outros existentes, por meio do uso da cláusula chamada **where**.

Vamos observar a estrutura do comando **select** com enfoque na cláusula **where**:

```
-- ESTRUTURA DO COMANDO (FILTRANDO RESULTADOS COM WHERE):  
select LISTA_DE_COLUNAS from NOME_DA_TABELA where  
  ( (CONDIÇÃO_FILTRO1) OPERADORES (CONDIÇÃO_FILTRO2) OPERADORES  
  ...);  
  
--EXEMPLOS:  
select * from produtos where preco > 10.0 and quantidade >  
8;  
select nome, email from usuarios where data_cadastro <=  
'2024-05-05';  
select * from estacionamento where saida is null;
```

Algumas observações sobre o uso da cláusula **where** em nossos exemplos:

- O uso do **where** é basicamente associado à seleção de uma coluna com algum dos inúmeros operadores existentes no MySQL (veja os operadores já abordados neste livro);
- Não existe limite para a quantidade de condições usadas em comandos com **where**;
- A junção de múltiplas condições em uma mesma cláusula **where** necessita obrigatoriamente de operadores lógicos de junção, como **and**, **or** e **not**.

Na **Figura 3.48**, vamos observar um resultado de seleção com **where**, selecionando apenas os filmes do gênero “Fantasia” e com nota de avaliação maior que 7,5.

65 • `select * from filmes streaming where genero = 'Fantasia' and avaliacao_usuario > 7.5;`

| id | titulo                                  | ano_lancamento | genero   | classificacao_indicativa | avaliacao_usuario | duracao_minutos | data_adicionado     |
|----|-----------------------------------------|----------------|----------|--------------------------|-------------------|-----------------|---------------------|
| 1  | O Senhor dos Anéis: A Sociedade do Anel | 2001           | Fantasia | 12                       | 10.0              | 178             | 2025-01-05 14:00:00 |
| 12 | Harry Potter e a Pedra Filosofal        | 2001           | Fantasia | Livre                    | 9.6               | 152             | 2025-01-13 12:30:00 |

**Figura 3.48** – Comando **select** em SQL para extração de dados das colunas selecionadas, com o uso do **where**.

Crédito: do elaborador, 2025.

Veremos mais exemplos dessa importante cláusula em consultas com o **select**, sobretudo nos casos de uso que exploraremos no final deste capítulo. Contudo, para abrir o horizonte do(a) aluno(a), mostraremos a seguir uma série de exemplos e possibilidades de uso:

```
-- EXEMPLOS DE USO DO WHERE EM DIFERENTES CENÁRIOS

-- selecionando preços acima de 100 reais
select nome, preco from produtos where preco > 100;

-- selecionando clientes com idade entre 18 e 35 anos
```

```
select nome, idade from clientes where idade between 18 and 35;

-- selecionando funcionários com status diferentes de 'ativo'
select nome, status from funcionarios where status != 'ativo';

-- selecionando produtos que comecem com a letra 'O' em seu nome
select nome from produtos where nome like 'O%';

-- selecionando pedidos realizados no ano de 2023
select id_pedido, data_pedido from pedidos where YEAR(data_pedido) = 2023;

-- selecionando usuários que possuem 'Silva' em seu nome e que estão ativos
select nome from usuarios where nome like '%Silva%' and status = 'ativo';

-- selecionando clientes que residem no estado de São Paulo e que não estão com status <pendente> e nem <excluído>
select nome, endereco from clientes where estado = 'SP' and status not in ('pendente', 'excluído');

-- selecionando fornecedores que não entregaram pedidos nos últimos 30 dias
select nome, data_ultima_entrega from fornecedores where data_ultima_entrega < CURDATE() - INTERVAL 30 day;
```

```

-- selecionando funcionários que não possuem telefone
cadastrado
select nome, departamento from funcionarios where telefone
is null;

-- selecionando funcionários que recebem salário maior que
5000 reais ou os funcionários que trabalham no departamento
de TI que possuem idade maior que 30
select nome, salario, departamento, idade from funcionarios
where salario > 5000 or (departamento = 'TI' and idade > 30);

```

order by – Ordenando os resultados

Você com certeza já se deparou, ao navegar em algum *site* ou aplicativo de compras, com aqueles filtros de classificação que, quando selecionados, alteram a ordenação dos produtos exibidos, como os filtros que ordenam por “Menor Preço”, “Mais Vendidos” ou “Últimos Lançamentos”. É principalmente para esse tipo de contexto que utilizamos o **order by**, ou seja, para selecionarmos o método de ordenação em que os resultados serão exibidos nas consultas.

A estrutura de uso do **order by** é bem simples, sendo associado com outras duas informações: **asc**, para ordenação ascendente, ou **desc**, classificação descendente:

```

-- ESTRUTURA DO COMANDO (ORDENANDO RESULTADO):
select LISTA_DE_COLUNAS from NOME_DA_TABELA order by LISTA_
DE_COLUNAS (ASC ou DESC);

-- EXEMPLOS:
-- ordenando remédios por nome em ordem descendente (do Z
ao A)
select * from remedios order by nome desc;

```

```
-- selecionando anúncios onde o preço está entre 500 e 1500
e ordenando por preço em ordem ascendente (do menor ao maior)
select * from anuncios where preco between 500 and 1500
order by preco asc;

-- ordenando livros primeiro pela data de publicação e
depois pelo autor em ordem ascendente (do mais antigo ao mais
novo, e do A ao Z)
select nome_livro, autor, data_publicacao from livros order
by data_publicacao desc, autor asc;
```

## IMPORTANTE

### Ordenações ascendentes

Se desejar ordenações ascendentes, você pode optar por omitir o **asc**, uma vez que, quando não informado, o MySQL adotará por padrão a classificação ascendente.

Outro ponto importante é que, quando utilizadas múltiplas ordenações, atente-se que o MySQL dará maior prioridade à sequência de colunas utilizada. Por exemplo, se fizermos um “**order by idade, nome**”, o banco de dados fará a ordenação prioritariamente pela idade e, em seguida, pelo nome, o que significa que, se houver duas pessoas com a mesma idade, o critério de classificação entre elas será dado pelo nome em ordem alfabética.

Como de costume, apresentamos um resultado prático do uso do **order by**, sendo possível observar que, no comando da **Figura 3.49**, a seleção dos filmes ordena primeiro aqueles mais bem avaliados (**desc**) e em sequência de ordem de lançamento, do mais antigo ao mais novo (**asc**).

```
67 • select *from filmes streaming order by avaliacao usuario desc, ano lancamento asc;
```

| Result Grid        |                                         |                |                   |                          |                   |                 |                     |
|--------------------|-----------------------------------------|----------------|-------------------|--------------------------|-------------------|-----------------|---------------------|
| Filter Rows:       |                                         |                |                   |                          |                   |                 |                     |
| Edit:              |                                         |                |                   |                          |                   |                 |                     |
| Export/Import:     |                                         |                |                   |                          |                   |                 |                     |
| Wrap Cell Content: |                                         |                |                   |                          |                   |                 |                     |
| id                 | titulo                                  | ano_lancamento | genero            | classificacao_indicativa | avaliacao_usuario | duracao_minutos | data_adicionado     |
| 1                  | O Senhor dos Anéis: A Sociedade do Anel | 2001           | Fantasia          | 12                       | 10.0              | 178             | 2025-01-05 14:00:00 |
| 12                 | Harry Potter e a Pedra Filosofal        | 2001           | Fantasia          | Livre                    | 9.6               | 152             | 2025-01-13 12:30:00 |
| 7                  | Planeta Terra                           | 2006           | Documentário      | Livre                    | 9.5               | 60              | 2025-01-11 08:00:00 |
| 2                  | Star Wars: O Império Contra-Ataca       | 1980           | Ficção Científica | 10                       | 9.4               | 124             | 2025-01-06 10:45:00 |
| 11                 | O Cavaleiro das Trevas                  | 2008           | Ação              | 14                       | 9.0               | 152             | 2025-01-13 10:00:00 |
| 8                  | Forrest Gump: O Contador de Histórias   | 1994           | Drama             | 12                       | 8.8               | 142             | 2025-01-12 11:40:00 |
| 3                  | Titanic                                 | 1997           | Romance           | 12                       | 8.7               | 195             | 2025-01-07 09:30:00 |
| 6                  | Matrix                                  | 1999           | Ação              | 14                       | 8.7               | 136             | 2025-01-10 13:50:00 |
| 9                  | De Volta para o Futuro                  | 1985           | Aventura          | 10                       | 8.5               | 116             | 2025-01-12 14:25:00 |
| 15                 | Gladiador                               | 2000           | Drama             | 16                       | 8.5               | 155             | 2025-01-13 19:40:00 |
| 14                 | Coringa                                 | 2019           | Drama             | 16                       | 8.5               | 122             | 2025-01-13 17:20:00 |
| 4                  | Jurassic Park: O Parque dos Dinossauros | 1993           | Aventura          | Livre                    | 8.1               | 127             | 2025-01-08 15:20:00 |
| 13                 | Os Vingadores                           | 2012           | Ação              | 12                       | 8.0               | 143             | 2025-01-13 14:45:00 |
| 10                 | Avatar                                  | 2009           | Ficção Científica | 12                       | 7.9               | 162             | 2025-01-12 17:30:00 |
| 5                  | It: A Coisa                             | 2017           | Terror            | 16                       | 7.3               | 135             | 2025-01-09 18:10:00 |

**Figura 3.49** – Comando em SQL para seleção e ordenação dos resultados de uma tabela de *streamings* – MySQL Workbench.

Crédito: do elaborador, 2025.

limit – Limitando consultas

Para explicar a utilidade do **limit** (e do **offset**, que veremos a seguir), recorreremos novamente ao exemplo das plataformas de *e-commerce*, já que elas consistem nos casos mais comuns de uso de tais comandos.

Considere a plataforma de *e-commerce* que você mais conhece ou utiliza e pense na quantidade gigantesca de produtos existentes que possivelmente estão cadastrados nos bancos de dados dela. Sem dúvida, é uma grande quantidade. Agora, imagine o que aconteceria se, ao acessar esse *site*, todos esses produtos tivessem que ser carregados na tela do usuário. É muito provável que você tenha chegado à conclusão correta: haveria uma grande e perceptível lentidão no carregamento de todos os produtos.

É para evitar esse tipo de situação que grandes *sites* de busca ou plataformas que lidam com quantidades consideráveis de itens acabam por utilizar o conceito de **paginação**. Ou seja, mesmo que existam mil itens a

serem exibidos, tais aplicações são configuradas para exibir na “página 1” apenas dez itens, por exemplo. Caso o usuário não encontre o que deseja, ele navega até a próxima página e lá pode visualizar outra parte dos itens, fazendo com que o carregamento e a usabilidade do *site* sejam muito mais ágeis e fluidos.

É pelo uso do **limit** que as buscas, mesmo que tenham o potencial de trazer uma grande quantidade de dados, passam a ficar restritas em uma quantidade delimitada, tornando o tempo de extração dos dados extremamente reduzido em relação ao total.

Vamos observar seu uso:

```
-- ESTRUTURA DO COMANDO (LIMITANDO RESULTADOS):
select LISTA_DE_COLUNAS from NOME_DA_TABELA limit
NÚMERO_DE_LINHAS;

-- EXEMPLOS:
-- selecionando os 25 primeiros itens de um menu
select * from menu limit 25;

-- selecionando nome, cpf e email dos 10 primeiros
professores por ordem alfabética
select nome, cpf, email from professores order by nome asc
limit 10

-- selecionando os 15 primeiros carros do tipo sedan
ordenados por preço crescente
select * from carros where categoria = 'SEDAN' order by
preço asc limit 15;
```

Veja na **Figura 3.50** um resultado prático de limitação de resultados, utilizando nosso exemplo de plataforma de *streamings*, em que, com a associação do **order by** + **limit**, conseguimos obter o “*top 5*” filmes mais bem avaliados na plataforma.

69 • `select *from filmes streaming order by avaliacao usuario desc limit 5;`

| Result Grid                                            |    |                                         |                |                   |                          |                   |                 |                     |
|--------------------------------------------------------|----|-----------------------------------------|----------------|-------------------|--------------------------|-------------------|-----------------|---------------------|
| Filter Rows: <input type="text"/>                      |    |                                         |                |                   |                          |                   |                 |                     |
| Edit:  Export/Import:  Wrap Cell Content:  Fetch rows: |    |                                         |                |                   |                          |                   |                 |                     |
|                                                        | id | titulo                                  | ano_lancamento | genero            | classificacao_indicativa | avaliacao_usuario | duracao_minutos | data_adicionado     |
| ▶                                                      | 1  | O Senhor dos Anéis: A Sociedade do Anel | 2001           | Fantasia          | 12                       | 10.0              | 178             | 2025-01-05 14:00:00 |
|                                                        | 12 | Harry Potter e a Pedra Filosofal        | 2001           | Fantasia          | Livre                    | 9.6               | 152             | 2025-01-13 12:30:00 |
|                                                        | 7  | Planeta Terra                           | 2006           | Documentário      | Livre                    | 9.5               | 60              | 2025-01-11 08:00:00 |
|                                                        | 2  | Star Wars: O Império Contra-Ataca       | 1980           | Ficção Científica | 10                       | 9.4               | 124             | 2025-01-06 10:45:00 |
|                                                        | 11 | O Cavaleiro das Trevas                  | 2008           | Ação              | 14                       | 9.0               | 152             | 2025-01-13 10:00:00 |

**Figura 3.50** – Comando em SQL para seleção e limitação dos resultados de uma tabela de *streamings* – MySQL Workbench.

Crédito: do elaborador, 2025.

offset – Paginando consultas

Como vimos no tópico sobre **limit**, o uso de **offset** está frequentemente associado à paginação de resultados. Em cenários nos quais um grande volume de dados precisa ser exibido em páginas, o **limit** define a quantidade de itens por página, enquanto o **offset** determina qual página exibir. Por exemplo, se uma busca retorna resultados em cinco páginas, o **offset** é responsável por selecionar os resultados específicos de cada página, de acordo com a página que o usuário escolheu.

Pense no **offset** como um “ponto de partida” na lista de resultados. Se você quer a página 1, o **offset** será 0 (começando do início da lista); se quiser a página 2, o **offset** será 10, “pulando” os dez primeiros resultados, e assim por diante!

Vamos dar uma olhada na estrutura do comando com o uso do **offset**:

```
-- ESTRUTURA DO COMANDO (PULANDO RESULTADOS):
select LISTA_DE_COLUNAS from NOME_DA_TABELA limit NÚMERO_DE_
LINHAS offset VALOR_INICIAL;

-- EXEMPLOS:
-- selecionando os 5 registros subsequentes a partir do 11º
select nome, data_nascimento, cidade from clientes limit 5
offset 10;
```



```
-- selecionando os próximos 20 produtos, ignorando os 5 mais caros.
select produto, preco from produtos order by preco desc
limit 20 offset 5;
```

Em seguida, apresentaremos um exemplo prático de uso do **offset**. No exemplo utilizado no tópico de **limit**, realizamos um comando para apresentação dos “*top 5*” filmes mais bem avaliados. Agora, dando continuidade ao contexto, utilizaremos o **offset** para apresentar os outros “*5 top*” filmes, como se o usuário da plataforma de *streamings* estivesse acessando a página seguinte dessa lista.

70 • `select *from filmes_streaming order by avaliacao_usuario desc limit 5 offset 5;`

| id | titulo                                | ano_lancamento | genero   | classificacao_indicativa | avaliacao_usuario | duracao_minutos | data_adicionado     |
|----|---------------------------------------|----------------|----------|--------------------------|-------------------|-----------------|---------------------|
| 8  | Forrest Gump: O Contador de Histórias | 1994           | Drama    | 12                       | 8.8               | 142             | 2025-01-12 11:40:00 |
| 3  | Titanic                               | 1997           | Romance  | 12                       | 8.7               | 195             | 2025-01-07 09:30:00 |
| 6  | Matrix                                | 1999           | Ação     | 14                       | 8.7               | 136             | 2025-01-10 13:50:00 |
| 9  | De Volta para o Futuro                | 1985           | Aventura | 10                       | 8.5               | 116             | 2025-01-12 14:25:00 |
| 14 | Coringa                               | 2019           | Drama    | 16                       | 8.5               | 122             | 2025-01-13 17:20:00 |

**Figura 3.51** – Comando em SQL para seleção e limitação dos resultados de uma tabela de *streamings* – MySQL Workbench.

Crédito: do elaborador, 2025.

Ao analisar o resultado mostrado na **Figura 3.51**, o aluno poderá comprovar como o **offset** modifica a saída da consulta, já que a lista dos “*top 5*” filmes resultante é significativamente diferente daquela obtida antes, quando o **offset** não foi aplicado. Essa comparação evidencia sua importância para selecionar diferentes conjuntos de resultados em consultas paginadas.

distinct – Seleção distinta

Em bancos de dados, muitas vezes nos deparamos com situações em que os resultados de nossas consultas contêm linhas duplicadas. Essas duplicatas, em alguns contextos, podem dificultar a análise dos dados e gerar informações indesejadas. É nesse cenário que a cláusula **distinct** se torna

uma ferramenta útil, permitindo que você filtre seus resultados e obtenha apenas os valores únicos e distintos.

Observe a estrutura e seu uso a seguir:

```
-- ESTRUTURA DO COMANDO (REMOVENDO DUPLICATAS):
select distinct NOME_DA_COLUNA from NOME_DA_TABELA;

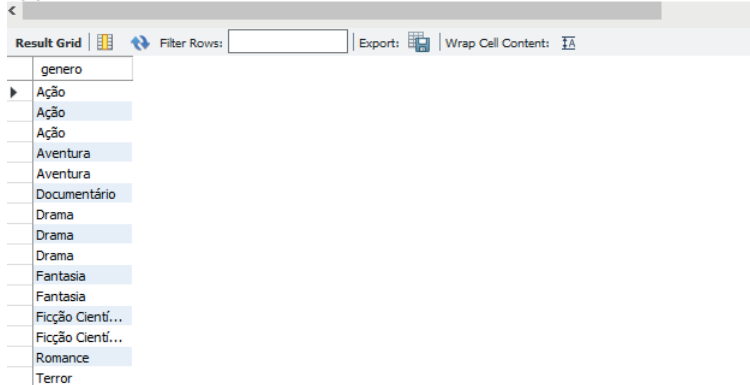
-- EXEMPLOS:
-- selecionando nomes não repetidos de carros, ordenados por
categoria
select distinct nome from carros order by categoria;

-- selecionando combinação única de cidade e estado de
clientes
select distinct cidade, estado from clientes;
```

Para esclarecer melhor os casos de uso do **distinct**, imagine que, em nosso exemplo de plataforma de *streamings*, você deseja saber quais os gêneros (categorias) existentes de filmes disponíveis para exibição na plataforma.

72 • `select genero from filmes_streaming order by genero asc;`

73



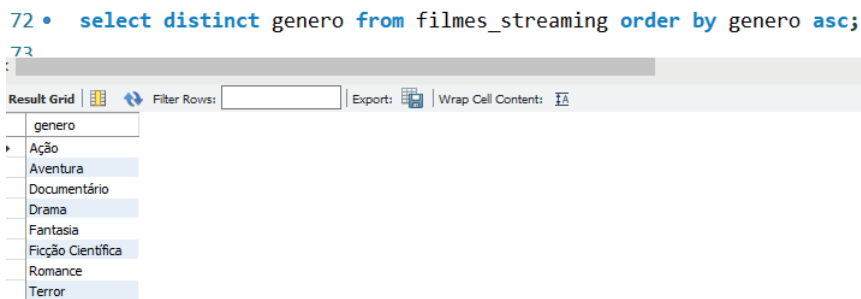
| genero            |
|-------------------|
| Ação              |
| Ação              |
| Ação              |
| Aventura          |
| Aventura          |
| Documentário      |
| Drama             |
| Drama             |
| Drama             |
| Fantasia          |
| Fantasia          |
| Ficção Científica |
| Ficção Científica |
| Romance           |
| Terror            |

**Figura 3.52** – Comando em SQL para seleção dos gêneros de uma tabela de *streamings* – MySQL Workbench.

Crédito: do elaborador, 2025.

Como se pode notar, conseguimos obter a relação de categorias de *streamings* em exibição, classificando-as em ordem alfabética. Contudo, obtivemos também no resultado várias repetições do mesmo gênero, devido ao fato de existir múltiplos *streamings* com a mesma categoria, algo que, para o objetivo inicial da consulta, torna-se indesejável, servindo apenas para “poluir” a informação obtida.

Agora, com a utilização do **distinct**, você poderá entender melhor a importância de seu uso, observando que, para a proposta de obtenção das categorias existentes, essa é a solução que melhor atende ao contexto.



**Figura 3.53** – Comando em SQL para seleção dos gêneros de uma tabela de *streamings*, removendo duplicatas com o uso do **distinct** – MySQL Workbench.

Crédito: do elaborador, 2025.

### Extraíndo dados calculados

Nos tópicos anteriores, abordamos comandos, cláusulas e palavras-chave específicas do MySQL, que possuem uma estrutura predefinida e bem documentada. Neste tópico, seguimos por um caminho diferente: o uso dinâmico do MySQL para gerar dados calculados.

Aqui, não há uma “receita fixa”, pois a extração de dados calculados depende diretamente da estrutura da tabela de origem e do tipo de cálculo necessário. O objetivo principal é mostrar ao(a) aluno(a) o poder do MySQL em criar informações a partir de colunas que não existem originalmente na estrutura da tabela, mas que podem ser derivadas em tempo de execução por meio de cálculos dinâmicos.

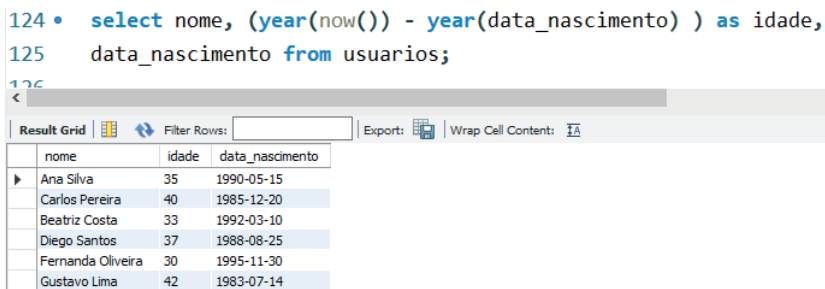
Para ilustrar, vamos refletir sobre uma questão prática: por que, na maioria dos sistemas que gerenciam cadastros de usuários, optamos por armazenar a data de nascimento em vez da idade?

A resposta é simples: a idade de um usuário muda a cada ano. Se armazenássemos a idade diretamente, seria necessário atualizá-la à mão periodicamente, um trabalho desnecessário e propenso a erros. Em vez disso, armazenamos a data de nascimento e calculamos a idade sempre que essa informação for requisitada.

Agora, imagine um cenário em que precisamos exibir a idade de cada usuário em uma consulta SQL, mas essa informação não está disponível diretamente na tabela. Nesse caso, é possível calcular a idade em tempo real com o comando **select**. Essa funcionalidade demonstra como o MySQL pode ser usado para resolver problemas práticos de forma eficiente.

Vamos explorar esse conceito com um exemplo prático para calcular a idade de cada usuário com base na data de nascimento. Na **Figura 3.54**, faremos uma consulta na tabela de usuários de um sistema, coletando os dados das colunas existentes **nome** e **data\_nascimento**. Com base na informação da data de nascimento, geramos dinamicamente uma nova coluna, que exibe o cálculo da idade de cada usuário em tempo real.

```
124 • select nome, (year(now()) - year(data_nascimento) ) as idade,
125     data_nascimento from usuarios;
126
```



| nome              | idade | data_nascimento |
|-------------------|-------|-----------------|
| Ana Silva         | 35    | 1990-05-15      |
| Carlos Pereira    | 40    | 1985-12-20      |
| Beatriz Costa     | 33    | 1992-03-10      |
| Diego Santos      | 37    | 1988-08-25      |
| Fernanda Oliveira | 30    | 1995-11-30      |
| Gustavo Lima      | 42    | 1983-07-14      |

**Figura 3.54** – Comando em SQL para seleção de nome, data de nascimento e idade calculada dinamicamente.

Crédito: do elaborador, 2025.

Alguns apontamentos importantes para esse exemplo:

- Além do nome e da idade, optamos por selecionar também a data de nascimento apenas para que o(a) aluno(a) possa comparar o resultado (considerando que a imagem foi gerada em janeiro de 2025);

- Utilizamos a função **now()** para obter o resultado da data e da hora do momento em que esse comando foi gerado;
- Aplicamos a função **year()** para extrair apenas o ano de datas completas.

### Exercício – *Startup* corretora de criptomoedas II

Você se recorda do exercício que fizemos sobre a corretora de criptomoedas, no tópico **insert – Inserindo dados em tabelas**? Daremos continuidade a ele, com foco na execução de consultas mais aprimoradas, já que agora você domina um pouco mais o **select**.

Caso você não o tenha feito ainda, não se preocupe. Vá até o final do tópico mencionado e execute primeiro o que foi pedido nos requisitos lá presentes.

Considerando nesse momento que você já tenha as entidades prontas de “usuários”, “criptomoedas”, “conta do usuário”, “criptomoedas do usuário” e “transações”, já com dados inseridos, faça as buscas com o **select** de acordo com os requisitos a seguir:

- Faça uma busca para mostrar as três criptomoedas que foram incluídas mais recentemente, usando **order by** e **limit**;
- Pesquise o nome de todas as criptomoedas que possuem alguma compra, ou seja, que foram associadas a algum usuário, porém, evite repetir o nome delas na consulta, usando o **distinct**;
- Faça uma busca para mostrar o nome e a idade da criptomoeda em anos, tomando como base a data de criação e utilizando as funções **year** e **now** para extração de dados calculados;
- Mostre todas as cinco criptomoedas mais antigas de um determinado usuário;
- Mostre todas as transações de venda de um determinado usuário;
- Mostre todas as transações de compra de um determinado usuário;

- Faça uma busca de todos os usuários que possuem uma criptomoeda específica.

## update – Atualizando dados

Depois de aprender a extrair informações de um banco de dados, chegou o momento de compreender como modificá-las. A atualização de dados é uma das funcionalidades mais importantes no gerenciamento de bancos de dados, pois permite corrigir informações, manter registros atualizados e refletir mudanças no mundo real. Seja em um sistema de estoque no qual o preço de um produto precisa ser ajustado devido a uma nova política comercial ou em um cadastro de clientes em que um número de telefone foi alterado, esses são exemplos simples de situações em que o comando **update** entra em cena, realizando mudanças em dados já armazenados.

### FIQUE ALERTA

#### Atenção redobrada

Você já deve ter ouvido esta frase: “com grandes poderes vêm grandes responsabilidades”.

O comando **update** requer atenção redobrada, especialmente quando aplicado em cenários reais (ambientes de produção), para evitar atualizações indesejadas ou generalizadas, que podem comprometer a integridade do banco de dados.

Por exemplo, imagine que você é um desenvolvedor de uma grande empresa de logística responsável por executar milhares de entregas ao redor do mundo. Devido a um contexto específico, você fica incumbido de atualizar o código de rastreio de uma determinada entrega, diretamente via bancos de dados. Contudo, por um descuido seu, em vez de atribuir o novo código apenas para uma entrega, você acaba atribuindo esse código para as dezenas de milhares de entregas existentes em seu banco. Sem dúvida, isso irá gerar muitos problemas, não é mesmo? Portanto, tenha sempre o hábito de revisar muito bem a execução dos comandos de **update**, evitando ao máximo executá-los em cenários de produção.

Observe a estrutura do comando **update**:

```
-- ESTRUTURA DO COMANDO:  
update NOME_DA_TABELA set  
    NOME_DA_COLUNA1 = VALOR1, ...  
    NOME_DA_COLUNA5 = VALOR5  
[where (CONDIÇÕES)];
```

O comando **update** possui uma estrutura relativamente simples, havendo a seleção da tabela e dos valores a serem atualizados, não sendo necessário informar todas as colunas, apenas as que receberão a modificação.

Além disso, temos a cláusula **where** para determinar a lógica de seleção dos registros a serem afetados pela atualização. Embora seja opcional, é sempre recomendado utilizá-la com o **update** para efeito de segurança e garantia da intencionalidade das operações executadas (em alguns casos, o próprio SGBD pode bloquear o uso de **update** sem as cláusulas **where**).

Nos tópicos a seguir, exploraremos em detalhes os diferentes aspectos do comando **update**, por meio do uso de atualizações condicionais e cálculos dinâmicos.

#### Atualizações condicionais

As atualizações condicionais fazem uso da cláusula **where**, possibilitando a utilização de uma infinidade de operações lógicas e outros operadores, de forma muito semelhante ao que já vimos com o **select**. São melhores e mais indicadas do que o uso dos comandos **update** sem as condicionais, já que estabelecem claramente a lógica definida de atualização, de forma intencional.

Vamos observar alguns exemplos de uso a seguir:

```

-- EXEMPLOS (CONDICIONAIS)
-- atualiza o preço do produto com id = 101
update produtos set preco = 29.90 where id = 101;

-- atualiza o status dos clientes com acesso antigo para
status inativo
update clientes set status = 'Inativo' where ultimo_acesso <
'2020-01-01';

-- atualiza um funcionário com determinado cpf para o cargo
de gerente
update funcionarios set cargo = 'Gerente' where cpf =
55577744422;

-- atualiza o estoque de todos os registros para quantidade
= zero e ativo = false
update estoque set quantidade = 0, ativo = false where nome
like '%registro%';

-- atualiza o estoque de todos os produtos com determinados
códigos
update estoque set quantidade = 20, ativo = true where
codigo in (5504,3301,8857,5547,2004,2025);

```

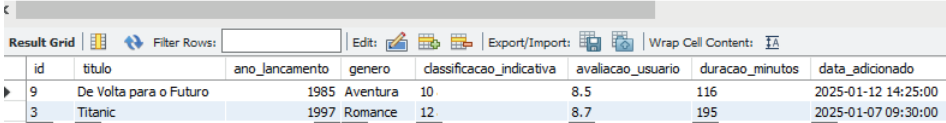
Como de costume, vamos treinar com uma situação prática, utilizando a plataforma de *streamings* que já apresentamos em alguns exemplos anteriores.

Em nosso cenário de treinamento, vamos supor que será necessário alterar as notas de avaliação dos usuários para os filmes *De volta para o futuro* e *Titanic*, respectivamente, com as notas 8,9 e 7,5.

Primeiro, faremos uma consulta antes da modificação mencionada, na qual poderemos constatar que as notas atuais possuem valores distintos: 8,5 e 8,7, respectivamente:



```
74 • select *from filmes_streaming where titulo like "%Futuro"  
75 or titulo like "Titanic" order by titulo asc;  
76
```



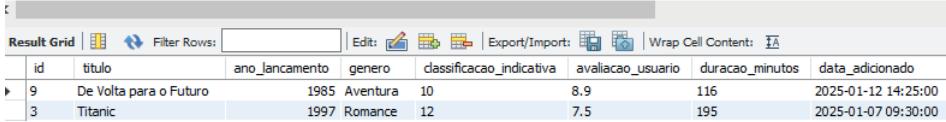
| id | titulo                 | ano_lancamento | genero   | classificacao_indicativa | avaliacao_usuario | duracao_minutos | data_adicionado     |
|----|------------------------|----------------|----------|--------------------------|-------------------|-----------------|---------------------|
| 9  | De Volta para o Futuro | 1985           | Aventura | 10                       | 8.5               | 116             | 2025-01-12 14:25:00 |
| 3  | Titanic                | 1997           | Romance  | 12                       | 8.7               | 195             | 2025-01-07 09:30:00 |

**Figura 3.55** – Comando em SQL para seleção dos filmes a serem atualizados.

Crédito: do elaborador, 2025.

Agora, providenciaremos a atualização dos valores, sendo possível identificar essas alterações com o comando **select**, conforme mostrado na **Figura 3.56**.

```
78 • update filmes_streaming set avaliacao_usuario = 8.9 where id = 9;  
79 • update filmes_streaming set avaliacao_usuario = 7.5 where id = 3;  
80  
81 • select *from filmes_streaming where titulo like "%Futuro"  
82 or titulo like "Titanic" order by titulo asc;  
83
```



| id | titulo                 | ano_lancamento | genero   | classificacao_indicativa | avaliacao_usuario | duracao_minutos | data_adicionado     |
|----|------------------------|----------------|----------|--------------------------|-------------------|-----------------|---------------------|
| 9  | De Volta para o Futuro | 1985           | Aventura | 10                       | 8.9               | 116             | 2025-01-12 14:25:00 |
| 3  | Titanic                | 1997           | Romance  | 12                       | 7.5               | 195             | 2025-01-07 09:30:00 |

**Figura 3.56** – Comando em SQL para atualização dos filmes com o **update**.

Crédito: do elaborador, 2025.

### Atualizações dinâmicas

As atualizações dinâmicas no MySQL são aquelas em que os valores atualizados dependem de cálculos ou operações realizadas diretamente no momento do comando **update**, ou seja, em tempo de execução.

Nos principais cenários em que tais atualizações são demandadas, não há um valor exato para ser atribuído aos registros, diferentemente das operações que fizemos antes. Por exemplo, imagine que uma grande empresa, após realizar os acordos sindicais, chega à conclusão de que fornecerá um reajuste salarial de 5% para todos os funcionários de sua folha de pagamento. Nesse caso, a atualização deve ser proporcional ao valor salarial atual de cada funcionário, não existindo, portanto, um valor fixo.

É por meio das atualizações dinâmicas que cenários como o do relato são facilmente resolvidos, já que elas permitem criar alterações mais flexíveis e adaptáveis, muitas vezes com base em informações já existentes.

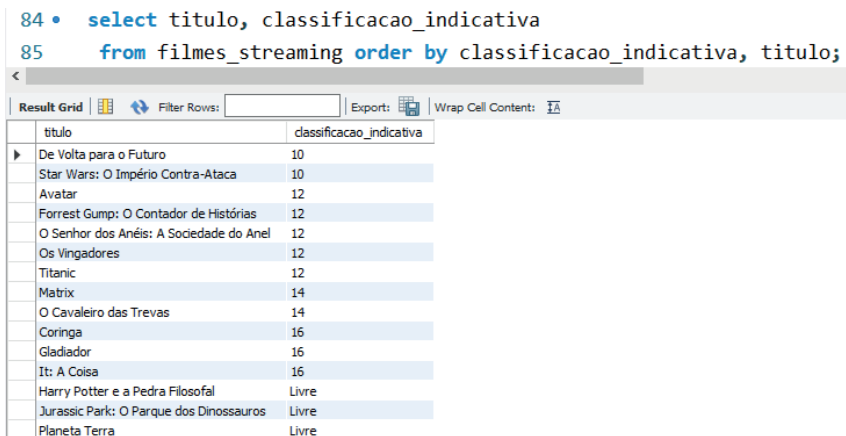
Em vez de simplesmente definir um novo valor fixo, as atualizações dinâmicas utilizam fórmulas, expressões e funções do MySQL para gerar resultados diretamente no comando. Isso é extremamente útil para cenários em que os valores precisam ser ajustados com base em percentuais, datas, incrementos ou outros critérios específicos.

Vamos explorar alguns exemplos para entender como funcionam as atualizações dinâmicas, não havendo um padrão de estrutura, já que em cada caso devemos adotar uma abordagem compatível com o desejado.

Retomando o exemplo de *streamings*, imagine que uma nova regra sobre classificação indicativa de filmes foi criada pelos órgãos reguladores, obrigando que todas as indicações iguais ou acima de 12 anos sejam aumentadas em mais um ano, ou seja, um filme anteriormente indicado para 14 anos se tornará, com a nova regra, indicado para 15 anos e assim por diante, limitado até os 18 anos de idade.

Na **Figura 3.57**, mostraremos o resultado das classificações antes da alteração.

```
84 • select titulo, classificacao_indicativa
85   from filmes_streaming order by classificacao_indicativa, titulo;
```



| titulo                                  | classificacao_indicativa |
|-----------------------------------------|--------------------------|
| De Volta para o Futuro                  | 10                       |
| Star Wars: O Império Contra-Ataca       | 10                       |
| Avatar                                  | 12                       |
| Forrest Gump: O Contador de Histórias   | 12                       |
| O Senhor dos Anéis: A Sociedade do Anel | 12                       |
| Os Vingadores                           | 12                       |
| Titanic                                 | 12                       |
| Matrix                                  | 14                       |
| O Cavaleiro das Trevas                  | 14                       |
| Coringa                                 | 16                       |
| Gladiador                               | 16                       |
| It: A Coisa                             | 16                       |
| Harry Potter e a Pedra Filosofal        | Livre                    |
| Jurassic Park: O Parque dos Dinossauros | Livre                    |
| Planeta Terra                           | Livre                    |

**Figura 3.57** – Comando em SQL para visualização dos filmes e de suas classificações antes da atualização.

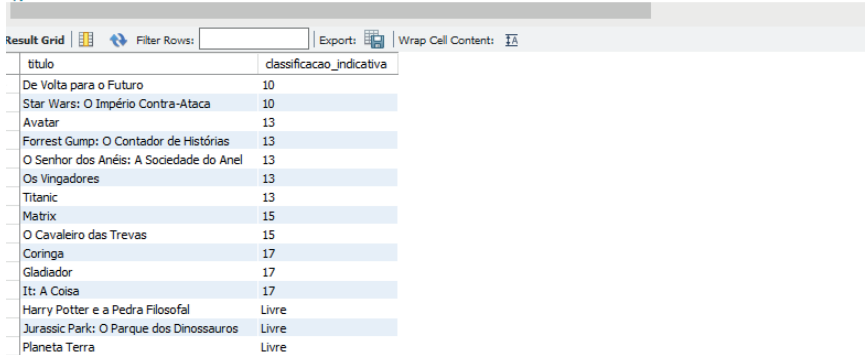
Crédito: do elaborador, 2025.

Agora executaremos o comando de atualização, utilizando como valores a serem atribuídos no **set** a operação de soma com a própria coluna, tornando o processo dinâmico, em vez de definir um valor fixo. Além disso, nos atentamos em definir as condições de exceção no processo de atualização, ou seja, ignorar classificações para menores que 12 anos, livre, ou maiores de 18 anos.

```

86 • update filmes_streaming
87     set classificacao_indicativa = (cast(classificacao_indicativa as unsigned) + 1)
88     where classificacao_indicativa not in ('Livre', '10', '18');
89
90 • select titulo, classificacao_indicativa
91     from filmes_streaming order by classificacao_indicativa, titulo;
92

```



| titulo                                  | classificacao_indicativa |
|-----------------------------------------|--------------------------|
| De Volta para o Futuro                  | 10                       |
| Star Wars: O Império Contra-Ataca       | 10                       |
| Avatar                                  | 13                       |
| Forrest Gump: O Contador de Histórias   | 13                       |
| O Senhor dos Anéis: A Sociedade do Anel | 13                       |
| Os Vingadores                           | 13                       |
| Titanic                                 | 13                       |
| Matrix                                  | 15                       |
| O Cavaleiro das Trevas                  | 15                       |
| Coringa                                 | 17                       |
| Gladiador                               | 17                       |
| It: A Coisa                             | 17                       |
| Harry Potter e a Pedra Filosofal        | Livre                    |
| Jurassic Park: O Parque dos Dinossauros | Livre                    |
| Planeta Terra                           | Livre                    |

**Figura 3.58** – Comandos em SQL para atualização dos filmes de acordo com as novas regras de classificação indicativa.

Crédito: do elaborador, 2025.

Algumas observações sobre esse processo executado, que podem gerar estranheza ao(à) aluno(a):

- Muito provavelmente não é novidade para você que, na maioria das linguagens de programação, somente é possível a execução de operadores matemáticos ou aritméticos se os dados envolvidos são do mesmo tipo. No MySQL não é diferente. Portanto, sendo a coluna **classificacao\_indicativa** uma sequência de caracteres (texto), para conseguirmos utilizar o operador matemático somando + 1, devemos primeiro converter os valores presentes na classificação para o tipo

inteiro positivo. É o que fizemos com o uso do **CAST(classificacao\_indicativa as UNSIGNED)**;

- Observe que, em operações de atualização dinâmica, é possível utilizar o valor atual de uma coluna como parte do cálculo para determinar o novo valor que será atribuído a ela. No exemplo apresentado, aproveitamos o valor existente na coluna **classificacao\_indicativa** para realizar a soma e, em seguida, atualizar a própria coluna com o resultado do cálculo;
- Por fim, para evitar que o processo de atualização afete toda a tabela, utilizamos a condicional **where**, determinando que somente registros com classificações que não sejam “Livres”, “10” ou “18” sejam modificados.

## delete e truncate – Deletando ou limpando dados

Se armazenar e consultar dados são pilares fundamentais de um banco de dados, a capacidade de removê-los ou limpá-los com eficiência não é menos importante. Afinal, os dados em um banco de dados não são estáticos: ao longo do tempo, registros podem se tornar irrelevantes, obsoletos ou simplesmente errados, exigindo sua exclusão.

É nesse contexto que os comandos **delete** e **truncate** entram em cena, oferecendo formas distintas de remover dados. Enquanto o **delete** nos dá a flexibilidade de trabalhar de forma seletiva, removendo apenas registros que atendam a certas condições, o **truncate** surge como uma solução poderosa para limpar tabelas inteiras de maneira rápida e eficiente.

Prepare-se para dominar mais um comando fundamental para a atuação no desenvolvimento com bancos de dados!

delete – Deletando dados

A estrutura do comando **delete** é muito semelhante em alguns pontos ao que já vimos em **select** e **update**, possuindo como principais elementos a indicação do nome da tabela a receber as deleções e as condições de exclusão.

Da mesma forma que com o comando **update**, é muito importante que se tenha total atenção ao realizar operações de deleção, especialmente em ambientes produtivos, sendo sempre recomendado o uso da cláusula **where** para indicar de forma clara e intencional quais os dados que serão removidos.

Observe a seguir a estrutura desse comando:

```
-- ESTRUTURA DO COMANDO:
delete from NOME_DA_TABELA [where (CONDIÇÕES)];

-- EXEMPLOS:
-- remove os produtos com data de validade vencida;
delete from produtos where data_validade <= now();

-- remove as roupas sem estoque e nota de avaliação inferior
a 5
delete from roupas where estoque = 0 and nota < 5;

-- remove os jogos comprados entre um período selecionado
delete from games where data_compra between '2023-01-01' and
'2023-06-09';
```

Vejamos um uso prático, envolvendo o exemplo da plataforma de *streamings* com o qual estamos trabalhando. Suponha que você recebeu a incumbência de remover do acervo de filmes disponíveis todos aqueles que possuam nota de avaliação inferior a 7,5 ou todos os filmes que tenham mais de 30 anos com nota de avaliação inferior a 8,5.

Inicialmente, faremos uma consulta atual para checar os resultados existentes antes de qualquer deleção.

67 • `select *from filmes_streaming order by avaliacao_usuario desc, ano_lancamento asc;`

| id | titulo                                  | ano_lancamento | genero            | classificacao_indicativa | avaliacao_usuario | duracao_minutos | data_adicionado     |
|----|-----------------------------------------|----------------|-------------------|--------------------------|-------------------|-----------------|---------------------|
| 1  | O Senhor dos Anéis: A Sociedade do Anel | 2001           | Fantasia          | 13                       | 10.0              | 178             | 2025-01-05 14:00:00 |
| 12 | Harry Potter e a Pedra Filosofal        | 2001           | Fantasia          | Livre                    | 9.6               | 152             | 2025-01-13 12:30:00 |
| 7  | Planeta Terra                           | 2006           | Documentário      | Livre                    | 9.5               | 60              | 2025-01-11 08:00:00 |
| 2  | Star Wars: O Império Contra-Ataca       | 1980           | Ficção Científica | 10                       | 9.4               | 124             | 2025-01-06 10:45:00 |
| 11 | O Cavaleiro das Trevas                  | 2008           | Ação              | 15                       | 9.0               | 152             | 2025-01-13 10:00:00 |
| 9  | De Volta para o Futuro                  | 1985           | Aventura          | 10                       | 8.9               | 116             | 2025-01-12 14:25:00 |
| 8  | Forrest Gump: O Contador de Histórias   | 1994           | Drama             | 13                       | 8.8               | 142             | 2025-01-12 11:40:00 |
| 6  | Matrix                                  | 1999           | Ação              | 15                       | 8.7               | 136             | 2025-01-10 13:50:00 |
| 15 | Gladador                                | 2000           | Drama             | 17                       | 8.5               | 155             | 2025-01-13 19:40:00 |
| 14 | Coringa                                 | 2019           | Drama             | 17                       | 8.5               | 122             | 2025-01-13 17:20:00 |
| 4  | Jurassic Park: O Parque dos Dinossauros | 1993           | Aventura          | Livre                    | 8.1               | 127             | 2025-01-08 15:20:00 |
| 13 | Os Vingadores                           | 2012           | Ação              | 13                       | 8.0               | 143             | 2025-01-13 14:45:00 |
| 10 | Avatar                                  | 2009           | Ficção Científica | 13                       | 7.9               | 162             | 2025-01-12 17:30:00 |
| 3  | Titanic                                 | 1997           | Romance           | 13                       | 7.5               | 195             | 2025-01-07 09:30:00 |
| 5  | It: A Coisa                             | 2017           | Terror            | 17                       | 7.3               | 135             | 2025-01-09 18:10:00 |

**Figura 3.59** – Comandos em SQL para exibição dos filmes atualmente cadastrados no banco de dados da plataforma de *streaming*.

Crédito: do elaborador, 2025.

Em seguida, realizaremos o comando de deleção, utilizando a estrutura apresentada, incluindo na cláusula **where** as condições de deleção, ou seja, remover todos os filmes com nota menor que 7,5 e os filmes com data de lançamento maior que 30 anos e com nota de avaliação inferior a 8,5.

Basicamente, o comportamento correto que devemos esperar para essas condições é a remoção dos filmes *It: a Coisa* (devido à sua baixa nota de avaliação) e *Titanic* (devido à sua data de lançamento e baixa nota de classificação).

64 • `delete from filmes_streaming where avaliacao_usuario < 7.5 or`  
 65 `( (year(now()) - ano_lancamento > 30) and avaliacao_usuario < 8.5) ;`  
 66  
 67 • `select *from filmes_streaming order by avaliacao_usuario desc, ano_lancamento asc;`

| id | titulo                                  | ano_lancamento | genero            | classificacao_indicativa | avaliacao_usuario | duracao_minutos | data_adicionado     |
|----|-----------------------------------------|----------------|-------------------|--------------------------|-------------------|-----------------|---------------------|
| 1  | O Senhor dos Anéis: A Sociedade do Anel | 2001           | Fantasia          | 13                       | 10.0              | 178             | 2025-01-05 14:00:00 |
| 12 | Harry Potter e a Pedra Filosofal        | 2001           | Fantasia          | Livre                    | 9.6               | 152             | 2025-01-13 12:30:00 |
| 7  | Planeta Terra                           | 2006           | Documentário      | Livre                    | 9.5               | 60              | 2025-01-11 08:00:00 |
| 2  | Star Wars: O Império Contra-Ataca       | 1980           | Ficção Científica | 10                       | 9.4               | 124             | 2025-01-06 10:45:00 |
| 11 | O Cavaleiro das Trevas                  | 2008           | Ação              | 15                       | 9.0               | 152             | 2025-01-13 10:00:00 |
| 9  | De Volta para o Futuro                  | 1985           | Aventura          | 10                       | 8.9               | 116             | 2025-01-12 14:25:00 |
| 8  | Forrest Gump: O Contador de Histórias   | 1994           | Drama             | 13                       | 8.8               | 142             | 2025-01-12 11:40:00 |
| 6  | Matrix                                  | 1999           | Ação              | 15                       | 8.7               | 136             | 2025-01-10 13:50:00 |
| 15 | Gladador                                | 2000           | Drama             | 17                       | 8.5               | 155             | 2025-01-13 19:40:00 |
| 14 | Coringa                                 | 2019           | Drama             | 17                       | 8.5               | 122             | 2025-01-13 17:20:00 |
| 13 | Os Vingadores                           | 2012           | Ação              | 13                       | 8.0               | 143             | 2025-01-13 14:45:00 |
| 10 | Avatar                                  | 2009           | Ficção Científica | 13                       | 7.9               | 162             | 2025-01-12 17:30:00 |
| 3  | Titanic                                 | 1997           | Romance           | 13                       | 7.5               | 195             | 2025-01-07 09:30:00 |

**Figura 3.60** – Comandos em SQL para remoção dos filmes cadastrados no banco de dados da plataforma de *streaming* de acordo com a condicional **where**.

Crédito: do elaborador, 2025.

Algumas observações:

- Observe que utilizamos os operadores de comparação para definir os valores a serem removidos de acordo com a avaliação de satisfação;
- Utilizamos a função **now()** para obtenção de data e hora atual do sistema; quando associada à função de nome **year()**, conseguimos obter os valores no formato anual numérico para execução da subtração entre eles.

truncate – Limpando dados

Da forma mais direta possível, o comando **truncate table** é utilizado para remover todos os registros de uma tabela de forma rápida e eficiente. Diferentemente do **delete**, o **truncate** não possibilita a remoção condicional de dados, ou seja, não pode ser associado a qualquer tipo de cláusula, como o **where**. Desse modo, dizemos que o **truncate** tem sempre atuação global na tabela, afetando todos os registros de uma só vez.

Você pode se questionar qual a diferença da deleção de registros com o **delete** (sem o **where**), em relação ao uso do **truncate**. Em termos práticos, ambos os comandos terão o mesmo efeito, contudo, para tabelas com uma grande quantidade de dados, o comando **truncate** apresenta uma velocidade de deleção muito maior, já que acaba removendo a tabela por completo e recriando-a.

```
-- ESTRUTURA DO COMANDO:
```

```
truncate NOME_DA_TABELA;
```

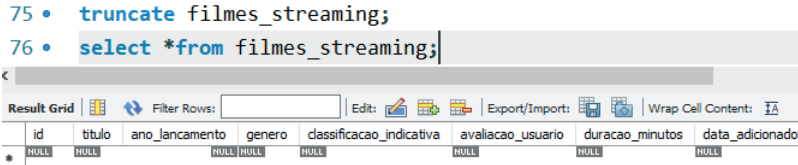
```
-- EXEMPLOS:
```

```
truncate pedidos;
```

```
truncate alarmes;
```

```
truncate usuarios;
```

Como exemplo final de nossa plataforma de *streamings*, faremos a execução do comando **truncate** para limpar toda a tabela, conforme vemos na **Figura 3.61**.



**Figura 3.61** – Comandos em SQL para limpeza e consulta dos filmes cadastrados no banco de dados da plataforma de *streaming*, utilizando o comando **truncate**.

Crédito: do elaborador, 2025.

## Resumo

Quanta coisa vimos até aqui, não é mesmo?

Neste capítulo, alcançamos um dos pontos mais empolgantes e essenciais de nosso estudo: a **manipulação de dados**.

O aluno teve a oportunidade de vivenciar o poder do **CRUD** (**Create**, **Read**, **Update**, **Delete**), um conjunto de operações que está no coração do trabalho com bancos de dados relacionais. E é por meio dessas operações que exploramos como criar, consultar, atualizar e excluir dados, consolidando os fundamentos que sustentam qualquer aplicação prática que faça uso de banco de dados.

Esperamos que, chegando até aqui, em sua visão, o banco de dados tenha deixado de ser apenas uma estrutura teórica para se tornar uma ferramenta prática e indispensável, permitindo que os dados armazenados sejam manipulados de maneira lógica, eficiente e orientada às necessidades reais de um sistema.

Mencionando sistemas reais, conseguimos experimentar neste capítulo uma infinidade de situações e casos de uso, justamente para possibilitar



o alinhamento teórico com o desenvolvimento prático, crucial para o verdadeiro processo de aprendizagem.

Contudo, embora este seja um marco significativo em sua jornada, é importante lembrar que ela ainda não está completa. Nos cenários reais, as operações de leitura, escrita, atualização e exclusão frequentemente envolvem desafios muito mais complexos, como consultas avançadas, integrações e regras de negócio específicas. O próximo capítulo é dedicado a expandir esses horizontes, abordando como lidar com situações mais elaboradas e aproveitando ao máximo o potencial dos bancos de dados.

Não pare por aqui! Ao avançar, você estará cada vez mais próximo de se tornar um excelente desenvolvedor de bancos de dados. Continue sua leitura e prepare-se para o que nos espera no próximo capítulo!

## CAPÍTULO 4

# MANIPULAÇÃO DE DADOS II – MODELO RELACIONAL

**P**arabéns! Você trilhou diversas etapas importantes para entender melhor o mundo do banco de dados, afinal, com seu conhecimento atual, você já é capaz de modelar dados de acordo com requisitos, criar bancos de dados e tabelas, inserir, buscar, atualizar e deletar dados. Sem dúvidas, com esse conhecimento você já possui uma excelente base para prosseguir. Contudo, embora tenhamos abordado diversos exemplos importantes e práticos, em projetos reais você poderá lidar com cenários ainda mais complicados, com diferentes tipos e quantidades de dados. Portanto, para cenários mais complexos, precisamos também de ferramentas mais complexas e robustas.

Neste capítulo, exploraremos novas funções do MySQL, como *triggers* e *procedures*, que aumentarão a complexidade e robustez de suas consultas. Mas não se preocupe, afinal, todo o conhecimento adquirido até aqui será de grande utilidade e com certeza contribuirá para diminuir sua curva de aprendizagem em relação a todos esses itens.

## Funções

Se você já codificou ao menos uma vez, utilizando alguma linguagem de programação, muito provavelmente deve ter se deparado com funções (algumas vezes também chamadas de **métodos**).

Em geral, funções são ferramentas, nativas ou criadas pelo desenvolvedor, que possuem a capacidade de realizar processos os mais diversos possíveis, podendo envolver transformações, operações de conversão, manipulação de valores e execução de regras de negócio.

As funções em MySQL seguem o mesmo princípio, sendo recursos nativos do sistema ou criados pelo usuário, com objetivos e usos definidos, fundamentais para cenários mais complexos, como para extrair o ano de uma coluna de data, converter um texto para maiúsculas, calcular a diferença entre duas datas ou arredondar valores numéricos, situações extremamente comuns.

Essas funções são categorizadas basicamente em **matemáticas** (para operações aritméticas), **manipulação de strings** (para trabalhar com texto), **estatísticas** (para cálculos agregados), **temporais** (para manipulação de datas e horários) e de **conversão** (para transformar tipos de dados), conforme veremos a seguir.

## Matemáticas

São funções destinadas à realização de operações matemáticas, como cálculos trigonométricos, arredondamentos, obtenção de valores absolutos etc. Vejamos no **Quadro 4.1** algumas delas.

**Quadro 4.1** – Funções matemáticas

| Função                  | Descrição                                                                       | Exemplo                                              | Resultado          |
|-------------------------|---------------------------------------------------------------------------------|------------------------------------------------------|--------------------|
| ABS(x)                  | Retorna o valor absoluto de um número.                                          | SELECT ABS(-4);                                      | 4                  |
|                         |                                                                                 | SELECT ABS (934.60);                                 | 934,60             |
| GREATEST (X,Y,Z,...)    | Retorna o maior valor entre a lista, ou o primeiro em ordem alfabética inversa. | SELECT GREATEST (55,8,74,2,44);                      | 74                 |
|                         |                                                                                 | SELECT GREATEST ("zebra","leão","camarão", "coala"); | zebra              |
| LEAST (X,Y,Z,...)       | Retorna o menor valor entre a lista, ou o primeiro em ordem alfabética.         | SELECT LEAST (55,8,74,2,44);                         | 2                  |
|                         |                                                                                 | SELECT LEAST ("zebra", "leão", "camarão","coala");   | camarão            |
| MOD(X,Y)<br>ou<br>X % Y | Retorna o resto da divisão inteira de X por Y.                                  | SELECT MOD(40,2);                                    | 0                  |
|                         |                                                                                 | SELECT MOD(34,4);                                    | 2                  |
|                         |                                                                                 | SELECT 15 %2;                                        | 1                  |
| POW(X,Y)                | Retorna o resultado de X elevado a Y.                                           | SELECT POW(4,2);                                     | 16                 |
|                         |                                                                                 | SELECT POW(3,3);                                     | 9                  |
| ROUND (X, Y )           | Arredonda o valor X com a quantidade de casas decimais Y.                       | SELECT ROUND (23.89,1);                              | 23,9               |
|                         |                                                                                 | SELECT ROUND (23.89);                                | 24                 |
|                         |                                                                                 | SELECT ROUND (1.9834,2);                             | 1,98               |
| SQRT(X)                 | Retorna a raiz quadrada de X.                                                   | SELECT SQRT(16);                                     | 4                  |
|                         |                                                                                 | SELECT SQRT(225);                                    | 15                 |
| TRUNCATE (X,Y)          | Elimina as casas decimais de X para a quantidade determinada de Y.              | SELECT TRUNCATE (23.779,2);                          | 23,77              |
|                         |                                                                                 | SELECT TRUNCATE (23.779,1);                          | 23,7               |
| RAND()                  | Gera um número aleatório entre 0 e 1.                                           | SELECT RAND();                                       | 0,4135775830723136 |
|                         |                                                                                 | SELECT RAND();                                       | 0,9110066778129029 |

Crédito: do elaborador, 2025.

## Manipulação de *strings*

É fato que um dos tipos de dados mais presentes em tabelas de bancos relacionais são os do tipo texto, também denominados de *strings* por inúmeras linguagens de programação. O MySQL oferece múltiplas possibilidades de manipulação envolvendo tais *strings*, o que veremos no **Quadro 4.2**.

**Quadro 4.2** – Funções de manipulação de *strings*

| Função          | Descrição                                                   | Exemplo                                | Resultado      |
|-----------------|-------------------------------------------------------------|----------------------------------------|----------------|
| UPPER(X)        | Converte todos os caracteres de X em letras maiúsculas.     | SELECT UPPER<br>("Loja Online");       | LOJA ONLINE    |
|                 |                                                             | SELECT UPPER<br>("tecnologia");        | TECNOLOGIA     |
| LOWER(X)        | Converte todos os caracteres de X em letras minúsculas.     | SELECT LOWER<br>("Loja Online");       | loja online    |
|                 |                                                             | SELECT LOWER<br>("TECNOLOGIA");        | tecnologia     |
| CONCAT<br>(X,Y) | Concatena a <i>string</i> Y em X.                           | SELECT CONCAT<br>("banco_", "dados");  | banco_dados    |
|                 |                                                             | SELECT CONCAT<br>("ABC", "DEFG");      | ABCDEFGF       |
| LENGTH(X)       | Retorna o comprimento de uma <i>string</i> .                | SELECT LENGTH<br>("dados");            | 5              |
|                 |                                                             | SELECT LENGTH<br>("sol");              | 3              |
| LEFT(X,Y)       | Retorna a quantidade Y de caracteres à esquerda de X.       | SELECT LEFT<br>("promessa", 2);        | pr             |
|                 |                                                             | SELECT LEFT<br>("banco de dados", 5);  | banco          |
| RIGHT(X,Y)      | Retorna a quantidade Y de caracteres à direita de X.        | SELECT RIGHT<br>("promessa", 2);       | sa             |
|                 |                                                             | SELECT RIGHT<br>("banco de dados", 5); | dados          |
| LTRIM(X)        | Elimina os espaços em branco à esquerda da <i>string</i> X. | SELECT LTRIM<br>("nosso lar");         | "nosso lar"    |
|                 |                                                             | SELECT LTRIM<br>(" híbrido 2025 ");    | "híbrido 2025" |

(continua)

(continuação)

| Função                                           | Descrição                                                                                                                 | Exemplo                                      | Resultado          |
|--------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|----------------------------------------------|--------------------|
| RTRIM(X)                                         | Elimina os espaços em branco à direita da <i>string</i> X.                                                                | SELECT RTRIM<br>(" nosso lar");              | " nosso lar"       |
|                                                  |                                                                                                                           | SELECT RTRIM<br>(" híbrido 2025 ");          | " híbrido<br>2025" |
| TRIM(X)                                          | Elimina os espaços em branco à esquerda ou direita da <i>string</i> X.                                                    | SELECT TRIM(" nosso<br>lar");                | "nosso lar"        |
|                                                  |                                                                                                                           | SELECT TRIM<br>(" híbrido 2025 ");           | "híbrido 2025"     |
| SUBSTRING<br>(X,Y)<br>ou<br>SUBSTRING<br>(X,Y,Z) | Retorna a <i>string</i> X cortada com base na quantidade de caracteres Y. Se Z for informado, define o tamanho retornado. | SELECT SUBSTRING<br>("subdimensionado",4);   | dimensionado       |
|                                                  |                                                                                                                           | SELECT SUBSTRING<br>("aquário",5);           | rio                |
|                                                  |                                                                                                                           | SELECT SUBSTRING<br>("subdimensionado",4,2); | di                 |
| REPLACE<br>(X,Y,Z)                               | Substitui o elemento Y, pelo Z, dentro da <i>string</i> X.                                                                | SELECT REPLACE<br>("bola","o","a");          | bala               |
|                                                  |                                                                                                                           | SELECT REPLACE<br>("prefixo","pre","");      | fixo               |
| LOCATE(X,Y)                                      | Retorna a posição da <i>string</i> X em Y.                                                                                | SELECT LOCATE<br>("o","morango");            | 2                  |
|                                                  |                                                                                                                           | SELECT LOCATE<br>("doce","lar doce lar");    | 5                  |

Crédito: do elaborador, 2025.

## Estatísticas

As funções estatísticas são essenciais para a análise de dados em bancos de dados, permitindo consolidá-los e transformá-los em informações úteis, seja para gerar relatórios do total de vendas realizadas durante um período em uma empresa, calcular a média de resultados de avaliações de uma escola ou até mesmo para encontrar desvios padrões para fomentar estudos estatísticos.

As funções estatísticas possuem papel fundamental, conforme apresentamos no **Quadro 4.3**.

Quadro 4.3 – Funções estatísticas

| Função   | Descrição                                                | Exemplo                           |
|----------|----------------------------------------------------------|-----------------------------------|
| MAX(X)   | Retorna o maior valor dos resultados da coluna X.        | SELECT MAX(CUSTO) FROM VENDAS;    |
|          |                                                          | SELECT MAX(NOTA) FROM AVALIACOES; |
| MIN(X)   | Retorna o menor valor dos resultados da coluna X.        | SELECT MIN(CUSTO) FROM VENDAS;    |
|          |                                                          | SELECT MIN(NOTA) FROM AVALIACOES; |
| SUM(X)   | Soma os resultados da coluna X.                          | SELECT SUM(CUSTO) FROM VENDAS;    |
|          |                                                          | SELECT SUM(NOTA) FROM AVALIACOES; |
| AVG(X)   | Calcula a média dos resultados da coluna X.              | SELECT AVG(CUSTO) FROM VENDAS;    |
|          |                                                          | SELECT AVG(NOTA) FROM AVALIACOES; |
| STD(X)   | Retorna o desvio padrão dos resultados da coluna X.      | SELECT STD(CUSTO) FROM VENDAS;    |
|          |                                                          | SELECT STD(NOTA) FROM AVALIACOES; |
| COUNT(*) | Conta a quantidade de linhas retornadas em uma consulta. | SELECT COUNT(*) FROM USUARIOS;    |
|          |                                                          | SELECT COUNT(*) FROM MAQUINAS;    |

Crédito: do elaborador, 2025.

## Conversão

Outra situação muito comum, ao se utilizar qualquer linguagem de programação que contenha tipagem de dados, é a necessidade de conversão de uma variável de um tipo para outro. Dessa forma, o MySQL apresenta duas possibilidades de conversão, mencionadas no **Quadro 4.4**.

Quadro 4.4 – Funções de conversão

| Função               | Descrição                                                                 | Exemplo                                          | Resultado           |
|----------------------|---------------------------------------------------------------------------|--------------------------------------------------|---------------------|
| CAST<br>(X AS TIPO)  | Converte X no tipo de dado indicado (não suporta conjunto de caracteres). | SELECT CAST<br>("4445" AS UNSIGNED);             | 4445                |
|                      |                                                                           | SELECT CAST<br>("2020-01-01" AS DATETIME);       | 2020-01-01 00:00:00 |
| CONVERT<br>(X, TIPO) | Converte X no tipo de dado indicado (suporta conjunto de caracteres).     | SELECT CONVERT<br>("4445", UNSIGNED);            | 4445                |
|                      |                                                                           | SELECT CONVERT<br>("Olá, mundo!" USING utf8mb4); | "Olá, mundo!"       |

Crédito: do elaborador, 2025.

## Temporais

Além das *strings*, outro tipo de dado muito frequente de se utilizar nas mais diversas aplicações é o que envolve unidade de medidas de tempo. Para auxiliar a trabalhar com eles, temos disponíveis inúmeras funções, conforme o **Quadro 4.5**.

**Quadro 4.5** – Funções temporais

| Função            | Descrição                                                                                  | Exemplo                                         | Resultado              |
|-------------------|--------------------------------------------------------------------------------------------|-------------------------------------------------|------------------------|
| NOW()             | Retorna a data e hora atual.                                                               | SELECT NOW();                                   | 2024-12-25<br>12:15:54 |
|                   |                                                                                            | SELECT NOW();                                   | 2025-01-08<br>02:55:00 |
| CURDATE()         | Retorna a data atual.                                                                      | SELECT CURDATE();                               | 2024-12-25             |
|                   |                                                                                            | SELECT CURDATE();                               | 2025-01-08             |
| CURTIME()         | Retorna o horário atual.                                                                   | SELECT CURTIME();                               | 12:15:54               |
|                   |                                                                                            | SELECT CURTIME();                               | 02:55:00               |
| DATEDIFF<br>(X,Y) | Retorna a diferença em dias entre as datas X e Y (se Y for maior que X, retorna negativo). | SELECT DATEDIFF<br>("2024-08-04","2024-08-14"); | -10                    |
|                   |                                                                                            | SELECT DATEDIFF<br>("2024-10-04","2024-10-01"); | 3                      |
| YEAR(X)           | Seleciona o ano da data X.                                                                 | SELECT YEAR("1991-11-11");                      | 1991                   |
|                   |                                                                                            | SELECT YEAR("2020-06-12");                      | 2020                   |
| MONTH(X)          | Seleciona o mês da data X                                                                  | SELECT MONTH("1991-11-11");                     | 11                     |
|                   |                                                                                            | SELECT MONTH("2020-06-12");                     | 6                      |
| DAY(X)            | Seleciona o dia da data X.                                                                 | SELECT DAY("1991-11-11");                       | 11                     |
|                   |                                                                                            | SELECT DAY("2020-06-12");                       | 12                     |

(continua)



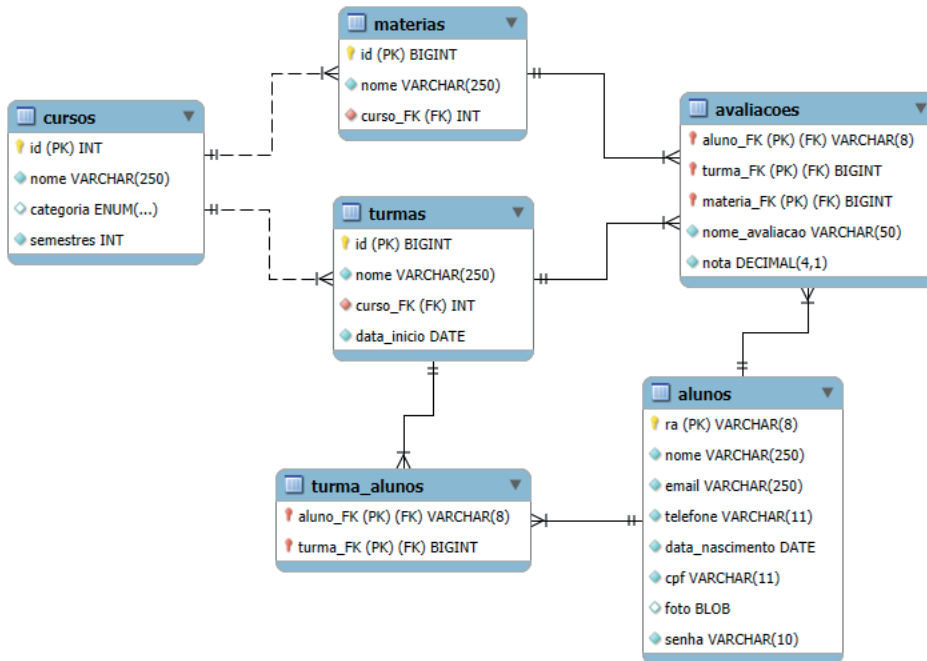
(continuação)

| Função           | Descrição                                                        | Exemplo                                      | Resultado    |
|------------------|------------------------------------------------------------------|----------------------------------------------|--------------|
| DAYOFWEEK<br>(X) | Retorna o dia da semana da data X (1: domingo, 2: segunda etc.). | SELECT DAYOFWEEK("1991-11-11");              | 2            |
|                  |                                                                  | SELECT DAYOFWEEK("2020-06-12");              | 6            |
| HOUR(X)          | Seleciona a hora do horário ou data-hora de X.                   | SELECT HOUR("1991-11-11 22:15:02");          | 22           |
|                  |                                                                  | SELECT HOUR("07:22:03");                     | 7            |
| MINUTE(X)        | Seleciona os minutos do horário ou data-hora de X.               | SELECT MINUTE("1991-11-11 22:15:02");        | 15           |
|                  |                                                                  | SELECT MINUTE("07:22:03");                   | 22           |
| SECOND(X)        | Seleciona os segundos do horário ou data-hora de X.              | SELECT SECOND("1991-11-11 22:15:02");        | 2            |
|                  |                                                                  | SELECT SECOND("07:22:03");                   | 3            |
| PERIOD_DIFF(X,Y) | Retorna os meses entre dois períodos X e Y.                      | SELECT PERIOD_DIFF(199106,199101);           | 5            |
|                  |                                                                  | SELECT PERIOD_DIFF(202001,202101);           | -12          |
| DATE_FORMAT(X,Y) | Retorna a data X no formato Y.                                   | SELECT DATE_FORMAT("1986-01-08","%d/%m/%Y"); | 08/01/1986   |
|                  |                                                                  | SELECT DATE_FORMAT("1986-01-08","%M/%Y");    | January/1986 |

Crédito: do elaborador, 2025.

## Exemplos e casos de uso – Plataforma educacional

Uma plataforma educacional foi desenvolvida para proporcionar a digitalização do processo de cadastramento de alunos, gerenciamento de notas por parte dos docentes e gestão das matérias existentes em cada curso de uma unidade escolar. Utilizando a associação entre diversas entidades, essa plataforma apresenta uma estrutura bem-definida entre alunos, cursos, matérias e avaliações, sendo possível identificá-la no diagrama UML da **Figura 4.1**.



**Figura 4.1** – Diagrama UML de uma plataforma educacional.

Crédito: do elaborador, 2025.

Embora possua um conjunto de associações bem intuitivas, vamos detalhar alguns pontos principais:

- As entidades “cursos” possuem um relacionamento de **um para muitos** (1 para N) tanto com “materias”, pois um curso geralmente possui várias matérias, quanto com “turmas”, já que é natural que diversos cursos formem inúmeras turmas;
- Os alunos são cadastrados em sua respectiva tabela e associados às turmas de que fazem parte por meio de outra entidade, denominada “turma\_alunos”, havendo a possibilidade de que o mesmo aluno possa estar matriculado em mais de uma turma;
- Finalmente, as avaliações devem estar sempre associadas a um conjunto de informações, compostas por aluno, matéria e turma, além, é claro, das notas e nomes das avaliações.

**IMPORTANTE****Contexto simplificado**

**Em um sistema educacional real, outras entidades provavelmente seriam criadas. Contudo, para dar mais enfoque à utilização das funções e facilitar o entendimento do(a) aluno(a), optamos por simplificar o contexto.**

Para visualizar algumas das avaliações já cadastradas, executaremos a consulta demonstrada na **Figura 4.2**.

| aluno_FK | turma_FK | materia_FK | nome_avaliacao   | nota |
|----------|----------|------------|------------------|------|
| RA000001 | 1        | 1          | Prova 1          | 8.5  |
| RA000001 | 1        | 2          | Trabalho 1       | 9.5  |
| RA000001 | 1        | 3          | Prova 2          | 8.0  |
| RA000001 | 9        | 14         | Projeto Dinâmico | 8.8  |
| RA000002 | 1        | 1          | Prova 1          | 9.0  |
| RA000002 | 1        | 2          | Trabalho 1       | 8.0  |
| RA000002 | 1        | 3          | Prova 2          | 9.2  |
| RA000002 | 7        | 11         | Prova 1          | 9.5  |
| RA000003 | 1        | 1          | Prova 1          | 7.5  |
| RA000003 | 1        | 2          | Trabalho 1       | 7.0  |
| RA000003 | 1        | 3          | Prova 2          | 7.8  |
| RA000003 | 7        | 11         | Prova 1          | 8.2  |
| RA000004 | 3        | 6          | Projeto Prático  | 8.5  |
| RA000004 | 5        | 8          | Prova 1          | 7.8  |
| RA000004 | 10       | 15         | Prova Prática    | 8.9  |
| RA000005 | 3        | 6          | Projeto Prático  | 9.0  |
| RA000005 | 5        | 8          | Prova 1          | 8.2  |
| RA000005 | 10       | 15         | Prova Prática    | 8.4  |
| RA000006 | 5        | 7          | Trabalho Final   | 8.3  |

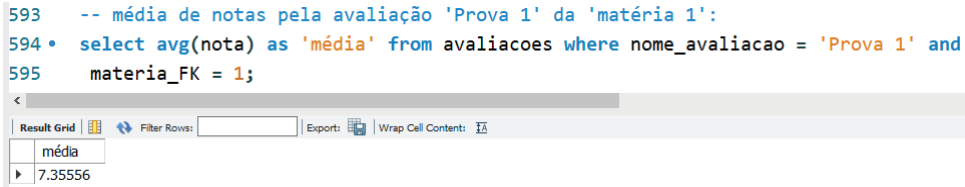
**Figura 4.2** – Consulta das avaliações cadastradas na plataforma educacional.

Crédito: do elaborador, 2025.

Seguindo sua trajetória desde os primeiros capítulos deste livro, você provavelmente não teria nenhuma dificuldade em exibir os resultados listados na **Figura 4.2** em uma ordenação diferente ou então filtrá-los seguindo uma lógica específica, não é mesmo? Contudo, o que você faria se fosse necessário identificar qual a média de notas dos alunos de uma determinada turma? Ou visualizar a maior nota de uma matéria? É nesse contexto que veremos a seguir exemplos de demandas e suas resoluções mediante o uso de algumas das principais funções disponíveis no MySQL.

### Média de notas

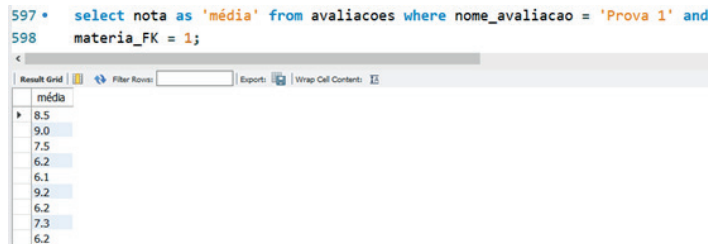
Supondo que um professor queira visualizar a média de notas tiradas por seus alunos em diferentes turmas para uma determinada prova e matéria, podemos aplicar nesse caso a função **avg**, seguida dos respectivos filtros para atendimento do contexto de busca. Vejamos um exemplo na **Figura 4.3**.



**Figura 4.3** – Consulta da média de notas em determinada avaliação e matéria.

Crédito: do elaborador, 2025.

Como você pode observar, aplicamos a função **avg()** na coluna de nota e, opcionalmente, renomeamos o seu resultado para “média”, indicando por meio do uso da cláusula **where** que desejamos obter a informação apenas referente à avaliação “Prova 1”, pertencente à “matéria 1”. Contudo, para comprovação da veracidade da média calculada, equivalente a “7.35556”, faremos uma consulta para obter os valores de notas nas mesmas condições de busca, conforme se observa na **Figura 4.4**.

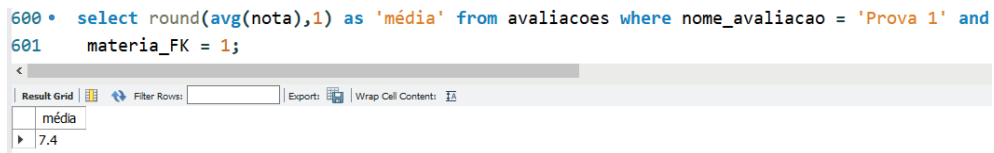


**Figura 4.4** – Notas obtidas pelos alunos na “Prova 1” e “matéria 1” da plataforma educacional.

Crédito: do elaborador, 2025.

Por meio da busca exibida na **Figura 4.4**, conseguimos obter o resultado de nove notas, sendo respectivamente: 8.5, 9.0, 7.5, 6.2, 6.1, 9.2, 6.2, 7.3, 6.2. Somando-se todas elas e dividindo pela sua quantidade, obtemos  $66.2 : 9 = 7.35556$ , ou seja, comprovamos a veracidade da média calculada pelo MySQL.

Entretanto, em um cenário escolar não é muito comum o uso de notas com essa precisão de casas decimais (7.35556), não é mesmo? Faremos agora a exemplificação do uso de outra função muito importante para exibição de resultados quando o assunto é arredondamento: a função **round**. Veja o exemplo da **Figura 4.5**.



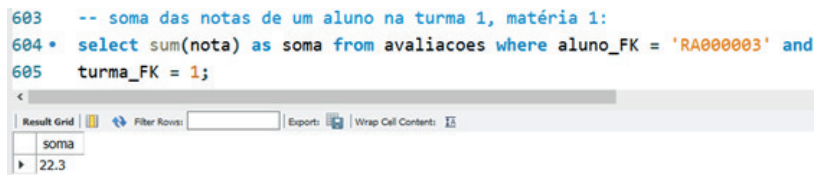
**Figura 4.5** – Arredondamento das notas obtidas pelos alunos na “Prova 1” e “matéria 1” da plataforma educacional.

Crédito: do elaborador, 2025.

Observe que, com o uso da função **round**, obtivemos a média arredondada para “7.4”, ou seja, com apenas uma casa decimal. Isso ocorre porque aplicamos diretamente à saída da função **avg** o arredondamento desejado, utilizando o **round** com o parâmetro 1, que indica a precisão de uma única casa decimal.

Soma de notas

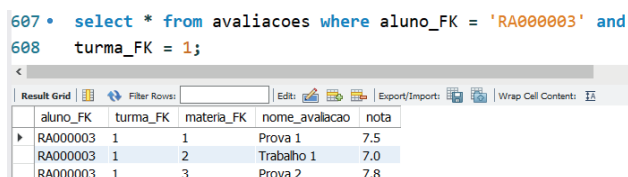
Dando continuidade ao nosso exemplo anterior, vamos supor que esse mesmo professor que faz uso da plataforma educacional necessite saber a soma das notas de um determinado aluno. Usaremos nesse caso a função **sum**, conforme a **Figura 4.6**.



**Figura 4.6** – Soma das notas obtidas por um determinado aluno da “turma 1” na plataforma educacional.

Crédito: do elaborador, 2025.

A seguir, faremos a comprovação do resultado, realizando uma nova consulta sem o uso da função de soma **sum**, conforme mostrado na **Figura 4.7**.



**Figura 4.7** – Notas obtidas por um determinado aluno da “turma 1” na plataforma educacional.

Crédito: do elaborador, 2025.

Finalmente, para comprovação dos resultados, faremos a soma dos valores de nota obtidos na consulta anterior, somando-se, portanto,  $7.5 + 7.0 + 7.8 = 22.3$ , ou seja, o mesmo valor obtido com o uso da função de soma.

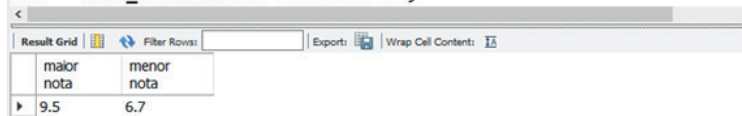
### Notas mínimas e máximas

Nesse mesmo contexto, vamos supor que um professor tenha a necessidade de saber quais foram a maior e a menor notas obtidas pelos seus alunos em um determinado trabalho escolar. Exemplificaremos agora as funções **max** e **min**, capazes de obter os maiores e os menores valores de uma coluna em uma determinada tabela, representadas na **Figura 4.8**.

```

613 -- maior e menor nota do trabalho 1
614 • select max(nota) as 'maior nota', min(nota) as 'menor nota'
615 from avaliacoes where materia_FK = 2 and turma_FK = 1 and
616 nome_avaliacao = 'Trabalho 1';

```



| maior nota | menor nota |
|------------|------------|
| 9.5        | 6.7        |

**Figura 4.8** – Maior e menor notas obtidas pela “turma 1”, “matéria 2”, na avaliação “Trabalho 1” da plataforma educacional.

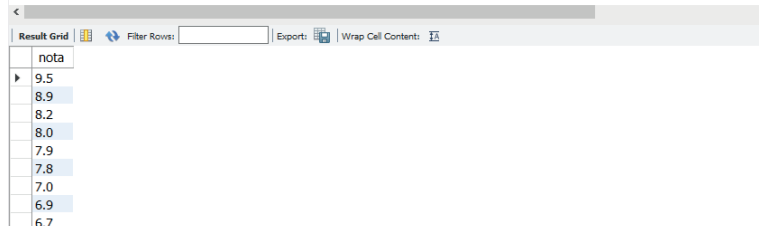
Crédito: do elaborador, 2025.

E, para não perdermos o costume, faremos uma comprovação dos resultados, melhorando assim o entendimento do aluno. Faremos na **Figura 4.9** a exibição de todas as notas com a mesma condição de busca, apenas adicionando uma ordenação descendente (do maior ao menor), para facilitar a visualização dos resultados.

```

618 • select nota from avaliacoes where materia_FK = 2 and turma_FK = 1 and
619 nome_avaliacao = 'Trabalho 1' order by nota desc;

```



| nota |
|------|
| 9.5  |
| 8.9  |
| 8.2  |
| 8.0  |
| 7.9  |
| 7.8  |
| 7.0  |
| 6.9  |
| 6.7  |

**Figura 4.9** – Notas obtidas pela “turma 1”, “matéria 2”, na avaliação “Trabalho 1” da plataforma educacional.

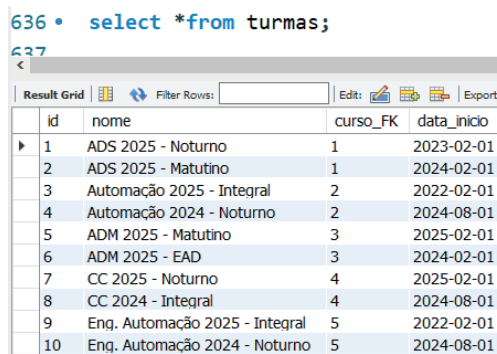
Crédito: do elaborador, 2025.

Como você pode verificar, a maior e a menor notas exibidas na **Figura 4.9** são exatamente as mesmas obtidas anteriormente com o uso das funções **max** e **min** (9.5 e 6.7), facilitando muito a obtenção desse tipo de informação.

### Cálculo de datas

Em nosso último exemplo, imagine que seja necessário identificar quantos semestres já foram concluídos em cada turma. Observe que, ao fazer uma busca na tabela de turmas, essa informação de semestres concluídos não está presente por padrão, conforme podemos identificar na **Figura 4.10**.

```
636 • select *from turmas;
```



| id | nome                           | curso_FK | data_inicio |
|----|--------------------------------|----------|-------------|
| 1  | ADS 2025 - Noturno             | 1        | 2023-02-01  |
| 2  | ADS 2025 - Matutino            | 1        | 2024-02-01  |
| 3  | Automação 2025 - Integral      | 2        | 2022-02-01  |
| 4  | Automação 2024 - Noturno       | 2        | 2024-08-01  |
| 5  | ADM 2025 - Matutino            | 3        | 2025-02-01  |
| 6  | ADM 2025 - EAD                 | 3        | 2024-02-01  |
| 7  | CC 2025 - Noturno              | 4        | 2025-02-01  |
| 8  | CC 2024 - Integral             | 4        | 2024-08-01  |
| 9  | Eng. Automação 2025 - Integral | 5        | 2022-02-01  |
| 10 | Eng. Automação 2024 - Noturno  | 5        | 2024-08-01  |

**Figura 4.10** – Turmas cadastradas na plataforma educacional.

Crédito: do elaborador, 2025.

Embora a informação requerida não esteja presente, podemos calculá-la tomando como base a diferença entre a data atual e a data de início de cada turma. Para isso, usaremos a função **period\_diff** para viabilizar o cálculo da diferença entre as datas referidas. No entanto, é importante considerar que a função **period\_diff** tem como retorno a diferença entre datas em meses, sendo necessário, portanto, que façamos o cálculo de conversão em semestres.

### IMPORTANTE

#### **date\_format**

É fundamental explicar que a função **period\_diff** tem um comportamento peculiar: ela precisa receber as datas no formato “AnoMês”, ou seja, a data de “2025-01-25” deve virar “202501”.

Usaremos então outra função chamada **date\_format** para adequar as datas a serem utilizadas.

```

647 • select id, nome, curso_FK, data_inicio,
648       curdate() as hoje,
649       round( period_diff(
650           date_format(curdate(), '%Y%m'),
651           date_format(data_inicio, '%Y%m')
652       ) / 6) as semestres from turmas;
653

```

| id | nome                           | curso_FK | data_inicio | hoje       | semestres |
|----|--------------------------------|----------|-------------|------------|-----------|
| 1  | ADS 2025 - Noturno             | 1        | 2023-02-01  | 2025-01-25 | 4         |
| 2  | ADS 2025 - Matutino            | 1        | 2024-02-01  | 2025-01-25 | 2         |
| 3  | Automação 2025 - Integral      | 2        | 2022-02-01  | 2025-01-25 | 6         |
| 4  | Automação 2024 - Noturno       | 2        | 2024-08-01  | 2025-01-25 | 1         |
| 5  | ADM 2025 - Matutino            | 3        | 2025-02-01  | 2025-01-25 | 0         |
| 6  | ADM 2025 - EAD                 | 3        | 2024-02-01  | 2025-01-25 | 2         |
| 7  | CC 2025 - Noturno              | 4        | 2025-02-01  | 2025-01-25 | 0         |
| 8  | CC 2024 - Integral             | 4        | 2024-08-01  | 2025-01-25 | 1         |
| 9  | Eng. Automação 2025 - Integral | 5        | 2022-02-01  | 2025-01-25 | 6         |
| 10 | Eng. Automação 2024 - Noturno  | 5        | 2024-08-01  | 2025-01-25 | 1         |

**Figura 4.11** – Semestres calculados para as turmas cadastradas na plataforma educacional.

Crédito: do elaborador, 2025.

Provavelmente, você deve ter se espantado com a quantidade de funções aplicadas para obter-se essa simples informação, então vamos a alguns comentários:

- Optamos por imprimir a data do momento da pesquisa para que o(a) aluno(a) consiga ter uma referência na comparação, ou seja, considere a data de hoje como “2025-01-25”. Utilizamos a função **curdate()** para coletá-la;
- A função **period\_diff** necessita que as datas a serem comparadas estejam em um formato “AnoMês” (juntos e sem caracteres separadores), portanto, utilizamos a função **date\_format** para que se possa formatar nesse padrão;
- Em seguida, após o cálculo retornado em meses pela função **period\_diff**, podemos dividi-los por 6 para obter os semestres;
- Finalmente, para evitar resultados com casas decimais, utilizamos a função **round** para executar o arredondamento final antes da exibição.



## Conclusão

Como o(a) aluno(a) pode perceber, dependendo do contexto, o uso de funções tem importância fundamental, possibilitando a obtenção de informações relevantes de forma confiável e fácil.

Sugerimos que você explore mais funções, além das apresentadas nesses exemplos, pois isso certamente o ajudará em sua atuação como desenvolvedor.

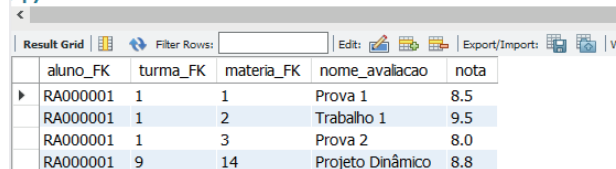
## Associação de tabelas

Até este momento, esperamos que você tenha se conscientizado da importância dos relacionamentos e associações entre tabelas quando o assunto é banco relacional. Afinal, em quase todos os casos de uso que abordamos, utilizamos algum tipo de associação e cardinalidade para atender às regras de negócio e assegurar a consistência da aplicação. Contudo, é possível que o(a) aluno(a) também tenha percebido que, nos casos em que essas associações são feitas, a visibilidade das informações das chaves estrangeiras durante consultas torna-se um pouco “simplificada”.

Para ilustrar esse contexto, vamos retomar o exemplo da plataforma de ensino. Se realizarmos uma consulta simples na tabela de avaliações existente nessa plataforma, podemos visualizar o resultado mostrado na **Figura 4.12**.

```
46 • select *from avaliacoes limit 4;
```

```
47
```



| aluno_FK | turma_FK | materia_FK | nome_avaliacao   | nota |
|----------|----------|------------|------------------|------|
| RA000001 | 1        | 1          | Prova 1          | 8.5  |
| RA000001 | 1        | 2          | Trabalho 1       | 9.5  |
| RA000001 | 1        | 3          | Prova 2          | 8.0  |
| RA000001 | 9        | 14         | Projeto Dinâmico | 8.8  |

**Figura 4.12** – Algumas avaliações registradas na plataforma educacional.

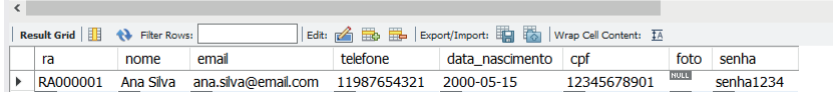
Crédito: do elaborador, 2025.

Observe que, por meio da pesquisa realizada, conseguimos obter diversas informações, tais como nome, nota, turma, matéria, e RA do aluno. Porém, como obter, nessa mesma consulta, informações adicionais sobre o aluno, como nome e *e-mail*?

Considerando que com o **select** utilizado temos disponível apenas a chave estrangeira do aluno (RA), se necessitarmos obter mais informações sobre ele, teríamos que realizar uma consulta separada. Vejamos um exemplo na **Figura 4.13**.

56 • **select \*from** alunos **where** ra = 'RA000001';

57



| ra       | nome      | email               | telefone    | data_nascimento | cpf         | foto | senha     |
|----------|-----------|---------------------|-------------|-----------------|-------------|------|-----------|
| RA000001 | Ana Silva | ana.silva@email.com | 11987654321 | 2000-05-15      | 12345678901 |      | senha1234 |

**Figura 4.13** – Informações da tabela de alunos para um determinado RA.

Crédito: do elaborador, 2025.

Contudo, agir dessa forma se tornaria inviável em situações que envolvessem uma grande quantidade de dados, não é mesmo? No entanto, não se preocupe, pois é exatamente nesse contexto que os comandos **join** nos oferecem a solução perfeita.

De maneira geral, os comandos do tipo **join**, embora possuam inúmeras variações, apresentam em comum a capacidade de realizar a união de associações entre tabelas, permitindo trazer os dados existentes entre elas. Considerando que a maioria das aplicações minimamente complexas possui algum tipo de associação, podemos presumir seu nível de importância e utilidade, o que veremos detalhadamente nos tópicos a seguir.

## inner join – Correspondência bidirecional

O **inner join** é uma variante do comando **join** que une tabelas, ou seja, também permite a consulta de valores unificados entre diferentes entidades. A diferença é que ele retorna apenas os dados que possuem correspondência em ambas as tabelas (correspondência **bidirecional**).

Vejamos exemplos práticos para melhor compreensão. Observe no bloco de código a seguir a estrutura de uso desse comando em SQL.

```
-- ESTRUTURA DO COMANDO:
select COLUNA1,COLUNA2,... from NOME_DA_TABELA1
  inner join NOME_DA_TABELA2 on COLUNA_TABELA1_FK =
COLUNA_TABELA2_PK;

-- EXEMPLOS:
-- exibe os filmes e as informações de seu diretor
select nome_filme, duracao, categoria, nome_diretor,
nascimento_diretor from filmes inner join diretores on
diretor_FK = diretor_id;

-- exibe todos os dados da tabela de automóveis e marcas
associados
select * from automoveis inner join marcas on marca_FK =
marca_id;

-- exibe dados de projetos e clientes associados em um
período, ordenando
select p.nome, p.data, p.valor, c.nome, c.cpf, c.email from
projetos p
  inner join clientes c on p.cliente_FK = c.id
where p.data >= '2025-01-01' and p.data <= '2025-02-02'
order by p.data, c.nome asc;
```

Vamos pontuar alguns itens importantes apresentados no bloco de código:

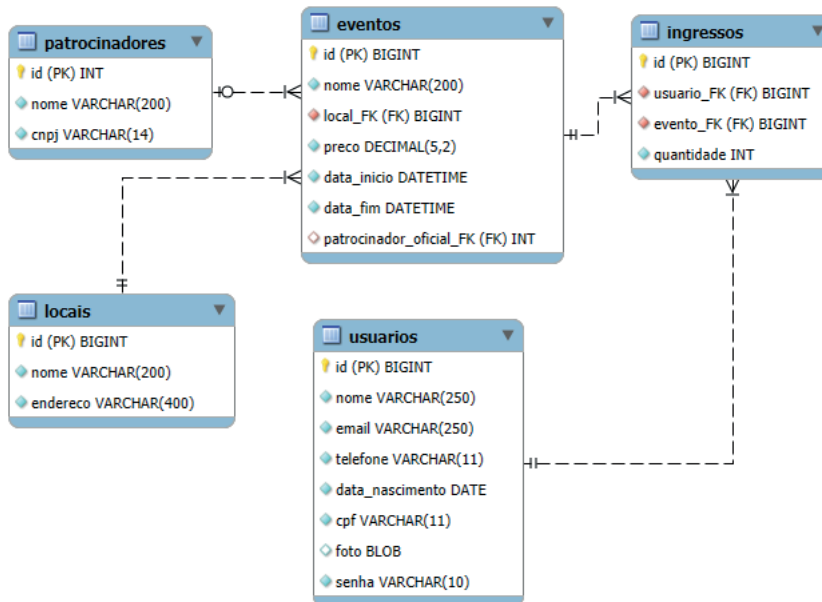
- Observe que a estrutura do comando **inner join** é relativamente simples, necessitando apenas da indicação da tabela a ser associada e da forma como essa associação será realizada, o que acaba representando, em outras palavras, a indicação das chaves estrangeiras e primárias envolvidas nessa relação;

- Em nosso primeiro exemplo de uso, envolvendo filmes e diretores, note que estamos indicando a tabela a ser associada após o **inner join** e, em seguida, fazemos a indicação da lógica de relação após o **on**. Ou seja, informamos que a coluna “diretor\_FK”, chave estrangeira presente na tabela de filmes, está diretamente relacionada com a coluna “diretor\_id”, existente na tabela de diretores;
- Observe também que, no último exemplo do bloco de código, envolvendo projetos e clientes, conseguimos constatar que o uso do **inner join** não afeta em nada a possibilidade de uso de outros comandos e cláusulas que já estudamos até aqui, sendo possível, portanto, a realização de filtros e ordenações em seus resultados.

## IMPORTANTE

### Especificação de tabela

Um fator muito importante a ser considerado em associações com o **join** é a existência de colunas com o mesmo nome nas tabelas em que são associadas. Vejamos um exemplo simples na **Figura 4.14**.



**Figura 4.14** – Diagrama UML de uma plataforma de eventos.

Crédito: do elaborador, 2025.

No diagrama UML, os atributos “id” e “nome” existem em diversas entidades. Nessa situação, ao tentarmos um **join** entre as entidades de “eventos” e “locais”, sem renomearmos as tabelas, ocorrerá o erro mostrado na **Figura 4.15**.

```
739 • select *from eventos inner join locais
740 on local_FK = id;
741
```



Output

Action Output

# Time Action Message

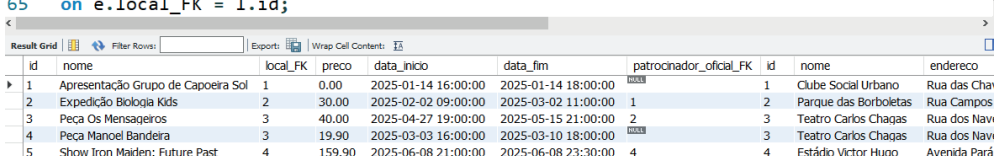
63 15:57:44 select \*from eventos inner join locais on local\_FK = id LIMIT 0, 500 Error Code: 1052. Column 'id' in on clause is ambiguous

**Figura 4.15** – Comando SQL com erro de ambiguidade – MySQL Workbench.

Crédito: do elaborador, 2025.

Isso acontece porque o MySQL não identifica qual tabela deve ser usada no **join**, já que a coluna “id” existe em ambas. Para corrigir, especifique a tabela no **join**, usando o nome completo ou um *alias*. Vejamos o exemplo de correção na **Figura 4.16**.

```
59 -- opção usando o nome completo da tabela:
60 • select *from eventos inner join locais
61 on eventos.local_FK = locais.id;
62
63 -- opção usando apelidos(alias):
64 • select *from eventos e inner join locais l
65 on e.local_FK = l.id;
```



|   | id | nome                               | local_FK | preco  | data_inicio         | data_fim            | patrocinador_oficial_FK | id | nome                  | endereco     |
|---|----|------------------------------------|----------|--------|---------------------|---------------------|-------------------------|----|-----------------------|--------------|
| 1 | 1  | Apresentação Grupo de Capoeira Sol | 1        | 0.00   | 2025-01-14 16:00:00 | 2025-01-14 18:00:00 | 1                       | 1  | Clube Social Urbano   | Rua das Char |
| 2 | 2  | Expedição Biologia Kids            | 2        | 30.00  | 2025-02-02 09:00:00 | 2025-03-02 11:00:00 | 1                       | 2  | Parque das Borboletas | Rua Campos   |
| 3 | 3  | Peca Os Mensageiros                | 3        | 40.00  | 2025-04-27 19:00:00 | 2025-05-15 21:00:00 | 2                       | 3  | Teatro Carlos Chagas  | Rua dos Navi |
| 4 | 4  | Peca Manoel Bandeira               | 3        | 19.90  | 2025-03-03 16:00:00 | 2025-03-10 18:00:00 | 2                       | 3  | Teatro Carlos Chagas  | Rua dos Navi |
| 5 | 5  | Show Iron Maiden: Future Past      | 4        | 159.90 | 2025-06-08 21:00:00 | 2025-06-08 23:30:00 | 4                       | 4  | Estádio Victor Hugo   | Avenida Pará |

**Figura 4.16** – Comando SQL e resultado com o uso de **inner join** – MySQL Workbench.

Crédito: do elaborador, 2025.

O **inner join** é muito útil em bancos de dados relacionais, especialmente em cenários com múltiplas tabelas associadas. No entanto, essa não é a única opção disponível para executar a união entre tabelas. Vamos entender a peculiaridade que o difere dos demais.

O comando **inner join** traz apenas os dados que possuem correspondência bidirecional (existem em ambas as tabelas), ou seja, ignoram, por exemplo, associações não obrigatórias, quando nulas.

Vamos analisar um exemplo na prática. Observe o resultado exibido na **Figura 4.17**, sobre os eventos da plataforma mencionada.

```
'67 * select * from eventos;
```

|   | id | nome                               | local_FK | preco  | data_inicio         | data_fim            | patrocinador_oficial_FK |
|---|----|------------------------------------|----------|--------|---------------------|---------------------|-------------------------|
| 1 | 1  | Apresentação Grupo de Capoeira Sol | 1        | 0.00   | 2025-01-14 16:00:00 | 2025-01-14 18:00:00 |                         |
| 2 | 2  | Expedição Biologia Kids            | 2        | 30.00  | 2025-02-02 09:00:00 | 2025-03-02 11:00:00 | 1                       |
| 3 | 3  | Peça Os Mensageiros                | 3        | 40.00  | 2025-04-27 19:00:00 | 2025-05-15 21:00:00 | 2                       |
| 4 | 4  | Peça Manoel Bandeira               | 3        | 19.90  | 2025-03-03 16:00:00 | 2025-03-10 18:00:00 |                         |
| 5 | 5  | Show Iron Maiden: Future Past      | 4        | 159.90 | 2025-06-08 21:00:00 | 2025-06-08 23:30:00 | 4                       |

**Figura 4.17** – Comando SQL e resultado exibindo os eventos disponíveis na plataforma – MySQL Workbench.

Crédito: do elaborador, 2025.

Na **Figura 4.17**, pode-se ver que existem cinco eventos cadastrados nessa plataforma e alguns deles não possuem nenhum patrocinador oficial (“patrocinador\_oficial\_FK” nulo), pois essa associação é opcional.

Na **Figura 4.18**, usaremos o comando **inner join** para unir os resultados da entidade de patrocinadores.

```
'69 * select * from eventos e inner join patrocinadores p
'70 on e.patrocinador_oficial_FK = p.id;
```

|   | id | nome                          | local_FK | preco  | data_inicio         | data_fim            | patrocinador_oficial_FK | id | nome              | cnpj           |
|---|----|-------------------------------|----------|--------|---------------------|---------------------|-------------------------|----|-------------------|----------------|
| 2 | 2  | Expedição Biologia Kids       | 2        | 30.00  | 2025-02-02 09:00:00 | 2025-03-02 11:00:00 | 1                       | 1  | Nova Energia Ltda | 12345678000123 |
| 3 | 3  | Peça Os Mensageiros           | 3        | 40.00  | 2025-04-27 19:00:00 | 2025-05-15 21:00:00 | 2                       | 2  | Tech Inova S.A.   | 98765432000198 |
| 5 | 5  | Show Iron Maiden: Future Past | 4        | 159.90 | 2025-06-08 21:00:00 | 2025-06-08 23:30:00 | 4                       | 4  | Food & Drinks Co. | 33445566000177 |

**Figura 4.18** – Comando SQL e resultado exibindo os eventos disponíveis com o **inner join** em patrocinadores na plataforma de eventos – MySQL Workbench.

Crédito: do elaborador, 2025.

A **Figura 4.18** demonstra o comportamento do **inner join**, trazendo as informações de cada patrocinador. Entretanto, observe também que **foram removidos os resultados de eventos que não continham associação com a tabela unida** (id 1 e 4), que, por não apresentarem patrocinador oficial, acabam ficando fora da exibição devido ao uso do **inner join**.

## FIQUE ALERTA

### Quantidade de resultados

O uso do **inner join** pode naturalmente diminuir a quantidade de resultados exibidos em relação ao total, devido ao fato de desconsiderar registros que não

possuem o relacionamento bidirecional entre as entidades unidas. Em situações em que seja exatamente essa a intenção, isto é, que seja necessário obter apenas os resultados que fazem uso do relacionamento, ele se torna o comando perfeito. Caso contrário, recomendamos o uso de outra variante **join**.

Agora que você já identificou o funcionamento e as principais características do **inner join**, vamos explorar as outras variantes desse tipo de comando nos tópicos seguintes.

## left join – Correspondência unidirecional à esquerda

O comando **left join**, também denominado **left outer join**, tem função muito parecida com o **inner join** que acabamos de ver, sendo também capaz de trazer as informações existentes em associações entre entidades.

A peculiaridade do **left join**, entretanto, está na sua capacidade de retornar tanto os dados com correspondência em ambas as tabelas quanto aqueles que possuem correspondência apenas na tabela à “esquerda” da relação definida pela cláusula **on**.

Antes de abordarmos os exemplos, vamos à estrutura do comando em SQL com **left join**, sendo basicamente a mesma do **inner Join**.

```
-- ESTRUTURA DO COMANDO:
select COLUNA1,COLUNA2,... from NOME_DA_TABELA1
  left join NOME_DA_TABELA2 on COLUNA_TABELA1_FK =
COLUNA_TABELA2_PK;

-- EXEMPLOS:
/* exhibe os filmes e as informações de seu diretor,
   exibindo inclusive os filmes que não possuem diretor */
select nome_filme, duracao, categoria, nome_diretor,
nascimento_diretor
  from filmes left join diretores on diretor_FK = diretor_id;

/* exhibe todos os dados da tabela de automóveis e laudos
   cautelares associados, exibindo inclusive os automóveis que
   não possuem laudo cautelar */
```

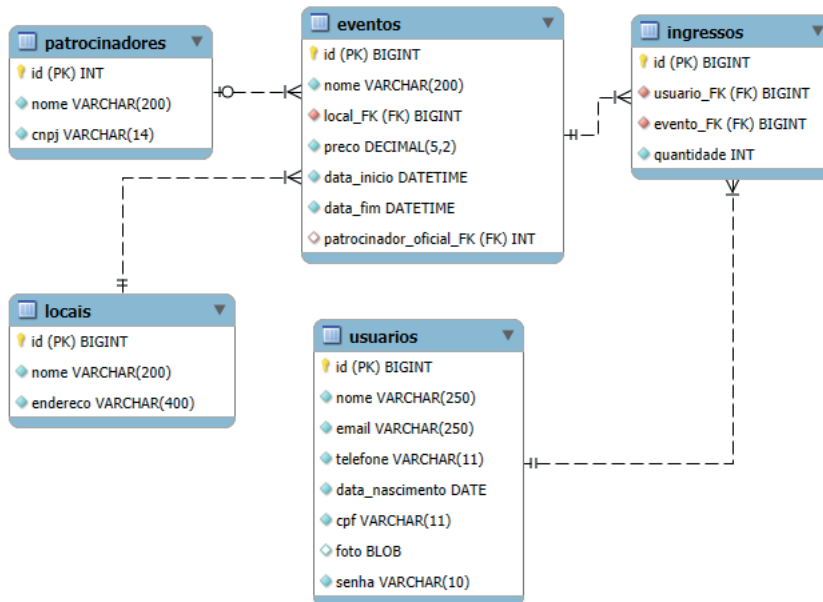
```

select * from automoveis a left join cautelar c
on a.cautelar_FK = c.id;

/* exibe dados de projetos e clientes associados em um
período, exibindo inclusive os projetos que não possuem
cliente */
select p.nome, p.data, p.valor, c.nome, c.cpf, c.email from
projetos p
left join clientes c on p.cliente_FK = c.id
where p.data >= '2025-01-01' and p.data <= '2025-02-02'
order by p.data, c.nome asc;

```

De certa forma, em cenários que envolvem apenas associações mandatórias, isto é, com chaves estrangeiras não nulas, o resultado de buscas com **inner join** ou **left join** serão exatamente iguais. Isso não ocorre em casos que envolvem associações opcionais, como veremos na **Figura 4.19**, retomando o exemplo da plataforma de eventos.



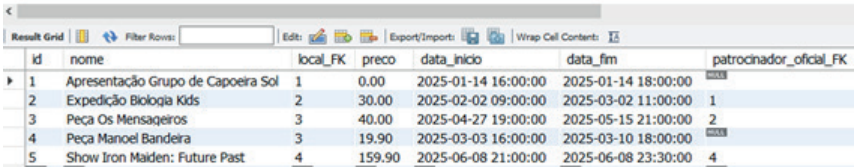
**Figura 4.19** – Diagrama UML de uma plataforma de eventos.

Crédito: do elaborador, 2025.



Note que a associação entre eventos e patrocinadores não é obrigatória, possibilitando a existência de eventos com essa chave estrangeira nula, conforme se vê na **Figura 4.20**.

```
'67 • select * from eventos;
```



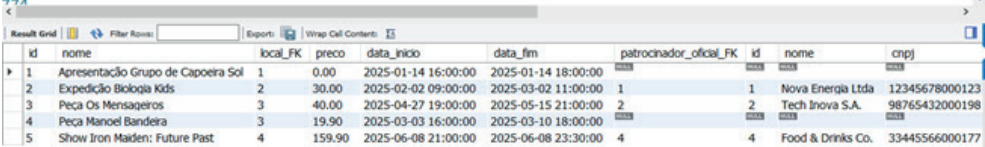
| id | nome                               | local_FK | preco  | data_inicio         | data_fim            | patrocinador_oficial_FK |
|----|------------------------------------|----------|--------|---------------------|---------------------|-------------------------|
| 1  | Apresentação Grupo de Capoeira Sol | 1        | 0.00   | 2025-01-14 16:00:00 | 2025-01-14 18:00:00 |                         |
| 2  | Expedição Biologia Kids            | 2        | 30.00  | 2025-02-02 09:00:00 | 2025-03-02 11:00:00 | 1                       |
| 3  | Peça Os Mensageiros                | 3        | 40.00  | 2025-04-27 19:00:00 | 2025-05-15 21:00:00 | 2                       |
| 4  | Peça Manoel Bandeira               | 3        | 19.90  | 2025-03-03 16:00:00 | 2025-03-10 18:00:00 |                         |
| 5  | Show Iron Maiden: Future Past      | 4        | 159.90 | 2025-06-08 21:00:00 | 2025-06-08 23:30:00 | 4                       |

**Figura 4.20** – Comando SQL e resultado exibindo os eventos disponíveis na plataforma de eventos – MySQL Workbench.

Crédito: do elaborador, 2025.

Agora, faremos o uso do **left join** entre os eventos e os patrocinadores e poderemos identificar melhor o comportamento desse comando, conforme se vê na **Figura 4.21**.

```
772 • select * from eventos e left join patrocinadores p  
773 on e.patrocinador_oficial_FK = p.id;
```



| id | nome                               | local_FK | preco  | data_inicio         | data_fim            | patrocinador_oficial_FK | id | nome              | cnpj           |
|----|------------------------------------|----------|--------|---------------------|---------------------|-------------------------|----|-------------------|----------------|
| 1  | Apresentação Grupo de Capoeira Sol | 1        | 0.00   | 2025-01-14 16:00:00 | 2025-01-14 18:00:00 |                         |    |                   |                |
| 2  | Expedição Biologia Kids            | 2        | 30.00  | 2025-02-02 09:00:00 | 2025-03-02 11:00:00 | 1                       | 1  | Nova Energia Ltda | 12345678000123 |
| 3  | Peça Os Mensageiros                | 3        | 40.00  | 2025-04-27 19:00:00 | 2025-05-15 21:00:00 | 2                       | 2  | Tech Inova S.A.   | 98765432000198 |
| 4  | Peça Manoel Bandeira               | 3        | 19.90  | 2025-03-03 16:00:00 | 2025-03-10 18:00:00 |                         |    |                   |                |
| 5  | Show Iron Maiden: Future Past      | 4        | 159.90 | 2025-06-08 21:00:00 | 2025-06-08 23:30:00 | 4                       | 4  | Food & Drinks Co. | 33445566000177 |

**Figura 4.21** – Comando SQL e resultado exibindo os eventos disponíveis com o **left join** em patrocinadores na plataforma de eventos – MySQL Workbench.

Crédito: do elaborador, 2025.

Pela **Figura 4.21**, podemos concluir que o uso do comando **left join** traz tanto os eventos com patrocinadores (correspondência bidirecional) quanto os eventos sem nenhum tipo de patrocinador (correspondência à esquerda), sendo, portanto, muito útil em cenários em que se deseja obter os resultados independentemente de estarem ou não associados.

## right join – Correspondência unidirecional à direita

O comando **right join**, também denominado **right outer join**, tem função muito parecida com o **left join** que acabamos de ver, sendo também capaz de trazer as informações existentes em associações entre entidades.

Como o(a) aluno(a) já deve imaginar, a peculiaridade do **right join**, entretanto, está na sua capacidade de retornar tanto os dados com correspondência em ambas as tabelas quanto aqueles que possuem correspondência apenas na tabela à “direita” da relação definida pela cláusula **on**.

Vamos à estrutura do comando em SQL com o **right join**, sendo basicamente a mesma dos demais.

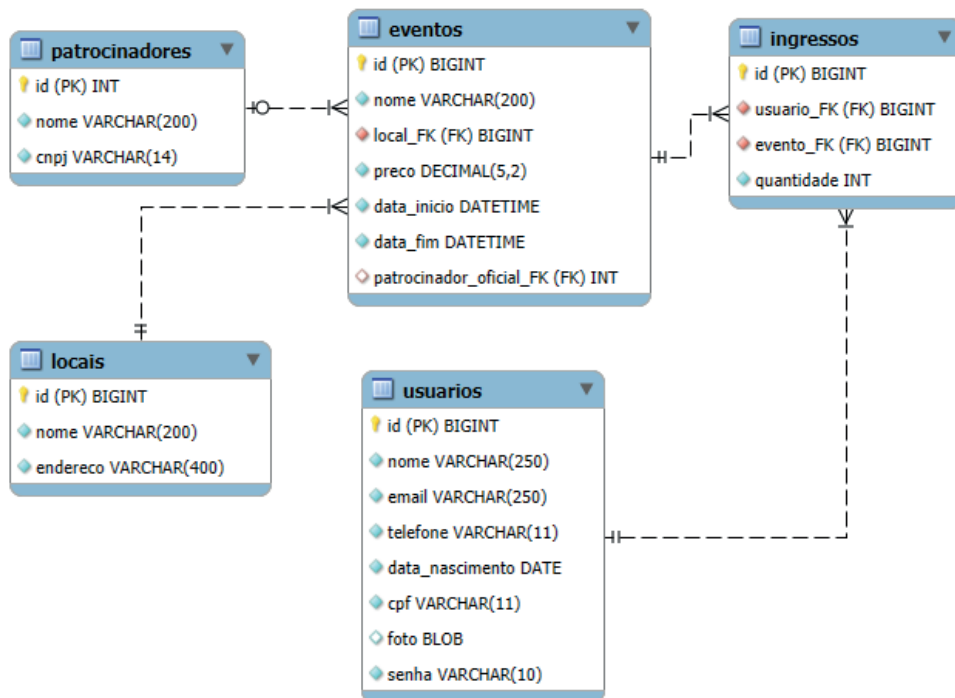
```
-- ESTRUTURA DO COMANDO:
select COLUNA1,COLUNA2,... from NOME_DA_TABELA1
  right join NOME_DA_TABELA2 on COLUNA_TABELA1_FK =
COLUNA_TABELA2_PK;

-- EXEMPLOS:
/* exibe os filmes e as informações de seu diretor,
   exibindo inclusive os diretores que não possuem filme */
select nome_filme, duracao, categoria, nome_diretor,
nascimento_diretor
  from filmes right join diretores on diretor_FK = diretor_id;

/* exibe todos os dados da tabela de automóveis e laudos
   cautelares associados, exibindo inclusive os laudos que não
   possuem automóveis */
select * from automoveis a right join cautelar c
  on a.cautelar_FK = c.id;

/* exibe dados de projetos e clientes associados em um
   período, exibindo inclusive os clientes que não possuem
   projeto */
select p.nome, p.data, p.valor, c.nome, c.cpf, c.email from
projetos p
  right join clientes c on p.cliente_FK = c.id
where p.data >= '2025-01-01' and p.data <= '2025-02-02'
order by p.data, c.nome asc;
```

Para dar maior clareza à explicação, veja a **Figura 4.22**, retomando o exemplo da plataforma de eventos.



**Figura 4.22** – Diagrama UML de uma plataforma de eventos.

Crédito: do elaborador, 2025.

Note que a associação entre eventos e patrocinadores não é obrigatória, possibilitando a existência de eventos com essa chave estrangeira nula, conforme se vê na **Figura 4.23**.

```
'67 • select * from eventos;
```

| id | nome                               | local_FK | preco  | data_inicio         | data_fim            | patrocinador_oficial_FK |
|----|------------------------------------|----------|--------|---------------------|---------------------|-------------------------|
| 1  | Apresentação Grupo de Capoeira Sol | 1        | 0.00   | 2025-01-14 16:00:00 | 2025-01-14 18:00:00 |                         |
| 2  | Expedição Biologia Kids            | 2        | 30.00  | 2025-02-02 09:00:00 | 2025-03-02 11:00:00 | 1                       |
| 3  | Peça Os Mensageiros                | 3        | 40.00  | 2025-04-27 19:00:00 | 2025-05-15 21:00:00 | 2                       |
| 4  | Peça Manoel Bandeira               | 3        | 19.90  | 2025-03-03 16:00:00 | 2025-03-10 18:00:00 |                         |
| 5  | Show Iron Maiden: Future Past      | 4        | 159.90 | 2025-06-08 21:00:00 | 2025-06-08 23:30:00 | 4                       |

**Figura 4.23** – Comando SQL e resultado exibindo os eventos disponíveis na plataforma de eventos – MySQL Workbench.

Crédito: do elaborador, 2025.

Agora, faremos o uso do **right join** entre os eventos e os patrocinadores e poderemos identificar melhor o comportamento deste comando, conforme se vê na **Figura 4.24**.

```

776 • select * from eventos e right join patrocinadores p
777 on e.patrocinador_oficial_FK = p.id;
778

```

| id | nome                          | local_FK | preco  | data_inicio         | data_fim            | patrocinador_oficial_FK | id | nome                | cnpj           |
|----|-------------------------------|----------|--------|---------------------|---------------------|-------------------------|----|---------------------|----------------|
| 2  | Expedição Biologia Kids       | 2        | 30,00  | 2025-02-02 09:00:00 | 2025-03-02 11:00:00 | 1                       | 1  | Nova Energia Ltda   | 12345678000123 |
| 3  | Peca Os Mensageiros           | 3        | 40,00  | 2025-04-27 19:00:00 | 2025-05-15 21:00:00 | 2                       | 2  | Tech Inova S.A.     | 98765432000198 |
| 5  | Show Iron Maiden: Future Past | 4        | 159,90 | 2025-06-08 21:00:00 | 2025-06-08 23:30:00 | 4                       | 4  | Food & Drinks Co.   | 33445566000177 |
|    |                               |          |        |                     |                     |                         | 5  | Luz e Som Produções | 55667788000199 |

**Figura 4.24** – Comando SQL e resultado exibindo os eventos disponíveis com o **right join** em patrocinadores, na plataforma de eventos – MySQL Workbench.

Crédito: do elaborador, 2025.

Conforme podemos identificar pela **Figura 4.24**, o uso do comando **right join** traz tanto os eventos com patrocinadores (correspondência bidirecional) quanto os patrocinadores sem nenhum tipo de evento (correspondência à direita), sendo, portanto, muito útil em cenários em que se deseja obter os resultados externos independentemente de estarem ou não associados à tabela atual.

## cross join – Combinação cartesiana

Você vai perceber que o comando **cross join** difere acentuadamente dos demais já vistos, tendo um uso muito mais específico.

O **cross join** é um tipo de **join** no SQL que retorna o produto cartesiano entre duas tabelas, ou seja, combina cada linha de uma tabela com todas as linhas da outra tabela, não exigindo nenhuma condição de relacionamento, o que significa que o número de linhas no resultado será igual ao número de linhas da primeira tabela multiplicado pelo número de linhas da segunda.

Dessa maneira, esse comando não é utilizado quando se deseja apenas obter informações adicionais em um processo de associação e sim quando se procura encontrar quais as combinações possíveis entre duas entidades.

Vamos à estrutura do comando **cross join** em SQL.

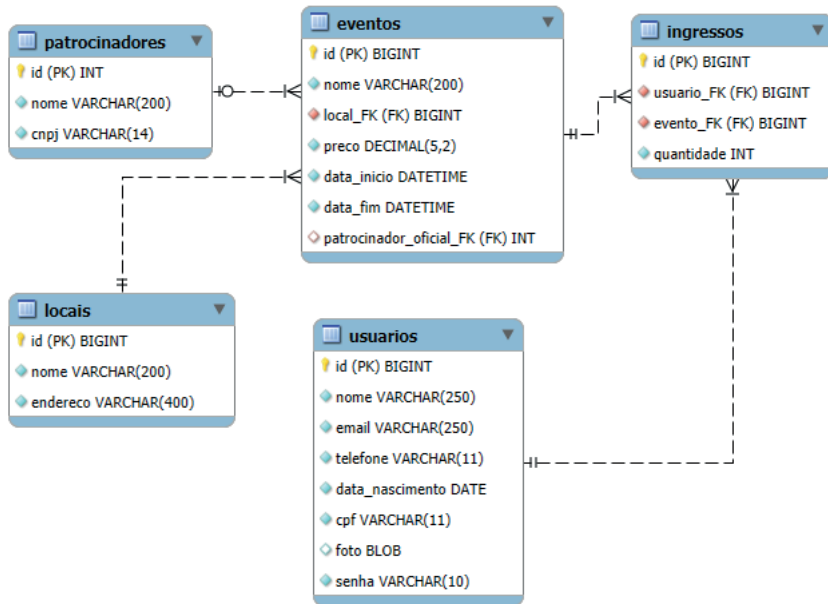
```
-- ESTRUTURA DO COMANDO:
select COLUNA1,COLUNA2,... from NOME_DA_TABELA1 cross join
NOME_DA_TABELA2;

-- EXEMPLOS:
-- exibe todos os filmes com todas as combinações possíveis
de diretores
select nome_filme, duracao, categoria, nome_diretor,
nascimento_diretor
from filmes cross join diretores;

/* exibe todos os automóveis com todas as combinações
possíveis de laudos cautelares */
select * from automoveis a cross join cautelar c;

/* exibe todos os dados de projetos com todas as combinações
possíveis de clientes que atendem a data específica, ordenados
por data e nome */
select p.nome, p.data, p.valor, c.nome, c.cpf, c.email from
projetos p
cross join clientes c
where p.data >= '2025-01-01' and p.data <= '2025-02-02'
order by p.data, c.nome asc;
```

Para dar maior clareza à explicação, veja a **Figura 4.25**, retomando o exemplo da plataforma de eventos.



**Figura 4.25** – Diagrama UML de uma plataforma de eventos.

Crédito: do elaborador, 2025.

Note que a associação entre eventos e patrocinadores não é obrigatória, possibilitando a existência de eventos com essa chave estrangeira nula, conforme se vê na **Figura 4.26**.

```
67 • select * from eventos;
```

| id | nome                               | local_FK | preco  | data_inicio         | data_fim            | patrocinador_oficial_FK |
|----|------------------------------------|----------|--------|---------------------|---------------------|-------------------------|
| 1  | Apresentação Grupo de Capoeira Sol | 1        | 0.00   | 2025-01-14 16:00:00 | 2025-01-14 18:00:00 | null                    |
| 2  | Expedição Biologia Kids            | 2        | 30.00  | 2025-02-02 09:00:00 | 2025-03-02 11:00:00 | 1                       |
| 3  | Peca Os Mensageiros                | 3        | 40.00  | 2025-04-27 19:00:00 | 2025-05-15 21:00:00 | 2                       |
| 4  | Peca Manoel Bandeira               | 3        | 19.90  | 2025-03-03 16:00:00 | 2025-03-10 18:00:00 | null                    |
| 5  | Show Iron Maiden: Future Past      | 4        | 159.90 | 2025-06-08 21:00:00 | 2025-06-08 23:30:00 | 4                       |

**Figura 4.26** – Comando SQL e resultado exibindo os eventos disponíveis na plataforma de eventos – MySQL Workbench.

Crédito: do elaborador, 2025.

Agora, faremos uso do **cross join** entre os eventos e os patrocinadores e poderemos identificar melhor o comportamento desse comando, conforme se vê na **Figura 4.27**.

782 • `select * from eventos e cross join patrocinadores p;`

783

| id | nome                               | local_fk | preco  | data_inicio         | data_fim            | patrocinador_oficial_fk | id | nome               | origem         |
|----|------------------------------------|----------|--------|---------------------|---------------------|-------------------------|----|--------------------|----------------|
| 5  | Show Iron Maiden: Future Past      | 4        | 159.90 | 2025-06-08 21:00:00 | 2025-06-08 23:30:00 | 4                       | 1  | Nova Energia Ltda  | 12345678000123 |
| 4  | Peca Manoel Bandeira               | 3        | 19.90  | 2025-03-03 16:00:00 | 2025-03-10 18:00:00 | 2025                    | 1  | Nova Energia Ltda  | 12345678000123 |
| 3  | Peca Os Mensageiros                | 3        | 40.00  | 2025-04-27 19:00:00 | 2025-05-15 21:00:00 | 2                       | 2  | Tech Inova S.A.    | 98765432000198 |
| 2  | Expedição Biologia Kids            | 2        | 30.00  | 2025-02-02 09:00:00 | 2025-03-02 11:00:00 | 1                       | 1  | Nova Energia Ltda  | 12345678000123 |
| 1  | Apresentação Grupo de Capoeira Sol | 1        | 0.00   | 2025-01-14 16:00:00 | 2025-01-14 18:00:00 | 2025                    | 1  | Nova Energia Ltda  | 12345678000123 |
| 5  | Show Iron Maiden: Future Past      | 4        | 159.90 | 2025-06-08 21:00:00 | 2025-06-08 23:30:00 | 4                       | 2  | Tech Inova S.A.    | 98765432000198 |
| 4  | Peca Manoel Bandeira               | 3        | 19.90  | 2025-03-03 16:00:00 | 2025-03-10 18:00:00 | 2025                    | 2  | Tech Inova S.A.    | 98765432000198 |
| 3  | Peca Os Mensageiros                | 3        | 40.00  | 2025-04-27 19:00:00 | 2025-05-15 21:00:00 | 2                       | 2  | Tech Inova S.A.    | 98765432000198 |
| 2  | Expedição Biologia Kids            | 2        | 30.00  | 2025-02-02 09:00:00 | 2025-03-02 11:00:00 | 1                       | 2  | Tech Inova S.A.    | 98765432000198 |
| 1  | Apresentação Grupo de Capoeira Sol | 1        | 0.00   | 2025-01-14 16:00:00 | 2025-01-14 18:00:00 | 2025                    | 2  | Tech Inova S.A.    | 98765432000198 |
| 5  | Show Iron Maiden: Future Past      | 4        | 159.90 | 2025-06-08 21:00:00 | 2025-06-08 23:30:00 | 4                       | 3  | Eventos Globais ME | 11223344000155 |
| 4  | Peca Manoel Bandeira               | 3        | 19.90  | 2025-03-03 16:00:00 | 2025-03-10 18:00:00 | 2025                    | 3  | Eventos Globais ME | 11223344000155 |
| 3  | Peca Os Mensageiros                | 3        | 40.00  | 2025-04-27 19:00:00 | 2025-05-15 21:00:00 | 2                       | 3  | Eventos Globais ME | 11223344000155 |
| 2  | Expedição Biologia Kids            | 2        | 30.00  | 2025-02-02 09:00:00 | 2025-03-02 11:00:00 | 1                       | 3  | Eventos Globais ME | 11223344000155 |
| 1  | Apresentação Grupo de Capoeira Sol | 1        | 0.00   | 2025-01-14 16:00:00 | 2025-01-14 18:00:00 | 2025                    | 3  | Eventos Globais ME | 11223344000155 |
| 5  | Show Iron Maiden: Future Past      | 4        | 159.90 | 2025-06-08 21:00:00 | 2025-06-08 23:30:00 | 4                       | 4  | Food & Drinks Co.  | 33445566000177 |
| 4  | Peca Manoel Bandeira               | 3        | 19.90  | 2025-03-03 16:00:00 | 2025-03-10 18:00:00 | 2025                    | 4  | Food & Drinks Co.  | 33445566000177 |
| 3  | Peca Os Mensageiros                | 3        | 40.00  | 2025-04-27 19:00:00 | 2025-05-15 21:00:00 | 2                       | 4  | Food & Drinks Co.  | 33445566000177 |
| 2  | Expedição Biologia Kids            | 2        | 30.00  | 2025-02-02 09:00:00 | 2025-03-02 11:00:00 | 1                       | 4  | Food & Drinks Co.  | 33445566000177 |

**Figura 4.27** – Comando SQL e resultado exibindo os eventos disponíveis com o **cross join** em patrocinadores na plataforma de eventos – MySQL Workbench.

Crédito: do elaborador, 2025.

Conforme podemos identificar na **Figura 4.27**, o uso do comando **cross join** realiza arbitrariamente a combinação entre todas as possibilidades que envolvem as entidades de eventos e patrocinadores, desconsiderando se tais combinações existem originalmente ou não. Dessa maneira, seu resultado será sempre a multiplicação entre a quantidade de colunas de cada uma das tabelas.

## Exemplos e casos de uso – Plataforma educacional

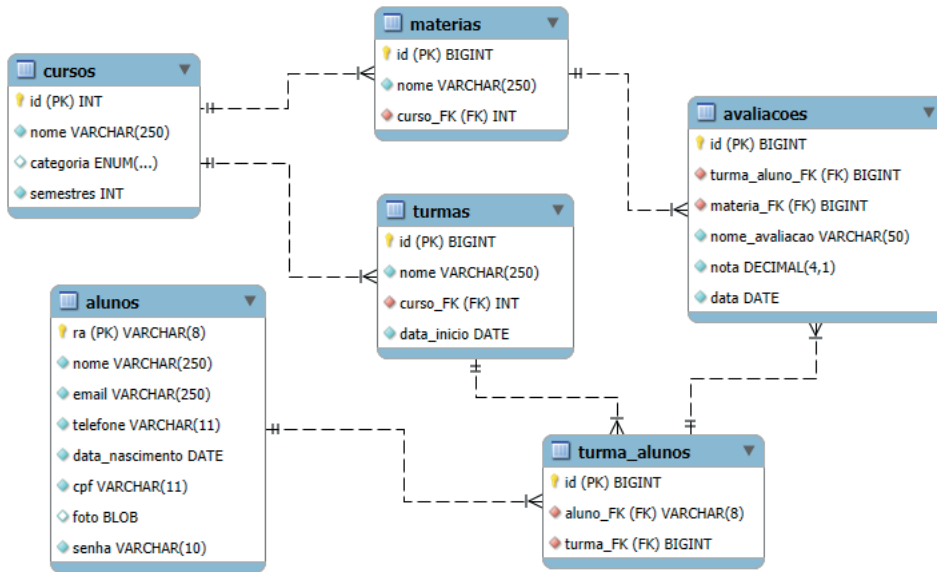
Aproveitaremos que o(a) aluno(a) já foi apresentado(a) ao exemplo que envolve uma plataforma educacional, em que tivemos a oportunidade de aplicar o uso de funções do MySQL (caso necessário, reveja os exemplos e casos de uso no tópico **Funções**, neste capítulo), e estenderemos o uso desse contexto para também exercitarmos os comandos **join** em uma condição mais prática.

Faremos apenas algumas readequações na modelagem do referido exemplo para deixá-lo mais fiel a um cenário real, uma vez que, continuamente, o(a) aluno(a) se encontra mais apto a interpretar contextos mais complexos.

Nesse cenário, uma plataforma educacional foi desenvolvida para proporcionar a digitalização do processo de cadastramento de alunos,

gerenciamento de notas por parte dos docentes e gestão das matérias existentes em cada curso de uma unidade escolar.

Utilizando a associação entre diversas entidades, essa plataforma apresenta uma estrutura bem-definida entre alunos, cursos, matérias e avaliações, sendo possível identificá-la no diagrama UML da **Figura 4.28**.



**Figura 4.28** – Diagrama UML de uma plataforma educacional.

Crédito: do elaborador, 2025.

Para objetividade desse caso de uso, não envolveremos todos os processos de criação e inserção dos dados, uma vez que estamos considerando que essa aplicação já está ativa (em produção). Necessitaremos, no entanto, de algumas consultas para obter informações do banco de dados dessa plataforma de ensino, de forma a atender a necessidade de cada tópico.

Veja no bloco de código a seguir como obter a relação dos nomes e RAs de alunos que estão matriculados em turmas que já tiveram início, além das informações da turma e do curso.

```

/* exibe os alunos e turmas/cursos que já se iniciaram */
select a.ra as 'RA Aluno', a.nome as 'Nome Aluno', a.email
as 'Email', t.id as 'Código da Turma', t.nome as 'Nome Turma',
c.nome as 'Curso'
  
```



```

from turma_alunos ta
inner join turmas t on t.id = ta.turma_FK
inner join alunos a on a.ra = ta.aluno_FK
inner join cursos c on c.id = t.curso_FK
where t.data_inicio <= curdate();

```

## IMPORTANTE

### Benefícios

Observe que os benefícios do uso de comandos **join** não são restritos apenas a trazer informações a serem exibidas, mas também possibilitam a manipulação dessas informações e até mesmo usá-las como filtros. Por exemplo, no comando SQL do bloco de código anterior, não estamos utilizando “**t.data\_inicio**” para ser exibida no resultado final da busca e sim para servir de filtro com a cláusula **where**.

| RA Aluno | Nome Aluno      | Email                      | Código da Turma | Nome Turma            | Curso                                 |
|----------|-----------------|----------------------------|-----------------|-----------------------|---------------------------------------|
| RA000001 | Ana Silva       | ana.silva@email.com        | 1               | ADS 2025 - Noturno    | Análise e Desenvolvimento de Sistemas |
| RA000002 | João Pereira    | joao.pereira@email.com     | 1               | ADS 2025 - Noturno    | Análise e Desenvolvimento de Sistemas |
| RA000003 | Maria Oliveira  | maria.oliveira@email.com   | 2               | ADS 2025 - Matutino   | Análise e Desenvolvimento de Sistemas |
| RA000004 | Pedro Santos    | pedro.santos@email.com     | 2               | ADS 2025 - Matutino   | Análise e Desenvolvimento de Sistemas |
| RA000005 | Carla Souza     | carla.souza@email.com      | 2               | ADS 2025 - Matutino   | Análise e Desenvolvimento de Sistemas |
| RA000006 | Lucas Lima      | lucas.lima@email.com       | 2               | ADS 2025 - Matutino   | Análise e Desenvolvimento de Sistemas |
| RA000007 | Fernanda Alves  | fernanda.alves@email.com   | 2               | ADS 2025 - Matutino   | Análise e Desenvolvimento de Sistemas |
| RA000008 | Rafael Costa    | rafael.costa@email.com     | 2               | ADS 2025 - Matutino   | Análise e Desenvolvimento de Sistemas |
| RA000009 | Juliana Martins | juliana.martins@email.com  | 2               | ADS 2025 - Matutino   | Análise e Desenvolvimento de Sistemas |
| RA000010 | Gabriel Rocha   | gabriel.rocha@email.com    | 2               | ADS 2025 - Matutino   | Análise e Desenvolvimento de Sistemas |
| RA000011 | Mário Silva     | mario.silva@email.com      | 2               | ADS 2025 - Matutino   | Análise e Desenvolvimento de Sistemas |
| RA000012 | Maurício Cuzco  | mauricio.cuzco@email.com   | 3               | Automação 2025 - I... | Automação Industrial                  |
| RA000013 | Maria Carla     | maria.carla@email.com      | 3               | Automação 2025 - I... | Automação Industrial                  |
| RA000014 | Luiz Carlos     | luiz.carlos@email.com      | 3               | Automação 2025 - I... | Automação Industrial                  |
| RA000015 | Thauane Jesus   | thauane.jesus@email.com    | 3               | Automação 2025 - I... | Automação Industrial                  |
| RA000016 | Filipe Roma     | filipe.roma@email.com      | 3               | Automação 2025 - I... | Automação Industrial                  |
| RA000017 | Lucas Oliveira  | lucas.oliveira@example.com | 3               | Automação 2025 - I... | Automação Industrial                  |

**Figura 4.29** – Resultado exibindo os alunos matriculados em turmas/cursos que já se iniciaram.

Crédito: do elaborador, 2025.

Veja no bloco de código a seguir como obter a relação dos nomes, números de telefone e *e-mails* de alunos que tiveram notas de avaliações menores que 5, realizadas nos últimos 30 dias, com nome da turma, matéria

e curso (a informação será usada para envio de alertas aos alunos e/ou responsáveis).

```

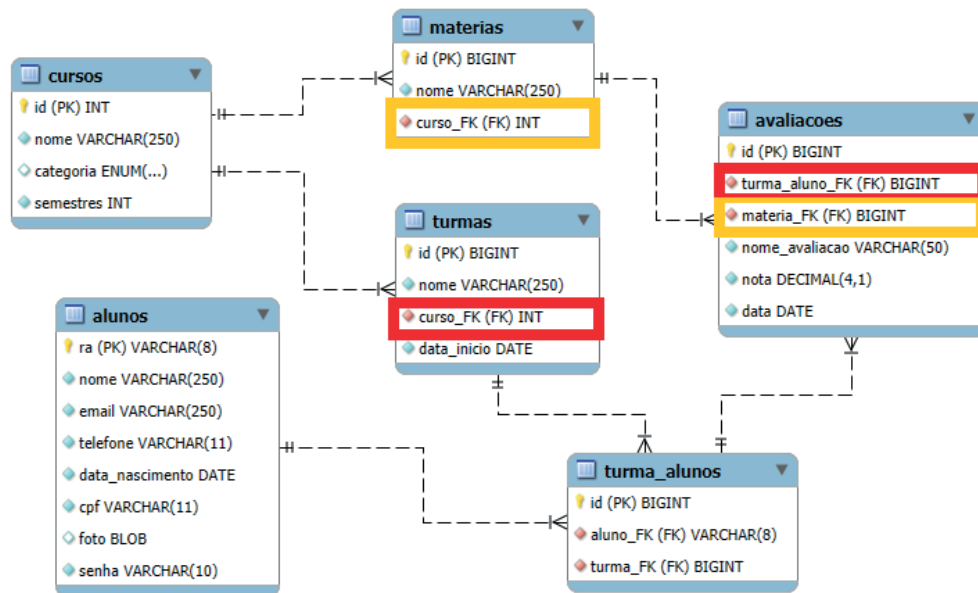
/* exibe os alunos com notas menores que 5 nas avaliações
realizadas nos últimos 30 dias */
select a.ra as 'RA Aluno', a.nome as 'Nome Aluno', a.email as
'Email',
a.telefone as 'Telefone', av.nota as 'Nota',
av.data as 'Data Avaliação', m.nome as 'Matéria',
t.id as 'Código da Turma', t.nome as 'Nome Turma', c.nome as
'Curso'
from avaliacoes av
inner join materias m on m.id = av.materia_FK
inner join turma_alunos ta on ta.id = av.turma_aluno_FK
inner join turmas t on t.id = ta.turma_FK
inner join alunos a on a.ra = ta.aluno_FK
inner join cursos c on c.id = t.curso_FK
inner join cursos c2 on c2.id = m.curso_FK
where datediff(curdate(),av.data) <= 30 and av.nota < 5
and c2.id = c.id order by c.nome, a.nome, m.nome;

```

## IMPORTANTE

### Possibilidade de inconsistência

Talvez você tenha se questionado sobre a razão de, no comando do bloco de código anterior, executarmos dois **join** diferentes para a mesma entidade **cursos** e posteriormente comparar seus **ids** durante a cláusula **where**. Em um cenário ideal, sem a possibilidade de inconsistências, esse tipo de comparação não seria necessário. Entretanto, devido à modelagem que adotamos para esse exemplo, há uma possibilidade de inconsistência entre o curso que foi utilizado para cadastrar o aluno e aquele atribuído indiretamente pela matéria usada na avaliação. Veja a demarcação da **Figura 4.30** para esclarecer esse fato.



**Figura 4.30** – Diagrama UML demarcando as interações entre os cursos na entidade de avaliações.

Crédito: do elaborador, 2025.

Nesse caso, para assegurar a consistência desses dados, adicionamos uma condição de igualdade entre os **ids**, uma vez que sempre devem se referir ao mesmo curso para cada avaliação/aluno/turma.

| RA Aluno | Nome Aluno      | Email                     | Telefone    | Nota | Data Avaliação | Matéria                         | Código da Turma | Nome Turma          | Curso                          |
|----------|-----------------|---------------------------|-------------|------|----------------|---------------------------------|-----------------|---------------------|--------------------------------|
| RA000001 | Ana Silva       | ana.silva@email.com       | 11987654321 | 2,4  | 2025-02-05     | Banco de Dados                  | 1               | ADS 2025 - Noturno  | Análise e Desenvolvimento de S |
| RA000001 | Ana Silva       | ana.silva@email.com       | 11987654321 | 1,6  | 2025-02-05     | Engenharia de Software          | 1               | ADS 2025 - Noturno  | Análise e Desenvolvimento de S |
| RA000001 | Ana Silva       | ana.silva@email.com       | 11987654321 | 3,0  | 2025-02-05     | Programação Orientada a Objetos | 1               | ADS 2025 - Noturno  | Análise e Desenvolvimento de S |
| RA000005 | Carla Souza     | carla.souza@email.com     | 31987654321 | 1,6  | 2025-02-05     | Engenharia de Software          | 2               | ADS 2025 - Matutino | Análise e Desenvolvimento de S |
| RA000005 | Carla Souza     | carla.souza@email.com     | 31987654321 | 3,5  | 2025-02-05     | Programação Orientada a Objetos | 2               | ADS 2025 - Matutino | Análise e Desenvolvimento de S |
| RA000010 | Gabriel Rocha   | gabriel.rocha@email.com   | 51998765432 | 3,9  | 2025-02-05     | Banco de Dados                  | 2               | ADS 2025 - Matutino | Análise e Desenvolvimento de S |
| RA000010 | Gabriel Rocha   | gabriel.rocha@email.com   | 51998765432 | 1,5  | 2025-02-05     | Banco de Dados                  | 2               | ADS 2025 - Matutino | Análise e Desenvolvimento de S |
| RA000002 | João Pereira    | joao.pereira@email.com    | 11998765432 | 0,9  | 2025-02-05     | Banco de Dados                  | 1               | ADS 2025 - Noturno  | Análise e Desenvolvimento de S |
| RA000002 | João Pereira    | joao.pereira@email.com    | 11998765432 | 1,4  | 2025-02-05     | Programação Orientada a Objetos | 1               | ADS 2025 - Noturno  | Análise e Desenvolvimento de S |
| RA000009 | Juliana Martins | juliana.martins@email.com | 51987654321 | 1,4  | 2025-02-05     | Banco de Dados                  | 2               | ADS 2025 - Matutino | Análise e Desenvolvimento de S |
| RA000009 | Juliana Martins | juliana.martins@email.com | 51987654321 | 1,2  | 2025-02-05     | Programação Orientada a Objetos | 2               | ADS 2025 - Matutino | Análise e Desenvolvimento de S |
| RA000006 | Lucas Lima      | lucas.lima@email.com      | 31998765432 | 1,3  | 2025-02-05     | Programação Orientada a Objetos | 2               | ADS 2025 - Matutino | Análise e Desenvolvimento de S |
| RA000003 | Maria Oliveira  | maria.oliveira@email.com  | 21987654321 | 0,5  | 2025-02-05     | Banco de Dados                  | 2               | ADS 2025 - Matutino | Análise e Desenvolvimento de S |
| RA000011 | Márcio Silva    | marcio.silva@email.com    | 11907454321 | 3,2  | 2025-02-05     | Banco de Dados                  | 2               | ADS 2025 - Matutino | Análise e Desenvolvimento de S |
| RA000004 | Pedro Santos    | pedro.santos@email.com    | 21998765432 | 2,9  | 2025-02-05     | Engenharia de Software          | 2               | ADS 2025 - Matutino | Análise e Desenvolvimento de S |
| RA000008 | Rafael Costa    | rafael.costa@email.com    | 41998765432 | 1,8  | 2025-02-05     | Banco de Dados                  | 2               | ADS 2025 - Matutino | Análise e Desenvolvimento de S |

**Figura 4.31** – Resultado exibindo os alunos com notas pendentes em avaliações feitas nos últimos 30 dias.

Crédito: do elaborador, 2025.

## Agrupamentos e agregações

Ao seguir nossa trajetória de estudos, você deve ter chegado à conclusão de que, de fato, o MySQL oferece ao desenvolvedor um repertório de comandos tão extenso que nos dá a sensação de ser quase impossível não atendermos a todos os cenários exigidos pelas mais diferentes aplicações.

Embora seja possível criar tabelas contendo uma grande diversidade de dados e manipulá-los utilizando inúmeras funções disponíveis, muitas vezes os cenários reais das aplicações demandam a compilação de informações que não correspondem exatamente à estrutura das entidades definidas em um projeto de banco de dados.

É nesse contexto que os comandos de agrupamentos e agregações se tornam indispensáveis, pois permitem deixar as estruturas modeladas ainda mais flexíveis por meio do uso de comandos que compilam, consolidam e transformam os dados conforme a necessidade de cada contexto. Nos tópicos a seguir, veremos o uso de tais recursos mais detalhadamente.

### group by – Agrupamentos simples

O comando **group by**, como sugere sua tradução literal, permite o agrupamento de colunas de uma tabela com o objetivo de consolidar informações, sendo amplamente utilizado em conjunto com funções de agregação, como **sum**, **avg**, **count**, entre outras, para resumir e analisar os dados de maneira mais significativa.

Vejamos no bloco de código a seguir a estrutura desse comando em SQL.

```
-- ESTRUTURA DO COMANDO:  
select COLUNA1,COLUNA2,... from NOME_DA_TABELA1 group by  
COLUNA1;
```

```

-- EXEMPLOS:
-- obtém todos os status de solicitações disponíveis
select status from solicitacoes group by status;

-- obtém a quantidade de vendas por produtos
select id_produto, nome_produto, count(*) as quantidade
from vendas group by id_produto;

-- obtém a soma de gols por times do estado de São Paulo no
campeonato
select id_time, nome_time, sum(gols) as total_gols
from campeonato where estado = 'SP' group by id_time;

```

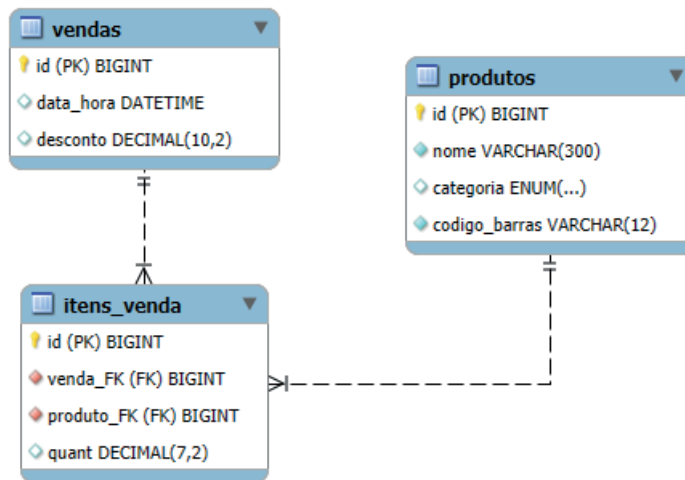
Embora o uso do **group by** seja relativamente intuitivo, alguns exemplos abordados no bloco de código anterior podem gerar certa estranheza ao(à) aluno(a), portanto, valem alguns apontamentos:

- O **group by** é aplicado em consultas com **select**, possibilitando a indicação das colunas a serem utilizadas como referência de agrupamento;
- É fundamental que a coluna a ser usada como referência de agrupamento seja utilizada no **select** para que não ocorram falhas de agrupamento, também conhecidas como erro de **nonaggregated columns**;
- O uso de funções de agregação não é obrigatório, sendo possível o uso do comando em seleção e agrupamento de uma mesma coluna. Para esses casos, o objetivo do comando consiste em coletar os valores presentes na tabela de uma forma resumida, como fizemos no primeiro exemplo;
- É possível aplicar o agrupamento a partir de dados pré-filtrados, como demonstramos no último exemplo, fazendo o uso das cláusulas **where**, seguindo o comportamento padrão de qualquer consulta.

De maneira simplificada, podemos evidenciar que o comando **group by** é usado, na maioria dos casos, com duas intenções possíveis:

- Consultar o resumo dos valores presentes de uma ou mais colunas em um banco de dados;
- Ou, se associado com funções de agregação, como **count**, **sum**, **avg**, tem a funcionalidade de realizar a consolidação de novas informações, muito utilizada em extração de relatórios.

Para melhor embasamento dessas explicações, nada melhor do que fazermos o uso de um exemplo prático. Possivelmente, você deve se lembrar de um exemplo envolvendo uma quitanda, utilizado no **Capítulo 3** deste livro.



**Figura 4.32** – Diagrama UML de uma aplicação envolvendo uma quitanda.

Crédito: do elaborador, 2025.

Essa aplicação, desenvolvida para iniciar o processo de digitalização de uma quitanda local, já está sendo capaz de registrar suas vendas e seus respectivos itens, de acordo com uma lista de produtos previamente cadastrados para comercialização.

Observe a lista da **Figura 4.33**, extraída utilizando-se o **join** entre as entidades, e note que em nosso exemplo é possível identificar tais negociações.

```

13 • select *from vendas v
14     inner join itens_venda iv on v.id = iv.venda_FK
15     inner join produtos p on p.id = iv.produto_FK;

```

|    | id | data_hora           | desconto | id | venda_FK | produto_FK | quant | id | nome            | categoria | codigo_barras |
|----|----|---------------------|----------|----|----------|------------|-------|----|-----------------|-----------|---------------|
| 1  | 1  | 2025-01-11 03:13:13 | 0.00     | 1  | 1        | 1          | 2.00  | 1  | Maçã Gala       | FRUTAS    | 789123456789  |
| 8  | 8  | 2025-02-06 05:41:12 | 2.75     | 16 | 8        | 1          | 1.00  | 1  | Maçã Gala       | FRUTAS    | 789123456789  |
| 21 | 21 | 2025-02-08 01:07:59 | 0.50     | 33 | 21       | 1          | 3.70  | 1  | Maçã Gala       | FRUTAS    | 789123456789  |
| 23 | 23 | 2025-02-08 01:07:59 | 4.80     | 38 | 23       | 1          | 0.90  | 1  | Maçã Gala       | FRUTAS    | 789123456789  |
| 2  | 2  | 2025-01-11 03:13:16 | 0.00     | 3  | 2        | 2          | 3.00  | 2  | Banana Prata    | FRUTAS    | 789987654321  |
| 9  | 9  | 2025-02-06 05:41:12 | 0.00     | 17 | 9        | 2          | 2.00  | 2  | Banana Prata    | FRUTAS    | 789987654321  |
| 21 | 21 | 2025-02-08 01:07:59 | 0.50     | 34 | 21       | 2          | 4.00  | 2  | Banana Prata    | FRUTAS    | 789987654321  |
| 1  | 1  | 2025-01-11 03:13:13 | 0.00     | 2  | 1        | 3          | 1.50  | 3  | Tomate Italiano | LEGUMES   | 789112233445  |
| 10 | 10 | 2025-02-06 05:41:12 | 7.25     | 19 | 10       | 3          | 0.75  | 3  | Tomate Italiano | LEGUMES   | 789112233445  |
| 20 | 20 | 2025-02-08 01:07:59 | 0.00     | 32 | 20       | 3          | 2.50  | 3  | Tomate Italiano | LEGUMES   | 789112233445  |
| 22 | 22 | 2025-02-08 01:07:59 | 0.00     | 35 | 22       | 3          | 0.70  | 3  | Tomate Italiano | LEGUMES   | 789112233445  |
| 23 | 23 | 2025-02-08 01:07:59 | 4.80     | 36 | 23       | 3          | 0.80  | 3  | Tomate Italiano | LEGUMES   | 789112233445  |
| 23 | 23 | 2025-02-08 01:07:59 | 4.80     | 39 | 23       | 3          | 1.20  | 3  | Tomate Italiano | LEGUMES   | 789112233445  |
| 3  | 3  | 2025-01-11 03:13:16 | 0.00     | 5  | 3        | 4          | 2.00  | 4  | Cenoura         | LEGUMES   | 789556677889  |

**Figura 4.33** – Comando SQL e resultado de uma quitanda – MySQL Workbench.

Crédito: do elaborador, 2025.

O fato é que, com o uso do **join**, podemos observar as vendas existentes de uma forma mais ampla, trazendo informações pertinentes de cada associação, em vez de nos restringirmos apenas às chaves estrangeiras de cada entidade.

No entanto, vamos supor que seja necessário obter quatro informações bem condizentes com o cenário a ser tratado, descritas a seguir:

- Obter os nomes/códigos de barras de produtos e suas respectivas quantidades totais de vendas;
- Obter as categorias de produto que já foram vendidas;
- Obter as categorias de produto e a quantidade de transações realizadas (apenas as existentes);
- Obter a quantidade de transações de venda por mês em 2025.

Sendo os tópicos bem coerentes com o cenário em que estamos trabalhando, como você os executaria? É o que veremos em cada tópico separadamente.

Veja no bloco de código a seguir como obter os nomes/códigos de barras de produtos e suas respectivas quantidades totais de vendas.

```
-- obtém a quantidade total de vendas de cada produto
select p.codigo_barras, p.nome, sum(iv.quant) as 'total de
vendas'
  from produtos p
 left join itens_venda iv on p.id = iv.produto_FK
 group by p.codigo_barras order by p.nome;
```

Para atender a essa necessidade, estamos utilizando o agrupamento por cada código de barras, já que possuímos a garantia dele ser único para cada produto. Estamos utilizando também a função **sum** para a soma dos valores agrupados de cada quantidade registrada de venda.

Outro detalhe importante é que decidimos utilizar, nesse exemplo, a união entre as entidades “produtos” e “itens\_venda” por meio do comando **left join**. Isso nos permite a visualização de todos os produtos, independentemente de possuírem ou não algum registro de venda.

Ao executar este comando, temos o resultado mostrado na **Figura 4.34**.

| codigo_barras | nome                     | total de vendas |
|---------------|--------------------------|-----------------|
| 789741852963  | Abobrinha Italiana       | 1.20            |
| 789135791357  | Alface Americana         | 5.00            |
| 789369121518  | Arroz Branco Tipo 1      | 7.00            |
| 789852741963  | Aveia em Flocos Finos    | 0.50            |
| 789987654321  | Banana Prata             | 9.00            |
| 789883455789  | Cajú                     | NULL            |
| 789556677889  | Cenoura                  | 6.20            |
| 789147258369  | Couve Manteiga           | 0.30            |
| 789246802468  | Espinafre                | 1.50            |
| 789481216202  | Feijão Carioca           | 3.00            |
| 789321654987  | Grão de Bico             | 1.00            |
| 789159357824  | Iogurte Natural Integ... | 1.00            |
| 789012345678  | Leite Integral           | 2.00            |
| 789123456789  | Maçã Gala                | 7.60            |
| 789654321098  | Manga Palmer             | 1.00            |
| 789258369147  | Melancia                 | 1.00            |
| 789753951864  | Muscarela                | 0.50            |

**Figura 4.34** – Resultado da quantidade total de vendas de cada produto, utilizando o comando **group by**.

Crédito: do elaborador, 2025.



## IMPORTANTE

### Função coalesce

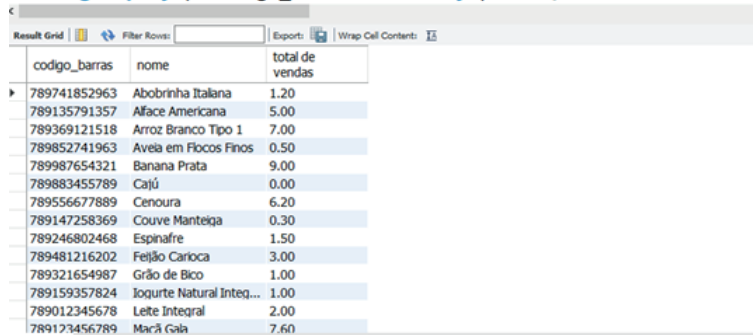
Você percebeu que na **Figura 4.34**, o produto “Caju”, o qual não possui nenhuma venda, ficou com um valor “null” ao invés de 0 na coluna “total de vendas”?

Supondo que essa consulta deva ser exportada e que o ideal seria os itens sem vendas aparecerem zerados no relatório, como você resolveria essa situação?

Para sua sorte, o MySQL tem uma função muito útil, denominada **coalesce**.

A função **coalesce** faz com que o MySQL mostre os valores originais quando estes não estiverem nulos e permite escolhermos um valor substituto para quando estiverem. Nesse caso, desejamos que os valores nulos da soma de vendas sejam substituídos pela expressão com valor 0, como se vê na **Figura 4.35**.

```
28 • select p.codigo_barras, p.nome, coalesce(sum(iv.quant), 0)
29 as 'total de vendas' from produtos p
30 left join itens_venda iv on p.id = iv.produto_FK
31 group by p.codigo_barras order by p.nome;
```



| codigo_barras | nome                     | total de vendas |
|---------------|--------------------------|-----------------|
| 789741852963  | Abobrinha Italiana       | 1.20            |
| 789135791357  | Afíface Americana        | 5.00            |
| 789369121518  | Arroz Branco Tipo 1      | 7.00            |
| 789852741963  | Aveia em Flocos Finos    | 0.50            |
| 789987654321  | Banana Prata             | 9.00            |
| 789883455789  | Caju                     | 0.00            |
| 789556677889  | Cenoura                  | 6.20            |
| 789147258369  | Couve Manteiga           | 0.30            |
| 789246802468  | Espinafre                | 1.50            |
| 789481216202  | Feijão Carioca           | 3.00            |
| 789321654987  | Grão de Bico             | 1.00            |
| 789159357824  | Iogurte Natural Integ... | 1.00            |
| 789012345678  | Leite Integral           | 2.00            |
| 789123456789  | Macã Gala                | 7.60            |

**Figura 4.35** – Utilização do **coalesce** para substituir valores nulos, obtidos com a soma, pelo valor zero.

Crédito: do elaborador, 2025.

Veja no bloco de código a seguir como obter as categorias de produto que já foram vendidas com a utilização do **group by**.

```
-- obtém todas as categorias já vendidas
select p.categoria from itens_venda iv
inner join produtos p on p.id = iv.produto_FK
group by p.categoria;
```

Aqui não temos nenhum segredo: estamos apenas agrupando por similaridade de categoria para que possamos exibir todas as que já foram registradas em algum tipo de transação da entidade de “itens\_venda”.

Ao executar esse comando, temos o resultado mostrado na **Figura 4.36**.

| categoria  |
|------------|
| FRUTAS     |
| LEGUMES    |
| VERDURAS   |
| LATICÍNIOS |
| GRÃOS      |

**Figura 4.36** – Resultado das categorias de produto já registradas em algum tipo de venda, utilizando o comando **group by**.

Crédito: do elaborador, 2025.

Veja no bloco de código a seguir como obter as categorias de produto e a quantidade de transações realizadas (apenas as existentes).

```
-- obtém a quantidade de transações envolvendo a venda de
categorias de produto
select p.categoria, count(*) as 'quantidade de transações'
from produtos p
inner join itens_venda iv on p.id = iv.produto_FK
inner join vendas v on v.id = iv.venda_FK
group by p.categoria;
```

Para atendermos a essa demanda, aplicamos o mesmo conceito já utilizado anteriormente, isto é, agrupamos por categoria de produto todos os dados que atendam às associações estabelecidas pelos comandos **inner join** e utilizamos também a função **count** para obter a quantidade de itens em cada agrupamento, conseguindo, desse modo, o resultado de transações/vendas para cada categoria.

Ao executar esse comando, temos o resultado mostrado na **Figura 4.37**.

| categoria  | quantidade de transações |
|------------|--------------------------|
| FRUTAS     | 9                        |
| LEGUMES    | 11                       |
| VERDURAS   | 7                        |
| LATICÍNIOS | 6                        |
| GRÃOS      | 6                        |

**Figura 4.37** – Resultado das categorias de produto e a quantidade de vendas registradas, utilizando o comando **group by**.

Crédito: do elaborador, 2025.

## group by – Agrupamentos múltiplos

Se você já entendeu o funcionamento dos agrupamentos simples com o **group by**, com toda certeza não terá dificuldades em também compreender os agrupamentos múltiplos, já que utilizam o mesmo comando. Na realidade, a única diferença entre os simples e os múltiplos é que os primeiros utilizam apenas uma coluna como referência, enquanto os segundos fazem uso de várias colunas para execução do agrupamento desejado.

Os agrupamentos múltiplos permitem obter mais detalhes, combinando registros que satisfazem múltiplas condições.

Da mesma forma que os agrupamentos simples, os múltiplos diferem muito pouco quanto a sua estrutura de comando e também permitem sua aplicação em conjunto com funções de agregação, como **sum**, **avg**, **count**, entre outras, para resumir e analisar os dados de maneira mais significativa.

```
-- ESTRUTURA DO COMANDO:
select COLUNA1,COLUNA2... from NOME_DA_TABELA1 group by
COLUNA1,COLUNA2...;

-- EXEMPLOS:
-- obtém os produtos vendidos e sua quantidade vendida em
cada ano
```

```

select produto, year(data_venda) as ano, count(*) as
'quantidade' from vendas group by produto, year(data_venda);

-- obtém os tipos de notas fiscais por status
select tipo_nota_fiscal, status, count(*) as 'quantidade' from
notas
group by tipo_nota_fiscal, status;

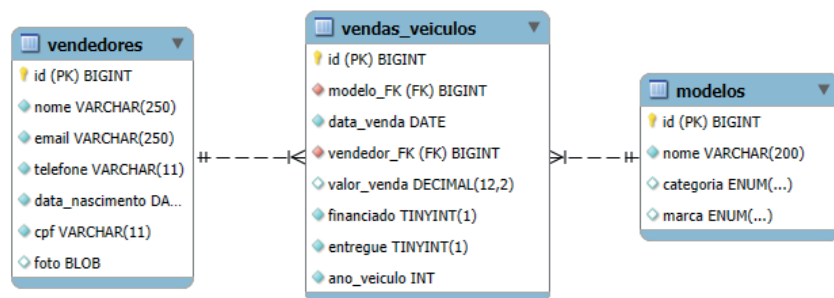
-- obtém as linhas de produto vendidas em notas fiscais por
revendedor e por mês
select lp.linha_produto, r.revendedor,
month(n.data_nota_fiscal) as mes, sum(ni.quantidade) as 'qtd.
vendas'
from notas n inner join nota_itens ni on n.id = ni.nota_FK
inner join produtos p on p.id = ni.produto_FK
inner join linhas_produtos lp on lp.id = p.linha_produto_FK
inner join revendedores r on r.id = n.revendedor_FK
group by lp.linha_produto, r.revendedor, month(n.
data_nota_fiscal);

-- obtém a quantidade de transações envolvendo a venda de
categorias de produto
select p.categoria, count(*) as 'quantidade de transações'
from produtos p
inner join itens_venda iv on p.id = iv.produto_FK
inner join vendas v on v.id = iv.venda_FK
group by p.categoria;

```

Como você pode perceber, a estrutura de uso de agrupamentos múltiplos é basicamente a mesma dos agrupamentos simples, apenas diferenciando-se pela quantidade de colunas referenciadas.

Vejamos uma aplicação prática, envolvendo uma plataforma de vendas em uma concessionária de veículos. O diagrama UML da **Figura 4.38** representa a estrutura básica dessa aplicação.



**Figura 4.38** – Diagrama UML de uma aplicação envolvendo uma concessionária de veículos.

Crédito: do elaborador, 2025.

Essa aplicação está dividida basicamente em três entidades, relacionando os vendedores e os modelos de veículos envolvidos em cada venda.

Fazendo uma simples consulta, obtemos os dados apresentados na **Figura 4.39**.

```
64 • select * from vendas_veiculos vv
65 inner join vendedores v on v.id = vv.vendedor_FK
66 inner join modelos m on m.id = vv.modelo_FK;
```

| id | modelo_FK | data_venda | vendedor_FK | valor_venda | financiado | entregue | ano_veiculo | id | nome           | email                    | telefone    | data_nascimento | cpf         |
|----|-----------|------------|-------------|-------------|------------|----------|-------------|----|----------------|--------------------------|-------------|-----------------|-------------|
| 1  | 1         | 2025-01-10 | 1           | 95000.00    | 1          | 1        | 2023        | 1  | João Silva     | joao.silva@email.com     | 11987654321 | 1985-03-15      | 12345678901 |
| 11 | 11        | 2025-02-09 | 1           | 95000.00    | 0          | 1        | 2025        | 1  | João Silva     | joao.silva@email.com     | 11987654321 | 1985-03-15      | 12345678901 |
| 21 | 11        | 2025-02-19 | 1           | 96000.00    | 1          | 0        | 2023        | 1  | João Silva     | joao.silva@email.com     | 11987654321 | 1985-03-15      | 12345678901 |
| 2  | 2         | 2025-01-12 | 2           | 72000.00    | 0          | 1        | 2025        | 2  | Maria Oliveira | maria.oliveira@email.com | 11912345678 | 1990-07-22      | 23456789012 |
| 12 | 12        | 2025-02-10 | 2           | 72000.00    | 1          | 0        | 2024        | 2  | Maria Oliveira | maria.oliveira@email.com | 11912345678 | 1990-07-22      | 23456789012 |
| 22 | 12        | 2025-02-20 | 2           | 73000.00    | 0          | 1        | 2023        | 2  | Maria Oliveira | maria.oliveira@email.com | 11912345678 | 1990-07-22      | 23456789012 |
| 3  | 3         | 2025-01-15 | 3           | 150000.00   | 1          | 0        | 2024        | 3  | Carlos Pereira | carlos.pereira@email.com | 21987654321 | 1988-05-10      | 34567890123 |
| 13 | 13        | 2025-02-11 | 3           | 155000.00   | 0          | 1        | 2024        | 3  | Carlos Pereira | carlos.pereira@email.com | 21987654321 | 1988-05-10      | 34567890123 |
| 23 | 13        | 2025-02-21 | 3           | 154000.00   | 1          | 1        | 2024        | 3  | Carlos Pereira | carlos.pereira@email.com | 21987654321 | 1988-05-10      | 34567890123 |
| 4  | 4         | 2025-01-20 | 4           | 110000.00   | 0          | 1        | 2023        | 4  | Ana Souza      | ana.souza@email.com      | 31987654321 | 1995-12-01      | 45678901234 |
| 14 | 14        | 2025-02-12 | 4           | 130000.00   | 1          | 1        | 2024        | 4  | Ana Souza      | ana.souza@email.com      | 31987654321 | 1995-12-01      | 45678901234 |
| 24 | 14        | 2025-02-22 | 4           | 133000.00   | 0          | 0        | 2023        | 4  | Ana Souza      | ana.souza@email.com      | 31987654321 | 1995-12-01      | 45678901234 |
| 5  | 5         | 2025-01-25 | 5           | 135000.00   | 1          | 1        | 2024        | 5  | Pedro Santos   | pedro.santos@email.com   | 41987654321 | 1983-11-18      | 56789012345 |
| 15 | 15        | 2025-02-13 | 5           | 85000.00    | 0          | 1        | 2025        | 5  | Pedro Santos   | pedro.santos@email.com   | 41987654321 | 1983-11-18      | 56789012345 |
| 25 | 15        | 2025-02-23 | 5           | 88000.00    | 1          | 1        | 2022        | 5  | Pedro Santos   | pedro.santos@email.com   | 41987654321 | 1983-11-18      | 56789012345 |
| 6  | 6         | 2025-01-30 | 6           | 190000.00   | 1          | 0        | 2023        | 6  | Fernanda Lima  | fernanda.lima@email.com  | 51987654321 | 1992-04-05      | 67890123456 |

**Figura 4.39** – Comando SQL e resultado de algumas colunas da aplicação de uma concessionária de veículos – MySQL Workbench.

Crédito: do elaborador, 2025.

Essa aplicação precisa fornecer dados para uma *dashboard* que auxiliará na tomada de decisões estratégicas, como identificar veículos com maior rotatividade para otimizar compras ou direcionar campanhas em períodos de menor venda.

Desse modo, o time de gestão solicitou as informações a seguir, mantendo a estrutura atual do projeto:

- Extração da quantidade de vendas realizadas por ano, mês, marca e categoria de veículo;
- Extração da quantidade de vendas realizadas por ano, mês, categoria e ano do veículo;
- Extração da soma das vendas realizadas por mês (do ano atual) e vendedor.

Sem dúvidas, a possibilidade de extração desses dados auxilia a tomada de diversas decisões estratégicas, revelando a importância da utilização do **group by** na manipulação de dados com SQL.

Vamos à prática. Veja no bloco de código a seguir como fazer a extração da quantidade de vendas realizadas por ano, mês, marca e categoria de veículo.

```
/* obtém a quantidade de vendas realizadas por ano, mês, marca
e categoria de veículo */
select year(vv.data_venda) as ano, month(vv.data_venda) as
mes, m.categoria, m.marca, count(*) as vendas from vendas_
veiculos vv
inner join vendedores v on vv.vendedor_FK = v.id
inner join modelos m on vv.modelo_FK = m.id
group by ano, mes, m.categoria, m.marca
order by ano desc, mes asc;
```

Nesse exemplo, estamos utilizando as funções **year** e **month** para dividir os resultados por ano e mês e, em seguida, realizamos o agrupamento por categoria e marca de cada veículo vendido. Finalmente, utilizamos a função **count** para registrar a quantidade de linhas agrupadas, o que, em outras palavras, significa a quantidade de vendas realizadas em cada combinação.

Ao executar esse comando, temos o resultado mostrado na **Figura 4.40**.

| ano  | mes | categoria   | marca      | vendas |
|------|-----|-------------|------------|--------|
| 2025 | 1   | HATCH       | HONDA      | 1      |
| 2025 | 1   | SEDAN       | HONDA      | 1      |
| 2025 | 1   | SEDAN       | NISSAN     | 1      |
| 2025 | 1   | SEDAN       | TOYOTA     | 1      |
| 2025 | 1   | SUV         | NISSAN     | 1      |
| 2025 | 1   | CAMINHONETE | TOYOTA     | 1      |
| 2025 | 2   | HATCH       | TOYOTA     | 2      |
| 2025 | 2   | HATCH       | VOLKSWAGEN | 2      |
| 2025 | 2   | SEDAN       | HONDA      | 2      |
| 2025 | 2   | SEDAN       | TOYOTA     | 1      |
| 2025 | 2   | SUV         | HONDA      | 2      |
| 2025 | 2   | SUV         | NISSAN     | 2      |
| 2025 | 2   | SUV         | VOLKSWAGEN | 1      |
| 2025 | 2   | SUV         | VOLVO      | 3      |
| 2025 | 2   | SUV         | GM         | 2      |
| 2025 | 2   | CAMINHONETE | NISSAN     | 2      |
| 2025 | 2   | CAMINHONETE | TOYOTA     | 1      |
| 2025 | 2   | CAMINHONETE | FORD       | 2      |

**Figura 4.40** – Resultado da quantidade de vendas de veículos por ano, mês, marca e categoria.

Crédito: do elaborador, 2025.

Na **Figura 4.40**, temos o resultado esperado, sendo possível identificar o número de vendas distribuído em cada atributo desejado, isto é, ano, mês, categoria e marca.

## SAIBA MAIS

### Função monthname

Sem dúvidas, extrair o relatório da **Figura 4.40**, com os dados detalhados de cada ano, mês, categoria e marca, deve ajudar a entender melhor o fluxo de vendas. Entretanto, supondo que seja necessário exibir o relatório em algum tipo de apresentação ou se desejar diminuir a manipulação de dados via código no *backend*, você também deve concordar que seria mais interessante mostrar os meses de forma nominal, ou seja, substituir o mês 2, por exemplo, por fevereiro e assim por diante.

Desse modo, como seria possível, de forma dinâmica, trocar os meses numéricos pelos literais? Felizmente o MySQL nos apresenta uma excelente solução!

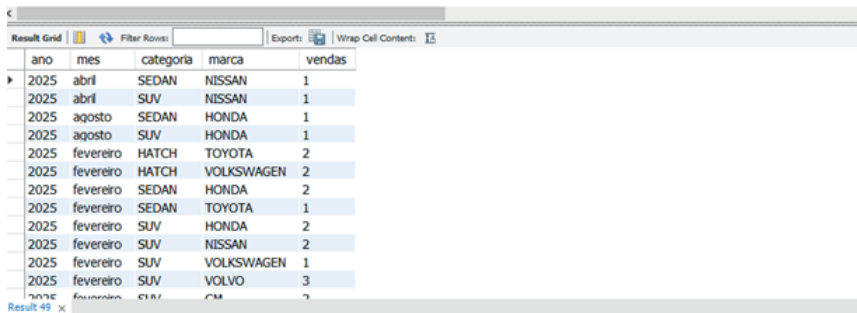
Vejamos os passos a seguir para executar essa tarefa:

1. Primeiro devemos indicar o idioma desejado para o servidor MySQL. Isso é facilmente configurável pelo comando a seguir, em que se atribui “pt\_BR”, indicando português do Brasil:

```
-- configura o idioma dos meses para português Brasil
SET lc_time_names = 'pt_BR';
```

2. Em seguida, utilizamos a função **monthname**, responsável por converter valores numéricos em seus nomes, usando o idioma que foi configurado no comando anterior:

```
22 • select year(data_venda) as ano, monthname(data_venda) as mes,
23       m.categoria, m.marca, count(*) as vendas from vendas_veiculos vv
24       inner join vendedores v on vv.vendedor_FK = v.id
25       inner join modelos m on vv.modelo_FK = m.id
26       group by year(data_venda), mes, m.categoria, m.marca
27       order by ano desc, mes asc;
```



| ano  | mes       | categoria | marca      | vendas |
|------|-----------|-----------|------------|--------|
| 2025 | abril     | SEDAN     | NISSAN     | 1      |
| 2025 | abril     | SUV       | NISSAN     | 1      |
| 2025 | agosto    | SEDAN     | HONDA      | 1      |
| 2025 | agosto    | SUV       | HONDA      | 1      |
| 2025 | fevereiro | HATCH     | TOYOTA     | 2      |
| 2025 | fevereiro | HATCH     | VOLKSWAGEN | 2      |
| 2025 | fevereiro | SEDAN     | HONDA      | 2      |
| 2025 | fevereiro | SEDAN     | TOYOTA     | 1      |
| 2025 | fevereiro | SUV       | HONDA      | 2      |
| 2025 | fevereiro | SUV       | NISSAN     | 2      |
| 2025 | fevereiro | SUV       | VOLKSWAGEN | 1      |
| 2025 | fevereiro | SUV       | VOLVO      | 3      |

**Figura 4.41** – Utilização do **monthname** para substituir valores numéricos, obtidos com o mês, pelo nome correspondente.

Crédito: do elaborador, 2025.

Veja no bloco de código a seguir como fazer a extração da quantidade de vendas realizadas por ano, mês, categoria e ano do veículo.

```
/* obtém a quantidade de vendas realizadas por ano, mês,
categoria e ano do veículo */
select year(vv.data_venda) as ano, monthname(vv.data_venda)
as mes, m.categoria, vv.ano_veiculo, count(*) as vendas
from vendas_veiculos vv
```



```

inner join vendedores v on vv.vendedor_FK = v.id
inner join modelos m on vv.modelo_FK = m.id
group by ano, mes, m.categoria, vv.ano_veiculo
order by ano desc, mes asc;

```

Já para esse exemplo, a resolução é muito similar à do anterior, apenas se diferenciando pelas colunas de referência de agrupamento, sendo ano, mês, categoria e ano do veículo.

Ao executá-lo, temos o resultado mostrado na **Figura 4.42**.

| ano  | mes       | categoria | ano_veiculo | vendas |
|------|-----------|-----------|-------------|--------|
| 2025 | abril     | SEDAN     | 2023        | 1      |
| 2025 | abril     | SUV       | 2024        | 1      |
| 2025 | agosto    | SEDAN     | 2023        | 1      |
| 2025 | agosto    | SUV       | 2023        | 1      |
| 2025 | fevereiro | HATCH     | 2022        | 2      |
| 2025 | fevereiro | HATCH     | 2025        | 2      |
| 2025 | fevereiro | SEDAN     | 2023        | 1      |
| 2025 | fevereiro | SEDAN     | 2024        | 1      |
| 2025 | fevereiro | SEDAN     | 2025        | 1      |
| 2025 | fevereiro | SUV       | 2022        | 2      |
| 2025 | fevereiro | SUV       | 2023        | 3      |
| 2025 | fevereiro | SUV       | 2024        | 5      |
| 2025 | fevereiro | CAMINH... | 2023        | 2      |
| 2025 | fevereiro | CAMINH... | 2024        | 5      |
| 2025 | janeiro   | HATCH     | 2025        | 1      |
| 2025 | janeiro   | SEDAN     | 2023        | 2      |
| 2025 | janeiro   | SEDAN     | 2024        | 1      |
| 2025 | janeiro   | SUV       | 2024        | 1      |

**Figura 4.42** – Resultado da quantidade de vendas de veículos por ano, mês, categoria e ano do veículo.

Crédito: do elaborador, 2025.

Veja no bloco de código a seguir como fazer a extração da soma das vendas realizadas por mês (do ano atual) e vendedor.

```

/* Extração da soma das vendas realizadas por mês (do ano
atual) e vendedor: */
select coalesce(monthname(vv.data_venda),'-') as mes, v.nome,
       concat('R$ ',coalesce(sum(vv.valor_venda),0)) as 'soma das
vendas'

```

```
from vendedores v
left join vendas_veiculos vv on v.id = vv.vendedor_FK
where year(vv.data_venda) = year(curdate())
or vv.data_venda is null
group by mes, v.nome order by v.nome;
```

No comando do bloco de código, conseguimos notar algumas diferenças em relação aos anteriores, nas quais vale a pena prestarmos atenção, detalhadas a seguir:

- Optamos por unir as entidades de vendedores e vendas com o **left join**, possibilitando que sejam mostrados os nomes de vendedores que não realizaram nenhuma venda no período selecionado;
- Adicionamos a função **coalesce** para selecionar um valor a ser preenchido caso a coluna esteja nula (vendedores sem nenhuma venda, por exemplo);
- Como o objetivo é ter a soma dos valores vendidos em vez do número de vendas, utilizamos a função **sum** para seu cálculo;
- Para deixarmos a exibição do resultado mais amigável, adicionamos o prefixo da moeda fiduciária (R\$) nos valores que envolvem a soma das vendas, por meio da função **concat**, responsável por concatenar (unir) valores;
- Como o requisito de extração é obter apenas os dados do ano atual, adicionamos a cláusula **where** para filtrar os valores antes de serem agrupados, considerando apenas os anos que sejam os mesmos da data atual, com a função **curdate**, obtendo os dados de forma dinâmica com o decorrer do tempo;

- Finalmente, vale ressaltar que, para possibilitarmos que vendedores sem nenhuma venda apareçam no relatório, adicionamos uma opção alternativa na cláusula **where**, considerando que a data de venda esteja nula (não exista), pois, sem essa condição, mesmo com o uso do **left join**, os dados seriam filtrados e não exibiriam tais vendedores, já que estamos utilizando a condição “year(vv.data\_venda) = year(curdate())”.

Feitas as considerações mais importantes, vamos observar o resultado do comando na **Figura 4.43**.

| mes       | nome            | soma das vendas |
|-----------|-----------------|-----------------|
| fevereiro | Carlos Pereira  | R\$ 309000.00   |
| abril     | Carlos Pereira  | R\$ 152000.00   |
| setembro  | Carlos Pereira  | R\$ 155500.00   |
| fevereiro | Fernanda Lima   | R\$ 395000.00   |
| janeiro   | Fernanda Lima   | R\$ 190000.00   |
| maio      | Fernanda Lima   | R\$ 195000.00   |
| janeiro   | João Silva      | R\$ 95000.00    |
| agosto    | João Silva      | R\$ 97000.00    |
| março     | João Silva      | R\$ 94000.00    |
| fevereiro | João Silva      | R\$ 191000.00   |
| junho     | Juliana Mach... | R\$ 88000.00    |
| fevereiro | Juliana Mach... | R\$ 262000.00   |
| junho     | Lucas Costa     | R\$ 248000.00   |
| fevereiro | Lucas Costa     | R\$ 745000.00   |
| -         | Luís Gaspetto   | R\$ 0.00        |
| agosto    | Maria Oliveira  | R\$ 74000.00    |
| março     | Maria Oliveira  | R\$ 72000.00    |

**Figura 4.43** – Resultado da soma de vendas de veículos por ano atual, mês e vendedor.

Crédito: do elaborador, 2025.

## having – Filtrando dados agrupados

Após explorarmos diversas formas de agrupamento de dados e analisarmos situações práticas nos tópicos anteriores, você já está ganhando domínio e autonomia para extrair e manipular informações de acordo com diferentes necessidades. Contudo, ainda falta um importante passo quando o assunto é agrupamentos: a possibilidade de filtrar dados “virtuais”, isto é, gerados apenas após a realização dos agrupamentos. Para esse caso, faremos uso de um novo comando: o **having**.

De forma simplificada, para que o(a) aluno(a) entenda mais facilmente, o comando **having** tem um comportamento muito semelhante à cláusula **where**, possibilitando a criação de condições para filtragem de resultados.

Contudo, é importante entender que a maior diferença entre ambos os comandos mencionados é que o **where** filtra os dados **antes do agrupamento**, enquanto o **having** realiza a filtragem dos valores **após o agrupamento**, possibilitando, desse modo, que sejam utilizados valores gerados por meio de cálculos ou funções dentro das condicionais.

### FIQUE ALERTA

#### Não confunda

O comando **having** somente pode ser usado em consultas que apresentam algum tipo de agregação de valores, ou seja, você o verá frequentemente com o **group by**.

Portanto, cuidado para não o confundir com o **where**.

No bloco de código a seguir, vejamos a estrutura do **having** e alguns exemplos de uso.

```
-- ESTRUTURA DO COMANDO:
select COLUNA1,COLUNA2... from NOME_DA_TABELA1
  group by COLUNA1,COLUNA2... having CONDIÇÃO;

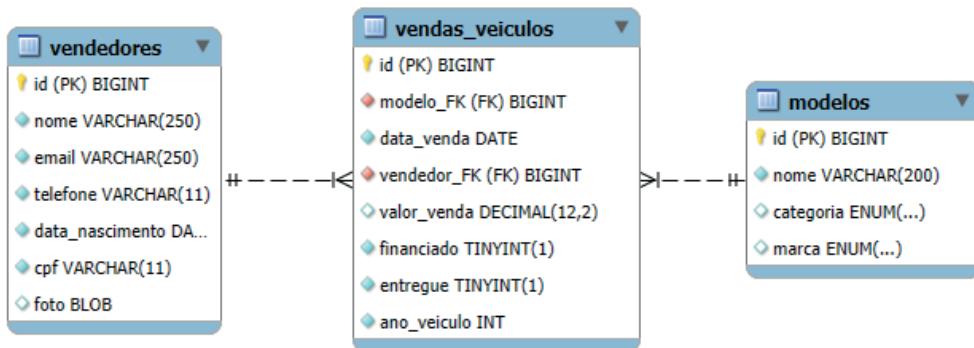
-- EXEMPLOS:
/* obtém os produtos vendidos e sua quantidade vendida em
cada ano, filtrando apenas aqueles com quantidade maior que
10 */
select produto, year(data_venda) as ano, count(*) as
quantidade from vendas
  group by produto, year(data_venda) having quantidade > 10;

/* obtém o cálculo do Produto Interno Bruto de cada país por
ano, filtrando apenas aqueles que tenham valores maiores que o
especificado */
```

```
select year(d.data_medicao) as ano, p.id as 'país id', p.nome,
sum((d.cf + d.ip + d.gg + d.bc)) as PIB
from países p
inner join dados d on p.id = d.pais_FK
group by ano, p.id having PIB > 8000000000.00
order by ano desc, PIB desc;
```

De fato, o uso do comando **having** não apresenta nenhuma dificuldade para aqueles que já se familiarizaram com o uso das cláusulas **where**. E, como se percebe nos exemplos mostrados no bloco de código anterior, seu uso é fundamental, uma vez que alguns atributos, obtidos mediante funções de agregação como **sum** e **count**, não poderiam ser utilizados em cláusulas **where**, já que antes do agrupamento tais atributos ainda não existem, sendo criados apenas após o processo de agrupamentos com o **group by**.

Para que o assunto fique menos subjetivo, vamos apresentar mais alguns casos de uso. Reaproveitaremos o mesmo exemplo envolvendo uma concessionária de carros, utilizado no tópico **group by – Agrupamentos múltiplos**.



**Figura 4.44** – Diagrama UML de uma aplicação envolvendo uma concessionária de veículos.

Crédito: do elaborador, 2025.

Da mesma forma, levaremos ao(à) aluno(a) alguns requisitos necessários para a extração dos dados dessa aplicação, com vistas a alimentar o time de gestão com relatórios contendo informações valiosas para a tomada

de decisões estratégicas, visando o aumento de vendas e a lucratividade da empresa. Antes de apresentarmos uma solução para cada requisito, incentivamos o(a) aluno(a) a se questionar e imaginar como resolver essa situação:

- Extração da soma das vendas realizadas no ano atual por vendedor, com valores acima de R\$ 500 000,00 (para efeito de premiação dos funcionários);
- Extração das marcas de veículos que tiveram 1 ou 0 vendas por ano (para efeito de evitar a compra dessas marcas nessa concessionária).

Veja no bloco de código a seguir como fazer a extração da soma das vendas realizadas no ano atual por vendedor, com valores acima de R\$ 500 000,00 (para efeito de premiação).

```
/*Extração da soma das vendas realizadas do ano atual por
vendedor, filtrando apenas as vendas com valores acima do
mínimo: */
select v.id as 'código vendedor', v.nome,
       concat('R$ ',sum(vv.valor_venda)) as 'soma das vendas'
from vendedores v
inner join vendas_veiculos vv on v.id = vv.vendedor_FK
where year(vv.data_venda) = year(curdate())
group by v.id having sum(vv.valor_venda) >= 500000.00
order by sum(vv.valor_venda) desc;
```

Observe que, conforme o bloco de código anterior, podemos utilizar inclusive a combinação de **where** + **having** no mesmo comando, já que possuem objetivos distintos, isto é, o primeiro está garantindo que os dados a serem usados antes da agregação sejam do ano atual e o segundo está filtrando os resultados de vendedores que tenham o somatório maior que o indicado, após o processo de agregação.

Na **Figura 4.45**, veremos o resultado e checaremos se está de acordo com o esperado.

| código vendedor | nome           | soma das vendas |
|-----------------|----------------|-----------------|
| 9               | Rafael Almeida | R\$ 1245500.00  |
| 7               | Lucas Costa    | R\$ 993000.00   |
| 6               | Fernanda Lima  | R\$ 780000.00   |
| 3               | Carlos Pereira | R\$ 766500.00   |
| 4               | Ana Souza      | R\$ 619000.00   |
| 5               | Pedro Santos   | R\$ 535000.00   |

**Figura 4.45** – Resultado da soma de vendas de veículos por ano atual e vendedor, que tenham alcançado a meta de premiação.

Crédito: do elaborador, 2025.

Veja no bloco de código a seguir como fazer a extração das marcas de veículos que tiveram 1 ou 0 vendas por ano (para efeito de evitar a compra dessas marcas nessa concessionária).

```
-- obtém as marcas de veículos com baixo ou nenhum volume de vendas
select coalesce(year(vv.data_venda), '-') as ano,
m.marca, count(vv.id) as vendas from modelos m
left join vendas_veiculos vv on m.id = vv.modelo_FK
group by ano, m.marca
having vendas <= 1
order by ano desc, vendas asc;
```

Em seguida, na **Figura 4.46**, executaremos o resultado.

| ano  | marca | vendas |
|------|-------|--------|
| 2024 | GM    | 1      |
| -    | FORD  | 0      |

**Figura 4.46** – Resultado das marcas que tiveram baixo volume de vendas anuais em uma concessionária.

Crédito: do elaborador, 2025.

## union – Unindo tabelas

O comando **union** é um operador utilizado para unir os resultados de duas ou mais instruções **select** em um único conjunto de resultados, eliminando automaticamente linhas duplicadas.

Diferentemente do que vimos em outros tipos de junções e agrupamentos, o objetivo do **union** não é compilar novos resultados e nem trazer informações adicionais vindas de chaves estrangeiras. Seu intuito é muito mais simples que isso: unir os resultados como se pertencessem à mesma tabela.

#### IMPORTANTE

##### Unificação dos resultados

**Para que o comando `union` consiga fazer a unificação dos resultados, é fundamental que a consulta possua o mesmo número de colunas, obedeça à mesma ordem e tenha tipos compatíveis entre si.**

Sua estrutura de uso é relativamente simples, conforme podemos identificar no bloco de código a seguir.

```
-- ESTRUTURA DO COMANDO:
select COLUNA1, COLUNA2, ... from TABELA1
union
select COLUNA1, COLUNA2, ... from TABELA2
...
;

--EXEMPLOS:
select email, telefone from fornecedores
union
select email, telefone from distribuidores;
```

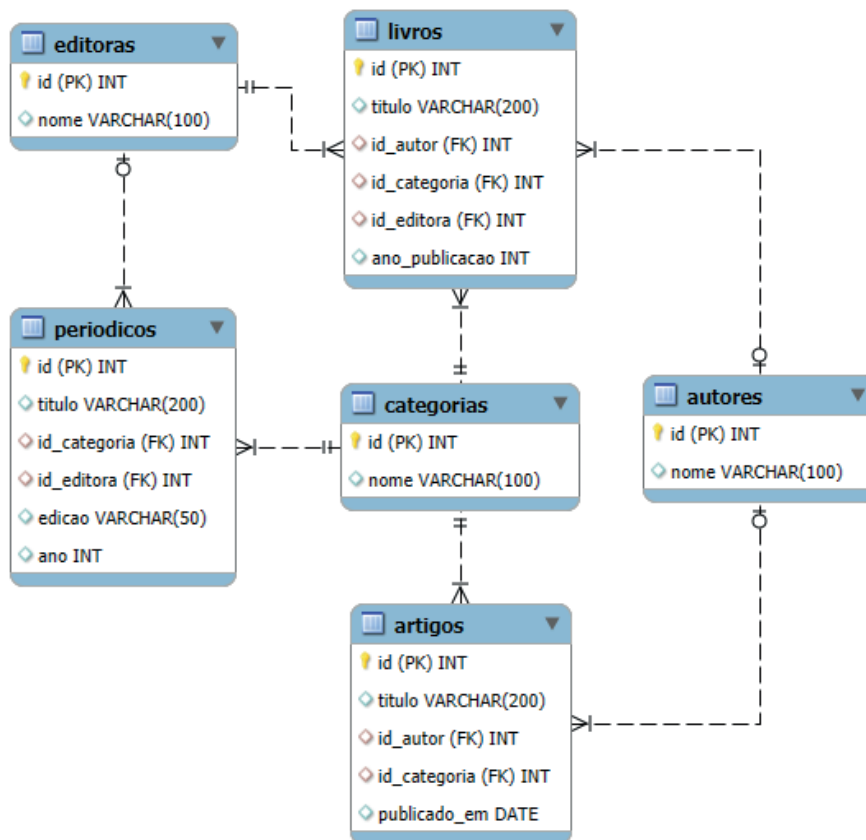
#### IMPORTANTE

##### union e union all

**O `union` realiza, naturalmente, a eliminação de dados duplicados. Contudo, esse comportamento pode ser removido, substituindo-se seu uso pelo comando `union all`.**



Para que esse conceito fique claro, apresentaremos o banco de dados do acervo de uma biblioteca, representada pelo diagrama UML da **Figura 4.47**.



**Figura 4.47** – Diagrama UML de uma aplicação envolvendo o acervo de uma biblioteca.

Crédito: do elaborador, 2025.

Nessa aplicação, os artigos, periódicos e livros são devidamente registrados em entidades separadas, cada uma com seu respectivo relacionamento. Essa organização por tipo de publicação é útil para manter a integridade e a clareza do modelo de dados, mas gera um desafio importante: as buscas por título, realizadas pelos usuários, precisam consultar cada tabela individualmente, de forma repetitiva e fragmentada. Para resolver esse problema de maneira eficiente, o uso do operador **union** torna-se essencial. Com ele, é possível unificar os resultados de múltiplas consultas em uma

única operação, facilitando a exibição conjunta de títulos, independentemente da tabela de origem.

No bloco de código a seguir, veremos como isso pode ser feito na prática.

```
select *
from (
  select titulo, 'Livro' as tipo from livros
  union
  select titulo, 'Artigo' as tipo from artigos
  union
  select titulo, 'Periódico' as tipo from periodicos
) as todas_publicacoes
where lower(titulo) like '%____%';
```

Optamos aqui por unificar cada entidade e aplicar um *alias*, para possibilitar o uso de uma única cláusula **where**, trazendo todos os resultados que satisfaçam a condição de busca.



**Figura 4.48** – União das entidades e seu resultado.

Crédito: do elaborador, 2025.

Conforme você pode identificar, por meio da utilização do **union** é possível estabelecer um sistema de buscas centralizado para cada título existente no acervo, independentemente de seu tipo, atendendo, assim, à necessidade dessa aplicação.

## Subconsultas

Em cenários reais, frequentemente nos deparamos com requisitos em que a consulta principal sozinha não é suficiente para extrair a informação desejada, ou até mesmo para realizar cálculos complexos que dependem de dados de outras tabelas ou de condições mais específicas.

São esses os casos em que as **subconsultas** são úteis, já que elas permitem que você insira uma consulta dentro de outra, tornando possível realizar buscas mais dinâmicas, além de diminuir a necessidade de manipulação de dados, seja no banco ou no *backend* das aplicações.

As subconsultas podem ser usadas em diferentes partes do **select**, como **where** e **from**, com comportamentos específicos.

Veremos a seguir os tipos de subconsultas e exemplos práticos de uso no MySQL.

### Subconsultas com where

Essas subconsultas, como se pode deduzir, são consultas internas realizadas com o objetivo de trazer algum resultado para servir como filtro condicional em uma cláusula **where**.

Seu uso se faz mais abundante em situações que envolvem uma lista dinâmica de valores a serem adicionados em uma cláusula **where**, ou na obtenção de um valor único, obtido por meio de uma outra consulta.

No bloco de código a seguir, podemos observar a estrutura desse processo com alguns exemplos.

```
-- ESTRUTURA DO COMANDO:
select T1.COLUNA1,T1.COLUNA2,T1.COLUNA3... from NOME_DA_TABELA1
T1
  .
  .
  .
```

```

where T1.COLUNA5 'OPERADOR' (select T2.COLUNA3 from NOME_DA_
TABELA2 T2
                                ... )
.
.
.
;

-- EXEMPLOS:
/* obtém os dados do funcionário ativo com maior salário */
select nome, salário from funcionarios
where salario = (
    select max(salario) from funcionarios where ativo = 1
);

/* coleta os dados dos cidadãos que já trabalharam como
mesários em processos eleitorais nos últimos 3 anos */
select c.nome, c.cpf, c.telefone, c.email from cidadaos c
where c.cpf in (
    select m.cpf from mesarios where year(m.data_eleicao) >
        (year(curdate()) - 3 )
);

-- lista as capitais cuja população excede a média
select nome_cidade, populacao from cidades
where populacao > (
    select avg(populacao) from cidades where capital = 1
);

```

Podemos observar na estrutura e nos exemplos do bloco de código anterior que o papel da cláusula **where**, mesmo fazendo uso de subconsultas, permanece o mesmo para filtrar os resultados com o uso dos mais diversos operadores disponíveis no MySQL. Além disso, todas as demais possibilidades de uso a serem adicionadas no comando **select** que você aprendeu até agora também permanecem disponíveis.

**FIQUE ALERTA****Múltiplas colunas**

Subconsultas em cláusulas **where** devem retornar apenas uma coluna. Do contrário, ocorrerá um erro. Note que retornar uma única coluna é diferente de retornar apenas um único resultado, já que uma subconsulta pode retornar uma lista de linhas contendo a mesma coluna/atributo.

Para exemplificar a necessidade de retornar apenas uma única coluna, vamos demonstrar um comando que causa um erro e, logo em seguida, apresentar a solução.

```
-- comando ERRADO: múltiplas colunas dentro da subconsulta
WHERE
select *from países p where p.id in (
    select d.pais_FK, d.data_medicao from dados d
);
```

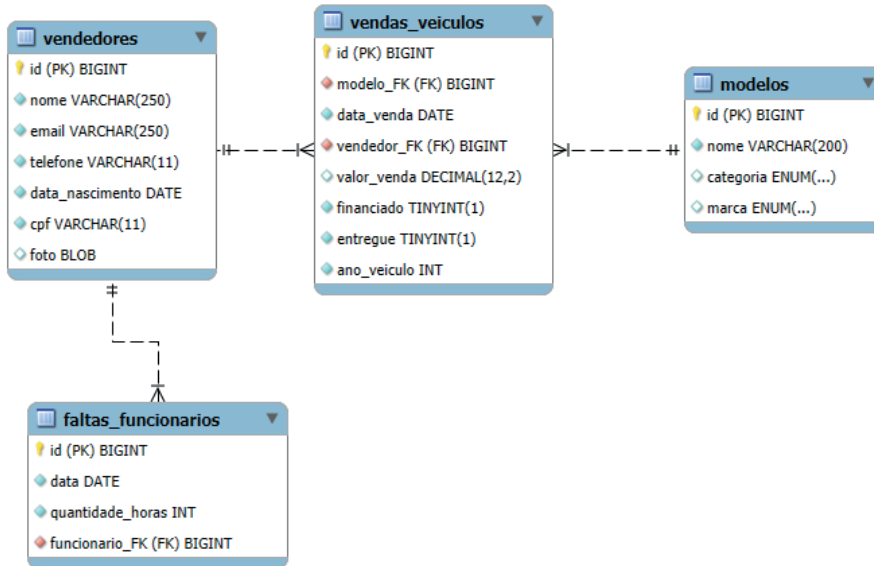
Note que o comando da subconsulta está retornando “d.pais\_FK” e “d.data\_medicao”, portanto, múltiplas colunas. Quando executado, isso ocasionará o seguinte erro: **“Error Code: 1241. Operand should contain 1 column(s)”**, indicando a necessidade da operação retornar apenas uma única coluna.

No bloco de código a seguir, o comando corrigido:

```
-- comando CORRETO: única coluna dentro da subconsulta WHERE
select *from países p where p.id in (
    select d.pais_FK from dados d
);
```

Agora que alguns pontos importantes foram esclarecidos, que tal entendermos um uso mais prático para essas subconsultas?

Vamos recuperar a aplicação de uma concessionária de veículos, abordada no tópico **Agrupamentos e agregações** deste capítulo, adicionando apenas mais uma entidade (“faltas\_funcionarios”), conforme diagrama UML da **Figura 4.49**.



**Figura 4.49** – Diagrama UML de uma aplicação envolvendo uma concessionária de veículos.

Crédito: do elaborador, 2025.

Conforme você deve se lembrar, essa plataforma foi criada com o objetivo de não somente registrar as vendas de veículos, como também gerar relatórios de dados contendo informações relevantes para a gestão da empresa, tais como fluxo de vendas, distribuição da quantidade de transações por marca e categoria, extração do montante negociado por cada vendedor, entre outras métricas.

Naturalmente, como ocorrem nos cenários reais, os sistemas continuam em expansão ao longo de seu ciclo de vida, tendo sido adicionada na base de dados mais uma tabela para registrar as faltas dos funcionários e assim melhorar o controle de absenteísmo por meio de incentivos e bonificações.

Desse modo, surgem outras demandas de extração de dados, representadas nos itens a seguir:

- Extração dos vendedores que ainda não faltaram no ano atual (para promover bonificação);

- Extração dos vendedores que já realizaram vendas no mês atual e que também não faltaram no mesmo mês (para promover maior bonificação).

Você deve concordar que tais requisitos possuem de fato utilidade, sendo essa uma das principais missões quando se envolve bancos de dados: a capacidade de transformá-los em informações úteis e valiosas. Você seria capaz de executá-las utilizando seu conhecimento atual? Caso sua resposta seja negativa, não se assuste, veremos como realizá-las a seguir.

Veja no bloco de código a seguir como fazer a extração dos vendedores que ainda não faltaram no ano atual (para promover bonificação).

```
-- obtém os vendedores que não estão na tabela de faltas
select *from vendedores v where
v.id not in (
  select f.funcionario_FK from faltas_funcionarios f
  where year(f.data) = year(curdate())
);
```

Observe que no bloco de código anterior temos uma consulta básica sendo realizada diretamente na tabela de vendedores, filtrada, porém, por todos os vendedores que possuem **id** (chave primária) que não estejam presentes no conjunto de linhas retornados pela subconsulta dentro de **not in**. A subconsulta, por sua vez, faz a extração da chave estrangeira de cada funcionário que está presente na tabela de faltas, fazendo uso das funções **year** e **curdate** para garantir que sejam retornados apenas os dados que coincidam com o ano atual.

#### SAIBA MAIS

##### Ordem de execução das consultas

Assim como ocorrem em expressões matemáticas que fazem uso de diversos parênteses, seguindo a ordem de execução das expressões internas para as externas, as subconsultas obedecem a mesma analogia. Ou seja, o MySQL

executará primeiro as consultas mais internas em direção às externas. Esse processo garante que, por exemplo, no bloco de código anterior, os vendedores sejam buscados apenas quando a consulta na tabela de faltas já tenha sido concluída.

Agora, executaremos o comando para verificar o resultado, conforme se vê na **Figura 4.50**.

| id | nome            | email                     | telefone    | data_nascimento | cpf         | foto |
|----|-----------------|---------------------------|-------------|-----------------|-------------|------|
| 5  | Pedro Santos    | pedro.santos@email.com    | 41987654321 | 1983-11-18      | 56789012345 | NULL |
| 6  | Fernanda Lima   | fernanda.lima@email.com   | 51987654321 | 1992-04-05      | 67890123456 | NULL |
| 7  | Lucas Costa     | lucas.costa@email.com     | 61987654321 | 1997-09-30      | 78901234567 | NULL |
| 8  | Juliana Machado | juliana.machado@email.com | 71987654321 | 1986-06-25      | 89012345678 | NULL |
| 11 | Luís Gaspetto   | luis.gaspetto@email.com   | 11988654333 | 1949-08-16      | 12125578933 | NULL |

**Figura 4.50** – Resultado dos vendedores que ainda não faltaram ao trabalho, em uma concessionária.

Crédito: do elaborador, 2025.

Veja no bloco de código a seguir como fazer a extração dos vendedores que já realizaram vendas no mês atual e que também não faltaram no mesmo mês (para promover maior bonificação).

```
-- obtém os vendedores que fizeram vendas e não faltaram no
mês atual
select *from vendedores v where
v.id not in (
  select f.funcionario_FK from faltas_funcionarios f
  where (month(f.data) = month(curdate())) and
  (year(f.data) = year(curdate())) )
)
and v.id in (
  select vv.vendedor_FK from vendas_veiculos vv where
  (month(vv.data_venda) = month(curdate())) and
  (year(vv.data_venda) = year(curdate())) )
);
```

Como detalhe aqui, demonstramos a possibilidade de uso de quantidade ilimitada de subconsultas no mesmo comando, desde que se atenda



ao requisito básico para evitar o retorno de múltiplas colunas. Veremos o resultado na **Figura 4.51**.

| id | nome            | email                     | telefone    | data_nascimento | cpf         | foto                                                                                |
|----|-----------------|---------------------------|-------------|-----------------|-------------|-------------------------------------------------------------------------------------|
| 1  | João Silva      | joao.silva@email.com      | 11987654321 | 1985-03-15      | 12345678901 |  |
| 2  | Maria Oliveira  | maria.oliveira@email.com  | 11912345678 | 1990-07-22      | 23456789012 |  |
| 3  | Carlos Pereira  | carlos.pereira@email.com  | 21987654321 | 1988-05-10      | 34567890123 |  |
| 4  | Ana Souza       | ana.souza@email.com       | 31987654321 | 1995-12-01      | 45678901234 |  |
| 5  | Pedro Santos    | pedro.santos@email.com    | 41987654321 | 1983-11-18      | 56789012345 |  |
| 6  | Fernanda Lima   | fernanda.lima@email.com   | 51987654321 | 1992-04-05      | 67890123456 |  |
| 7  | Lucas Costa     | lucas.costa@email.com     | 61987654321 | 1997-09-30      | 78901234567 |  |
| 8  | Juliana Machado | juliana.machado@email.com | 71987654321 | 1986-06-25      | 89012345678 |  |
| 10 | Camila Ribeiro  | camila.ribeiro@email.com  | 91987654321 | 1998-02-20      | 01234567890 |  |

**Figura 4.51** – Resultado dos vendedores que ainda não faltaram no mês atual ao trabalho e que já tiveram algum tipo de venda em uma concessionária.

Crédito: do elaborador, 2025.

## Subconsultas como tabelas com **from/join**

Assim como no **where**, as subconsultas no **from/join** executam buscas internas antes da consulta principal. No entanto, há uma diferença importante: elas atuam como tabelas temporárias e, portanto, podem retornar múltiplas colunas.

Como de costume, nada melhor do que analisarmos a estrutura do comando, com alguns exemplos, no bloco de código a seguir.

```
-- ESTRUTURA DO COMANDO:
select T1.COLUNA1,T1.COLUNA2,T1.COLUNA3... from (
  select T2.COLUNA3, T2.COLUNA5 from NOME_DA_TABELA2 T2 ...
) as T1
.
.
.
;

-- EXEMPLOS:
/* obtém os dados do departamento que possui media salarial
maior que 5000 */
select d.nome_departamento, sub.media_salarial
```

```

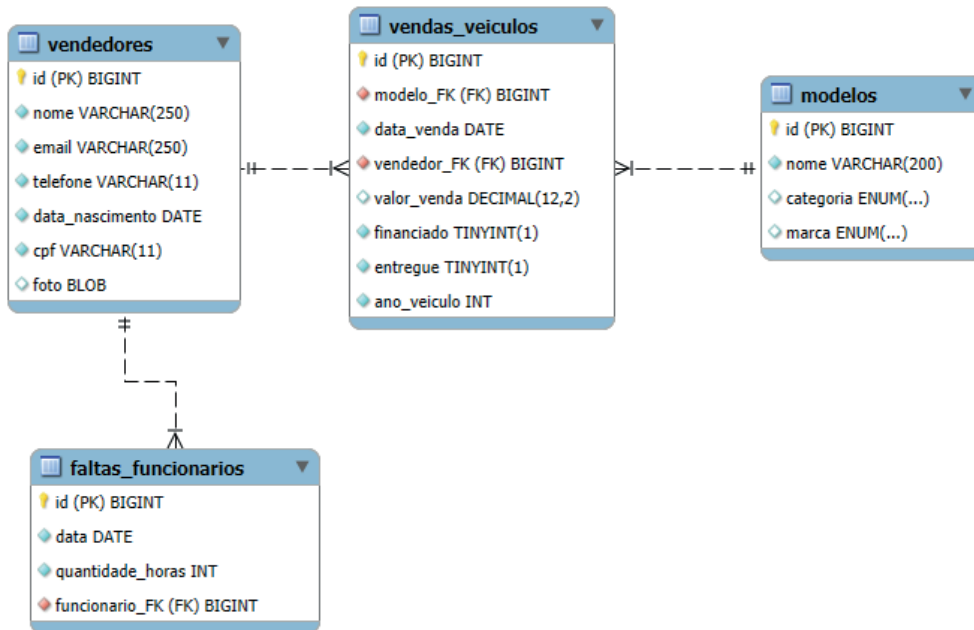
from departamentos d
join (
  select departamento_FK, avg(salario) as media_salarial
  from funcionarios
  group by departamento_FK
) as sub
on d.id = sub.departamento_FK
where sub.media_salarial > 5000;

/* soma as médias de faturamento de cada unidade de negócio
para saber o total estimado da empresa */
select sum(tabela_medias.media) as 'total faturamento' from (
  select f.unidade_negocio, avg(f.faturamento) as media
  from faturamentos f
  where f.data between '2024-01-01' and '2024-12-31'
  group by f.unidade_negocio
) as tabela_medias;

```

À primeira vista, esses comandos um tanto quanto extensos podem parecer difíceis de compreender, não é mesmo? Entretanto, mude seu foco de atenção e perceba que em todos os exemplos do bloco de código as subconsultas são tratadas exatamente como se fossem tabelas, ganhando inclusive um nome. De fato, foi exatamente isso o que ocorreu: foram geradas tabelas temporárias que servem como referência para outras consultas, sendo obrigatório nomeá-las, como fizemos no primeiro exemplo (subtabela “sub”) e no segundo (subtabela “tabela\_medias”), caso contrário um erro seria gerado pelo MySQL.

Para finalizarmos com “chave de ouro” e checarmos a utilidade desse tipo de comando, vamos fazer novamente uma imersão para um contexto prático, reaproveitando a aplicação da concessionária de veículos que você tem acompanhado em sua jornada até aqui.



**Figura 4.52** – Diagrama UML de uma aplicação envolvendo uma concessionária de veículos.

Crédito: do elaborador, 2025.

Agora, essa concessionária de veículos deseja aumentar sua capacidade de extração de dados, já que as informações anteriores tiveram um excelente resultado, sobretudo na performance dos colaboradores por meio da utilização de bonificações e reconhecimentos de sua dedicação e desempenho. Para isso, será realizada uma premiação especial para todos os vendedores que atendam à seguinte extração: extração dos vendedores que tenham faltas menores que cinco no ano e tenham vendido acima da média anual por vendedor. Note que, para ambos os casos, não possuímos essas informações prontamente disponíveis em apenas um único agrupamento, já que estamos nos referindo a uma comparação da média de todas as médias. Dessa forma, deveremos executar o comando do bloco de código a seguir.

```

-- seleciona os vendedores cuja média supera a média das médias
select v2.id, v2.nome, sum(vv2.valor_venda) as vendas2
  from vendedores v2
 inner join vendas_veiculos vv2 on vv2.vendedor_FK = v2.id
 where year(vv2.data_venda) = year(curdate())
 group by v2.id
 having vendas2 > (
   -- calcula a média de todas as médias
   select avg(media_vendas.vendas) as media_das_medias from (
     -- calcula a média de cada vendedor:
     select v.id, v.nome, sum(vv.valor_venda) as vendas
     from vendedores v
    inner join vendas_veiculos vv
      on vv.vendedor_FK = v.id
     where year(vv.data_venda) = year(curdate())
     group by v.id ) as media_vendas
 ) and v2.id not in (
   select ff.funcionario_FK from faltas_funcionarios ff
   where year(data) = year(curdate())
 );

```

Da mesma forma que fizemos nos processos anteriores, estamos calculando cada média individualmente, em seguida calculamos a média geral e, por fim, filtramos apenas os vendedores que superaram o valor de referência. Por último, mas não menos importante, consideramos apenas aqueles que não tiveram nenhuma falta no ano atual.

Na **Figura 4.53**, podemos ver os resultados.

| id | nome          | vendas2   |
|----|---------------|-----------|
| 6  | Fernanda Lima | 880000.00 |
| 7  | Lucas Costa   | 993000.00 |

**Figura 4.53** – Resultado dos vendedores que superaram a média e não faltaram no trabalho no ano atual.

Crédito: do elaborador, 2025.

## Resumo

Neste capítulo, aprofundamos alguns dos conceitos mais poderosos do MySQL, explorando funções, **joins**, agrupamentos com **group by** e subconsultas. Todos esses tópicos são essenciais para qualquer aplicação que precise extrair e manipular dados de forma eficiente e estruturada.

Começamos aprendendo como as funções do MySQL podem simplificar o trabalho com dados, tornando tarefas como cálculos e transformações mais ágeis. Depois, mergulhamos nos **joins**, que nos permitem combinar dados de diferentes tabelas, criando visões mais completas e enriquecedoras. Em seguida, exploramos os agrupamentos com **group by**, uma ferramenta fundamental para sumarizar informações, o que é especialmente útil quando trabalhamos com grandes volumes de dados que precisam ser analisados em grupos. Por fim, as subconsultas nos abriram um leque de possibilidades, permitindo que realizássemos consultas complexas e aninhadas, em que uma consulta depende do resultado de outra, aumentando a flexibilidade e o poder das nossas operações no banco de dados.

Com o conhecimento adquirido até aqui, você deve estar mais confiante para utilizar o banco de dados em diferentes situações práticas e, sem dúvida, mais preparado para atender às necessidades do mercado. Entretanto, embora este capítulo tenha sido fundamental, ainda há muitos outros temas a serem explorados para que você continue ampliando seu domínio sobre o uso de bancos de dados MySQL, o que exigirá sua dedicação no próximo capítulo.



## CAPÍTULO 5

# GESTÃO DO BANCO DE DADOS – MODELO RELACIONAL

**F**inalmente, chegamos ao último capítulo da nossa jornada pelo mundo do MySQL. Ao longo deste livro, você percorreu uma trajetória repleta de conceitos essenciais sobre bancos de dados, aprendendo desde os fundamentos até a construção de consultas mais elaboradas. A essa altura, você já sabe modelar dados, estruturar tabelas, manipular informações e até mesmo otimizar consultas para melhorar o desempenho das suas aplicações.

Certamente, essa caminhada trouxe muitos desafios, mas cada um deles fortaleceu seu conhecimento e sua capacidade de trabalhar com o MySQL de forma mais confiante.

No entanto, se observarmos o mundo real, veremos que, muitas vezes, o simples uso de consultas SQL não é suficiente para lidar com a complexidade dos sistemas modernos. Empresas lidam diariamente com grandes volumes de dados, regras de negócio bem-definidas e a necessidade de garantir que certas ações aconteçam de maneira automática e segura.

Pense em processos complexos que precisam ser executados de forma padronizada, regras de validação que devem ser aplicadas sempre que um dado for inserido ou modificado, ou ações que precisam ser disparadas automaticamente em resposta a um evento específico no banco. Como assegurar que operações críticas, como transferências bancárias, sejam concluídas integralmente ou, em caso de falhas, revertidas de maneira segura e controlada? E de que forma administrar, com precisão e segurança, os diferentes níveis de permissão de acesso ao banco de dados, especialmente em sistemas com múltiplos usuários e funções bem-definidas?

Para atender a essas demandas, os bancos de dados oferecem recursos mais sofisticados que ajudam a automatizar processos, garantir a integridade dos dados e reforçar a segurança do sistema.

Neste capítulo final, vamos explorar três temas essenciais para quem deseja dar um passo além na administração de bancos de dados: *procedures*, *triggers* e os comandos **DCL** (*Data Control Language*) e **TCL** (*Transaction Control Language*). Com eles, você aprenderá a criar rotinas automatizadas dentro do banco, definir ações que ocorrem de forma reativa às operações e controlar permissões e transações de maneira eficiente.

Se você chegou até aqui, significa que já possui uma base sólida e está preparado para entender esses conceitos mais avançados. Então, vamos juntos encerrar este aprendizado com chave de ouro!

## Triggers

O funcionamento dos *triggers* é facilmente compreensível ao se analisar sua tradução do inglês: **gatilhos**.

Podemos defini-los como funções que são executadas quando determinados gatilhos são iniciados. Em geral, esses gatilhos consistem em operações tradicionais do SQL, como **insert**, **update** ou **delete**.

O uso dos *triggers*, na maioria das vezes, está presente quando se deseja obter histórico de transações ou executar determinadas operações em cadeia.

Para diminuir a subjetividade do assunto, vamos recorrer a um exemplo. Imagine um banco de dados de uma loja de materiais de construção, contando evidentemente com uma quantidade expressiva de itens a serem comercializados em seu estoque e fazendo uso de um sistema para controle desse estoque e de suas respectivas vendas.

Dentro desse contexto, você deve concordar que, após a efetivação de uma venda, os itens que foram comercializados devem sofrer uma atualização em sua quantidade no estoque, correto? Por exemplo, se foram vendidas cinco torneiras de um total de 30 que estavam disponíveis, o sistema deveria, durante o registro dessa venda, atualizar a quantidade real disponível, isto é, contar agora com apenas 25 torneiras no estoque.

Observe que, em nosso exemplo, o processo de atualização do estoque deve sempre ocorrer após a inserção de um registro de venda, ou seja, problema facilmente resolvido por meio da criação de um *trigger* associado com o **insert** da tabela de vendas. Portanto, para esse caso, o **insert** seria o gatilho dessa operação de atualização.



## IMPORTANTE

**Frameworks backend**

Você verá ao longo deste capítulo que os *triggers* possuem grande utilidade, realizando papel fundamental em inúmeras operações. Contudo, vale ressaltar ao(à) aluno(a) que, com o avanço da capacidade tecnológica dos *frameworks backend*, as aplicações têm transferido algumas das responsabilidades anteriormente executadas pelos *triggers* para a camada do *backend*. *Frameworks* como Spring Boot (Java), Django (Python), Express (Node.js) e Laravel (PHP), por exemplo, permitem que validações, atualizações automáticas e regras de negócio sejam implementadas diretamente no código da aplicação.

Esse processo centraliza a lógica de negócio fora do banco de dados, proporcionando maior controle, flexibilidade e integração com outras funcionalidades da aplicação, além de facilitar testes, manutenção e escalabilidade do sistema como um todo, diminuindo a necessidade de alterações diretas no banco de dados.

## create trigger – Criando *triggers*

Antes de mergulharmos no código do exemplo da loja de materiais de construção, apresentaremos a você a estrutura principal do comando de criação dos *triggers* em SQL.

```
-- ESTRUTURA DO COMANDO:
delimiter $$
create trigger NOME_DO_TRIGGER [BEFORE / AFTER]
[INSERT / UPDATE / DELETE] ON NOME_DA_TABELA FOR EACH ROW
BEGIN
    -- COMANDOS SQL QUANDO O TRIGGER FOR EXECUTADO....
END;
$$
delimiter ;
```

```

-- EXEMPLOS:
/* criação do trigger para registrar em uma tabela os novos
clientes cadastrados
(para que a empresa possa oferecer planos ou serviços
adicionais) */
delimiter $$
create trigger trigger_novos_clientes AFTER INSERT ON clientes
for each row
BEGIN
    insert into novos_clientes (nome, cliente_FK, data_inclusao)
    values (NEW.nome, NEW.id, now());
END;
$$
delimiter ;

```

Nesse momento, após analisar as informações do bloco de código anterior, você provavelmente deve estar repleto de dúvidas, afinal, estamos utilizando uma estrutura de comando com novos elementos e um pouco mais complexa do que as que vimos anteriormente.

De modo a esclarecer tais dúvidas, vamos focar no exemplo abordado no bloco de código, sobre novos clientes, e explicá-lo em partes.

Primeiro, nos atentaremos ao uso do comando **delimiter**, conforme se pode ver no bloco de código a seguir.

```

-- EXEMPLOS:
/* criação do trigger para registrar em uma tabela os novos
clientes cadastrados (para que a empresa possa oferecer
planos ou serviços adicionais) */
delimiter $$
create trigger trigger_novos_clientes AFTER INSERT ON
clientes
for each row

```

```

BEGIN
insert into novos_clientes (nome, cliente_FK, data_inclusao)
  values (NEW.nome, NEW.id, now());
END;
$$
delimiter ;

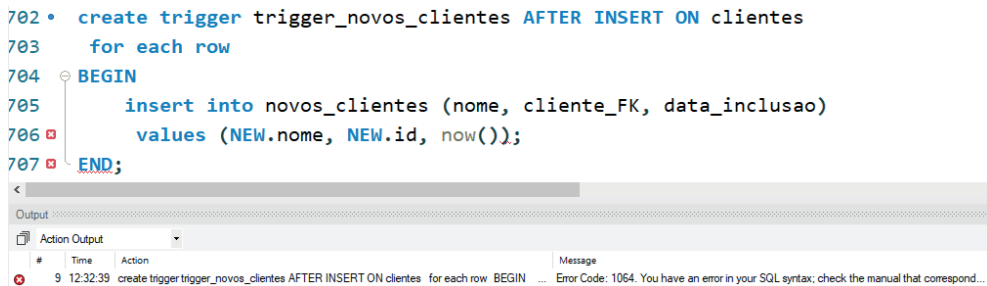
```

O **delimiter** não afeta diretamente o funcionamento do *trigger*; sua função é permitir que o MySQL interprete corretamente onde começa e termina a definição do gatilho. Isso é necessário porque o MySQL, por padrão, usa o caractere “;” (ponto e vírgula) para encerrar comandos. No entanto, dentro do próprio *trigger*, podem existir múltiplos pontos e vírgulas que não indicam o fim do gatilho, mas sim a conclusão de cada instrução dentro dele.

Desse modo, o comando **delimiter** age de forma a redefinir, temporariamente, outro caractere como finalizador de comando (neste caso, empregamos \$\$) a ser usado pelo MySQL.

Ao final do comando *trigger*, restauramos o ponto e vírgula como o finalizador de comando padrão.

Na **Figura 5.1**, observe o erro gerado, quando não utilizamos o recurso mencionado em um *trigger*.



**Figura 5.1** – Erro gerado devido ao não uso do **delimiter**.

Crédito: do elaborador, 2025.

Agora que você já entendeu o uso do **delimiter**, vamos nos atentar às condições de gatilho, conforme se vê no bloco de código a seguir.

```
-- EXEMPLOS:
/* criação do trigger para registrar em uma tabela os novos
clientes cadastrados
(para que a empresa possa oferecer planos ou serviços
adicionais) */
delimiter $$
create trigger trigger_novos_clientes AFTER INSERT ON
clientes
for each row
BEGIN
insert into novos_clientes (nome, cliente_FK,
data_inclusao)
values (NEW.nome, NEW.id, now());
END;
$$
delimiter ;
```

Como podemos observar no bloco de código anterior, estamos criando um gatilho que será disparado sempre após (**after**) a inserção de algum registro na tabela de clientes (**insert on clientes**).

Como já é de seu conhecimento, em um único comando **insert** é possível incluir múltiplas linhas, e é por esse motivo que incluímos o **for each row**, para determinar que para cada linha envolvida no evento de gatilho haverá uma execução correspondente do *trigger*.

E, por fim, mas não menos importante, observaremos no bloco de código a seguir as instruções a serem executadas após o acionamento do *trigger*.

```
-- EXEMPLOS:
/* criação do trigger para registrar em uma tabela os novos
clientes cadastrados
```

```

    (para que a empresa possa oferecer planos ou serviços
    adicionais) */
    delimiter $$
    create trigger trigger_novos_clientes AFTER INSERT ON
    clientes
    for each row
    BEGIN
        insert into novos_clientes (nome, cliente_FK,
    data_inclusao)
        values (NEW.nome, NEW.id, now());
    END;
    $$
    delimiter ;

```

Encapsuladas entre o **begin** e **end**, podemos definir o conjunto de instruções a serem executadas quando o gatilho for acionado, havendo a possibilidade de uso de praticamente qualquer comando SQL conhecido (embora seja mais comum o uso de **insert**, **update** ou **delete**).

Outro ponto importante a ser considerado: aqui temos acesso completo aos dados envolvidos na operação de gatilho. Basta utilizarmos as palavras **new** ou **old** antes do atributo desejado.

Em nosso exemplo, como desejamos obter **nome** e **id** do novo cliente, envolvidos na operação **insert** que disparou o gatilho, temos acesso completo a essas informações utilizando “NEW.nome” e “NEW.id”.

Para esse caso específico, não teríamos acesso a nenhuma informação utilizando o **old**, já que ele é utilizado para recuperar os dados antigos, antes da operação de gatilho. No entanto, o **old** é muito utilizado em situações em que os gatilhos ocorrem com os comandos **delete** ou **update**.

Para deixar mais claro, em uma operação na qual o gatilho é um **update**, utilizar o comando **old** nos dará acesso aos dados antes da atualização; já com o emprego do **new**, possuímos os dados novos durante o processo de atualização.

Usando o mesmo contexto do exemplo abordado, imagine que um cliente que acaba de se cadastrar deseja fazer uma correção em seu nome. Nesse caso, o nome anterior ainda permanecerá na tabela de “novos\_clientes” se não providenciarmos um *trigger* de atualização.

Para atender a essa situação, criaremos mais um gatilho, porém agora vinculado com o comando **update**, adicionando uma cláusula **where** que deverá ser executada apenas se o nome for alterado.

```
delimiter $$
create trigger trigger_atualiza_clientes AFTER UPDATE ON
clientes
for each row
BEGIN
    update novos_clientes set nome = NEW.nome where
        OLD.nome != NEW.nome and id = NEW.id;
END;
$$
delimiter ;
```

## show triggers – Exibindo os *triggers* existentes

Vimos até agora diversas variantes do comando **show**, de grande utilidade para conhecermos tabelas, índices e demais recursos criados no MySQL.

Apresentaremos agora mais uma opção disponível, para podermos visualizar os *triggers* existentes.

```
show triggers;
```

### IMPORTANTE

#### **Schema selecionado**

O comando **show triggers** exibirá apenas os existentes no *schema* selecionado, portanto, confirme se você está utilizando a base de dados desejada.

## drop trigger – Removendo *triggers*

De forma muito semelhante ao que ocorre com as tabelas, utilizamos o comando **drop** para remover os *triggers* existentes.

No bloco de código a seguir, vejamos sua estrutura e exemplos.

```
-- ESTRUTURA DO COMANDO:  
drop trigger NOME_DO_TRIGGER;  
  
-- EXEMPLOS:  
drop trigger backup_trigger;  
drop trigger atualizacao;
```

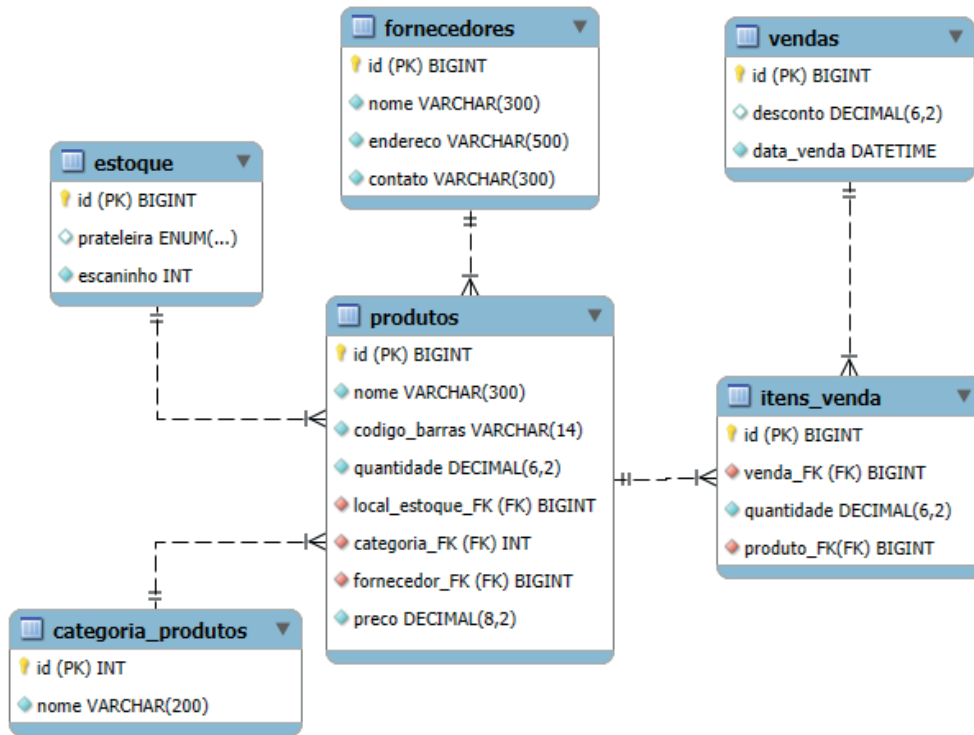
Agora que você já entende um pouco melhor o funcionamento e a estrutura dos *triggers*, nada melhor do que utilizarmos alguns exemplos com contextos mais realistas para promover a fixação do conteúdo.

## Exemplos e casos de uso – Depósito de materiais de construção

Nesta situação-problema, retomaremos o exemplo da loja de materiais de construção, abordado no início deste capítulo. Essa loja conta atualmente com um sistema de vendas que não se comunica com seu estoque, havendo a necessidade de atualizá-lo de forma manual, situação que frequentemente ocasiona divergências devido a preenchimentos incorretos.

Por esse motivo, chega até você a necessidade de desenvolvimento de um sistema capaz de gerenciar todo o processo de registro de vendas, como também o gerenciamento de estoque dos respectivos produtos. Além disso, será necessário considerar o requisito fundamental: para cada venda efetuada, o sistema deve atualizar automaticamente a quantidade disponível no estoque.

Seu time, alinhado com o cliente, chegou à modelagem do banco, conforme se vê na **Figura 5.2**.



**Figura 5.2** – Diagrama UML de uma loja de materiais de construção.

Crédito: do elaborador, 2025.

Como se pode perceber nessa modelagem, temos uma estrutura capaz de registrar os produtos, com suas respectivas categorias e fornecedores, bem como informações de localização no estoque, além também do relacionamento que se espera para efetuar as vendas.

Agora, serão criadas as entidades fortes dessa modelagem, isto é, as tabelas que não dependem de nenhuma chave estrangeira.

```

-- cria a tabela de categoria de produtos:
create table categoria_produtos (
    id int primary key auto_increment,
    nome varchar(200) not null
);

-- cria a tabela de fornecedores:
create table fornecedores (

```



```

    id bigint primary key auto_increment,
    nome varchar(300) not null,
    endereco varchar(500) not null,
    contato varchar(300) not null
);

-- cria a tabela de estoque:
create table estoque (
    id bigint primary key auto_increment,
    prateleira enum('A','B','C','D','E'),
    escaninho integer not null,
    check(escaninho > 0)
);

```

No bloco de código a seguir, faremos a criação das tabelas faltantes, utilizando como chaves estrangeiras as tabelas já existentes.

```

create table produtos (
    id bigint primary key auto_increment,
    nome varchar(300) not null,
    codigo_barras varchar(14) not null,
    quantidade decimal(6,2) not null default 0.0,
    local_estoque_FK bigint not null,
    categoria_FK int not null,
    fornecedor_FK bigint not null,
    preco decimal(8,2) not null,
    foreign key(categoria_FK) references categoria_produtos(id),
    foreign key(fornecedor_FK) references fornecedores(id),
    foreign key(local_estoque_FK) references estoque(id),
    check(length(codigo_barras) = 14),
    check(quantidade >= 0.0),
    check(preco > 0.0)
);

```

```

create table vendas (
    id bigint primary key auto_increment,
    desconto decimal(6,2) default 0.0,
    data_venda datetime not null default now(),
    check(desconto >= 0.0)
);

create table itens_venda (
    id bigint primary key auto_increment,
    venda_FK bigint not null,
    quantidade decimal(6,2) not null default 1.0,
    produto_FK bigint not null,
    foreign key(venda_FK) references vendas(id),
    foreign key(produto_FK) references produtos(id),
    check(quantidade > 0)
);

```

Finalmente, criaremos o *trigger* para solucionar a questão de atualização das quantidades disponíveis de produtos. Seu gatilho deve estar associado com o processo de inclusão de itens de uma venda.

```

delimiter $$
create trigger atualiza_quantidade_produtos after insert on
itens_venda
for each row
begin
    update produtos p set p.quantidade = ( p.quantidade -
NEW.quantidade )
    where p.id = NEW.produto_FK and p.quantidade >= NEW.
quantidade;
end

$$
delimiter ;

```

Sendo os *triggers* o foco desta situação-problema, vamos aos pormenores dessa criação:

- Definimos como gatilho a inclusão de cada item de uma venda, pois é nesse momento que possuímos a quantidade e o produto comercializado;
- Cada vez que o gatilho for acionado, temos a confirmação de que o produto já existe (caso contrário, o gatilho nem será ativado, devido à restrição referencial existente em “itens\_venda” ao utilizar a chave estrangeira “produto\_FK”). Desse modo, utilizamos o comando **update** para possibilitar a atualização desse produto já presente no banco de dados;
- Na operação de **update**, estamos redefinindo a quantidade disponível do produto, subtraindo da quantidade atual o valor envolvido na operação de venda;
- Para garantir a seleção do produto correto, utilizamos a chave estrangeira “produto\_FK” dentro da cláusula **where**, obtendo apenas o produto com o “id” que está presente na operação de venda;
- Finalmente, adicionamos também, dentro da cláusula **where**, uma condição para atualizar apenas os produtos que possuem quantidade suficiente para atender ao processo de venda, garantindo assim que nenhum item fique com quantidade negativa (“p.quantidade >= NEW.quantidade”).

Agora que já criamos toda a estrutura necessária, é fundamental testarmos para checar se o comportamento está de acordo com o esperado. Para isso, faremos a inclusão de uma carga de dados para testes, começando também com as inclusões nas tabelas independentes (entidades fortes).

```

insert into categoria_produtos (nome) values
('Cimento e Argamassa'), ('Tijolos e Blocos'),
('Ferragens e Fixadores'),('Hidráulica'),
('Elétrica');

insert into fornecedores (nome, endereco, contato) values
('Construmat Distribuidora', 'Rua das Pedras, 123, São Paulo
- SP', '(11) 98765-4321'),
('Fortalece Materiais', 'Avenida Concreto, 456, Rio de Janeiro
- RJ', '(21) 99876-5432'),
('MaxObra Suprimentos', 'Rua dos Tijolos, 789, Belo Horizonte
- MG', '(31) 91234-5678'),
('Hidrosul Tubos e Conexões', 'Praça das Águas, 101, Curitiba
- PR', '(41) 92345-6789'),
('EletroCon Construção', 'Estrada dos Fios, 202, Porto Alegre
- RS', '(51) 93456-7890');

insert into estoque (prateleira, escaninho) values
('A', 1), ('A', 2), ('B', 3), ('B', 4),
('C', 5), ('C', 6), ('D', 7), ('E', 8);

```

Em seguida, faremos a inserção nas entidades de produtos e vendas, com atenção para usar as chaves estrangeiras com ids já existentes.

```

insert into produtos (nome, codigo_barras, quantidade, local_
estoque_FK, categoria_FK, fornecedor_FK, preco) values
('Cimento CP IV 50kg', '78912345678901', 100.00, 1, 1, 1, 45.90),
('Argamassa ACIII 20kg', '78912345678902', 80.00, 2, 1, 2,
25.50),
('Tijolo Cerâmico 6 furos', '78912345678903', 500.00, 3, 2, 3,
1.20),
('Bloco de Concreto 39x14x19cm', '78912345678904', 300.00, 4, 2,
3, 3.50),
('Parafuso 10mm c/ 100un', '78912345678905', 50.00, 5, 3, 4,
15.75),

```

```
(‘Tubo PVC 50mm 6m’, ‘78912345678906’, 200.00, 6, 4, 4, 32.90),
(‘Conexão PVC Joelho 90° 50mm’, ‘78912345678907’, 150.00, 7, 4,
4, 8.60),
(‘Fio Elétrico 2,5mm 100m’, ‘78912345678908’, 60.00, 8, 5, 5,
125.00),
(‘Tomada 10A Branco’, ‘78912345678909’, 120.00, 1, 5, 5, 9.90),
(‘Disjuntor 20A’, ‘78912345678910’, 90.00, 2, 5, 5, 22.50);

insert into vendas (desconto) values (1.50);
```

Antes de iniciarmos a inclusão dos itens vendidos, o que ocasionará o gatilho de atualização, consultaremos a quantidade atual disponível, para checagem do resultado.

```
66 • select *from produtos where id in (1,4);
```

| id | nome                         | codigo_barras  | quantidade | local_estoque_FK | categoria_FK | fornecedor_FK | preco |
|----|------------------------------|----------------|------------|------------------|--------------|---------------|-------|
| 1  | Cimento CP IV 50kg           | 78912345678901 | 100.00     | 1                | 1            | 1             | 45.90 |
| 4  | Bloco de Concreto 39x14x19cm | 78912345678904 | 300.00     | 4                | 2            | 3             | 3.50  |

**Figura 5.3** – Exibição dos produtos antes da inclusão da venda.

Crédito: do elaborador, 2025.

Finalmente, incluiremos os itens a serem vendidos, o que disparará o gatilho para a atualização.

```
66 • insert into itens_venda (venda_FK, quantidade, produto_FK)
67 values
68 (1, 15.00, 1), -- venda de 15kg do Cimento (ID 1)
69 (1, 100.00, 4); -- venda de 100un de Blocos de Concreto (ID 4)
70
71 • select *from produtos where id in (1,4);
```

| id | nome                         | codigo_barras  | quantidade | local_estoque_FK | categoria_FK | fornecedor_FK | preco |
|----|------------------------------|----------------|------------|------------------|--------------|---------------|-------|
| 1  | Cimento CP IV 50kg           | 78912345678901 | 85.00      | 1                | 1            | 1             | 45.90 |
| 4  | Bloco de Concreto 39x14x19cm | 78912345678904 | 200.00     | 4                | 2            | 3             | 3.50  |

**Figura 5.4** – Comandos em SQL para inserção e consulta após o processo de uma venda.

Crédito: do elaborador, 2025.

Note que, após a inclusão dos itens vendidos correspondentes aos produtos 1 e 4, as novas quantidades foram atualizadas corretamente.

Para auxiliar o(a) aluno(a) a interpretar o resultado, essa atualização seguiu o seguinte cálculo:

- Produto 1, antes da atualização: 100.00; após a atualização: 85.00;
- Produto 4, antes da atualização: 300.00; após a atualização: 200.00.

## Exercícios – Depósito de materiais de construção: continuação

Agora que você já sabe como usá-los, aplique os *triggers* no mesmo contexto da loja de materiais de construção, mas agora com foco em atender às seguintes demandas:

1. Existem situações em que a quantidade de um item vendido precisa ser alterada (por exemplo, em devoluções, trocas de itens, ou para corrigir valores errados). Desse modo, crie um *trigger* que será acionado com o comando **update** na tabela de “itens\_venda” para atualizar a quantidade disponível no estoque, caso tenha havido alguma alteração na quantidade vendida.
2. Caso alguma venda tenha que ser cancelada ou estornada, a quantidade disponível de produtos deve ser atualizada. Portanto, crie um *trigger* que será disparado com o comando **delete** na tabela de “itens\_venda”, reajustando o produto para a quantidade correta.

Após a criação de tais *triggers*, faça a validação do funcionamento por meio de testes, simulando operações de alteração e remoção de itens vendidos.

## *Procedures*

Se você já programou alguma linguagem de programação que possui funções (também chamadas de métodos), essa é a associação mais simples para que também entenda o que são as *procedures*.

As *procedures*, também denominadas *stored procedures*, são trechos de código SQL cuja funcionalidade é executar alguma lógica útil para a aplicação, podendo possuir ou não parâmetros de entrada e/ou retornos de saída, sendo invocadas sempre que se deseja executar esse processamento.

Diferentemente dos *triggers*, elas não são obrigatoriamente associadas a eventos padrão do MySQL, ou seja, em inserções, deleções ou atualizações. Porém, quando necessário, podem ser invocadas dentro dos *triggers*.

Na prática, as *procedures* podem ser invocadas da mesma forma como ocorrem nas funções nativas do MySQL, ou até mesmo chamadas pelo próprio *backend*.

Sua funcionalidade é bem variada, podendo servir para reduzir a quantidade de código a ser utilizado em operações repetitivas (uma única chamada pode executar centenas de comandos), corrigir inconsistências em bases de dados, reproduzir regras de negócio e compilar dados.

De maneira geral, o uso de *procedures* também contribui para a performance do sistema, já que evita múltiplas idas e vindas entre a aplicação e o banco de dados; reforça a segurança, permitindo controlar melhor o acesso a dados sensíveis por meio de permissões específicas; e melhora a organização do código, centralizando regras de negócio em um único ponto e facilitando a manutenção e a reutilização.

#### IMPORTANTE

##### ***Stored procedures e frameworks backend***

**Assim como informamos no tópico sobre *triggers*, o uso das *stored procedures* vem sendo constantemente diminuído devido aos avanços nos *frameworks backend*, já que dentro deles é possível executar lógicas complexas com maior flexibilidade e facilidade, em comparação ao que está disponível no banco de dados. Contudo, em situações em que não é possível ou não é vantajoso alterar o *backend*, o uso das *procedures* passa a ser a solução adotada.**

**Outra utilização da *procedure*, ainda nos contextos atuais, é a compilação assíncrona de dados, em casos em que existe uma grande quantidade de registros a serem manipulados, o que resultaria em uma demora prejudicial no tempo de resposta síncrona, isto é, em condições em que se deseja o resultado de imediato.**

Assim como ocorre com as funções em linguagens de programação, a complexidade das *procedures* pode variar de acordo com a necessidade, não havendo um padrão a se seguir, podendo conter desde centenas de comandos até mesmo um simples **select**. Contudo, normalmente, você não verá *procedures* executando tarefas muito simples, a menos que sejam casos específicos (a impossibilidade de colocá-las no *backend* da aplicação, por exemplo).

Abordaremos nos tópicos seguintes algumas *procedures* de acordo com o nível de sua complexidade e uso. Embora os comandos de criação de todas essas variantes sejam praticamente os mesmos, essa divisão nos parece mais adequada para seu entendimento.

## create procedure – Criando *procedures* simples

As *stored procedures* mais simples, ou seja, as que não possuem parâmetros de entrada e nem respostas de saída, são quase que exclusivamente utilizadas quando se deseja a manipulação, a compilação ou o ajuste de dados da aplicação, uma vez que não reproduzem um retorno como resultado.

Antes de nos aprofundarmos nas situações de uso, em nossos costumados exemplos, observe no bloco de código a seguir a estrutura de uso dessas *procedures* simplificadas.

```
-- ESTRUTURA DO COMANDO:
delimiter $$
create procedure NOME_DA_PROCEDURE()
begin
    -- COMANDOS SQL QUANDO A PROCEDURE FOR EXECUTADA....
```



```

    end
$$
delimiter ;

-- EXEMPLO:
-- criação de uma procedure para atualização dos status dos
pedidos
delimiter $$
create procedure atualizaStatusPedidos()
begin
    -- atualiza os pedidos com mais de 30 dias pendentes para
    expirado.
    update pedidos set status = 'Expirado' where status =
    'Pendente'
    and data_pedido < DATE_SUB(CURDATE(), interval 30 day);

    /* bloqueia clientes que possuem pedidos com pendência de
    pagamento há mais de 60 dias */
    update clientes set status = 'Bloqueado'
    where id in (
        select distinct p.cliente_id from pedidos p
        where p.status = 'Pagamento Pendente'
        and p.data_pedido < DATE_SUB(CURDATE(), interval 60 day)
    );
end $$
delimiter ;

```

Analisando o bloco de código anterior, você deve ter chegado à conclusão de que, de fato, a estrutura de uso é realmente simplificada, ficando a complexidade quase que exclusivamente a cargo da implementação da lógica a ser executada dentro da *procedure*, o que deve variar de acordo com cada necessidade.

Vale pontuar também que, de modo semelhante ao que já vimos nos *triggers*, aplicamos aqui o uso do **delimiter**, para que o servidor MySQL possa interpretar corretamente o início e fim do comando de criação.

No bloco de código anterior, visualizamos em “atualizaStatusPedidos” algumas operações envolvendo uma aplicação com a necessidade de modificação dos *status* de seus respectivos pedidos, sendo possível concluir que entender as *procedures* está mais relacionado com todo o conhecimento já visto até este capítulo do que necessariamente com elas.

Antes de mergulharmos em nossos costumeiros exemplos e casos de uso, nos quais exploraremos as *procedures* de modo mais minucioso, devemos reconhecer que não há grande utilidade em saber criá-las sem saber como invocá-las, não é mesmo?

Para fazer isso, é muito simples: utilizamos o comando **call** com o nome da *procedure*, conforme o bloco de código a seguir.

```
-- ESTRUTURA DO COMANDO:  
call NOME_DA_PROCEDURE();  
  
-- EXEMPLOS:  
call atualizaStatusPedidos();  
call imprimeRelatorio();
```

Agora sim, vamos entender um pouco mais das *stored procedures* por meio de um exemplo apresentado no tópico seguinte.

### Exemplos e casos de uso – *E-commerce*

Um *e-commerce* de bolsas e acessórios femininos deseja adicionar ao seu sistema digital já existente algumas tratativas para melhorar o controle de estoque de seus produtos e para impulsionar as vendas.

De maneira geral, a ideia é, ao final de cada dia e após cada venda efetuada, o banco de dados adotar algumas rotinas de atualização, conforme os **requisitos** a seguir:

1. Itens com quantidade inferior a dez unidades devem ter ao final de sua descrição o texto chamativo “\*\*ESTOQUE BAIXO!”, pois isso ajudará os funcionários da empresa a priorizarem a reposição do estoque.

Ao mesmo tempo, essa informação também é útil ao cliente, para que ele possa efetuar a compra o mais rápido possível antes que a quantidade acabe;

2. Itens com nenhuma quantidade no estoque devem ser imediatamente desabilitados para venda (coluna “ativo = false”), evitando assim o cancelamento de uma ordem de venda por falta de estoque;
3. Itens com quantidade de reclamações maior que quatro também devem ter sua venda suspensa (coluna “ativo = false”), pois assim abrem espaço para que os funcionários da empresa analisem melhor o produto e tentem resolver a situação, antes que outros clientes o comprem;
4. Itens que foram repostos no estoque devem voltar a estarem ativos e sem nenhuma descrição de volume baixo no estoque;
5. Após cada inserção de venda, a quantidade disponível do produto deve ser atualizada;
6. Ao final do dia, deve ser gerado um relatório de reposição de estoque com os produtos que estão em baixa quantidade, porém desconsiderando os que possuem a quantidade de reclamações excedidas (maior que quatro reclamações). Para facilitar o processo da empresa, nesse relatório deve ser exibida a quantidade que deve ser reposta e não a quantidade atual disponível, considerando que em cada reposição o estoque deve estar com no mínimo 25 itens. Ou seja, se um produto estiver com apenas cinco itens no estoque, então devem ser exibidas no relatório de compras 20 unidades.

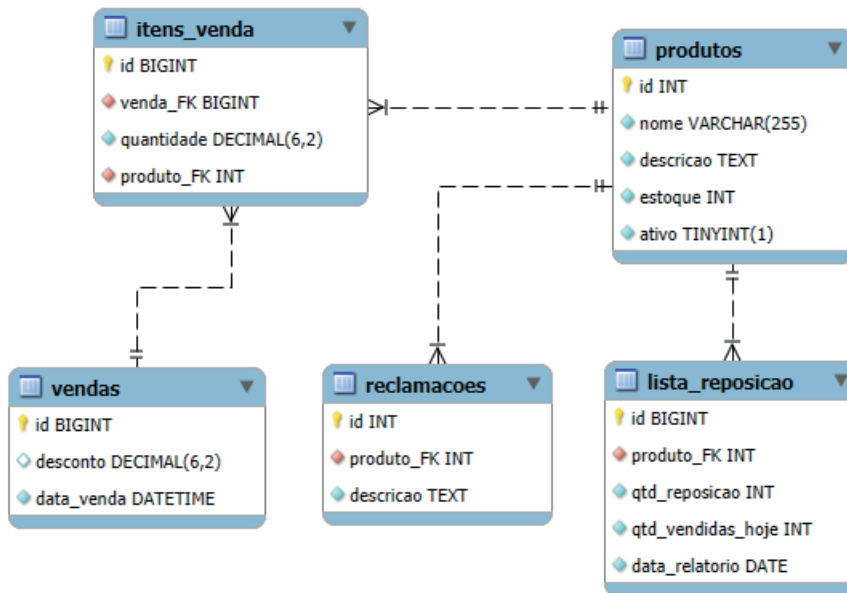
#### FIQUE ALERTA

##### Combinação de soluções

Observe que diante dos requisitos é necessária a adoção de ações usando SQL ao final de cada venda. Neste momento, você deve ter ficado tentado a resolver essa situação apenas com o uso dos *triggers*. Contudo, é importante

ressaltar que o conjunto de operações em SQL que envolvem essa situação-problema requer uma abordagem mais ampla, já que as operações também devem ser executadas ao final de cada dia (mesmo que nenhuma venda tenha sido efetuada), por exemplo, durante a geração do relatório de vendas. Desse modo, você verá que combinaremos algumas soluções envolvendo *triggers*, *procedures* e comandos de consulta mais avançados.

Agora que você já entendeu os requisitos funcionais dessa aplicação, vamos interpretar a estrutura das tabelas e seus respectivos relacionamentos, de acordo com a **Figura 5.5**.



**Figura 5.5** – Modelagem UML de um e-commerce de bolsas e acessórios femininos, em que será necessária a atuação envolvendo *procedures*.

Crédito: do elaborador, 2025.

Na modelagem apresentada, podemos identificar uma estrutura relativamente simples envolvendo “produtos”, “vendas” e “itens\_venda”, muito similar ao que já exploramos ao longo deste livro, ou seja, uma venda possui diversos itens, todos presentes na tabela de produtos cadastrados. Adicionalmente, possuímos uma tabela para registro das reclamações dos clientes, em que há a indicação do produto a receber a reclamação. E, por fim,

temos uma entidade para guardar a lista de reposição dos itens que devem ser comprados para repor o estoque.

Como existem inúmeros requisitos, vamos proceder com a resolução em partes.

Primeiro, digitaremos a estrutura da *procedure* que será utilizada para atualização da descrição dos itens com estoque baixo, conforme se vê no bloco de código a seguir.

```
delimiter $$
create procedure atualiza_estoque()
begin

    /* atualiza os produtos com quantidade abaixo de 10 e que
    ainda não foram atualizados */
    update produtos set descricao =
        concat(descricao, ' **ESTOQUE BAIXO!') where estoque < 10
        and descricao not like '%ESTOQUE BAIXO!%';

end
$$
delimiter ;
```

Nesse bloco de código, estamos utilizando o comando **update** para concatenar, por meio da função “concat”, a descrição de cada produto que atender à condição com estoque inferior a dez unidades.

Como essa *procedure* será executada diversas vezes, adicionamos também uma condição dentro da cláusula **where** para não modificar os itens já atualizados, ou seja, que já tiverem o texto “\*\*ESTOQUE BAIXO!”.

Em seguida, damos continuidade à mesma *procedure*, adicionando também o comando que removerá o aviso de estoque reduzido para aqueles casos em que ocorreu alguma reposição, conforme se observa no bloco de código a seguir.

```

delimiter $$
create procedure atualiza_estoque()
begin

/* atualiza os produtos com quantidade abaixo de 10 e que
ainda não foram atualizados */
update produtos set descricao =
concat(descricao, ' **ESTOQUE BAIXO!') where estoque < 10
and descricao not like '%ESTOQUE BAIXO!%';

/* atualiza os produtos com quantidade acima ou igual a 10
mas que ainda estavam com o texto de 'ESTOQUE BAIXO!' */
update produtos set descricao =
replace(descricao, ' **ESTOQUE BAIXO!', '') where estoque
>= 10
and descricao like '%ESTOQUE BAIXO!%';

end
$$
delimiter ;

```

Aqui utilizamos a lógica inversa do comando anterior de **update**, ou seja, removemos o texto “\*\*ESTOQUE BAIXO!” dos itens que não estão mais com quantidade inferior a dez, usando a função “replace”, capaz de substituir um texto por outro (no caso, estamos substituindo por um texto vazio).

Agora, no bloco de código a seguir, faremos a ativação dos itens que foram desativados por estarem com baixa quantidade de estoque.

```

delimiter $$
create procedure atualiza_estoque()
begin

```

```

/* atualiza os produtos com quantidade abaixo de 10 e que
ainda não foram atualizados */
update produtos set descricao =
concat(descricao, ' **ESTOQUE BAIXO!') where estoque < 10
and descricao not like '%ESTOQUE BAIXO!%';

/* atualiza os produtos com quantidade acima ou igual a 10
mas que ainda estavam com o texto de 'ESTOQUE BAIXO!' */
update produtos set descricao =
replace(descricao, ' **ESTOQUE BAIXO!', '') where estoque
>= 10
and descricao like '%ESTOQUE BAIXO!%';

/* reativa todos os produtos que foram repostos no estoque */
update produtos set ativo = true
where estoque > 0 and ativo = false;

end
$$
delimiter ;

```

No bloco de código seguinte, faremos a lógica de desativação de todos os itens que estão com quantidade zerada no estoque ou que possuem quantidade de reclamações superior a quatro.

```

delimiter $$
create procedure atualiza_estoque()
begin

/* atualiza os produtos com quantidade abaixo de 10 e que
ainda não foram atualizados */

```

```

update produtos set descricao =
    concat(descricao, ' **ESTOQUE BAIXO!') where estoque < 10
and descricao not like '%ESTOQUE BAIXO!%';

/* atualiza os produtos com quantidade acima ou igual a 10
mas que ainda estavam com o texto de 'ESTOQUE BAIXO' */
update produtos set descricao =
    replace(descricao, ' **ESTOQUE BAIXO!', '') where estoque
>= 10
and descricao like '%ESTOQUE BAIXO!%';

/* reativa todos os produtos que foram repostos no estoque */
update produtos set ativo = true
where estoque > 0 and ativo = false;

/* desativa os itens que estejam ativos e que possuam o id
dentro da relação de produtos com quantidade de
reclamações maior que 4, ou que estejam com estoque = 0 */
update produtos p set p.ativo = false where p.ativo = true
and p.id in (
    select r.produto_FK from reclamacoes r group by
r.produto_FK
    having count(*) > 4
) or estoque = 0;

end
$$
delimiter ;

```



Para atender à necessidade de desativação dos itens com nenhum estoque ou que possuam quantidade de reclamações superior ao estipulado, adicionamos no código o último comando de nossa *procedure*.

Esse comando necessitou de uma complexidade um pouco maior, utilizando uma subconsulta dentro do **update** para coletar apenas os “ids” dos produtos que estiverem na tabela de reclamações em uma quantidade maior do que quatro vezes.

Com essa última inclusão, finalizamos a proposta dessa *procedure*, uma vez que ela atende aos requisitos 1, 2, 3 e 4, mencionados no início deste tópico.

Para atendermos ao requisito 6, isto é, atualizar a quantidade disponível em cada venda, necessitaremos do uso dos *triggers*, associados com o evento de inserção, conforme o bloco de código a seguir.

```
delimiter $$
create trigger atualiza_quantidade_produtos after insert
on itens_venda for each row
begin
    -- subtrai do estoque a quantidade vendida:
    update produtos p set p.estoque = ( p.estoque - NEW.
quantidade )
    where p.id = NEW.produto_FK and p.estoque >= NEW.
quantidade;

    -- chama a procedure de atualização após cada evento de
venda
    call atualiza_estoque();
end
$$
delimiter ;
```

Observe que aqui criamos um *trigger* cujo evento de gatilho é a inserção na tabela de “itens\_venda”, em que, para cada registro incluído, será

atualizado o volume disponível em estoque de acordo com a quantidade envolvida na operação de venda.

Aproveitamos também o gatilho do *trigger* para chamar o “atualiza\_estoque”, fazendo com que as operações contidas nessa *procedure* sejam executadas dinamicamente em cada operação de venda. Desse modo, possibilitamos que um item seja imediatamente detectado como “ESTOQUE BAIXO!” assim que a transação de sua venda é realizada e conseguimos atender ao requisito 6 da aplicação.

Agora, faremos outra *procedure*, com o objetivo de gerar o relatório de reposição de itens do estoque. Para isso usaremos um recurso oferecido pelo MySQL, que é o **insert into select**.

#### SAIBA MAIS

##### insert into select

O comando **insert into select** possibilita que seja feita uma operação de inserção dos dados retornados durante uma consulta, sendo geralmente utilizado em situações em que os registros a serem inseridos são dinâmicos e, portanto, desconhecidos.

Por exemplo, imagine uma situação em que uma grande empresa, presente em diversos países, ao final de cada mês deseje salvar em uma tabela as 30 maiores vendas efetuadas naquele mês. Poderíamos saber quais são essas vendas por meio de uma simples consulta **select**, mas isso não resolveria o processo de inclusão. Sem dúvidas, optarmos pela cópia manual desses valores em um comando **insert** não seria a opção mais rápida e segura.

Desse modo, resolveríamos facilmente essa situação por meio do uso do comando apresentado no bloco de código a seguir.

```
insert into maiores_vendas (venda_FK, data_venda)
select v.id, v.data_venda from vendas v where
    year(v.data_venda) = year(now()) and month(v.data_venda) =
month(now())
order by v.valor desc limit 30;
```

Como você pode notar, o **insert into select** é, de fato, a união do funcionamento entre ambos os comandos que você já conhece, tendo grande utilidade para inclusão de registros de forma dinâmica e ágil.

Retomando o exemplo de gerenciamento de estoque, detalharemos a criação de mais uma *procedure*. Devido à sua complexidade, faremos isso em etapas, conforme o bloco de código a seguir.

```
delimiter $$
create procedure relatorio_compras()
begin
insert into lista_reposicao (produto_FK, qtd_reposicao,
qtd_vendidas_hoje)
select p.id, ('QUANTIDADE A SER REPOSTA'), ('QUANTIDADE
VENDIDA')
from produtos p where p.estoque < 10 and p.id not in
('PRODUTOS COM MUITA RECLAMAÇÃO');
end
$$
delimiter ;
```

Obviamente, o que vemos no bloco de código anterior não é o comando completo. Deixamos algumas partes essenciais, que serão substituídas por subconsultas, indicadas entre aspas: “QUANTIDADE A SER REPOSTA”, “QUANTIDADE VENDIDA” e “PRODUTOS COM MUITA RECLAMAÇÃO”.

Basicamente, a intenção aqui é inserirmos na “lista\_reposicao” o resultado da consulta na tabela de “produtos”, porém algumas informações não estão prontamente disponíveis. Para isso, precisaremos de cálculos e subconsultas, o que faremos a seguir.

No bloco de código a seguir, substituiremos o texto “QUANTIDADE A SER REPOSTA” por meio do cálculo da diferença entre a quantidade atual disponível no estoque e a quantidade mínima desejada, isto é, 25.

```

delimiter $$
create procedure relatorio_compras()
begin
insert into lista_reposicao (produto_FK, qtd_reposicao,
qtd_vendas_hoje)
select p.id, (25 - p.estoque), ("QUANTIDADE VENDIDA")
from produtos p where p.estoque < 10 and p.id not in
("PRODUTOS COM MUITA RECLAMAÇÃO");

end
$$
delimiter ;

```

No próximo bloco de código, faremos a subconsulta para obter a “QUANTIDADE VENDIDA” de itens durante o dia.

```

delimiter $$
create procedure relatorio_compras()
begin
insert into lista_reposicao (produto_FK, qtd_reposicao,
qtd_vendas_hoje)
select p.id, (25 - p.estoque),
(select count(*) from itens_venda iv join vendas v on
v.id = iv.venda_FK where date(v.data_venda) =
current_date()
and iv.produto_FK = p.id)
from produtos p where p.estoque < 10 and p.id not in
("PRODUTOS COM MUITA RECLAMAÇÃO");

end
$$
delimiter ;

```

Embora essa adição tenha deixado o comando volumoso, a subconsulta não apresenta nenhum comando desconhecido, afinal, estamos apenas

retornando a quantidade de resultados presentes para cada produto que tenha sido vendido na data atual de execução.

Por fim, faremos a substituição da última subconsulta faltante, ou seja, o comando que obterá os “ids” dos produtos que possuem um excesso de reclamações para servir de filtro à consulta principal, evitando assim que tais itens “problemáticos” sejam colocados novamente em estoque.

```
delimiter $$
create procedure relatorio_compras()
begin
insert into lista_reposicao (produto_FK, qtd_reposicao,
qtd_vendidas_hoje)
select p.id, (25 - p.estoque),
(select count(*) from itens_venda iv join vendas v on
v.id = iv.venda_FK where date(v.data_venda) =
current_date()
and iv.produto_FK = p.id)
from produtos p where p.estoque < 10 and p.id not in
(select r.produto_FK from reclamacoes r group by
produto_FK
having count(*) > 4);
end
$$
delimiter ;
```

Estamos praticamente chegando ao final de nosso desenvolvimento. Faremos agora a criação de um evento em que haverá a chamada das duas *procedures* que criamos, sempre ao final do dia (às 23h59min).

```
delimiter $$
create event if not exists evento_relatorio_compras
on schedule every 1 day
starts timestamp(curdate() + interval 1 day) + interval 23
hour +
```

```

interval 59 minute + interval 0 second
do
begin
    call atualiza_estoque();
    call relatorio_compras();
end
$$
delimiter ;

```

Com esse comando, estamos gerando um evento no servidor MySQL, acionado diariamente no horário de 23 horas, 59 minutos e 0 segundos, realizando a execução das duas *procedures* criadas anteriormente. Dessa forma, conseguimos atender a todos os requisitos exigidos pela aplicação.

Convidamos o(a) aluno(a) a fazer o teste do sistema, realizando a inclusão de itens nas tabelas da aplicação, a fim de verificar se o comportamento está de acordo com o esperado.

#### SAIBA MAIS

##### create event

Conforme pudemos identificar no contexto deste exemplo, o comando **create event** é extremamente útil quando se deseja automatizar a execução de operações no banco de dados de forma agendada.

Existem basicamente dois tipos de eventos, aqueles com execução única e os recorrentes. Observe a seguir sua estrutura de uso, com mais alguns exemplos interessantes.

```

-- ESTRUTURA DO COMANDO:
delimiter $$
create event NOME_DO_EVENTO on schedule
[at HORÁRIO_EVENTO] / [every VALOR PERÍODO starts
HORÁRIO_EVENTO]
do
begin

```

```

-- Comandos SQL do Evento...
end
$$
delimiter ;

-- evento executado uma única vez no dia e horário
estipulado:
delimiter $$
create event atualizar_status_pedidos on schedule
at '2025-04-15 00:00:00'
do
begin
update pedidos set status = 'vencido' where data_entrega
< now();
end
$$
delimiter ;

-- evento executado diariamente no horário em que foi criado
delimiter $$
create event limpar_logs_antigos on schedule
every 1 day starts current_timestamp
do
begin
delete from logs where data < now() - interval 30 day;
end
$$
delimiter ;

```

Embora o uso de eventos tenha diminuído com os avanços nos *frameworks backend*, ainda são utilizados quando se deseja simplificar a aplicação ou quando a modificação no banco de dados é a única opção disponível.

### Exercícios – *E-commerce*: continuação

Considerando que você já tem intimidade com o sistema de *e-commerce* que abordamos no caso de uso, identificamos duas oportunidades de melhorias em relação ao sistema atual, o que levará você a praticar os conceitos estudados:

1. Implemente uma modificação no projeto de modo a identificar os produtos que tiverem mais de 15 unidades vendidas no mês atual, adicionando em sua descrição o texto:

‘ \*\*MAIS VENDIDOS!’

2. Adicione uma coluna “em\_promocao” na tabela de produtos, do tipo booleano. Essa coluna deverá estar “true” para os produtos ativos que não foram vendidos há mais de 60 dias e que possuem quantidade de estoque maior que zero.

### create procedure – Criando *procedures* com entradas de parâmetros

Vimos no tópico anterior que as *stored procedures* podem ser muito úteis, oferecendo a possibilidade de execução de um conjunto de operações SQL para atender às mais diversas necessidades das aplicações, sobretudo quando associadas com eventos e até mesmo *triggers*.

Contudo, existem situações em que as operações que serão executadas dentro de uma *procedure* dependem de fatores externos, e é nesses casos que utilizamos os chamados parâmetros de entrada de uma *procedure*. Esse recurso é frequentemente utilizado em funções, dando a elas o grau de flexibilidade necessário para cada contexto.

Observe que, no bloco de código a seguir, esse recurso não altera demasiadamente a estrutura do comando que já vimos.



```

-- ESTRUTURA DO COMANDO:
delimiter $$
    create procedure NOME_DA_PROCEDURE(in NOME_PARÂMETRO1
TIPO_PARÂMETRO1,
                                     in NOME_PARÂMETRO2 TIPO_
PARÂMETRO2, ...)
    begin
        -- COMANDOS SQL QUANDO A PROCEDURE FOR EXECUTADA....
    end
$$
delimiter ;

-- EXEMPLO:
/* criação de uma procedure para apenas inserir um item de
venda se o produto tiver quantidade o suficiente e não
estiver desativado. caso passe na condição, executará
a adição da venda e subtração da quantidade do estoque,
chamando outra procedure com implementações de mais
regras de negócio da aplicação
*/

delimiter $$
    create procedure insere_item_venda(in in_produto_FK bigint,
                                     in in_quantidade int,
                                     in in_venda_FK bigint)

    begin
        if exists (select 1 from produtos p where p.id =
in_produto_FK
                  and p.estoque >= in_quantidade and p.ativo = true )
        then

            insert into itens_venda (venda_FK, quantidade, produto_FK)
            values (in_venda_FK, in_quantidade, in_produto_FK);

```

```

    update produtos p set p.estoque = ( p.estoque - in_
quantidade )
    where p.id = in_produto_FK;

    call atualiza_estoque();

end if;
end
$$
delimiter ;

```

De modo muito semelhante ao que ocorre com funções nas mais diversas linguagens de programação, as *stored procedures* com parâmetros de entrada possibilitam o recebimento de um número ilimitado de informações, exigindo, no entanto, que se nomeie cada variável recebida com seu respectivo tipo, assim como é feito ao se criar colunas em uma tabela.

Utilizamos, no bloco de código anterior, o reaproveitamento do contexto do caso de uso de um *e-commerce*, em que havia a necessidade de gerenciamento de seu estoque. Nessa situação, criamos uma *procedure* para substituir o *trigger* criado para atualização do estoque, uma vez que, com o código atual, conseguimos garantir que a venda de um item somente ocorra se houver estoque e se o produto não estiver desativado. Contudo, essa melhoria somente é possível com a inclusão dos parâmetros de entrada necessários para essas operações, ou seja, os “ids” do produto e da venda e a quantidade do item a ser vendida.

Exemplos e casos de uso – Operadora de crédito

Uma nova operadora de crédito está entrando no mercado financeiro com a proposta de facilitar e desburocratizar a contratação de empréstimos aos

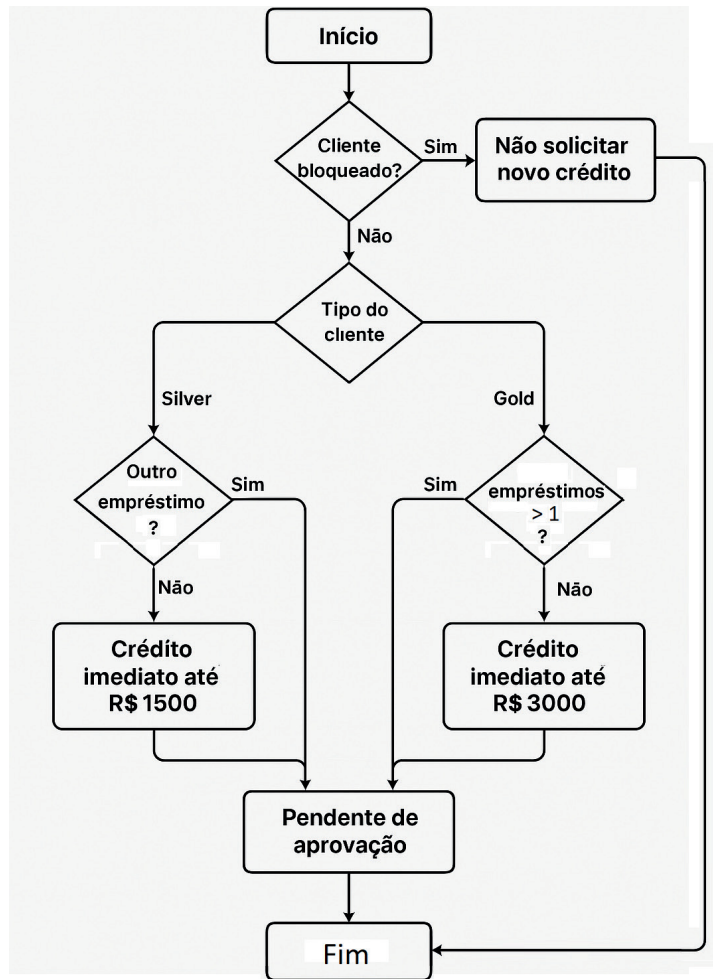
seus clientes. Para isso, seu sistema está passando por diversos desenvolvimentos, e é nesse contexto que sua atuação se faz necessária.

Conforme mencionado, a principal proposta dessa empresa é reduzir a burocracia na obtenção de crédito por meio da criação de regras de negócio que visam aumentar a agilidade da concessão de empréstimos de acordo com o perfil do cliente e dos valores envolvidos.

Para auxiliar o processo de desenvolvimento, foram elaborados os seguintes **requisitos**:

1. Devem existir duas categorias de clientes, “Silver” e “Gold”, em que estes últimos apresentam mais benefícios e maior facilidade para aquisição de empréstimos;
2. Clientes do tipo “Silver” possuem crédito imediato até R\$ 1 500,00, não necessitando de aprovadores, desde que não tenham nenhum outro empréstimo em andamento;
3. Clientes do tipo “Gold” possuem crédito imediato até R\$ 3 000,00, não necessitando de aprovadores, desde que possuam até um empréstimo em andamento;
4. Para valores acima ou condições diferentes dos limites mencionados, o cliente poderá submeter sua solicitação de empréstimo, porém ela ficará pendente de aprovação;
5. Clientes que estejam bloqueados não poderão solicitar novo crédito em nenhuma hipótese.

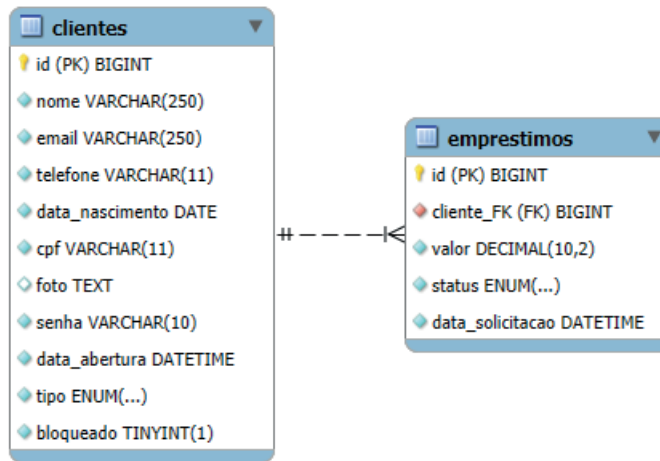
Para ajudar no entendimento desse conjunto de regras de negócio, as colocamos em um fluxograma, conforme **Figura 5.6**.



**Figura 5.6** – Fluxograma representando o funcionamento das regras de negócio da operadora de crédito.

Crédito: do elaborador, 2025.

Finalmente, antes de iniciarmos as etapas de desenvolvimento, vejamos a modelagem UML das duas entidades envolvidas nessas regras de negócio (obviamente, uma aplicação desse porte exigirá muitas outras tabelas, porém será feita a supressão delas, já que não estão diretamente relacionadas com os requisitos abordados).



**Figura 5.7** – Modelagem UML de duas entidades de uma operadora de crédito do mercado financeiro.

Crédito: do elaborador, 2025.

Agora, faremos o desenvolvimento de uma *procedure* para atender às regras de negócio dessa aplicação, descrita nos passos a seguir.

No bloco de código a seguir, criaremos a estrutura da *procedure* com o nome “solicita\_emprestimo”, que receberá dois parâmetros de entrada: o valor do empréstimo solicitado e o cliente.

```

delimiter $$
create procedure solicita_emprestimo(in in_valor
decimal(10,2),
                                     in in_cliente_FK bigint)
begin
    -- (“LÓGICA DA PROCEDURE”)
end
$$
delimiter ;
  
```

Em seguida, faremos a primeira validação, usando o **if exists**, comando capaz de checar se uma determina condição possui ocorrência verdadeira. Caso possua, ele direcionará o servidor para a execução do bloco de código após o **then**.

```

delimiter $$
create procedure solicita_emprestimo(in in_valor
decimal(10,2),
                                     in in_cliente_FK bigint)

begin
    if exists (select 1 from clientes c where c.id =
in_cliente_FK
              and bloqueado = 0 and not exists
                (select 1 from empréstimos e where
                  e.cliente_FK = in_cliente_FK and e.status =
'VENCIDO')
              )
    then
        -- ("LÓGICA PARA INSERÇÃO DOS EMPRÉSTIMOS")
    end if;
end
$$
delimiter ;

```

Para esse caso, criamos algumas condicionais principais dentro do **if exists**, de modo a validar alguns requisitos obrigatórios antes de efetuar qualquer solicitação de empréstimo. Em outras palavras, estamos checando se existe ocorrência de um cliente que:

- Atende ao **id** recebido como parâmetro de entrada (“c.id = in\_cliente\_FK”);
- Não está bloqueado (“bloqueado = 0”);
- Não está presente na tabela de empréstimos com *status* vencido (subconsulta na tabela de empréstimos com “e.status = ‘VENCIDO’”).

No bloco de código a seguir, faremos a inclusão na tabela de empréstimos, usando o parâmetro de “id” do cliente, o valor a ser emprestado e o mais importante: mais uma condicional para decidir se o crédito será concedido imediatamente ou se necessitará de uma etapa de aprovação.

```

delimiter $$
create procedure solicita_emprestimo(in in_valor
decimal(10,2),
                                     in in_cliente_FK bigint)

begin
    if exists (select 1 from clientes c where c.id =
in_cliente_FK
              and bloqueado = 0 and not exists
                (select 1 from empréstimos e where
                  e.cliente_FK = in_cliente_FK and e.status =
'VENCIDO'))
    then
        insert into empréstimos (cliente_FK, valor, status)
values
    (in_cliente_FK, in_valor,
     case
       when ("CLIENTE SILVER EXCEDE LIMITE OU JÁ POSSUI 1
            EMPRÉSTIMO")
       then 'EM APROVAÇÃO'

       when ("CLIENTE GOLD EXCEDE LIMITE OU JÁ POSSUI MAIS
QUE 1 EMPRÉSTIMO")
       then 'EM APROVAÇÃO'

       else 'EM ANDAMENTO'
     end
    );
    end if;
end
$$
delimiter ;

```

No bloco de código anterior, conseguimos identificar a utilização de mais um importante recurso condicional: o **case/when**, aplicado para

decidir de forma dinâmica o resultado a ser usado na coluna de “status” da entidade de empréstimo.

De modo a auxiliar o(a) aluno(a) nesse entendimento, observe que adicionamos um descritivo de qual lógica será operada em cada condição **when**. Nas duas primeiras, definimos que o *status* do empréstimo necessitará de uma aprovação adicional caso o cliente exceda o limite permitido pela sua categoria (*Silver* ou *Gold*) ou caso já possua uma quantidade de empréstimos não concluídos também acima do permitido. E, finalmente, caso nenhuma das condições anteriores seja atendida, concluímos que o empréstimo deve ser concedido imediatamente.

```
delimiter $$
create procedure solicita_emprestimo(in in_valor
decimal(10,2),
                                     in in_cliente_FK bigint)
begin
    if exists (select 1 from clientes c where c.id =
in_cliente_FK
              and bloqueado = 0 and not exists
              (select 1 from empréstimos e where
                e.cliente_FK = in_cliente_FK and e.status =
'VENCIDO'))
    then
        insert into empréstimos (cliente_FK, valor, status)
values
    (in_cliente_FK, in_valor,
     case
       when (select cl.tipo from clientes cl where cl.id =
in_cliente_FK) = 'SILVER' and
            ( in_valor > 1500 or
              (select count(*) from empréstimos e
```



```

        where e.status != 'CONCLUÍDO' and
        e.cliente_FK = in_cliente_FK) > 0
    )
    then 'EM APROVAÇÃO'

    when (select cl.tipo from clientes cl where cl.id =
        in_cliente_FK) = 'GOLD' and
        ( in_valor > 3000 or
        (select count(*) from emprestimos e
        where e.status != 'CONCLUÍDO' and
        e.cliente_FK = in_cliente_FK) > 1
        )
    then 'EM APROVAÇÃO'

    else 'EM ANDAMENTO'
end
);
end if;
end
$$
delimiter ;

```

Finalmente, concluímos nossa *procedure* com todos os requisitos mencionados no exemplo. Sugerimos ao(à) aluno(a) que reproduza essa estrutura e faça os devidos testes.

#### Exercícios – Operadora de crédito: continuação

Para que você possa aplicar os conhecimentos adquiridos, implemente mais uma importante regra de negócio para a operadora de crédito mencionada anteriormente.

Agora, com o objetivo de beneficiar clientes que já possuem empréstimos quitados, deve-se oferecer um limite adicional de empréstimo pré-aprovado num valor 10% acima do que os demais que não possuem empréstimos quitados. Ou seja, se o limite atual pré-aprovado de um cliente *Silver* é de

R\$ 1 500,00, aqueles que já possuem empréstimos anteriores quitados devem possuir um limite 10% maior, isto é, equivalente a R\$ 1 650,00.

Faça a implementação dessa lógica adicional na *procedure* que criamos anteriormente, consultando o histórico do cliente como um auxiliar no processo de decisão de aprovação.

## create procedure – Criando *procedures* com variáveis

Você já deve ter percebido em nossos exemplos que as *stored procedures* podem se tornar um tanto quanto complexas, dependendo da quantidade de requisitos e regras de negócio envolvidas.

É para esses casos que o uso de variáveis se torna indispensável, diminuindo, em certas ocasiões, a quantidade de código presente na *procedure*, aumentando também a organização e a legibilidade do *script*.

Observe alguns exemplos de uso no bloco de código a seguir.

```
-- ESTRUTURA DO COMANDO:
delimiter $$
create procedure NOME_DA_PROCEDURE(in NOME_PARÂMETRO1
TIPO_PARÂMETRO1,
                                in NOME_PARÂMETRO2 TIPO_
PARÂMETRO2, ...)
begin
    -- CRIANDO VARIÁVEIS
    declare NOME_DA_VARIAVEL1 TIPO_DE_DADO1 default
VALOR_INICIAL1;
    declare NOME_DA_VARIAVEL2 TIPO_DE_DADO2;
    ...

    -- ARMAZENANDO UM VALOR NA VARIÁVEL
    set NOME_DA_VARIAVEL2 = VALOR_DESEJADO;

    -- CRIA UMA VARIÁVEL DE SESSÃO E ATRIBUI UM VALOR
    set @NOME_DA_VARIAVEL3 = VALOR_DESEJADO;
```

```

-- ARMAZENANDO O RESULTADO DE UMA CONSULTA NA VARIÁVEL
select COLUNA4 into NOME_DA_VARIAVEL1 from TABELA where
....;

if NOME_DA_VARIAVEL1 > NOME_DA_VARIAVEL2 then
...
end if;
end
$$
delimiter ;

```

## IMPORTANTE

### Uso de variáveis

**O uso de variáveis não está restrito apenas a *stored procedures* com parâmetros de entrada, mas sim disponível em praticamente todos os tipos de *procedures*, *triggers* e até mesmo comandos triviais do SQL, ao se usar variáveis de sessão.**

Existem basicamente dois tipos de variáveis, as locais e as de sessão. As primeiras podem ser utilizadas apenas dentro do contexto em que foram criadas, ou seja, se uma variável local é criada em uma *procedure*, usá-la externamente ocasionará um erro. Já as variáveis de sessão podem ser criadas e usadas em qualquer contexto, desde que se mantenha a mesma sessão do usuário SQL.

Exemplos e casos de uso – Operadora de crédito

Retomaremos o mesmo exemplo trabalhado no tópico anterior, em que uma operadora de crédito está entrando no mercado financeiro com a proposta de facilitar e desburocratizar a contratação de empréstimos aos seus clientes. Nesse contexto, implementamos uma *procedure* com diversas operações e parâmetros de entrada, mas sem nenhuma variável. Contudo, como agora

as variáveis já foram apresentadas, faremos uso delas para otimizar o código da *procedure* que desenvolvemos anteriormente.

No bloco de código a seguir, apresentaremos a mesma *procedure*, modificada apenas com a criação das variáveis locais.

```

delimiter $$
create procedure solicita_emprestimo(in in_valor
decimal(10,2),
                                in in_cliente_FK bigint)

begin
    -- variáveis inseridas:
    declare var_tipo_cliente enum ('SILVER','GOLD');
    declare var_numero_emprestimos int;
    declare var_pendencia boolean;

    if exists (select 1 from clientes c where c.id =
in_cliente_FK
                and bloqueado = 0 and not exists
                (select 1 from empréstimos e where
                    e.cliente_FK = in_cliente_FK and e.status =
'VENCIDO')
            )
    then
        insert into empréstimos (cliente_FK, valor, status)
values
    (in_cliente_FK, in_valor,
    case
        when (select cl.tipo from clientes cl where cl.id =
in_cliente_FK) = 'SILVER' and
        ( in_valor > 1500 or
        (select count(*) from empréstimos e
            where e.status != 'CONCLUÍDO' and
            e.cliente_FK = in_cliente_FK) > 0
        )
        then 'EM APROVAÇÃO'
    )

```



```

-- variáveis inseridas:
declare var_tipo_cliente enum ('SILVER','GOLD');
declare var_numero_emprestimos int;
declare var_pendencia boolean;

-- armazena a categoria do cliente na variável
"var_tipo_cliente":
select c.tipo into var_tipo_cliente from clientes c
where c.id = in_cliente_FK;

if exists (select 1 from clientes c where c.id =
in_cliente_FK
        and bloqueado = 0 and not exists
        (select 1 from emprestimos e where
        e.cliente_FK = in_cliente_FK and e.status =
'VENCIDO')
        )
then
    insert into emprestimos (cliente_FK, valor, status)
values
    (in_cliente_FK, in_valor,
    case
        when var_tipo_cliente = 'SILVER' and
        ( in_valor > 1500 or
        (select count(*) from emprestimos e
        where e.status != 'CONCLUÍDO' and
        e.cliente_FK = in_cliente_FK) > 0
        )
        then 'EM APROVAÇÃO'

        when var_tipo_cliente = 'GOLD' and
        ( in_valor > 3000 or
        (select count(*) from emprestimos e
        where e.status != 'CONCLUÍDO' and
        e.cliente_FK = in_cliente_FK) > 1
        )
    )

```

```

        then 'EM APROVAÇÃO'

        else 'EM ANDAMENTO'
        end
    );
end if;
end
$$
delimiter ;

```

Após o salvamento da informação do tipo de categoria do cliente na variável, substituímos as consultas anteriores que extraíam a mesma funcionalidade pelo uso da variável.

Da mesma forma que fizemos anteriormente, faremos a inserção de valores na variável “var\_numero\_emprestimos”, em que usaremos o **select into** para armazenar a quantidade de empréstimos existentes para o cliente em questão.

```

delimiter $$
create procedure solicita_emprestimo(in in_valor
decimal(10,2),
                                in in_cliente_FK bigint)

begin
    -- variáveis inseridas:
    declare var_tipo_cliente enum ('SILVER','GOLD');
    declare var_numero_emprestimos int;
    declare var_pendencia boolean;

    -- armazena a categoria do cliente na variável
    “var_tipo_cliente”:
    select c.tipo into var_tipo_cliente from clientes c
    where c.id = in_cliente_FK;

```

```

/* armazena a quantidade de empréstimos na variável
“var_numero_emprestimos”: */
select count(*) into var_numero_emprestimos from
emprestimos e
    where e.status != 'CONCLUÍDO' and e.cliente_FK =
in_cliente_FK;

    if exists (select 1 from clientes c where c.id =
in_cliente_FK
        and bloqueado = 0 and not exists
            (select 1 from empréstimos e where
                e.cliente_FK = in_cliente_FK and e.status =
'VENCIDO')
        )
    then
        insert into empréstimos (cliente_FK, valor, status)
values
    (in_cliente_FK, in_valor,
        case
            when var_tipo_cliente = 'SILVER' and
                (in_valor > 1500 or var_numero_emprestimos > 0)
            then 'EM APROVAÇÃO'

            when var_tipo_cliente = 'GOLD' and
                (in_valor > 3000 or var_numero_emprestimos) > 1
            then 'EM APROVAÇÃO'

            else 'EM ANDAMENTO'
        end
    );
    end if;
end
$$
delimiter ;

```



Após o salvamento da informação do número de empréstimos do cliente na variável, substituímos as consultas anteriores que extraíam a mesma funcionalidade pelo uso da variável. Observe que, com esse processo, a *procedure* fica com o código mais organizado e legível.

Finalmente, no bloco de código a seguir, faremos a inserção de valores na variável “var\_pendencia”, também utilizando o **select into** para definir se existe alguma restrição impeditiva para o cliente obter seu empréstimo.

```
delimiter $$
create procedure solicita_emprestimo(in in_valor
decimal(10,2),
                                     in in_cliente_FK bigint)
begin
    -- variáveis inseridas:
    declare var_tipo_cliente enum ('SILVER','GOLD');
    declare var_numero_emprestimos int;
    declare var_pendencia boolean;

    -- armazena a categoria do cliente na variável
    “var_tipo_cliente”:
    select c.tipo into var_tipo_cliente from clientes c
    where c.id = in_cliente_FK;

    /* armazena a quantidade de empréstimos na variável
    “var_numero_emprestimos”: */
    select count(*) into var_numero_emprestimos from
    empréstimos e
    where e.status != 'CONCLUÍDO' and e.cliente_FK =
    in_cliente_FK;

    /* armazena o estado lógico se o cliente possui algum
    empréstimo vencido ou está bloqueado, na variável “var_
    pendencia”: */
    select exists (
        select 1 from clientes c left join empréstimos e on
```

```

        c.id = e.cliente_FK where e.cliente_FK = in_cliente_FK
and
        (e.status = 'VENCIDO' or c.bloqueado = 1)
    ) into var_pendencia;

if var_pendencia = 0 then
    insert into emprestimos (cliente_FK, valor, status)
    values (in_cliente_FK, in_valor,
        case
            when var_tipo_cliente = 'SILVER' and
                (in_valor > 1500 or var_numero_emprestimos > 0)
            then 'EM APROVAÇÃO'

            when var_tipo_cliente = 'GOLD' and
                (in_valor > 3000 or var_numero_emprestimos > 1)
            then 'EM APROVAÇÃO'

            else 'EM ANDAMENTO'
        end
    );
end if;

end
$$
delimiter ;

```

Graças ao estado lógico da variável “var\_pendencia”, conseguimos utilizá-la dentro de uma estrutura **if**, de modo a permitir que os empréstimos sejam elegíveis apenas se o cliente não possuir nenhuma pendência.

#### IMPORTANTE

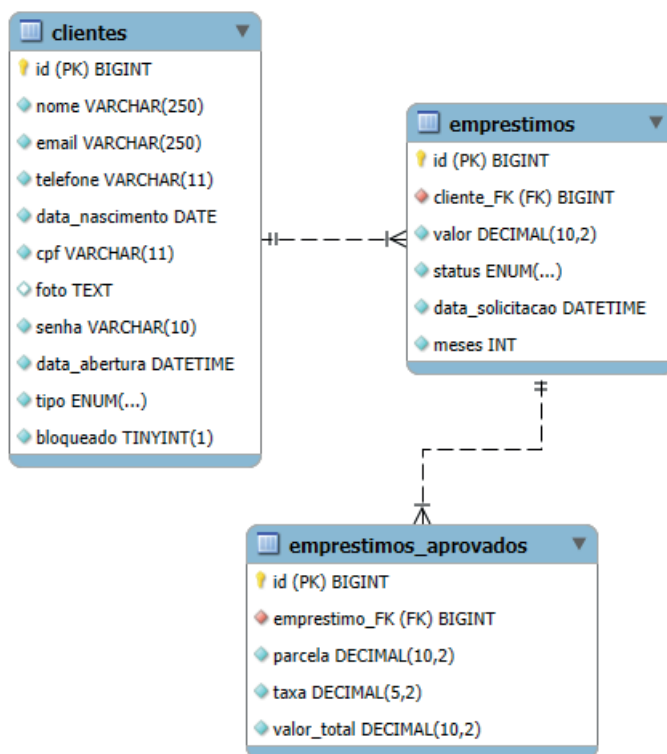
##### Variáveis e o tamanho do código

Note que nem sempre o uso de variáveis irá reduzir o tamanho de seu código, contudo, se bem aplicadas, irão contribuir para simplificar a

**interpretação e a organização de seu *script* SQL. Vale ressaltar também que o bom senso é fundamental, afinal, o uso em excesso de variáveis pode trazer o efeito contrário. De maneira geral, usamos variáveis para informações estratégicas e que serão usadas diversas vezes.**

Exercícios – Operadora de crédito: continuação

Dando continuidade à aplicação da operadora de crédito, você deve implementar agora o fluxo de aprovação ou reprovação da solicitação de empréstimos. Dessa forma, será necessário alterar a estrutura das tabelas atuais, criando uma nova entidade denominada “empréstimos\_aprovados”, que se relacionará com “empréstimos”. Além disso, você deve também criar uma nova coluna na tabela de “empréstimos”, que será chamada “meses”, utilizada para registrar o prazo solicitado da quantidade de parcelas, conforme diagrama UML da **Figura 5.8**.



**Figura 5.8** – Modelagem UML das entidades de uma operadora de crédito do mercado financeiro.

Crédito: do elaborador, 2025.

Para realizar o processo de aprovação/reprovação desses empréstimos, detalharemos os requisitos necessários a seguir:

- Você deve criar uma nova *procedure*, que receberá o “emprestimo\_FK” e a decisão de aprovação ou reprovação como parâmetros;
- Se reprovado, o *status* do empréstimo deve ficar “REPROVADO”, não havendo nenhuma inclusão na tabela de “emprestimos\_aprovados”;
- Se aprovado, a *procedure* deve calcular a taxa e o valor total do empréstimo, registrando essa operação na tabela de “emprestimos\_aprovados”;
- A operadora de crédito desse projeto utiliza o método de juro simples em suas negociações, portanto, faça os cálculos considerando esse conceito;
- A taxa de juros deve ser diferenciada de acordo com o nível do solicitante: por exemplo, para usuários *Silver*, a taxa será de 3% ao mês, enquanto para usuários *Gold* será uma taxa menor, equivalente a 2% ao mês;
- Ao final do cálculo, salve o valor da parcela, a taxa e o valor total do empréstimo na tabela de “emprestimos\_aprovados”.

#### SAIBA MAIS

##### Juro simples

Para calcular juro simples, utilize a fórmula a seguir:

$$J = P \cdot i \cdot t$$

Em que:

- J: juros;
- P: capital inicial;
- i: taxa de juros;
- t: tempo (meses).

## create procedure – Criando *procedures* com retornos

Para finalizar nosso estudo sobre as *procedures*, abordaremos mais uma importante variante desse comando: a que possui parâmetros de saída, também chamados de retornos.

De forma semelhante ao que ocorre com as funções nas mais diversas linguagens de programação, retornos são respostas de um determinado processamento executado pela função. Seguindo esse mesmo raciocínio, as *procedures* com variáveis de saída são geralmente empregadas quando se deseja executar operações (lógicas ou cálculos) e receber de volta um resultado.

Observe que, no bloco de código a seguir, a estrutura de uso dessas *procedures* é muito semelhante às que apresentam parâmetros de entrada.

```
-- ESTRUTURA DO COMANDO:
delimiter $$
create procedure NOME_DA_PROCEDURE(in NOME_PARÂMETRO1
TIPO_PARÂMETRO1,
                                in NOME_PARÂMETRO2
TIPO_PARÂMETRO2,
                                ...
                                out NOME_SAÍDA1
TIPO_DA_SAÍDA,
                                out NOME_SAÍDA2
TIPO_DA_SAÍDA,
                                ...)
begin
    -- CÓDIGO DE EXECUÇÃO DA PROCEDURE
end
$$
delimiter ;
```

**IMPORTANTE****Entradas e saídas**

**É importante deixar claro que, embora o exemplo apresentado no bloco de código contenha parâmetros de entrada, não há um vínculo direto com os parâmetros de saída, ou seja, podem existir *procedures* que contenham apenas saídas e nenhuma entrada.**

Exemplos e casos de uso – Concessionária de veículos

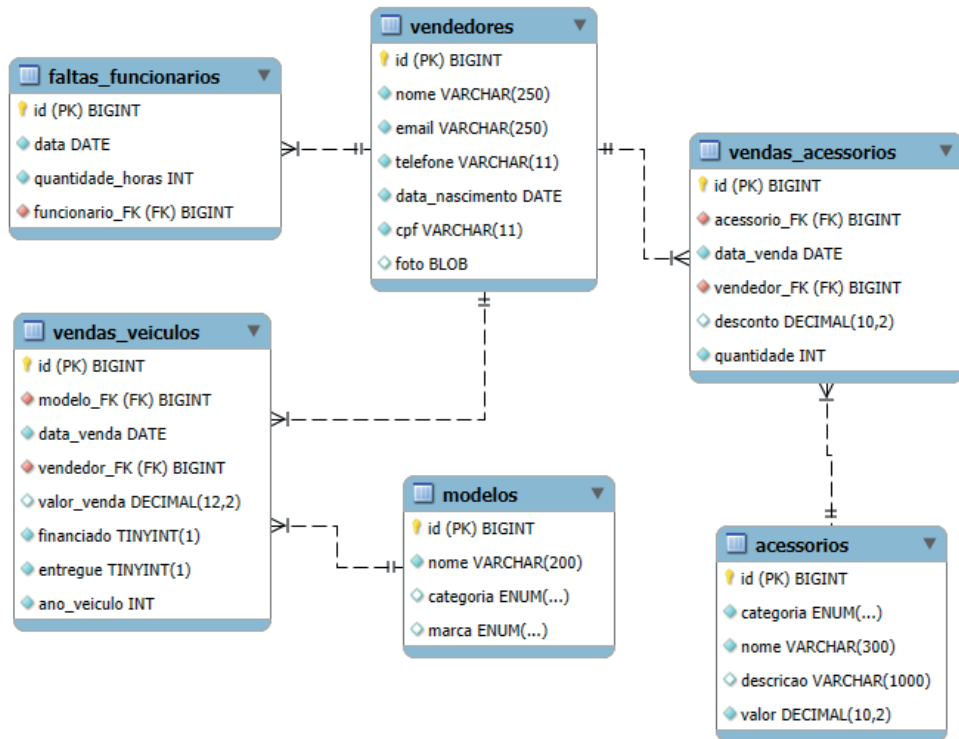
Já abordamos no **Capítulo 4** um caso de uso de uma concessionária de veículos quando estávamos estudando as subconsultas. Retomaremos o mesmo contexto, porém adicionando mais algumas regras de negócio que justifiquem o uso de *procedures* com retornos de saída.

Nessa concessionária, o salário dos vendedores é composto por uma base fixa e um complemento variável, sendo proporcional à quantidade de vendas efetuadas individualmente pelo vendedor.

Embora o cálculo final do rendimento a ser recebido por um colaborador seja feito pelo departamento financeiro, os vendedores dessa concessionária solicitaram a possibilidade de poderem acompanhar em tempo real a estimativa de recebimento, sem esperar o fechamento do mês.

Dessa forma, para proporcionar um ambiente mais transparente e incentivar seus funcionários, essa concessionária aceitou modificar seu sistema, adicionando a possibilidade de cada vendedor entrar em um portal *web* e visualizar a estimativa de salário que receberá até o dia da consulta.

Para atender a esse contexto, a empresa em que você trabalha foi contratada para modificar o sistema atual, adicionando essa possibilidade. Observe na **Figura 5.9** o diagrama UML dessa aplicação.



**Figura 5.9** – Diagrama UML da concessionária de veículos que solicitou uma alteração em seu sistema.

Crédito: do elaborador, 2025.

Você deverá desenvolver uma *procedure* capaz de receber as informações necessárias, como o “id” do vendedor, mês e ano, e calcular o valor a ser recebido segundo as regras definidas pela concessionária, conforme os **requisitos** a seguir:

1. Todo vendedor recebe um salário fixo de R\$ 4 000,00, independentemente da quantidade de vendas realizadas por ele naquele mês;
2. Como complemento variável, cada vendedor recebe uma comissão de 3% pelo valor total das vendas de veículos realizadas naquele mês;
3. Outro complemento variável: cada vendedor recebe uma comissão de 6% pelo valor total das vendas de acessórios veiculares realizadas naquele mês;

4. E, por fim, como bonificação, o vendedor recebe a quantia adicional de R\$ 250,00, caso suas faltas ao trabalho não excedam quatro horas naquele mês.

Para atender a esses requisitos, criaremos uma *procedure* seguindo os passos a seguir.

Primeiro, criaremos a estrutura da *procedure*, com os parâmetros de entrada “id do vendedor”, “mês” e “ano” de referência na consulta. O parâmetro de saída será o valor final a ser recebido pelo vendedor.

```
delimiter $$
create procedure calcula_salario(in in_vendedor_id bigint,
                                in in_mes int,
                                in in_ano int,
                                out out_salario
                                decimal(10,2))
begin
  -- criação dos valores fixos:
  declare var_salario_fixo decimal(8,2) default 4000.00;
  declare var_taxa_carros decimal(5,2) default 0.03;
  declare var_taxa_acessorios decimal(5,2) default 0.06;
  declare var_bonus_faltas decimal(8,2) default 250.00;

  -- criação das variáveis:
  declare var_horas_faltas int default 0;
  declare var_vendas_carros decimal(12,2) default 0.00;
  declare var_vendas_acessorios decimal(12,2) default 0.00;
end
$$
delimiter ;
```

Agora, faremos a consulta para obter a soma das horas correspondentes às faltas dos vendedores no mês e ano de consulta.



```

delimiter $$
create procedure calcula_salario(in in_vendedor_id bigint,
                                in in_mes int,
                                in in_ano int,
                                out out_salario
decimal(10,2))
begin
    -- criação dos valores fixos:
    declare var_salario_fixo decimal(8,2) default 4000.00;
    declare var_taxa_carros decimal(5,2) default 0.03;
    declare var_taxa_acessorios decimal(5,2) default 0.06;
    declare var_bonus_faltas decimal(8,2) default 250.00;

    -- criação das variáveis:
    declare var_horas_faltas int default 0;
    declare var_vendas_carros decimal(12,2) default 0.00;
    declare var_vendas_acessorios decimal(12,2) default 0.00;

    -- soma das faltas do funcionário no mês e ano de
consulta:
    select coalesce(sum(f.quantidade_horas),0) into
var_horas_faltas
    from faltas_funcionarios f where f.funcionario_FK =
in_vendedor_id
    and month(f.data) = in_mes and year(f.data) = in_ano;

end
$$
delimiter ;

```

Note que no bloco de código anterior, tivemos o cuidado de adicionar a função **coalesce** para garantir que, caso o vendedor não possua nenhuma falta, o valor na variável não fique nulo e sim zerado.

Em seguida, criaremos a consulta para obter a soma das vendas de veículos realizadas pelo vendedor.

```

delimiter $$
create procedure calcula_salario(in in_vendedor_id bigint,
                                in in_mes int,
                                in in_ano int,
                                out out_salario
decimal(10,2))
begin
    -- criação dos valores fixos:
    declare var_salario_fixo decimal(8,2) default 4000.00;
    declare var_taxa_carros decimal(5,2) default 0.03;
    declare var_taxa_acessorios decimal(5,2) default 0.06;
    declare var_bonus_faltas decimal(8,2) default 250.00;

    -- criação das variáveis:
    declare var_horas_faltas int default 0;
    declare var_vendas_carros decimal(12,2) default 0.00;
    declare var_vendas_acessorios decimal(12,2) default 0.00;

    -- soma das faltas do funcionário no mês e ano de
consulta:
    select coalesce(sum(f.quantidade_horas),0) into
var_horas_faltas
    from faltas_funcionarios f where f.funcionario_FK =
in_vendedor_id
    and month(f.data) = in_mes and year(f.data) = in_ano;

    /* soma das vendas veiculares do funcionário no mês e ano
de consulta:*/
    select coalesce(sum(vv.valor_venda), 0.00) into
var_vendas_carros
    from vendas_veiculos vv where vv.vendedor_FK =
in_vendedor_id
    and month(vv.data_venda) = in_mes and year(vv.data_venda)
= in_ano;

end

```

```
$$
delimiter ;
```

Da mesma forma, faremos a soma do valor das vendas de acessórios no bloco de código a seguir.

```
delimiter $$
create procedure calcula_salario(in in_vendedor_id bigint,
                                in in_mes int,
                                in in_ano int,
                                out out_salario
decimal(10,2))
begin
    -- criação dos valores fixos:
    declare var_salario_fixo decimal(8,2) default 4000.00;
    declare var_taxa_carros decimal(5,2) default 0.03;
    declare var_taxa_acessorios decimal(5,2) default 0.06;
    declare var_bonus_faltas decimal(8,2) default 250.00;

    -- criação das variáveis:
    declare var_horas_faltas int default 0;
    declare var_vendas_carros decimal(12,2) default 0.00;
    declare var_vendas_acessorios decimal(12,2) default 0.00;

    -- soma das faltas do funcionário no mês e ano de
consulta:
    select coalesce(sum(f.quantidade_horas),0) into
var_horas_faltas
    from faltas_funcionarios f where f.funcionario_FK =
in_vendedor_id
    and month(f.data) = in_mes and year(f.data) = in_ano;

    /* soma das vendas veiculares do funcionário no mês e ano
de consulta:*/
```

```

select coalesce(sum(vv.valor_venda), 0.00) into
var_vendas_carros
  from vendas_veiculos vv where vv.vendedor_FK =
in_vendedor_id
  and month(vv.data_venda) = in_mes and year(vv.data_venda)
= in_ano;

/* soma das vendas dos acessórios do funcionário no mês e
ano de consulta:*/
select coalesce(sum((va.quantidade * a.valor) - va.desconto
), 0.00)
into var_vendas_acessorios from vendas_acessorios va
join acessorios a on a.id = va.acessorio_FK where
va.vendedor_FK = in_vendedor_id and month(va.data_venda) =
in_mes
and year(va.data_venda) = in_ano;

end
$$
delimiter ;

```

Em seguida, calcularemos o valor do salário, considerando o que foi obtido em cada variável, adicionando também a condição de bonificação.

```

delimiter $$
create procedure calcula_salario(in in_vendedor_id bigint,
                                in in_mes int,
                                in in_ano int,
                                out out_salario
decimal(10,2))
begin
  -- criação dos valores fixos:
  declare var_salario_fixo decimal(8,2) default 4000.00;
  declare var_taxa_carros decimal(5,2) default 0.03;
  declare var_taxa_acessorios decimal(5,2) default 0.06;

```

```

declare var_bonus_faltas decimal(8,2) default 250.00;

-- criação das variáveis:
declare var_horas_faltas int default 0;
declare var_vendas_carros decimal(12,2) default 0.00;
declare var_vendas_acessorios decimal(12,2) default 0.00;

-- soma das faltas do funcionário no mês e ano de
consulta:
select coalesce(sum(f.quantidade_horas),0) into
var_horas_faltas
from faltas_funcionarios f where f.funcionario_FK =
in_vendedor_id
and month(f.data) = in_mes and year(f.data) = in_ano;

/* soma das vendas veiculares do funcionário no mês e ano
de consulta:*/
select coalesce(sum(vv.valor_venda), 0.00) into
var_vendas_carros
from vendas_veiculos vv where vv.vendedor_FK =
in_vendedor_id
and month(vv.data_venda) = in_mes and year(vv.data_venda)
= in_ano;

/* soma das vendas dos acessórios do funcionário no mês e
ano de consulta:*/
select coalesce(sum((va.quantidade * a.valor) - va.desconto
), 0.00)
into var_vendas_acessorios from vendas_acessorios va
join acessorios a on a.id = va.acessorio_FK where
va.vendedor_FK = in_vendedor_id and month(va.data_venda) =
in_mes
and year(va.data_venda) = in_ano;

-- cálculo do salário:
set out_salario = (var_salario_fixo + (var_vendas_carros *

```

```

    var_taxa_carros) + (var_vendas_acessorios *
var_taxa_acessorios));

-- adição da bonificação:
if var_horas_faltas <= 4 then
    set out_salario = (out_salario + var_bonus_faltas);
end if;
end
$$
delimiter ;

```

Agora, mostraremos ao(a) aluno(a) como esse código pode ser testado. Para isso, será necessário que você use o que aprendeu até este momento, a fim de criar as tabelas de acordo com a modelagem dessa aplicação e inserir dados em todas elas.

Antes de usarmos nossa *procedure*, faremos as consultas individuais para obter a soma das vendas de veículos e acessórios do vendedor que usaremos para o teste. Não podemos nos esquecer também de realizar as consultas de suas respectivas faltas. Essa etapa é importante para podermos comparar os resultados obtidos com o da *procedure* e assim ter a certeza de que as operações executadas por ela estão corretas.

```

04 * select coalesce(sum(f.quantidade_horas),0) as faltas from faltas_funcionarios f
05     where f.funcionario_FK = 5 and month(f.data) = 2 and year(f.data) = 2025;

faltas
0

07 * select (coalesce(sum(vv.valor_venda), 0.00)*0.03) as 'comissão veículos'
08     from vendas_veiculos vv where vv.vendedor_FK = 5
09     and month(vv.data_venda) = 2 and year(vv.data_venda) = 2025;

comissão
veículos
5190.0000

11 * select (coalesce(sum( (va.quantidade * a.valor) - va.desconto ), 0.00)*0.06) as
12     'comissão acessórios' from vendas_acessorios va join acessorios a on a.id = va.acessorio_FK
13     where va.vendedor_FK = 5 and month(va.data_venda) = 2 and year(va.data_venda) = 2025;

comissão
acessórios
0.0000

```

**Figura 5.10** – Código em SQL com o resultado de consultas para cálculo do salário a ser recebido por um determinado vendedor em um mês e ano, para servir de comparação com o valor a ser calculado pela *procedure*.

Crédito: do elaborador, 2025.

Os resultados da **Figura 5.10** nos mostram que o vendedor a ser usado no teste, com “id” 5, obteve no mês de fevereiro de 2025:

- Nenhuma falta no período selecionado;
- Comissão de R\$ 5 190,00 nas vendas de veículos;
- Nenhuma comissão na venda de acessórios.

Dessa maneira, podemos seguir com a soma dos seguintes valores:

- R\$ 250,00 de bonificação por não possuir nenhuma falta no período;
- R\$ 5 190,00 de comissão de vendas de veículos;
- R\$ 4 000,00 de base fixa do salário de vendedor.

Finalmente, por meio desse cálculo manual, sabemos que o vendedor 5, em fevereiro de 2025, recebeu a quantia de R\$ 9 440,00.

Agora que já temos o valor a ser comparado, usaremos a nossa *procedure* para que esse mesmo cálculo seja feito com facilidade muito maior.

```
'15 -- cria uma variável de sessão para armazenar o resultado:
'16 • set @salario = 0;
'17 -- chama a procedure para o vendedor 5, no mês 2, ano de 2025 e com a variável de saída:
'18 • call calcula_salario(5, 2, 2025, @salario);
'19 -- consulta o resultado:
'20 • select @salario;
```



| Result Grid | Filter Rows: | Export: | Wrap Cell Content: |
|-------------|--------------|---------|--------------------|
| @salario    |              |         |                    |
| 9440.00     |              |         |                    |

**Figura 5.11** – Código em SQL com o resultado do salário a ser recebido por um determinado vendedor em um mês e ano por meio da *procedure* criada.

Crédito: do elaborador, 2025.

Como você pode constatar, obtemos com a *procedure* “calcula\_salario” exatamente o mesmo valor calculado manualmente, o que comprova sua eficiência e facilidade de uso.

Exercícios – Concessionária de veículos: continuação

A solução implementada na concessionária de veículos para que os funcionários possam acompanhar seus recebimentos foi um sucesso! Contudo,

um ponto de melhoria que foi levantado é a possibilidade de visualização de cada parte que compôs o cálculo do salário final do funcionário. Em outras palavras, se um salário calculado foi de R\$ 8 000,00, a aplicação deve permitir que o empregado consiga visualizar cada parte utilizada para chegar nesse rendimento.

Desse modo, altere a *procedure* que foi criada para também retornar mais quatro variáveis de saída, equivalentes a:

- Total da comissão de vendas de veículos;
- Total da comissão de vendas de acessórios;
- Total da bonificação recebida;
- Total do valor fixo do salário do funcionário.

Lembre-se de realizar os testes para validação de seu funcionamento após executar as modificações na *procedure*.

## drop procedure – Removendo *procedures*

De forma semelhante ao que ocorre com diversos recursos do MySQL, usamos o comando **drop** para remoção de *procedures* criadas. Observe a estrutura desse comando no bloco de código a seguir.

```
-- ESTRUTURA DO COMANDO:  
drop procedure NOME_DA_PROCEDURE;  
  
-- EXEMPLOS:  
drop procedure calcula_salario;  
drop procedure atualiza_metas;
```

Devido à sua simplicidade, não implementaremos nenhum caso de uso para esse comando.



## Views

No MySQL, as *views* são tabelas virtuais geradas por meio de consultas, desenvolvidas com o intuito de segregar parte de uma ou mais tabelas existentes. Por esse motivo elas acabam se tornando uma excelente opção para evitar algumas repetições de cláusulas em consultas de determinadas aplicações, ou até mesmo simplificar a união entre tabelas.

Para que o assunto fique mais concreto, imagine que você possui uma aplicação com uma tabela de usuários e nela há o *status* de ativação do usuário. Nessa mesma aplicação, como uma regra de negócio, quase todas as consultas e operações realizadas no banco de dados devem ser realizadas apenas para usuários ativos, fazendo com que a cláusula “WHERE status = 1”, seja repetida em diversos comandos. Em casos como esse, o uso de *views* se aplica perfeitamente.

### create view – Criando views

Para criarmos uma *view*, utilizamos o comando **create view**. Vejamos a estrutura desse comando no bloco de código a seguir, com um exemplo sobre o contexto de usuários que mencionamos anteriormente.

```
-- ESTRUTURA DO COMANDO:
create view NOME_DA_VIEW as
select [distinct] LISTA_DE_COLUNAS
from NOME_DA_TABELA
[join NOME_DA_TABELA_2 on CONDIÇÃO_DE_JUNÇÃO]
...
[join NOME_DA_TABELA_2 on CONDIÇÃO_DE_JUNÇÃO]
[where (CONDIÇÕES_FILTRO..)]
[group by LISTA_DE_COLUNAS]
[having (CONDIÇÕES_FILTRO_CONSOLIDAÇÃO...)]
[order by LISTA_DE_COLUNAS [asc/desc] ]
```

```

[limit NÚMERO_DE_LINHAS [offset VALOR_INICIAL] ];

-- EXEMPLOS:
-- view com apenas usuários ativos
create view usuarios_ativos as
select * from usuarios where ativo = 1;

-- view com os pedidos atrasados (que passaram de 20 dias)
create view pedidos_atrasados as
select v.numero_pedido, v.data_pedido, f.email as
email_vendedor,
       c.nome as nome_cliente, c.email as email_cliente,
       c.telefone as telefone_cliente from vendas v
join funcionarios f on v.vendedor_FK = f.id
join clientes c on v.cliente_FK = c.id
where v.entregue = 0 and
datediff(current_date(), v.data_pedido) > 20;

```

De acordo com o bloco de código anterior, podemos constatar que o comando de criação da *view* se divide basicamente em duas partes: a criação do nome da *view* e a consulta de seleção que vai popular a tabela virtual. Na primeira parte, não há nenhuma complexidade, resumindo-se apenas pela definição do nome dado. Já na segunda parte, ela obedece a todas as exigências e possibilidades existentes nos comandos **select** tradicionais, não havendo nada de especial.

## IMPORTANTE

### Views e join

**Observe que, conforme o exemplo mostrado no bloco de código, o uso de views também pode servir para dispensar o uso de junções repetitivas com o **join**, ou até mesmo certos agrupamentos.**

Agora que você percebeu que a criação das *views* não é nada complicada – aliás, a complexidade que podem apresentar está exclusivamente

relacionada com o **select** a ser utilizado para construí-las –, provavelmente, deve estar se perguntando como usá-las.

Embora uma *view* seja considerada uma tabela virtual, seu modo de uso é exatamente como se ela fosse uma tabela tradicional, com a exceção de que, evidentemente, ela não poderá receber comandos de escrita ou deleção de valores, como **update** ou **delete**, da mesma forma que as tabelas comuns. Para esses casos, sempre que for necessário atualizar ou deletar registros, é recomendado que os comandos sejam executados diretamente nas tabelas que originaram a *view*.

Contudo, existem algumas possibilidades de se executar os comandos **update** ou **delete** diretamente nas *views*, caso elas sigam estes requisitos:

- Tenham sido construídas por meio de uma única tabela (sem envolver processos de união);
- Tenham sido construídas sem o uso de agrupamentos, seleções distintas, funções de agregações ou uniões ou tenham seus valores paginados (ou seja, foram criadas sem o uso de **join**, **group by**, **having**, **limit**, **offset**, **distinct**, **sum**, **avg** etc.);
- Não tenham utilizado subconsultas para sua geração;
- Contenham todas as colunas da tabela original.

De forma simplificada, o que queremos dizer é que, para a *view* do bloco de código seguinte, podemos executar as operações de **select**, **update** e **delete**, se assim quisermos, pois ela atende a todas as restrições mencionadas.

```
-- view com apenas usuários ativos, criada com uma única
tabela
create view usuarios_ativos as
  select * from usuarios where ativo = 1;
```

```

-- busca os usuários ativos cadastrados à partir de
01/01/2025
select * from usuarios_ativos where data_cadastro >=
'2025-01-01';

-- atualiza o email de um usuário diretamente na view
update usuarios_ativos set email = 'novo_email@email.com'
where id = 32;

-- deleta um usuário ativo com um determinado cpf.
delete from usuarios_ativos where cpf = '33333344444';

```

Em contrapartida, a *view* do próximo bloco de código não dá esse suporte aos comandos **update** ou **delete**, já que não atende a todos os requisitos, uma vez que não foi criada com todas as colunas e utilizou mais de uma tabela no processo de criação (uso do **join**), havendo apenas a possibilidade de uso do **select**.

```

-- view com os pedidos atrasados (que passaram de 20 dias)
create view pedidos_atrasados as
select v.numero_pedido, v.data_pedido, f.email as
email_vendedor,
c.nome as nome_cliente, c.email as email_cliente,
c.telefone as telefone_cliente from vendas v
join funcionarios f on v.vendedor_FK = f.id
join clientes c on v.cliente_FK = c.id
where v.entregue = 0 and
datediff(current_date(), v.data_pedido) > 20;

-- busca os 10 pedidos mais atrasados
select * from pedidos_atrasados order by data_pedido asc
limit 10;

-- busca a relação dos clientes e a quantidade de pedidos
atrasados

```

```
select nome_cliente, email_cliente, count(*) as  
quantidade_pedidos_atrasados  
from pedidos_atrasados group by email_cliente;
```

## drop view – Removendo views

Para removermos uma *view* criada, utilizamos o comando **drop view**. Observe a estrutura do bloco de código a seguir, com alguns exemplos de uso.

```
-- ESTRUTURA DO COMANDO  
drop view NOME_DA_VIEW;  
  
-- EXEMPLOS:  
drop view pedidos_atrasados;  
drop view produtos_nacionais_ativos;
```

Como você pode perceber, esse comando apresenta o mesmo funcionamento de suas variantes vistas anteriormente para remoção de *procedures*, *triggers*, tabelas etc.

### SAIBA MAIS

#### create or replace view

Em algumas situações, você pode desejar alterar uma *view* já existente. Para esses casos, você pode optar por excluí-la para depois recriá-la. Contudo, para evitar que sejam dados dois comandos, é interessante que o(a) aluno(a) saiba que é possível utilizar o **create or replace view**, fazendo com que o MySQL crie ou substitua uma *view* já existente, conforme o bloco de código a seguir.

```

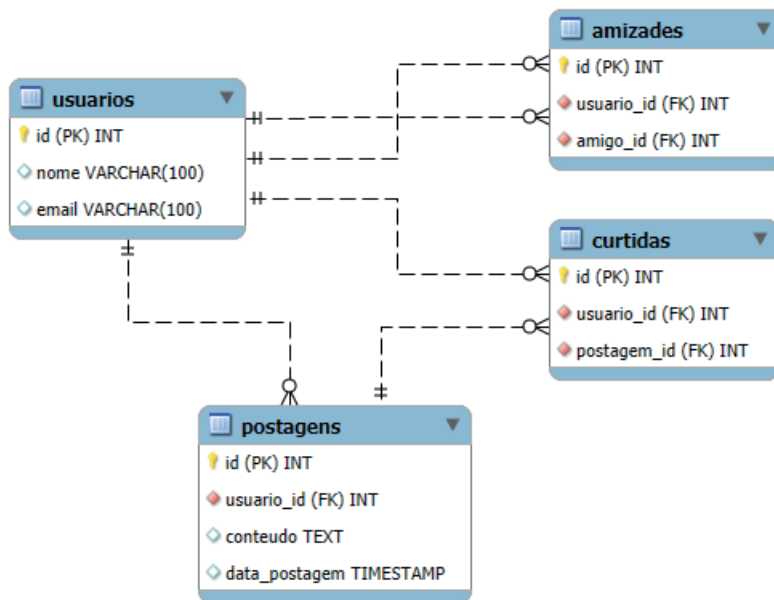
/* criação ou substituição de uma view para coletar os nomes
dos funcionários que são gestores e que trabalham na unidade
de uma empresa no Brasil */
create or replace view gestores_brasil as
select f.id, f.nome, f.email, f.data_contratacao from
funcionarios f
join cargos c on f.cargo_FK = c.id
where c.categoria = 'GESTOR' and f.unidade = 'BRASIL';

```

## Exemplos e casos de uso – Rede social

Uma nova rede social está surgindo e você foi escalado para atuar em seu desenvolvimento. Uma das premissas dessa rede é otimizar o desempenho da aplicação para que seus usuários tenham uma navegação fluida e boa experiência do usuário.

Portanto, chega até você a necessidade de desenvolver uma *view* para simplificar a exibição do “total de curtidas” de cada postagem.



**Figura 5.12** – Diagrama UML de uma rede social.

Crédito: do elaborador, 2025.

Observe que, de acordo com a modelagem apresentada na **Figura 5.12**, essa parte da aplicação está organizada em quatro entidades que se relacionam: “usuarios”, “amizades”, “postagens” e “curtidas”.

A problemática apresentada é que, para que se exiba na tela o *feed* de postagens, a aplicação precisa realizar diversas consultas de agregação, fazendo com que os comandos fiquem maiores e repetitivos apenas para se obter a informação do número de curtidas de cada postagem na rede social.

Inicialmente, no bloco de código a seguir, faremos a criação de todas as tabelas, seguindo a modelagem apresentada na **Figura 5.12**.

```
create table usuarios (  
  id int primary key auto_increment,  
  nome varchar(100),  
  email varchar(100) unique  
);  
  
create table amizades (  
  id int primary key auto_increment,  
  usuario_id int,  
  amigo_id int,  
  foreign key (usuario_id) references usuarios(id),  
  foreign key (amigo_id) references usuarios(id)  
);  
  
create table postagens (  
  id int primary key auto_increment,  
  usuario_id int,  
  conteudo text,  
  data_postagem timestamp default current_timestamp,  
  foreign key (usuario_id) references usuarios(id)  
);  
  
create table curtidas (  
  id int primary key auto_increment,
```

```

    usuario_id int,
    postagem_id int,
    foreign key (usuario_id) references usuarios(id),
    foreign key (postagem_id) references postagens(id)
);

```

Assim, para solucionar esse problema, desenvolveremos uma *view* para obter as informações desejadas de modo mais direto, conforme detalhado no bloco de código a seguir.

```

/* cria uma view com as curtidas de cada postagem */
create or replace view feed_com_curtidas as
    select p.id as postagem_id, p.conteudo, p.usuario_id, u.nome
as autor,
    count(c.id) as total_curtidas
from postagens p
left join curtidas c on p.id = c.postagem_id
join usuarios u on p.usuario_id = u.id
group by p.id, p.conteudo, p.usuario_id, u.nome;

```

Note que, para compilar as informações básicas de cada postagem, realizamos a junção entre as entidades de postagens, curtidas e usuários, agrupando-as.

Observe também que tivemos o cuidado de realizar um **left join** entre as tabelas de postagens e curtidas, uma vez que desejamos exibir inclusive as postagens que ainda não receberam nenhuma curtida, ou seja, relacionamento unidirecional.

Por fim, estamos utilizando a função **count** para contabilizar a quantidade de vezes que “ids” de curtidas aparecem no resultado para cada postagem agrupada.

Agora, para testarmos nossa *view*, faremos a inserção de dados fictícios.

```

insert into usuarios (nome, email) values
    ('João Silva', 'joao@email.com'),
    ('Maria Oliveira', 'maria@email.com'),

```



```

('Carlos Souza', 'carlos@email.com'),
('Ana Mendes', 'ana@email.com'),
('Pedro Santos', 'pedro@email.com');

insert into amizades (usuario_id, amigo_id) values
(1, 2), (1, 3), (2, 4), (3, 1);

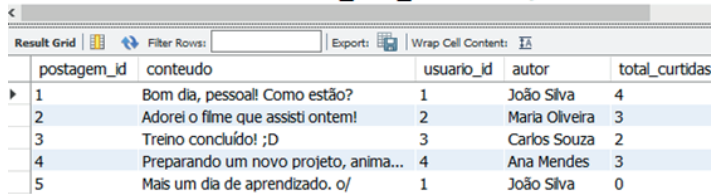
insert into postagens (usuario_id, conteudo) values
(1, 'Bom dia, pessoal! Como estão?'),
(2, 'Adorei o filme que assisti ontem!'),
(3, 'Treino concluído! ;D'),
(4, 'Preparando um novo projeto, animada!'),
(1, 'Mais um dia de aprendizado! \o/');

insert into curtidas (usuario_id, postagem_id) values
(2, 1), (3, 1), (4, 1), (5, 1), (1, 2), (3, 2),
(4, 2), (1, 3), (2, 3), (1, 4), (2, 4), (3, 4);

```

Finalmente, faremos a consulta da *view* criada, conforme se vê na **Figura 5.13**.

94 • **SELECT \* FROM feed\_com\_curtidas;**



| postagem_id | conteudo                             | usuario_id | autor          | total_curtidas |
|-------------|--------------------------------------|------------|----------------|----------------|
| 1           | Bom dia, pessoal! Como estão?        | 1          | João Silva     | 4              |
| 2           | Adorei o filme que assisti ontem!    | 2          | Maria Oliveira | 3              |
| 3           | Treino concluído! ;D                 | 3          | Carlos Souza   | 2              |
| 4           | Preparando um novo projeto, anima... | 4          | Ana Mendes     | 3              |
| 5           | Mais um dia de aprendizado. o/       | 1          | João Silva     | 0              |

**Figura 5.13** – Comando para consulta das postagens e suas respectivas curtidas.

Crédito: do elaborador, 2025.

## Gerenciamento de usuários

Ao longo deste livro, percorremos uma vasta quantidade de assuntos, em que podemos executar diversos exercícios e casos de uso. Em todos eles, caso você tenha seguido as orientações existentes no início do **Capítulo 3**,

fizemos a utilização do usuário raiz para a conexão com o servidor MySQL, denominado “root”.

Contudo, em ambientes reais de desenvolvimento, isto é, em empresas em que você possa atuar profissionalmente como um desenvolvedor, é muito comum que não seja dado livre acesso a esse tipo de usuário “mestre”, por medidas de segurança, já que, com ele, você é capaz de excluir todos os registros, alterar estruturas de tabelas ou removê-las e, até mesmo, deletar todo o banco de dados.

#### IMPORTANTE

##### Usuários

**Quando nos referimos a “usuários” do MySQL, é importante que não se confunda com possíveis tabelas de usuários existentes em determinadas aplicações. Aqui, estamos nos referindo aos usuários de conexão com o servidor do MySQL e não aos pertencentes à aplicação que faz uso do banco de dados.**

Diante desses fatos, é muito comum que as empresas realizem a criação de usuários específicos, delegando-lhes o nível de permissão compatível com o contexto em que será usado. Embora não seja uma regra, é muito comum haver a separação de usuários destinados a operações mais simples de consulta, outros para consultas e atualizações, alguns capazes de executar modificações ou deleções mais profundas na base de dados e, finalmente, algum detentor do privilégio máximo, podendo inclusive criar, remover ou conceder acessos a outros usuários.

#### IMPORTANTE

##### Motivos da existência de diversos usuários

**Vale ressaltar que a existência de diversos usuários não é, necessariamente, com a intenção de serem usados em conexões realizadas por pessoas (desenvolvedores, por exemplo), mas sim também serem usados pelas próprias aplicações. Também é muito comum que o usuário utilizado para conexão com a aplicação não apresente todos os privilégios máximos.**

Vejamos alguns benefícios obtidos por meio dessa organização entre usuários:

- Diminuição da possibilidade de modificações acidentais no banco de dados (restringindo usuários com maiores privilégios apenas para desenvolvedores mais experientes);
- Redução da criticidade em ataques cibernéticos (as aplicações geralmente não utilizam usuários com privilégios máximos);
- Melhoria na organização e na rastreabilidade das operações.

#### IMPORTANTE

##### Concessão de privilégios

**É de extrema importância que o(a) aluno(a) entenda que apenas criar um usuário, embora seja parte primordial, não garante automaticamente os benefícios mencionados. Para isso, é fundamental utilizar os comandos de concessão de privilégios, pertencentes a uma categoria de comandos denominada DCL (Linguagem de Controle de Dados), a qual veremos mais adiante.**

## create user – Criando um usuário no MySQL

Agora que você percebeu a importância dos usuários, sobretudo em cenários reais de desenvolvimento, é hora de aprendermos a criá-los. Para isso, é fundamental que você esteja utilizando o usuário raiz de seu servidor, assim como orientamos no início do **Capítulo 3**, ou algum outro com privilégio equivalente.

Usaremos o comando **create user**, obedecendo à estrutura do bloco de código a seguir.

```
-- ESTRUTURA DO COMANDO:  
create user 'NOME_DO_USUÁRIO'@'ENDEREÇO_CONEXÃO' identified by  
    'SENHA_DO_USUÁRIO';
```

```

-- EXEMPLOS
/* cria um usuário com nome de <suporte>, que pode se
conectar com o mesmo endereço/local do servidor: */
create user 'suporte'@'localhost' identified by
'2025-suporte-pass';

/* cria um usuário com nome de 'observer', que pode se
conectar através de qualquer origem: */
create user 'observer'@'%' identified by 'read-only1122';

/* cria um usuário com nome de 'plataforma_eventos_user',
que pode se conectar apenas através do IP 192.168.15.155: */
create user 'plataforma_eventos_user'@'192.168.15.155'
identified by
'eventos$1961_r1';

```

Note que, de acordo com o bloco de código anterior, para criarmos um novo usuário, precisamos passar três informações importantes: nome, senha e endereço de origem permitido para a conexão com o servidor. Esse último parâmetro é utilizado para servir como uma camada adicional de segurança, ou seja, mesmo que se obtenha o nome e a senha do usuário, somente as origens especificadas serão permitidas, sendo comum para esses casos algumas empresas utilizarem endereços de IP dentro de uma VPN (*virtual private network* ou rede virtual privada). Em situações em que não se opte por essa proteção de origem, podemos utilizar o símbolo “%” para permitir todas as origens.

## MySQL – Conectando-se com um usuário

Após criados os usuários, podemos estabelecer a conexão com o servidor MySQL utilizando o mesmo processo já abordado no tópico **Conectando-se ao servidor**, do **Capítulo 3**, atentando-se, no entanto, para a utilização de nome, usuário e origem especificada durante a criação de seu usuário.

Para exemplificação, faremos agora a criação de um usuário e sua conexão por meio do terminal de comando.

Primeiro, realizaremos a criação do usuário, partindo do uso de um que possua o privilégio máximo (no caso, usuário “root”), conforme se vê na **Figura 5.14**.

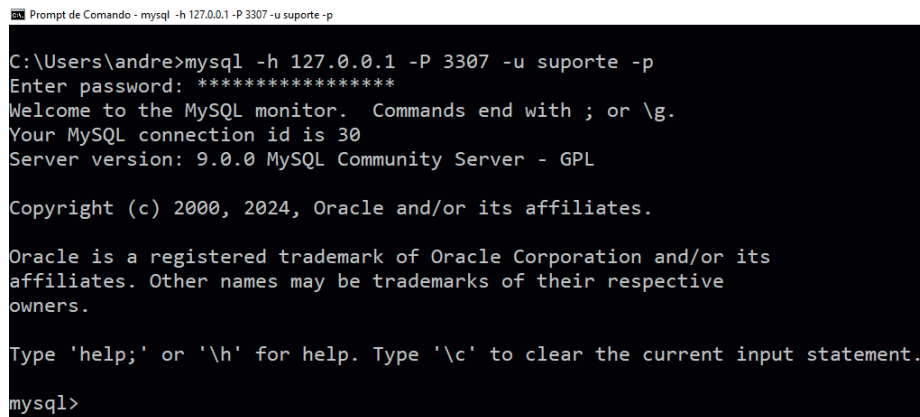
```
98 • create user 'suporte'@'%' identified by '2025-suporte-pass';
```



**Figura 5.14** – Criação de um usuário chamado “suporte”.

Crédito: do elaborador, 2025.

Em seguida, abriremos o terminal para estabelecer a conexão, digitando o comando MySQL e passando os parâmetros necessários, conforme **Figura 5.15**.



**Figura 5.15** – Conexão com o servidor MySQL, utilizando o usuário criado, por meio de terminal de comando.

Crédito: do elaborador, 2025.

Observe que, após passada a informação de que desejamos estabelecer a conexão usando o usuário “suporte”, deveremos digitar a mesma senha definida anteriormente.

Note também que alguns dos parâmetros usados nesse comando dependerão da forma como o seu servidor MySQL está configurado. Havendo dúvidas, sugerimos que o(a) aluno(a) reveja o tópico **Conectando-se ao servidor**, do **Capítulo 3**.

Porém, de maneira geral, ao digitarmos o comando “mysql -h 127.0.0.1 -P 3307 -u suporte -p”, estamos informando que:

- O servidor MySQL está sendo executado na máquina local (**localhost** ou **127.0.0.1**);
- O servidor MySQL não está utilizando a porta padrão e sim a **porta 3307**;
- Desejamos utilizar o usuário com nome “suporte”;
- Digitaremos a senha do respectivo usuário após a confirmação do comando.

## drop user – Removendo um usuário no MySQL

Para remoção dos usuários criados, usaremos mais uma variante de um comando já muito conhecido: o **drop user**.

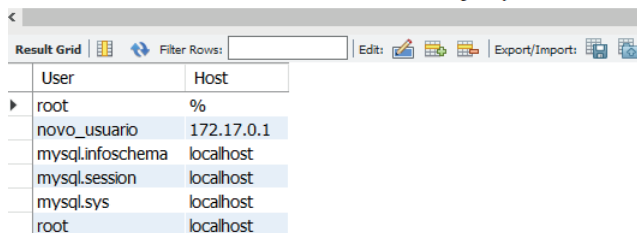
```
-- ESTRUTURA DO COMANDO:  
drop user 'NOME_DO_USUÁRIO'@'ENDEREÇO_CONEXÃO';  
  
-- EXEMPLOS  
drop user 'suporte'@'127.0.0.1';  
drop user 'master-admin'@'%';
```

## select mysql.user – Exibindo usuários do MySQL

Os usuários criados para conexão com o MySQL são também armazenados em uma tabela chamada “user”, presente no *schema* “mysql”.

Dessa forma, para visualizarmos os usuários disponíveis, basta que consultemos essa entidade usando o comando **select** padrão. Veja o exemplo da **Figura 5.16**.

**36 • SELECT User, Host FROM mysql.user;**



| User             | Host       |
|------------------|------------|
| root             | %          |
| novo_usuario     | 172.17.0.1 |
| mysql.infoschema | localhost  |
| mysql.session    | localhost  |
| mysql.sys        | localhost  |
| root             | localhost  |

**Figura 5.16** – Exibição dos usuários existentes e o endereço autorizado para conexão.

Crédito: do elaborador, 2025.

#### SAIBA MAIS

##### Tabela mysql.user

A tabela “mysql.user” possui uma quantidade significativa de colunas, apresentando diversas informações, tais como nome do usuário, endereço autorizado de conexão, quantidade de conexões ativas, data de alteração da senha, privilégios para execução de operações e muitos outros dados. Aqui evidenciamos apenas o nome do usuário e “host” (endereço autorizado de conexão), mas convidamos o(a) aluno(a) a explorar mais os recursos dessa tabela.

## Exemplos e casos de uso – Indústria automotiva

Você já deve ter visto alguma notícia ou ouvido falar sobre o ambiente sigiloso envolvendo indústrias ou montadoras automotivas durante o desenvolvimento de um novo projeto até seu lançamento. Basicamente, é por meio desse sigilo que tais empresas alcançam o pioneirismo entre suas concorrentes e surpreendem seus clientes com novos produtos.

Diante disso, uma empresa deseja compartimentar o acesso a alguns recursos do banco de dados por meio da criação de usuários específicos:

- Um usuário para acesso aos projetos elétricos, chamado “usuario\_eletrica”;
- Um usuário para acesso aos projetos mecânicos, chamado “usuario\_mecanica”;
- Um usuário para acesso aos projetos estéticos e de acabamentos, chamado “usuario\_design”;
- Um usuário para acesso aos resultados de testes, chamado “usuario\_testes”;
- Um usuário para acesso gerencial com todas as tabelas, chamado “usuario\_gestao”.

Primeiro, criaremos os usuários, conforme bloco de código a seguir. Usamos aqui uma faixa de IP de um servidor específico, mas o(a) aluno(a) pode substituir pelo **localhost** ou colocar o %.

```
/* um usuário para acesso aos projetos elétricos, chamado
   “usuario_eletrica”; */
create user 'usuario_eletrica'@'172.17.0.1' identified by
  'eletrica#64rosa';

/* um usuário para acesso aos projetos mecânicos, chamado
   “usuario_mecanica”; */
create user 'usuario_mecanica'@'172.17.0.1' identified by
  'mec2025-proj';

/* um usuário para acesso aos projetos estéticos e de
   acabamentos, chamado “usuario_design”; */
create user 'usuario_design'@'172.17.0.1' identified by
  'auto_91design';

/* um usuário para acesso aos resultados de testes, chamado
   “usuario_testes”; */
```



```
create user 'usuario_testes'@'172.17.0.1' identified by
'Teste1703';

/* um usuário para acesso gerencial com todas as tabelas,
chamado "usuario_gestao"; */
create user 'usuario_gestao'@'172.17.0.1' identified by
'manager2025@gi';
```

Em seguida, faremos o teste dos usuários criados, fazendo a conexão com o servidor MySQL por meio do *login* e da senha definidos (mostraremos a conexão com apenas um deles), conforme a **Figura 5.17**.

```
C:\Users\andre>mysql -h 127.0.0.1 -P 3307 -u usuario_eletrica -p
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 41
Server version: 9.0.0 MySQL Community Server - GPL

Copyright (c) 2000, 2024, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
```

**Figura 5.17** – Teste de conexão com o usuário criado.

Crédito: do elaborador, 2025.

## DCL – Linguagem de Controle de Dados

A Linguagem de Controle de Dados (DCL, *Data Control Language*) é composta por comandos que fazem o gerenciamento de privilégios de acesso aos objetos de um banco de dados. É por meio de seus comandos que, de fato, conseguimos ampliar ou reduzir a quantidade de operações permitidas para cada usuário existente no servidor.

São dois tipos de comandos: **grant** (para concessão de privilégios) e **revoke** (para revogação de privilégios).

## grant – Concessão de privilégios

Os comandos do tipo **grant** têm o objetivo de conceder privilégios aos usuários especificados, definindo inclusive as operações que serão possíveis, conforme bloco de código a seguir.

```
-- ESTRUTURA DO COMANDO:
grant PERMISSÃO1, PERMISSÃO2, ... on NOME_DO_BANCO.
NOME_DO_OBJETO
to USUÁRIO1, USUÁRIO2, ... [with grant option];

-- EXEMPLOS:
/* concede apenas acesso de consulta ao 'usuario_observador'
em todas as tabelas do banco de dados <rede_social' */
grant select on rede_social.* to usuario_observador;

/* concede acesso de consulta, atualização e inserção ao 'u_
corretora', na tabela de apólices do banco corretora */
grant select, update, insert on corretora.apolices to
u_corretora;

/* concede todos os privilégios ao 'usuario_master' em todas
as tabelas dos bancos, com possibilidade de replicação de
privilégios para outros usuários
*/
grant all privileges on *.* to usuario_master with grant
option;

/* concede o privilégio de executar a procedure 'atualiza_
preco' do banco <mercado> ao usuário <u_manutencao' */
grant execute on mercado.atualiza_preco to u_manutencao;
```

Algumas observações importantes sobre o bloco de código anterior:

- Para se especificar todas as tabelas de um determinado banco de dados, utilizamos a estrutura: “NOME\_DO\_BANCO.\*”;
- Para definição de todos os bancos e suas tabelas, utilizamos o símbolo “\*.\*”;
- A opção “with grant option” define se o usuário mencionado terá a capacidade de replicar concessões para outros usuários;
- Para se executar *procedures*, utilizamos o **grant execute**.

## revoke – Revogação de privilégios

O comando **revoke** tem função contrária ao **grant**, ou seja, é utilizado para remover concessões de privilégio. Vejamos no bloco de código a seguir alguns exemplos de uso.

```
-- EXEMPLOS:

/* remove apenas acesso de consulta ao 'usuario_observador'
em todas as tabelas do banco de dados 'rede_social' */
revoke select on rede_social.* to usuario_observador;

/* remove acesso de consulta, atualização e inserção ao 'u_
corretora', na tabela de apólices do banco corretora */
revoke select, update, insert on corretora.apolices to
u_corretora;

/* remove todos os privilégios ao 'usuario_master' em todas
as tabelas dos bancos, com possibilidade de replicação de
privilégios para outros usuários
*/
revoke all privileges on *.* to usuario_master cascade;
```

```
/* remove o privilégio de executar a procedure 'atualiza_
preco' do banco 'mercado' ao usuário 'u_manutencao' */
revoke execute on mercado.atualiza_preco to u_manutencao;
```

Algumas observações importantes sobre o bloco de código anterior:

- Para especificar todas as tabelas de um determinado banco de dados, utilizamos a estrutura: “NOME\_DO\_BANCO.\*”;
- Para definição de todos os bancos e suas tabelas, utilizamos o símbolo “\*.\*”;
- A opção “cascade” define se a operação de remoção do privilégio será em cascata para removê-lo também dos usuários criados a partir dele;
- Para remover a execução de *procedures*, utilizamos o **revoke execute**.

## show grants – Exibindo os privilégios

O comando **show grants** nos permite visualizar os privilégios do usuário atualmente conectado. Na **Figura 5.18**, observe um exemplo de uso.

41 • **show grants;**

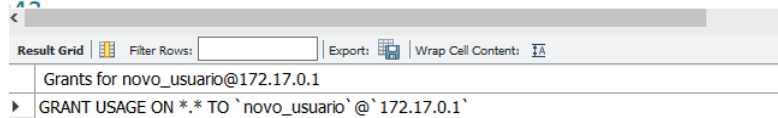
| Grants for root@% |                                                                                                                                        |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| ▶                 | GRANT SELECT, INSERT, UPDATE, DELETE, CREATE, DROP, RELOAD, SHUTDOWN, PROCESS, FILE, REFERENCES, INDEX, ALTER, SHOW DATABASES, SUPE... |
| ▶                 | GRANT ALLOW_NONEXISTENT_DEFINER, APPLICATION_PASSWORD_ADMIN, AUDIT_ABORT_EXEMPT, AUDIT_ADMIN, AUTHENTICATION_POLICY_ADMIN, BACK...     |

**Figura 5.18** – Demonstração do comando **show grants** para exibição dos privilégios do usuário atualmente conectado.

Crédito: do elaborador, 2025.

É também de grande utilidade que o(a) aluno(a) conheça a possibilidade de visualizar os privilégios de outros usuários por meio do comando **show grants for**, conforme **Figura 5.19**. Contudo, vale ressaltar que, para executá-lo, o usuário atual deve ter privilégios administrativos.

```
42 • show grants for 'novo_usuario'@'172.17.0.1';
```



| Result Grid                                       | Filter Rows: | Export: | Wrap Cell Content: |
|---------------------------------------------------|--------------|---------|--------------------|
| Grants for novo_usuario@172.17.0.1                |              |         |                    |
| GRANT USAGE ON *.* TO `novo_usuario`@`172.17.0.1` |              |         |                    |

**Figura 5.19** – Demonstração do comando **show grants for** para exibição dos privilégios de outros usuários.

Crédito: do elaborador, 2025.

## Exemplos e casos de uso – Indústria automotiva

Por ser pertinente ao assunto atual, retomaremos o mesmo contexto abordado no tópico anterior, **Exemplos e casos de uso – Indústria automotiva**, sobre a criação de usuários. Nele, pudemos trabalhar em uma situação-problema envolvendo uma indústria/montadora automotiva, em que criamos usuários para resolver a necessidade de segurança e sigilo de novos projetos.

Na etapa anterior dessa situação-problema, criamos diversos usuários. Agora, no entanto, focaremos em atribuir as devidas permissões para cada um, seguindo os critérios a seguir:

- O usuário chamado “usuario\_eletrica” terá acessos de consulta, atualização, inserção e execução em todas as tabelas do banco de dados “projetos\_eletricos”;
- O usuário chamado “usuario\_mecanica” terá acessos de consulta, atualização, inserção e execução em todas as tabelas do banco de dados “projetos\_mecanicos”;
- O usuário chamado “usuario\_design” terá acessos de consulta, atualização e inserção em todas as tabelas do banco de dados “projetos\_design”;
- O usuário chamado “usuario\_testes” terá acessos de consulta, atualização e inserção em todas as tabelas do banco de dados “homologacao”;

- O usuário chamado “usuario\_gestao” terá acesso máximo em todos os bancos.

No bloco de código a seguir, aplicaremos a concessão de acessos para cada usuário, seguindo as regras estabelecidas (caso você não os tenha, precisará criá-los antes).

```
grant select, update, insert, execute on projetos_eletricos.*
to 'usuario_eletrica'@'172.17.0.1';

grant select, update, insert, execute on projetos_mecanicos.*
to 'usuario_mecanica'@'172.17.0.1';

grant select, update, insert on projetos_design.* to
'usuario_design'@'172.17.0.1';

grant select, update, insert on homologacao.* to
'usuario_testes'@'172.17.0.1';

grant all privileges on *.* to 'usuario_gestao'@'172.17.0.1'
with grant option;
```

## IMPORTANTE

### localhost

No bloco de código anterior, utilizamos o IP “172.17.0.1” por se tratar do endereço onde o servidor está sendo utilizado no *container docker*. Contudo, a menos que o(a) aluno(a) tenha alterado intencionalmente, seu endereço será o “localhost”.

Agora, faremos a conexão com o primeiro usuário, conforme **Figura 5.20**.

```
C:\Users\andre>mysql -h 127.0.0.1 -P 3307 -u usuario_eletrica -p
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 41
Server version: 9.0.0 MySQL Community Server - GPL

Copyright (c) 2000, 2024, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
```

**Figura 5.20** – Teste de conexão com o usuário criado.

Crédito: do elaborador, 2025.

Em seguida, faremos a exibição dos bancos em que esse usuário possui acesso, conforme **Figura 5.21**.

```
mysql> show databases;
+-----+
| Database |
+-----+
| information_schema |
| performance_schema |
| projetos_eletricos |
+-----+
3 rows in set (0.17 sec)
```

**Figura 5.21** – Comando para exibição dos bancos em que o “usuario\_eletrica” possui acesso.

Crédito: do elaborador, 2025.

Note que, para o “usuario\_eletrica”, está disponível apenas o banco de dados que foi definido durante o processo de concessão de privilégio.

Agora, verificaremos se o usuário está capacitado a realizar alguma das operações autorizadas (**select**, **update**, **insert** ou **execute**), conforme **Figura 5.22**.

```
mysql> insert into componentes (nome,valor) values ('Módulode Injeção XTZ9822',5300.00);
Query OK, 1 row affected (0.02 sec)

mysql> select *from componentes;
+-----+-----+-----+
| id | nome | valor |
+-----+-----+-----+
| 1 | Módulo de Injeção XTZ9822 | 5300.00 |
+-----+-----+-----+
1 row in set (0.00 sec)

mysql> update componentes set nome = 'Módulo de Injeção ABC123' where id = 1;
Query OK, 1 row affected (0.02 sec)
Rows matched: 1 Changed: 1 Warnings: 0
```

**Figura 5.22** – Teste dos comandos autorizados para o “usuario\_eletrica”.

Crédito: do elaborador, 2025.

Podemos constatar que, de acordo com a **Figura 5.22**, esse usuário tem autorização para a execução das operações que foram definidas com o **grant**.

Agora, provocaremos um erro tentando executar uma operação não autorizada, isto é, um **delete** na tabela de “componentes” pertencente ao banco de dados “projetos\_eletricos”, conforme **Figura 5.23**. Se o erro ocorrer devido a restrições de acesso, o funcionamento está seguindo de acordo com o planejado.

```
mysql> delete from componentes where id = 1;
ERROR 1142 (42000): DELETE command denied to user 'usuario_eletrica'@'172.17.0.1' for table 'componentes'
mysql>
```

**Figura 5.23** – Teste dos comandos não autorizados para o “usuario\_eletrica”.

Crédito: do elaborador, 2025.

Como você pode perceber, esse usuário não foi autorizado a fazer uma operação de deleção, comprovando o correto funcionamento durante a concessão dos privilégios com o comando **grant**.

Agora, faremos a conexão com o “usuario\_gestao”, conforme **Figura 5.24**, e tentaremos executar a mesma operação de deleção que foi negada anteriormente, sendo esperado que esse comando seja executado sem problemas, uma vez que esse usuário é autorizado a esse tipo de operação.

```
C:\Users\andre>mysql -h 127.0.0.1 -P 3307 -u usuario_gestao -p
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 46
Server version: 9.0.0 MySQL Community Server - GPL

Copyright (c) 2000, 2024, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> use projetos_eletricos;
Database changed
mysql> delete from componentes where id = 1;
Query OK, 1 row affected (0.02 sec)
```

**Figura 5.24** – Teste do comando de deleção para o “usuario\_gestao”.

Crédito: do elaborador, 2025.

Conforme esperado, conseguimos constatar na **Figura 5.24**, que o comando **delete** foi executado com sucesso, uma vez que fizemos a conexão

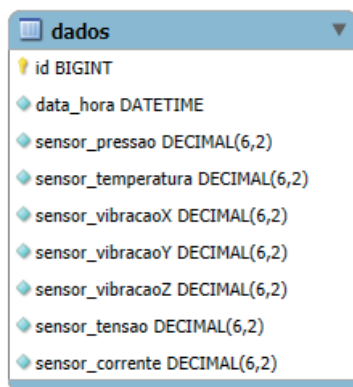


utilizando um usuário com nível de acesso correspondente, evidenciando, assim, a importância da criação de usuários e da definição de seus privilégios.

## Exercícios – *Machine Learning*

Um professor está ensinando *Machine Learning* (Aprendizagem de Máquina) para a turma de um curso de Inteligência Artificial. Para que seus alunos possam realizar testes com seus algoritmos, foram criadas tabelas contendo uma grande quantidade de dados, retirados de uma máquina disponível no laboratório da instituição.

Nessa tabela, são registradas as medições dos sensores dessa máquina para cada peça produzida. O objetivo é detectar as variações ocorridas durante o processo produtivo e, assim, potencializar a predição de falhas.



**Figura 5.25** – Diagrama UML da tabela do banco “machine\_learning”.

Crédito: do elaborador, 2025.

Para possibilitar que os alunos utilizem essa tabela com segurança, você deve ajudar na criação de três tipos de usuários do banco de dados, conforme definição a seguir:

- Um usuário denominado “alunos”, com a capacidade de realizar apenas leitura dos dados da tabela (evitando inserções ou remoções de registros de forma incorreta);
- Um usuário denominado “maquina”, com a capacidade de realizar a inserção de dados;

- Um usuário denominado “professor”, com todos os privilégios existentes.

Faça a criação dos usuários conforme o requisito e realize testes para verificar se os acessos estão de acordo com o necessário.

## TCL – Linguagem de Controle de Transações

A Linguagem de Controle de Transações – TCL (*Transaction Control Language*) é uma categoria de comandos focados no gerenciamento das transações efetuadas pelo banco de dados. Transações, por sua vez, é como chamamos o processo de execução de um comando do início ao fim.

Por exemplo, quando executamos um comando de inserção de um registro em uma tabela, uma transação é então iniciada, com um ciclo de vida, isto é, com começo, meio e fim, cujo objetivo é garantir a **unicidade** do comando executado sem afetar a **integridade** da base de dados existente.

Embora essas duas palavras destacadas possam inicialmente parecer complexas, seu efeito é mais fácil de se entender do que parece. A unicidade de uma transação é a garantia de que será executada por completo apenas uma única vez, afinal, ao inserir uma ordem de venda em uma tabela, você com certeza não deseja que esse registro fique duplicado. Já quanto à integridade, uma transação deve respeitar as restrições (*constraints*) definidas durante a criação de uma tabela, o que também é bastante compreensível.

### Transações – Conceito

Acabamos de definir no tópico anterior alguns pontos importantes sobre transações. Você pôde entender que todos os comandos utilizados no decorrer deste livro acabaram por originar transações que, em seu ciclo de vida,

cuidaram que as operações de inserção, consulta, deleção e outras pudessem ser executadas sem causar inconsistências no banco de dados. Podemos considerar as transações como unidades atômicas de trabalho, ou seja, não podem ser quebradas ou divididas, com a garantia de que possuem começo, meio e fim.

### Entendendo transações na prática

Para exemplificar mais claramente o efeito de uma transação, vamos recorrer a um cenário simples, mas eficiente para entender o tema abordado. Imagine que possuímos uma tabela de cidades, em que há uma *constraint* do tipo chave primária composta para garantir que não se repita o mesmo nome de cidade para o mesmo estado.

```
create table cidades (  
  cidade varchar(200) not null,  
  estado char(2) not null,  
  numero_habitantes bigint not null,  
  primary key(cidade,estado)  
) engine=InnoDB;
```

### IMPORTANTE

#### InnoDB

Usamos no bloco de código anterior a “engine=InnoDB” para declaradamente definirmos que a tabela “cidades” deve usar o mecanismo de armazenamento chamado InnoDB, pois ele dá suporte às transações. Na realidade, não havíamos utilizado de forma explícita essa definição antes, pois, normalmente, o próprio MySQL já define o uso desse mecanismo como padrão. Além disso, o uso ou não de suporte às transações nos exercícios anteriores era indiferente. Agora, no entanto, definiremos explicitamente o uso dessa *engine* para garantir o comportamento das transações conforme o esperado.

Agora, considere que precisamos executar a inserção de algumas cidades, conforme o bloco de código a seguir.

```
insert into cidades (cidade,estado,numero_habitantes) values
('Campos do Jordão','SP',46974), ('Pedro Leopoldo','MG',62580),
('Santa Bárbara d'Oeste','SP',183347), ('Paranavaí','PR',91950),
('Campos do Jordão','SP',46974);
```

Preste atenção: caso o(a) aluno(a) não tenha notado, a expectativa é que esse comando ocasione um erro de execução, uma vez que estamos infringindo a restrição de unicidade entre a cidade e o estado (o nome “Campos do Jordão”, “SP” está em duplicidade). Fato é que, nesse caso, o erro não deve gerar nenhuma surpresa, não é mesmo?

Error Code: 1062. Duplicate entry 'Campos do Jordão-SP' for key 'cidades.PRIMARY'

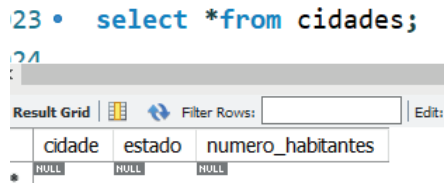
**Figura 5.26** – Erro gerado durante a inserção de chave primária duplicada na tabela de cidades.

Crédito: do elaborador, 2025.

Entretanto, fica o seguinte questionamento ao(à) aluno(a): sendo o item duplicado apenas o último elemento entre os cinco existentes dentro do comando **insert** que foi dado, em sua opinião, **o servidor incluiu os outros quatro e apenas acusou o erro no último? Ou você acredita que, devido ao erro, nada foi feito?**

```
insert into cidades (cidade,estado,numero_habitantes) values
('Campos do Jordão','SP',46974), ('Pedro Leopoldo','MG',62580),
('Santa Bárbara d'Oeste','SP',183347), ('Paranavaí','PR',91950),
('Campos do Jordão','SP',46974);
```

Para respondermos ao questionamento levantado, faremos simplesmente uma consulta nessa tabela, na qual poderemos verificar se os quatro elementos duplicados foram ou não adicionados, conforme **Figura 5.27**.



**Figura 5.27** – Consulta na tabela de cidades.

Crédito: do elaborador, 2025.

Observe que o comportamento adotado pelo servidor MySQL foi o de cancelar a operação inteira de inserção, não adicionando nenhum dos elementos presentes no comando **insert**. Esse comportamento é a comprovação de que, mesmo havendo múltiplos elementos, o comando **insert** que utilizamos é considerado uma única transação e, conceitualmente, uma transação deve ser inteiramente válida ou então não é executada. É por esse motivo que até mesmo os itens não duplicados não foram adicionados, sendo essa uma ação comportamental da transação para garantir a consistência do banco de dados, ignorando um comando problemático.

Antes de concluirmos essa explicação, demonstraremos que, ao optarmos por adicionar os valores em comandos **insert** separados, conforme o bloco de código a seguir, o comportamento será diferente.

```
insert into cidades (cidade,estado,numero_habitantes) values
('Campos do Jordão','SP',46974);

insert into cidades (cidade,estado,numero_habitantes) values
('Pedro Leopoldo','MG',62580);

insert into cidades (cidade,estado,numero_habitantes) values
('Santa Bárbara d'Oeste','SP',183347);

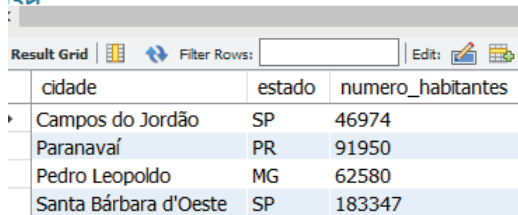
insert into cidades (cidade,estado,numero_habitantes) values
('Paranavaí','PR',91950);

insert into cidades (cidade,estado,numero_habitantes) values
('Campos do Jordão','SP',46974);
```

Agora, checaremos novamente o conteúdo da tabela, conforme **Figura 5.28**.

29 • `select *from cidades;`

30



| cidade                | estado | numero_habitantes |
|-----------------------|--------|-------------------|
| Campos do Jordão      | SP     | 46974             |
| Paranavaí             | PR     | 91950             |
| Pedro Leopoldo        | MG     | 62580             |
| Santa Bárbara d'Oeste | SP     | 183347            |

**Figura 5.28** – Consulta na tabela de cidades.

Crédito: do elaborador, 2025.

Note que, agora, todos os dados anteriores à duplicidade foram incluídos. Isso ocorre porque, ao separarmos os comandos **insert**, o servidor considerará cada um deles como uma transação separada, desconsiderando apenas a que contém o “problema”.

### ACID – Conceito

Agora que você entendeu melhor o comportamento das transações, é importante que conheça também um outro conceito que as envolve.

Já dissemos anteriormente que as transações garantem a integridade e a confiabilidade dos bancos de dados, contudo, podemos apenas afirmar que uma transação atende a esses objetivos se cumprir com quatro critérios fundamentais, denominados ACID.

ACID corresponde às iniciais de:

- **Atomicidade (*Atomicity*)**: a transação deve ser finalizada por inteiro ou deve ser completamente cancelada, associando-se ao conceito atômico como uma unidade mínima indivisível;
- **Consistência (*Consistency*)**: nenhuma transação, por mais importante ou complexa que seja, deve infringir as regras e restrições que impactam a consistência do banco de dados;
- **Isolamento (*Isolation*)**: uma transação não deve influenciar ou ser influenciada por outra;

- **Durabilidade (*Durability*):** os dados devem ser armazenados permanentemente após a transação ser completada.

É por meio dos requisitos mencionados que obtemos a garantia que uma transação será válida, possibilitando que até em contextos mais complexos, com milhares de transações executadas por minuto, a estrutura do banco de dados se mantenha consistente e intacta.

## commit – Confirmando uma transação

Nesse momento, você já compreendeu que o MySQL utiliza as transações como unidade de trabalho para executar cada operação com segurança, de forma automática.

Contudo, existem situações em que você deseja mudar o comportamento do MySQL para substituir essas transações que ocorrem automaticamente para cada comando por uma única transação que envolva um conjunto deles, de forma a apenas efetivá-la se todos esses comandos atingirem o propósito desejado.

É para esses casos que o MySQL nos oferece a possibilidade de criarmos nossa própria transação e definirmos em qual momento ela será confirmada. E é nesse contexto que utilizamos o comando **commit**, responsável por indicar que a transação deverá ser finalizada e as ações geradas por ela deverão ser permanentemente salvas.

Observe no bloco de código a seguir a estrutura de uso nesse contexto.

```
/* ESTRUTURA DO COMANDO: */  
-- inicia a transação:  
start transaction;  
  
-- COMANDOS ENVOLVIDOS NA MESMA TRANSAÇÃO...  
  
-- fim da transação, salvando permanentemente as operações  
realizadas;
```

```

commit;

/* EXEMPLO: */
-- registra uma venda, os itens vendidos e o status do
pagamento,
-- dentro de uma mesma transação
start transaction;
insert into vendas (id,usuario_FK) values ('348a12t1', 552);

insert into venda_item (venda_FK, produto_FK, quantidade)
values
('348a12t1', 887, 2), ('348a12t1', 960, 10), ('348a12t1', 541,
8);

insert into pagamentos (venda_FK, status) values
('348a12t1','PENDENTE');
commit;

```

Em suma, o uso dessas transações é relativamente simples, basta iniciarmos a transação, realizarmos as operações desejadas e, quando precisarmos efetuar a confirmação das operações, inserirmos o comando **commit**.

## IMPORTANTE

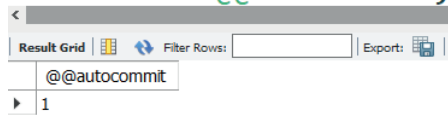
### **commit automático**

Embora seu uso seja simples, existe aqui um detalhe fundamental: precisamos garantir que seu banco não esteja configurado para fazer o **commit** automático para cada comando (normalmente, isso vem como padrão no MySQL), caso contrário, cada instrução executada já será permanentemente salva. Em outros modelos de bancos de dados, como no Oracle, é mais comum essa configuração estar desativada.

Para checarmos se nosso banco está com o **commit** automático, basta consultarmos o comando da **Figura 5.29**.



```
19 • select @@autocommit;
```



The screenshot shows a SQL query result in a table with one column labeled '@@autocommit' and one row containing the value '1'. The interface includes a 'Result Grid' button, a 'Filter Rows' input field, and an 'Export' button.

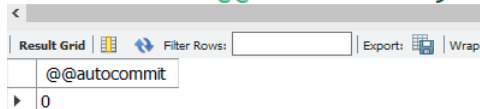
**Figura 5.29** – Consulta da configuração de **commit** automático.

Crédito: do elaborador, 2025.

Ao executar o comando da **Figura 5.29**, se seu resultado for **1**, então seu banco irá finalizar a transação após cada instrução individual, o que com certeza não atenderá aos casos em que se deseja criar sua própria transação e poder finalizá-la conforme sua necessidade.

Para isso, faremos agora a mudança desse comportamento, executando o comando da **Figura 5.30**. Na sequência, consultaremos novamente o “autocommit”.

```
20 • set @@autocommit = 0;
21 • select @@autocommit;
```



The screenshot shows a SQL query result in a table with one column labeled '@@autocommit' and one row containing the value '0'. The interface includes a 'Result Grid' button, a 'Filter Rows' input field, and an 'Export' button.

**Figura 5.30** – Alteração da configuração de **commit** automático.

Crédito: do elaborador, 2025.

Agora você será capaz de utilizar as transações corretamente, contudo, é importante lembrá-lo(a) que, dessa forma, mesmo após a conclusão de sua transação personalizada, cada comando que você utilizar não será salvo permanentemente, a menos que você utilize o **commit** ou volte à configuração inicial com “set @@autocommit = 1”.

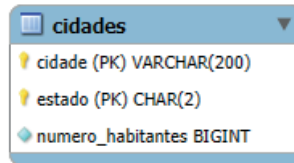
Entendendo as transações não confirmadas

Nesse momento, você já deve ter compreendido que as operações dentro de transações que não receberam o comando **commit** ainda não estão permanentemente definitivas no banco de dados.

Contudo, o que talvez você ainda não saiba – e que com certeza pode gerar dúvidas – é que as operações dessas transações, embora ainda não impermanentes, aparecem na sessão do usuário que as criou como se estivessem definitivas. Já para outras sessões, essas operações estarão invisíveis.

Nada melhor do que testarmos isso na prática, não é mesmo?

Na **Figura 5.31**, utilizaremos como exemplificação uma tabela de “cidades”, devido a sua simplicidade, já que o foco aqui é entendermos de que forma os registros não transacionados ficam visíveis de acordo com a sessão.



**Figura 5.31** – Diagrama UML de uma tabela de cidades.

Crédito: do elaborador, 2025.

Agora, na **Figura 5.32**, desabilitaremos o “autocommit” e, em seguida, consultaremos as cidades atualmente existentes no banco.

```

.63 • set @@autocommit = 0;
.64 • select * from cidades;
.65

```

| cidade                | estado | numero_habitantes |
|-----------------------|--------|-------------------|
| Campos do Jordão      | SP     | 46974             |
| Paranavaí             | PR     | 91950             |
| Pedro Leopoldo        | MG     | 62580             |
| Santa Bárbara d'Oeste | SP     | 183347            |

**Figura 5.32** – Configuração de “autocommit” e consulta de cidades.

Crédito: do elaborador, 2025.

Finalmente, iniciaremos uma transação e, dentro dela, faremos a inserção do registro de uma nova cidade, **sem executar**, no entanto, sua confirmação com o **commit**.

```

start transaction;

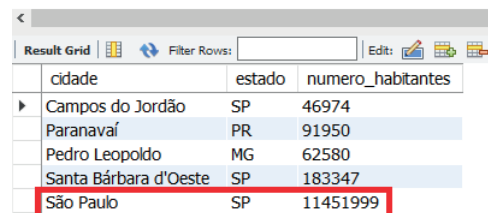
insert into cidades (cidade,estado,numero_habitantes)
values ('São Paulo','SP',11451999);
  
```

Nesse momento, levamos ao(à) aluno(a) um importante questionamento: essa inserção realizada dentro de uma transação ainda não finalizada poderá ser visualizada antes de ser executado o comando **commit**? De uma

maneira lógica, você provavelmente é levado(a) a crer que, uma vez que essa inserção ainda não foi confirmada, ela não estará visível de forma nenhuma, não é mesmo? E para responder a esse questionamento, vejamos o próximo passo deste estudo.

Nada melhor do que checarmos na prática se a cidade de “São Paulo”, adicionada dentro de uma transação não finalizada, estará ou não disponível para visualização por meio de um comando **select**.

**70 • select \* from cidades;**



| cidade                | estado | numero_habitantes |
|-----------------------|--------|-------------------|
| Campos do Jordão      | SP     | 46974             |
| Paranavaí             | PR     | 91950             |
| Pedro Leopoldo        | MG     | 62580             |
| Santa Bárbara d'Oeste | SP     | 183347            |
| São Paulo             | SP     | 11451999          |

**Figura 5.33** – Consulta de todos os dados de uma entidade com transações pendentes.

Crédito: do elaborador, 2025.

## IMPORTANTE

### Visualização de transações não confirmadas

O ponto-chave que queremos mostrar ao(à) aluno(a) com esse exemplo é que mesmo as transações ainda não confirmadas podem ser visualizadas como se estivessem confirmadas apenas **para a sessão do usuário do banco de dados que as criou**. Já para os outros usuários conectados no banco de dados, esses dados ainda não poderão ser visualizados, uma vez que não receberam o **commit**.

Outro ponto importante é que esses registros não transacionados ficam visíveis ao usuário que os criou desde que se mantenha a sessão de **login**, o que quer dizer que, caso ocorra a desconexão do servidor e posterior reconexão usando o mesmo usuário, esses dados não estarão mais visíveis.

Para confirmarmos se realmente a cidade de “São Paulo” ainda não está visível para todos no banco, abriremos um novo terminal do sistema operacional (sem fechar ou se desconectar do MySQL que criou a transação,

pois daremos continuidade nos comandos posteriormente). Nesse novo terminal, podemos fazer o *login* com o mesmo usuário ou com outro existente em seu MySQL, já que ambos os casos consistem em uma nova sessão (optaremos pelo uso do mesmo usuário).

```
C:\Users\andre>mysql -h 127.0.0.1 -u root -p
Enter password: ****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 21
Server version: 9.0.0 MySQL Community Server - GPL
```

**Figura 5.34** – Nova sessão de conexão com o servidor MySQL, via terminal de comando.

Crédito: do elaborador, 2025.

## FIQUE ALERTA

### Terminal de comando ou MySQL Workbench

Utilizamos aqui o terminal de comando apenas por facilidade, porém, caso seja a preferência do(a) aluno(a), pode-se optar por abrir uma nova conexão com o MySQL Workbench, desde que não se remova a atual. Além disso, alertamos o(a) aluno(a) a seguir o endereço de IP e o usuário que já utiliza em seu laboratório ou computador pessoal.

Em seguida, faremos a utilização do banco e consultaremos a tabela de cidades.

```
mysql> use censo;
Database changed
mysql> select *from cidades;
+-----+-----+-----+
| cidade          | estado | numero_habitantes |
+-----+-----+-----+
| Campos do Jordão | SP     | 46974              |
| Paranavaí       | PR     | 91950              |
| Pedro Leopoldo  | MG     | 62580              |
| Santa Bárbara d'Oeste | SP    | 183347             |
+-----+-----+-----+
4 rows in set (0.00 sec)
```

**Figura 5.35** – Consulta da tabela de cidades por meio de uma nova sessão de conexão com o servidor MySQL.

Crédito: do elaborador, 2025.

Observe que, de acordo com a **Figura 5.35**, temos a importante comprovação de que, de fato, a cidade de “São Paulo”, adicionada em uma

operação não transacionada, torna-se visível apenas na sessão em que ela foi criada, uma vez que seu estado é temporário.

Agora, para encerrarmos o ciclo de nosso estudo sobre esse assunto, voltaremos aos comandos de nossa transação e faremos sua finalização por meio do comando **commit**, conforme se vê no bloco de código a seguir.

```
start transaction;

insert into cidades (cidade,estado,numero_habitantes)
values ('São Paulo','SP',11451999);

commit;
```

Finalmente, faremos agora uma nova consulta à tabela de cidades, utilizando prioritariamente o terminal de comando que antes não exibia o registro da cidade de “São Paulo”, para constatarmos a eficácia do comando **commit**.

```
mysql> select *from cidades;
+-----+-----+-----+
| cidade | estado | numero_habitantes |
+-----+-----+-----+
| Campos do Jordão | SP | 46974 |
| Paranavaí | PR | 91950 |
| Pedro Leopoldo | MG | 62580 |
| Santa Bárbara d'Oeste | SP | 183347 |
| São Paulo | SP | 11451999 |
+-----+-----+-----+
5 rows in set (0.00 sec)
```

**Figura 5.36** – Consulta da tabela de cidades por meio de uma nova sessão de conexão com o servidor MySQL, após o comando **commit**.

Crédito: do elaborador, 2025.

Sem dúvidas, foram passos trabalhosos, mas de elevada importância para que o aluno entenda a dinâmica das transações e como elas ficam visíveis quando não confirmadas.

## rollback – Cancelando uma transação

Nesse momento, você já compreendeu como é possível utilizar as transações do MySQL e confirmá-las com o uso do comando **commit**.

Agora apresentaremos a você o comando **rollback**, utilizado para o cancelamento das transações ainda não confirmadas.

Sua utilização é bem simples, conforme podemos ver no bloco de código a seguir.

```
/* ESTRUTURA DO COMANDO: */
-- inicia a transação:
start transaction;

-- COMANDOS ENVOLVIDOS NA MESMA TRANSAÇÃO...

-- cancelamento da transação, removendo as operações
realizadas:
commit;

/* EXEMPLO: */
-- registra uma venda e seus itens vendidos, porém cancela a
operação
start transaction;
insert into vendas (id,usuario_FK) values ('348a12t1', 552);

insert into venda_item (venda_FK, produto_FK, quantidade)
values
('348a12t1', 887, 2), ('348a12t1', 960, 10), ('348a12t1', 541,
8);

rollback;
```

## IMPORTANTE

### Utilização do rollback

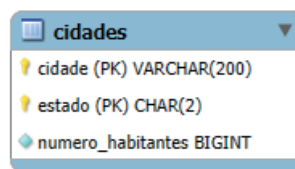
Geralmente, a utilização do comando **rollback** não é realizada de forma intencional para que ele seja sempre executado como um caminho “natural”. Nesses casos, é mais comum que ele seja utilizado com o objetivo de

**desfazer tudo o que foi feito, caso algum erro ocorra ou exista alguma regra de negócio em específico (um usuário solicitando na aplicação o cancelamento de uma operação, por exemplo). Em algumas linguagens de programação para *backend*, o *rollback* é utilizado justamente em certas operações que interagem com o banco de dados durante o tratamento de exceções.**

Observando o funcionamento do *rollback*

Da mesma forma como fizemos com o **commit**, usaremos uma tabela de “cidades”, simples o suficiente para exemplificar a ação do **rollback** durante o cancelamento das operações dentro de uma transação.

Usaremos em nosso exemplo a tabela responsável por armazenar cidades e a quantidade de seus respectivos habitantes, conforme o diagrama UML da **Figura 5.37**.



**Figura 5.37** – Diagrama UML de uma tabela de cidades.

Crédito: do elaborador, 2025.

Agora, desabilitaremos o “autocommit” para garantir que o servidor MySQL não confirme automaticamente as operações executadas. Em seguida, consultaremos as cidades atualmente existentes no banco.

```

1 • set @@autocommit = 0;
2 • select * from cidades;
  
```

| cidade                | estado | numero_habitantes |
|-----------------------|--------|-------------------|
| Campos do Jordão      | SP     | 46974             |
| Paranavai             | PR     | 91950             |
| Pedro Leopoldo        | MG     | 62580             |
| Santa Bárbara d'Oeste | SP     | 183347            |
| São Paulo             | SP     | 11451999          |

**Figura 5.38** – Configuração de autocommit e consulta de cidades.

Crédito: do elaborador, 2025.

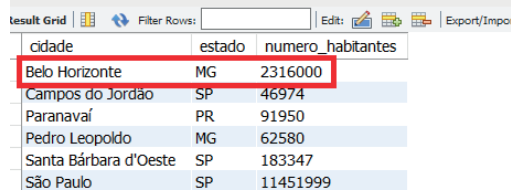
Finalmente, iniciaremos uma transação e, dentro dela, faremos a inserção do registro de uma nova cidade, **sem executar**, no entanto, sua confirmação com o **commit** ou o cancelamento com o **rollback**.

```
start transaction;

insert into cidades (cidade,estado,numero_habitantes)
values ('Belo Horizonte','MG',2316000);
```

Mantendo-se a sessão usada para inserir o registro anterior, consultaremos para checar se a nova cidade foi incluída devidamente na tabela (embora essa inclusão esteja apenas visível para a sessão do usuário atual no servidor MySQL, conforme explicamos detalhadamente no tópico anterior).

2. **select \* from cidades;**



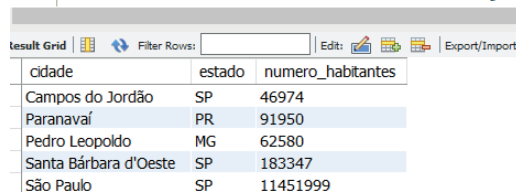
| cidade                | estado | numero_habitantes |
|-----------------------|--------|-------------------|
| Belo Horizonte        | MG     | 2316000           |
| Campos do Jordão      | SP     | 46974             |
| Paranavaí             | PR     | 91950             |
| Pedro Leopoldo        | MG     | 62580             |
| Santa Bárbara d'Oeste | SP     | 183347            |
| São Paulo             | SP     | 11451999          |

**Figura 5.39** – Consulta de todos os dados de uma entidade com transações pendentes.

Crédito: do elaborador, 2025.

Faremos agora a utilização do comando **rollback** para cancelarmos a operação de inserção que está pendente. E, por fim, checaremos se a transação foi realmente retornada ao estado original.

3. **rollback;**  
4. **select \* from cidades;**



| cidade                | estado | numero_habitantes |
|-----------------------|--------|-------------------|
| Campos do Jordão      | SP     | 46974             |
| Paranavaí             | PR     | 91950             |
| Pedro Leopoldo        | MG     | 62580             |
| Santa Bárbara d'Oeste | SP     | 183347            |
| São Paulo             | SP     | 11451999          |

**Figura 5.40** – Consulta de todos os dados de uma entidade ao cancelar a transação pendente.

Crédito: do elaborador, 2025.



Como você pode observar, a cidade “Belo Horizonte”, incluída na transação pendente, agora não é mais exibida, já que estornamos a operação ao utilizar o **rollback**.

## savepoint – Ponto de salvamento de uma transação

Durante nossa jornada rumo ao entendimento dos comandos TCL, já abordamos a importância das transações e como confirmá-las ou estorná-las por meio do uso de **commit** e **rollback**. Entretanto, o fato é que o comportamento apresentado com o uso desses comandos faz com que as transações sejam 100% ou 0% executadas, ou seja, com um efeito de “tudo ou nada”.

Contudo, nem sempre as aplicações devem se comportar dessa maneira, havendo situações em que, perante um erro, uma exceção, uma falha de conexão ou outra situação inesperada, deve-se recuperar parte das operações transações. É nesse contexto que podemos recorrer ao comando **savepoint**.

O **savepoint** é um recurso disponível dentro do repertório de comandos para controle de transações, em que, fazendo jus à sua tradução literal, permite criar um ponto de salvamento das operações dentro de uma transação.

Para que fique um pouco mais associativo, imagine o **savepoint** como um ponto de *backup*, sendo inclusive essa uma das situações em que pode ser utilizado, permitindo o retorno do banco de dados a um determinado estado no momento em que foi criado, dentro do contexto de sua transação, é claro.

```
/* ESTRUTURA DO COMANDO: */
-- inicia a transação:
start transaction;

-- CONJUNTO 1 DE COMANDOS ENVOLVIDOS NA MESMA TRANSAÇÃO...

savepoint CONJUNTO_1;

-- CONJUNTO 2 DE COMANDOS ENVOLVIDOS NA MESMA TRANSAÇÃO...
```

```

-- estorno das transações após o 'CONJUNTO_1':
rollback to CONJUNTO_1;

/* EXEMPLO: */
-- registra uma venda e seus itens vendidos
start transaction;
insert into vendas (id,usuario_FK) values ('348a12t1', 552);
insert into venda_item (venda_FK, produto_FK, quantidade)
values
('348a12t1', 887, 2), ('348a12t1', 960, 10), ('348a12t1', 541,
8);

savepoint venda_registrada;

/* outras operações ...*/

/* situação de tratamento de exceção (retorno ao estado de
venda registrada) */

rollback to venda_registrada;
commit;

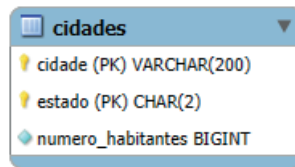
```

Note que, de acordo com o bloco de código anterior, para criarmos um ponto de salvamento em uma transação, basta utilizarmos o comando **savepoint**, informando o nome que desejamos atribuir a esse ponto. Além disso, percebemos também que, para resgatar o ponto salvo, utilizamos o comando **rollback to**, informando o nome utilizado anteriormente com o **savepoint**.

Observando o funcionamento do savepoint

Para nos concentrarmos de fato no funcionamento do **savepoint**, usaremos o mesmo contexto utilizado para exemplificar os comandos **commit** e **rollback**, envolvendo uma tabela responsável por armazenar algumas cidades e seus respectivos habitantes.

Observe o diagrama UML da **Figura 5.41**, em que podemos identificar a presença de uma tabela chamada “cidades”.



**Figura 5.41** – Diagrama UML de uma tabela de cidades.

Crédito: do elaborador, 2025.

Agora, desabilitaremos o “autocommit”, para garantir que o servidor MySQL não confirme automaticamente as operações executadas. Em seguida, consultaremos as cidades atualmente existentes no banco.

```

1 • set @@autocommit = 0;
2 • select * from cidades;
  
```

| cidade                | estado | numero_habitantes |
|-----------------------|--------|-------------------|
| Campos do Jordão      | SP     | 46974             |
| Paranavaí             | PR     | 91950             |
| Pedro Leopoldo        | MG     | 62580             |
| Santa Bárbara d'Oeste | SP     | 183347            |
| São Paulo             | SP     | 11451999          |

**Figura 5.42** – Configuração de autocommit e consulta de cidades.

Crédito: do elaborador, 2025.

Iniciaremos uma transação e, dentro dela, faremos a inserção de registros das capitais dos estados do sul do Brasil, criando a seguir um ponto de salvamento. Em seguida, faremos a inserção de uma outra cidade após o ponto de salvamento.

```

start transaction;

insert into cidades (cidade,estado,numero_habitantes)
values ('Curitiba','PR',1774000);

insert into cidades (cidade,estado,numero_habitantes)
values ('Florianópolis','SC',576000);

insert into cidades (cidade,estado,numero_habitantes)
  
```

```

values ('Porto Alegre','RS',1333000);

savepoint capitais_sul;

insert into cidades (cidade,estado,numero_habitantes)
values ('Campinas','SP',1139000);

```

Mantendo-se a sessão usada para inserir o registro anterior, faremos uma consulta para checar se as novas cidades foram incluídas devidamente na tabela (embora essa inclusão esteja apenas visível para a sessão do usuário atual no servidor MySQL, conforme explicamos detalhadamente no tópico anterior).

2. `select * from cidades;`

| cidade                | estado | numero_habitantes |
|-----------------------|--------|-------------------|
| Campinas              | SP     | 1139000           |
| Campos do Jordão      | SP     | 46974             |
| Curitiba              | PR     | 1774000           |
| Florianópolis         | SC     | 576000            |
| Paranavai             | PR     | 91950             |
| Pedro Leopoldo        | MG     | 62580             |
| Porto Alegre          | RS     | 1333000           |
| Santa Bárbara d'Oeste | SP     | 183347            |
| São Paulo             | SP     | 11451999          |

**Figura 5.43** – Consulta de todos os dados de uma entidade com transações pendentes.

Crédito: do elaborador, 2025.

Faremos agora a utilização do comando **rollback to**, para podermos recuperar o estado da transação durante o ponto de salvamento, o que, em termos práticos, provocará a exclusão da cidade de “Campinas”, já que ela foi inserida após o **savepoint**, o que não deve afetar a inserção das cidades anteriores (as capitais do sul do país, por exemplo). Em seguida, faremos uma consulta, para confirmar esse comportamento.

9. `rollback to capitais_sul;`  
0. `select * from cidades;`

| cidade                | estado | numero_habitantes |
|-----------------------|--------|-------------------|
| Campos do Jordão      | SP     | 46974             |
| Curitiba              | PR     | 1774000           |
| Florianópolis         | SC     | 576000            |
| Paranavai             | PR     | 91950             |
| Pedro Leopoldo        | MG     | 62580             |
| Porto Alegre          | RS     | 1333000           |
| Santa Bárbara d'Oeste | SP     | 183347            |
| São Paulo             | SP     | 11451999          |

**Figura 5.44** – Consulta de todos os dados de uma entidade ao recuperar o ponto de salvamento.

Crédito: do elaborador, 2025.

Como você pode observar, a cidade de “Campinas” foi excluída, já que tinha sido inserida após o ponto de salvamento, mantendo-se, no entanto, as capitais do sul incluídas anteriormente.

## transaction – Propriedades de uma transação

Durante o decorrer de todos os tópicos deste capítulo, você pôde observar que as operações realizadas em uma transação, mesmo quando não confirmadas com o comando **commit**, podem ser visualizadas desde que se mantenha a sessão do usuário. Isso ocorre porque, até o momento do **commit**, as alterações estão restritas à transação ativa, mas acessíveis dentro daquela mesma conexão.

### IMPORTANTE

#### Concorrência de usuários

**Vale ressaltar que, na maioria dos casos envolvendo aplicações reais em ambientes produtivos, quando nos referimos a “usuários” do MySQL, estamos nos referindo a conexões estabelecidas por partes da própria aplicação, como serviços de *backend*, e não diretamente por pessoas físicas. Assim, múltiplas conexões simultâneas podem estar ocorrendo de diferentes pontos da aplicação, gerando cenários de concorrência que precisam ser tratados com atenção.**

Para levar o(a) aluno(a) ao entendimento do que será abordado neste tópico, imagine um sistema bancário moderno, acessível por dispositivos móveis, navegadores *web* e caixas eletrônicos. Você deve concordar que, em um sistema desse porte, a consistência dos dados é fundamental. Isso ocorre porque existem pontos de acesso distintos, que permitem simultaneidade nas transações.

Por exemplo, um cliente pode estar movimentando sua conta bancária pelo celular, enquanto, ao mesmo tempo, realiza uma transação pelo computador. Além disso, essa mesma conta pode estar recebendo interações externas, como débito automático de contas ou transferências recebidas.

Nesse cenário mencionado, convidamos o(a) aluno(a) a pensar sobre uma delicada questão: imagine que duas operações bancárias distintas estão ocorrendo quase que simultaneamente na mesma conta e, como é de seu conhecimento, ambas estão criando duas *transactions* individuais no banco de dados. Uma dessas operações é a adição de saldo na conta por meio de uma transferência bancária que, se finalizada sem erros, será recebida. A outra, é um débito automático por meio do pagamento de um boleto, que deverá ser subtraído do saldo, caso a operação de pagamento seja concluída com sucesso.

Em sua opinião, você acha correto que o saldo adicionado seja imediatamente considerado para o pagamento, mesmo que a transação de adição de saldo ainda não tenha sido confirmada com o **commit**?

Vamos refletir: se a operação de pagamento considerar o saldo da transferência antes de sua confirmação, isso pode causar um problema sério de inconsistência. Imagine, por exemplo, que a transferência falhe por algum motivo após o pagamento já ter sido aprovado. O resultado seria um saldo negativo na conta, ou, pior ainda, um pagamento que efetivamente não poderia ter ocorrido.

Por esse motivo, o conceito de **isolamento das transações** torna-se fundamental. A transação de adição de saldo e a transação de pagamento precisam ser tratadas de forma completamente **independente e isolada**. Isso significa que o saldo da transferência só deve ser considerado disponível para o pagamento após a conclusão bem-sucedida da transação com o **commit**.

Portanto, é essencial que o banco de dados seja configurado com um nível de isolamento apropriado para garantir que uma transação não interfira na outra, impedindo que o saldo adicionado por uma operação seja utilizado por outra enquanto ainda não foi definitivamente confirmado.

## Níveis de isolamento de uma transação

Conforme mencionamos anteriormente, o isolamento das transações pode influenciar muito a saúde e a consistência das aplicações. Contudo, certamente você deve ter compreendido, ao longo dos inúmeros estudos de caso que fizemos no decorrer deste livro, que existem diversos contextos para cada aplicação, o que aumenta ou diminui o rigor para com essas transações.

Dessa forma, o MySQL nos disponibiliza basicamente quatro níveis diferentes de isolamento de uma transação:

- **“read uncommitted” (leitura não confirmada):** permite a leitura de dados que ainda não foram confirmados, não sendo recomendado para casos em que reversões afetam negativamente a consistência dos dados;
- **“read committed” (leitura confirmada):** permite a leitura de dados apenas confirmados por outras transações, havendo a possibilidade de mudarem no decorrer da atual, se alterados novamente por meio de novos **commits**, sendo indicado para situações em que essa mudança não impacta negativamente na consistência da aplicação e em que esse dinamismo em tempo real é desejado;
- **“repeatable read” (leitura repetível):** é o padrão do MySQL, permite a leitura de dados apenas confirmados, mas que permanecem inalterados do início ao fim da transação atual, indicado para a maioria dos casos por ser uma solução intermediária entre os extremos;
- **“serializable”:** as transações são totalmente isoladas e independentes, evitando problemas de simultaneidade, bloqueando leituras e escritas concorrentes, sendo indicado para situações delicadas e complexas, em que se objetiva o maior nível de consistência.

```
/* ESTRUTURA DO COMANDO: */  
set transaction isolation level TIPO_DE_ISOLAMENTO;  
  
/* EXEMPLO: */  
set transaction isolation level serializable;  
start transaction  
  update saldo set valor = (valor - 500.00) where conta_id =  
55789;
```

```
insert into operacoes (conta_FK,valor,data) values
(55789,500.00,now());
commit;
```

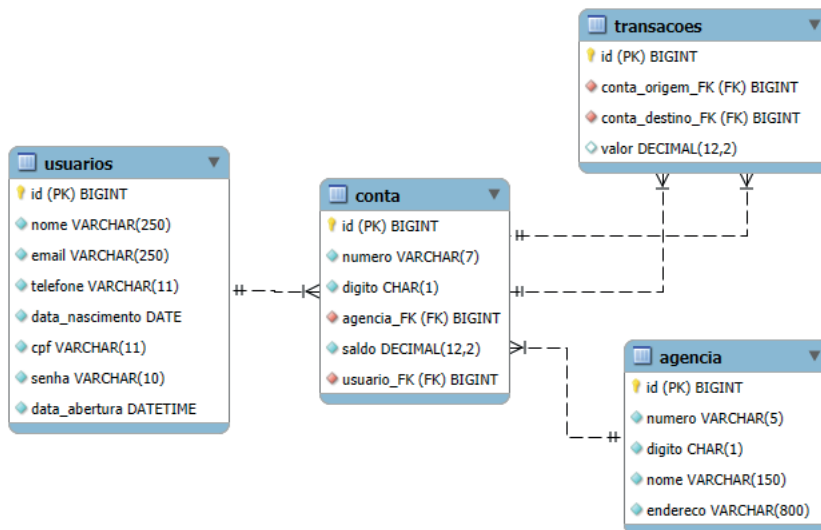
Observe que, de uma forma simples, conseguimos definir qual o tipo de isolamento desejado para a transação a ser iniciada, com vistas a garantir a melhor integridade dos dados da aplicação.

Contudo, talvez o assunto ainda esteja um pouco abstrato em termos de sua importância nas aplicações do dia a dia, não é mesmo? Por isso, recorreremos a um exemplo de uso.

## Exemplos e casos de uso – Transferências bancárias

Você com certeza já deve ter ouvido falar que, em aplicações envolvendo contextos bancários, a segurança e a confiabilidade devem ser preocupações constantes, já que erros e inconsistências podem representar grandes prejuízos financeiros. É nesse cenário que, para exemplificar de forma prática, faremos uma pequena abstração de um sistema bancário envolvendo quatro tabelas: usuários, agências, contas e transações.

Na **Figura 5.45**, observe o diagrama UML de uma parte dessa aplicação.



**Figura 5.45** – Diagrama UML de uma aplicação bancária.

Crédito: do elaborador, 2025.



Nessa aplicação, para exemplificarmos o uso e a importância das transações e, consequentemente, o comando **commit**, iremos simular o processo envolvendo uma operação bancária de transferência entre contas.

Inicialmente, **não** faremos o uso dos recursos estudados neste capítulo (transações e **commits**), para demonstrarmos melhor a problemática.

Para que uma transferência seja efetuada, deveremos adicionar o valor na conta de destino, subtrair o valor da conta de origem e registrar na tabela de transações bancárias.

Como você já aprendeu sobre *procedures* neste livro, faremos o uso desse recurso nessa aplicação.

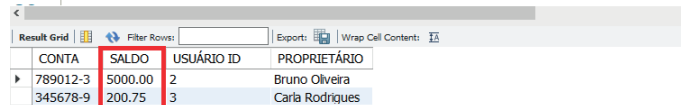
```
drop procedure if exists transferir_saldo;
delimiter $$
create procedure transferir_saldo(
    in in_conta_origem_FK bigint,
    in in_conta_destino_FK bigint,
    in in_valor decimal(12,2)
)
begin
    -- adiciona o valor na conta de destino
    update conta set saldo = (saldo + in_valor)
        where usuario_FK = in_conta_destino_FK;

    -- retira o valor da conta de origem
    update conta set saldo = (saldo - in_valor)
        where usuario_FK = in_conta_origem_FK;

    -- registra a transação bancária
    insert into transacoes (conta_origem_FK, conta_destino_FK,
valor)
        values (in_conta_origem_FK, in_conta_destino_FK,
in_valor);
    end
    $$
delimiter ;
```

Antes de utilizarmos nossa *procedure*, faremos a consulta prévia dos saldos das contas de dois usuários que escolhemos para o exemplo, a fim de que possamos ter o comparativo após as transferências.

```
.77 • select concat(c.numero,'-',c.digito) as 'CONTA',
.78 c.saldo as 'SALDO', c.usuario_FK as 'USUÁRIO ID',
.79 u.nome as 'PROPRIETÁRIO' from conta c
.80 join usuarios u on u.id = c.usuario_FK
.81 where usuario_FK in (2,3);
```



| CONTA    | SALDO   | USUÁRIO ID | PROPRIETÁRIO    |
|----------|---------|------------|-----------------|
| 789012-3 | 5000.00 | 2          | Bruno Oliveira  |
| 345678-9 | 200.75  | 3          | Carla Rodrigues |

**Figura 5.46** – Consulta dos saldos das contas dos usuários “Bruno” e “Carla” antes da transferência.

Crédito: do elaborador, 2025.

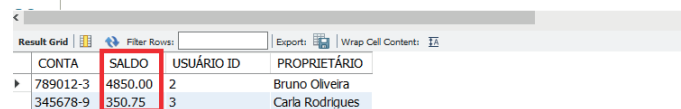
Como podemos observar, o saldo inicial do usuário “Bruno” é R\$ 5 000,00, enquanto o de “Carla” é R\$ 200,75.

Agora, faremos nossa primeira transferência, no valor de R\$ 150,00 entre as contas dos usuários mencionados. Adotaremos o processo da transferência com todas as operações sendo executadas corretamente, em um cenário ideal.

```
-- transferência do proprietário id 2 para o id 3, no valor de
R$150,00
call transferir_saldo(2,3,150.00);
```

Em seguida, consultaremos novamente os saldos para checarmos se a transferência saiu conforme o esperado.

```
.77 • select concat(c.numero,'-',c.digito) as 'CONTA',
.78 c.saldo as 'SALDO', c.usuario_FK as 'USUÁRIO ID',
.79 u.nome as 'PROPRIETÁRIO' from conta c
.80 join usuarios u on u.id = c.usuario_FK
.81 where usuario_FK in (2,3);
```



| CONTA    | SALDO   | USUÁRIO ID | PROPRIETÁRIO    |
|----------|---------|------------|-----------------|
| 789012-3 | 4850.00 | 2          | Bruno Oliveira  |
| 345678-9 | 350.75  | 3          | Carla Rodrigues |

**Figura 5.47** – Consulta dos saldos das contas dos usuários “Bruno” e “Carla” após a transferência.

Crédito: do elaborador, 2025.

Note que o processo de transferência ocorreu com sucesso, já que possuímos exatamente a diferença de R\$ 150,00 em cada conta em relação ao seu saldo anterior.

No entanto, simulamos até agora apenas o “caminho feliz” do processo, isto é, com todas as operações sendo executadas corretamente, o que nem sempre ocorre na prática. Entretanto, agora simularemos uma operação problemática por meio do uso de uma transferência com valor maior que o saldo disponível, forçando a geração de um erro de *constraint* durante o processo.

Desse modo, invocaremos novamente nossa *procedure*, mas envolvendo uma tentativa de transferência de R\$ 20 000,00, saldo que nenhum dos dois usuários possui. Isso claramente ocasionará um erro, já que a tabela de conta possui uma *constraint* em que o saldo deve sempre permanecer maior ou igual a zero.

## IMPORTANTE

### Erros

**É importante evidenciar que o erro que será provocado aqui se baseia em um *check constraint* da coluna saldo. Contudo, em cenários reais, os erros gerados podem ser os mais diversos, como conexões perdidas com o servidor MySQL, outras *constraints*, invalidações por regras de negócio, problemas de sintaxe etc.**

```
-- transferência do proprietário id 2 para o 3, no valor de  
R$20.000,00  
call transferir_saldo(2,3,20000.00);
```

Devido à *constraint* de checagem de “saldo  $\geq$  0” existente na tabela de conta, o primeiro comando dentro da *procedure* deve ocasionar um erro (já que o saldo do usuário “Bruno” é de apenas R\$ 4 850,00).

|   |    |          |                                                                                              |                                                               |
|---|----|----------|----------------------------------------------------------------------------------------------|---------------------------------------------------------------|
| ✓ | 46 | 22:10:27 | call transferir_saldo(2,3,150.00)                                                            | 1 row(s) affected                                             |
| ✓ | 47 | 22:10:32 | select concat(c.numero,'-',c.digito) as 'CONTA', c.saldo as 'SALDO', c.usuario_FK as 'USU... | 2 row(s) returned                                             |
| ✗ | 48 | 22:33:09 | call transferir_saldo(2,3,20000.00)                                                          | Error Code: 3819. Check constraint 'conta_chk_1' is violated. |

**Figura 5.48** – Status dos comandos executados em uma transferência bancária com saldo insuficiente.

Crédito: do elaborador, 2025.

Entretanto, podemos constatar também que, embora um erro de *constraint* tenha sido gerado durante o débito do saldo da conta de origem, o comando para adição de saldo foi concluído com sucesso, o que de fato é um grande problema, pois nesse momento acaba de se gerar uma grande inconsistência na aplicação. Observe a **Figura 5.49** e notará esse problema.

```

77 • select concat(c.numero,'-',c.digito) as 'CONTA',
78 c.saldo as 'SALDO', c.usuario_FK as 'USUÁRIO ID',
79 u.nome as 'PROPRIETÁRIO' from conta c
80 join usuarios u on u.id = c.usuario_FK
81 where usuario_FK in (2,3);

```

| CONTA    | SALDO    | USUÁRIO ID | PROPRIETÁRIO    |
|----------|----------|------------|-----------------|
| 789012-3 | 4850.00  | 2          | Bruno Oliveira  |
| 345678-9 | 20350.75 | 3          | Carla Rodrigues |

**Figura 5.49** – Consulta dos saldos das contas dos usuários após a transferência com saldo insuficiente.

Crédito: do elaborador, 2025.

Como podemos constatar, foi adicionado o valor da transferência no saldo do destinatário, mesmo sem existir quantia suficiente na conta de origem.

## IMPORTANTE

### Soluções

É claro que, para solucionarmos a situação retratada quando não há saldo suficiente, podemos recorrer às mais diversas soluções. Dentre elas, poderíamos criar uma checagem com **select** para somente executar a transferência se houver a comprovação de saldo, ou então realizarmos essa validação dentro de condicionais no próprio *backend* da aplicação (o que é mais comum). Contudo, alertamos o(a) aluno(a) a não focar sua atenção apenas na questão do saldo, uma vez que outros tipos de erros poderiam ser gerados até mesmo ao se usar essas soluções, como falhas de conexão com o banco, violações de outras *constraints*, erros

**matemáticos etc., e em todos esses casos haveria a possibilidade de geração de inconsistências semelhantes ao que foi apresentado.**

Agora que você já percebeu a gravidade dessa situação, vamos à sua solução. Esse problema é resolvido facilmente se realizarmos as tratativas devidas de transações no MySQL, pois englobariam, desse modo, tanto os erros gerados por *constraints* quanto as instabilidades no servidor, as falhas de conexão etc., protegendo a integridade da aplicação.

Ajustaremos então nossa *procedure*, criando uma *transaction*.

```
drop procedure if exists transferir_saldo;
delimiter $$
create procedure transferir_saldo(
  in in_conta_origem_FK bigint,
  in in_conta_destino_FK bigint,
  in in_valor decimal(12,2)
)
begin
  -- cria um tratamento para as exceções geradas
  declare exit handler for sqlexception
  begin
    -- faz o estorno/cancelamento das operações da transação
    rollback;
  end;

  -- inicia uma transação isolada
  set transaction isolation level serializable;
  start transaction;

  -- adiciona o valor na conta de destino
  update conta set saldo = (saldo + in_valor)
  where usuario_FK = in_conta_destino_FK;

  -- retira o valor da conta de origem
  update conta set saldo = (saldo - in_valor)
```

```

where usuario_FK = in_conta_origem_FK;

-- registra a transação bancária
insert into transacoes (conta_origem_FK, conta_destino_FK,
valor)
  values (in_conta_origem_FK, in_conta_destino_FK,
in_valor);

-- finaliza a transação se nenhum problema ocorreu
commit;
end
$$
delimiter ;

```

Conforme destacamos no bloco de código anterior, adicionamos alguns pontos importantes:

- Iniciamos uma transação envolvendo os comandos de adição e remoção de saldo, bem como o registro da transferência, de modo apenas a realizar a confirmação se todos forem executados normalmente, por meio do comando **commit**;
- Para tratarmos os possíveis erros gerados, utilizamos um gerenciador dessas exceções (“handlers”), definindo que, para esses casos, o MySQL deverá cancelar todas as operações realizadas na transação em aberto por meio do comando **rollback**.

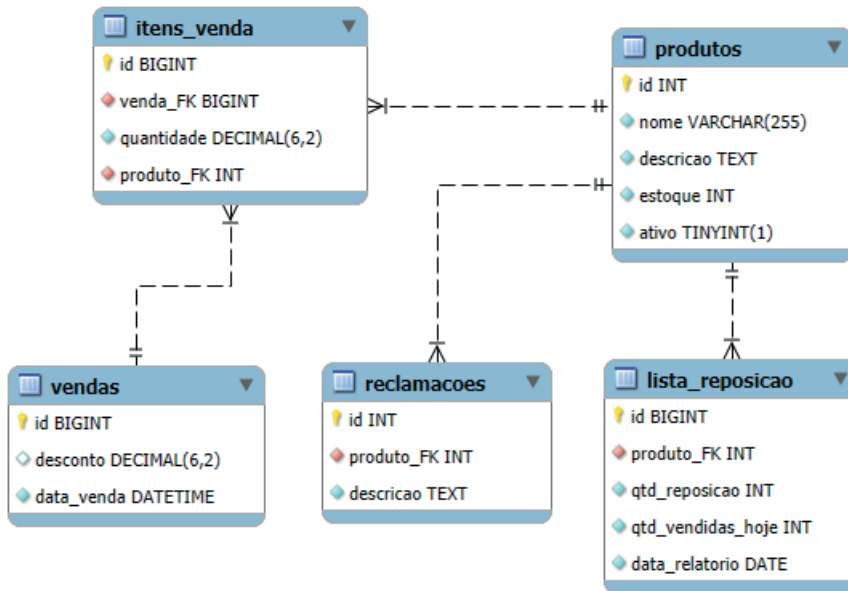
#### IMPORTANTE

##### **rollback das operações**

Note que a utilização de comandos em uma *transaction* não garante automaticamente o **rollback** das operações, tendo sido necessário, nesse caso, explicitar a situação em que elas serão canceladas.

## Exercícios – *E-commerce*

Agora que você já aprendeu a lidar com as transações do banco de dados para deixar certas operações críticas mais seguras e confiáveis, retomaremos a aplicação de *e-commerce* abordada anteriormente, com o objetivo de torná-la mais consistente. Na **Figura 5.50**, observe o diagrama UML dessa aplicação.



**Figura 5.50** – Modelagem UML de um e-commerce de bolsas e acessórios femininos.

Crédito: do elaborador, 2025.

Na modelagem apresentada, podemos identificar uma estrutura relativamente simples envolvendo “produtos”, “vendas” e “itens\_venda”, muito similar ao que já exploramos ao longo deste livro, ou seja, uma venda possui diversos itens, todos presentes na tabela de produtos cadastrados. Adicionalmente, possuímos uma tabela para registro das reclamações dos clientes, em que há a indicação do produto a receber a reclamação. E, por fim, temos uma entidade para guardar a lista de reposição dos itens que devem ser comprados para repor o estoque.

Sua atuação agora será criar uma *procedure* que utilize os conceitos de TCL aprendidos para garantir que a inserção de uma venda e de seus itens ocorra de forma segura na mesma transação. O objetivo principal dessa modificação é evitar problemas de inconsistência gerados, por exemplo, por inserções incompletas dos itens de uma venda.

Faça a alteração do projeto e teste suas modificações, garantindo o comportamento desejado.

## Migração de dados

Você deve ter percebido que trabalhar com bancos de dados vai muito além do que projetá-los, criá-los e usá-los, havendo também a necessidade de gerenciar esses bancos.

É nesse contexto que a migração de dados se enquadra: para atender aos casos em que a exportação e a importação de bancos e tabelas se torna necessária. Essas migrações ocorrem frequentemente quando se deseja transferir dados de um servidor para outro, executar *backups*, ou transferir dados de produção para ambientes de teste.

Felizmente, o MySQL também oferece recursos para facilitar esse tipo de operação, o que veremos a seguir.

## Exportação de dados

`mysqldump` – Exportação de dados com terminal de comando

Para realizar a exportação dos dados com o terminal do sistema operacional, utilizamos o comando **mysqldump**, conforme o bloco de código a seguir.

```
-- ESTRUTURA DO COMANDO:  
mysqldump -h ENDEREÇO_CONEXÃO -P PORTA_SERVIDOR
```



```
-u USUARIO_DO_BANCO -p NOME_DO_BANCO [TABELA1 TABELA2 ...] >
NOME_DO_ARQUIVO.sql

-- EXEMPLOS:
-- exportando o banco de dados "estacionamento"
mysqldump -h 127.0.0.1 -P 3307 -u root -p estacionamento >
estacionamento_backup.sql

-- exportando a tabela "alunos" do banco de dados
"plataforma_educacional"
mysqldump -h 127.0.0.1 -P 3306 -u root -p plataforma_educacional
alunos > extracao_alunos.sql
```

Conforme podemos identificar no bloco de código anterior, a estrutura de uso do **mysqldump** possui semelhança com o modo de conexão com o servidor MySQL via terminal de comando (conforme vimos nos capítulos iniciais deste livro). Para utilizá-lo, precisamos fornecer endereço e porta de conexão, usuário e senha e definir quais banco ou tabelas serão exportados.

### IMPORTANTE

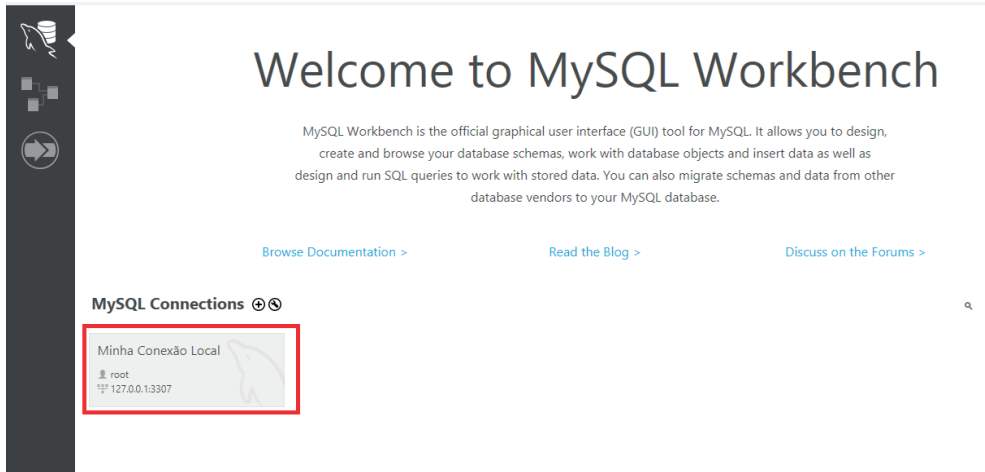
#### Cópia dos dados

**Vale ressaltar que o **mysqldump** realiza apenas a cópia dos dados, ou seja, não apaga os dados da fonte de origem para o destino. Se necessário, o desenvolvedor deve executar a operação de deleção manualmente.**

Exportação de dados com MySQL Workbench

Se você deseja uma opção mais visual e interativa, é possível utilizar o MySQL Workbench para executar o processo de exportação, conforme os passos a seguir.

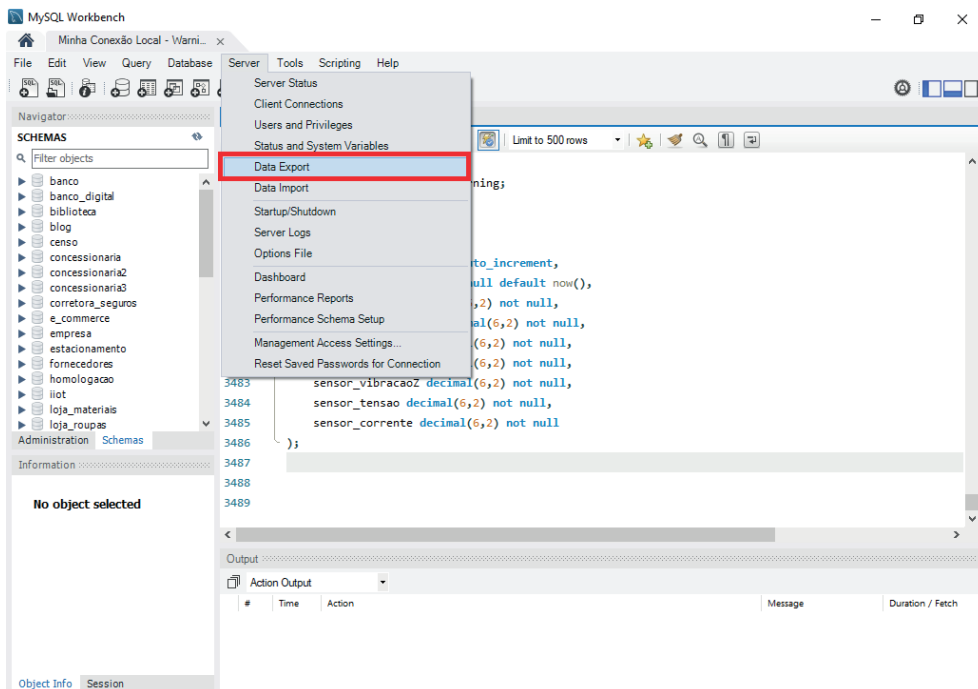
Abra o MySQL Workbench e faça a conexão com o servidor desejado, conforme já abordamos anteriormente neste livro.



**Figura 5.51** – Conexão com o servidor no MySQL Workbench.

Crédito: do elaborador, 2025.

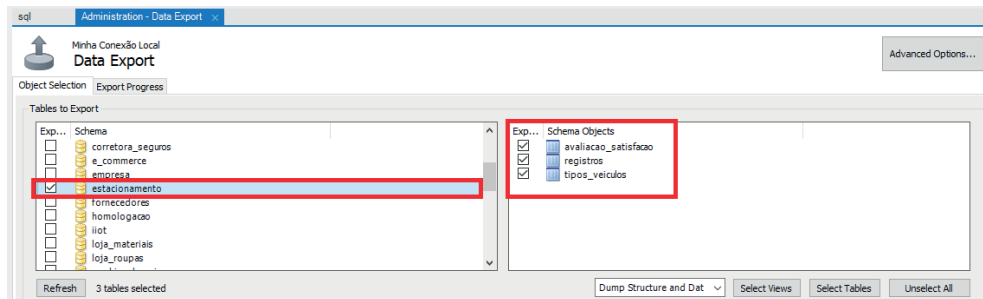
Selecione no menu superior as opções “Server” e “Data Export”, conforme se vê na **Figura 5.52**.



**Figura 5.52** – Seleção da opção “Data Export” no MySQL Workbench.

Crédito: do elaborador, 2025.

Selecione os bancos e as tabelas desejadas para exportação, conforme mostrado na **Figura 5.53**.



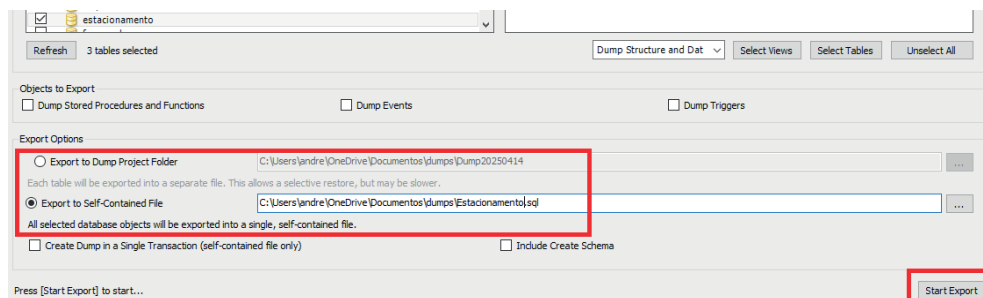
**Figura 5.53** – Seleção dos bancos e das tabelas a serem exportados com o MySQL Workbench.

Crédito: do elaborador, 2025.

Em seguida, selecione uma das duas alternativas de exportação disponíveis e o caminho para onde deve ser enviada a migração, considerando as opções seguintes:

- **“Export to Dump Project Folder”**: cria arquivos separados para cada tabela e banco de dados, localizados dentro de uma mesma pasta de exportação;
- **“Export to Self-Contained File”**: cria todos os *scripts* de exportação em um único arquivo.

Por fim, clique no botão de exportação (“Start Export”), conforme se vê na **Figura 5.54**.



**Figura 5.54** – Seleção da opção de exportação no MySQL Workbench.

Crédito: do elaborador, 2025.

## Exportação de dados com condicionais

Suponha que você trabalha em um grande banco internacional, com milhares de clientes ao redor do mundo, e que precisa implementar alguns testes apenas com os dados dos clientes do seu país. Com toda certeza, não seria nada seguro executar os testes referidos na base de dados original (ambiente produtivo), e exportar a tabela de todos os clientes existentes ao redor do mundo demandaria um tempo adicional desnecessário, já que você somente necessita de uma pequena parcela desses dados.

Felizmente, essa situação é de fácil solução, pois o comando **mysqldump** possibilita a exportação condicional dos dados, ou seja, para esse caso, poderíamos exportar apenas os clientes da localidade desejada por meio da cláusula **where**.

Observe a estrutura de uso, conforme o bloco de código a seguir.

```
-- ESTRUTURA DO COMANDO:
mysqldump -h ENDEREÇO_CONEXÃO -P PORTA_SERVIDOR
-u USUARIO_DO_BANCO -p NOME_DO_BANCO TABELA1
--where="CONDIÇÃO_WHERE" > NOME_DO_ARQUIVO.sql

-- EXEMPLOS:
-- exportando os clientes com localidade 'Brazil'
mysqldump -h 127.0.0.1 -P 3307 -u myUser -p sistema_financeiro
clientes
--where="local='Brazil'" > clientes_br.sql
-- exportando os alunos que ainda não finalizaram o curso
mysqldump -h 127.0.0.1 -P 3306 -u root -p plataforma_
educacional alunos
--where="status='EM ANDAMENTO'" > extracao_alunos.sql
```

Como podemos observar no bloco de código anterior, conseguimos definir uma cláusula **where** para determinar que a exportação obedeça a uma condicional estabelecida, colaborando dessa forma para que os dados estejam devidamente filtrados de acordo com a necessidade.

## Importação de dados

mysql – Importação de dados com terminal de comando

De forma muito semelhante à utilizada para exportar os dados, para realizar sua importação com o terminal do sistema operacional utilizamos o comando **mysql**, conforme o bloco de código a seguir.

```
-- ESTRUTURA DO COMANDO:
mysql -h ENDEREÇO_CONEXÃO -P PORTA_SERVIDOR
  -u USUARIO_DO_BANCO -p NOME_DO_BANCO [TABELA1] < NOME_DO_
ARQUIVO.sql

-- EXEMPLOS:
-- importante o banco de dados “estacionamento”
mysql -h 127.0.0.1 -P 3307 -u root -p
  estacionamento < estacionamento_backup.sql

-- importante a tabela “alunos” do banco de dados
“plataforma_educacional”
mysql -h 127.0.0.1 -P 3306 -u root -p plataforma_educacional
  alunos < extracao_alunos.sql
```

Na estrutura de uso do **mysql** para importação, precisamos fornecer endereço e porta de conexão, usuário e senha e definir quais banco ou tabelas serão importados.

### IMPORTANTE

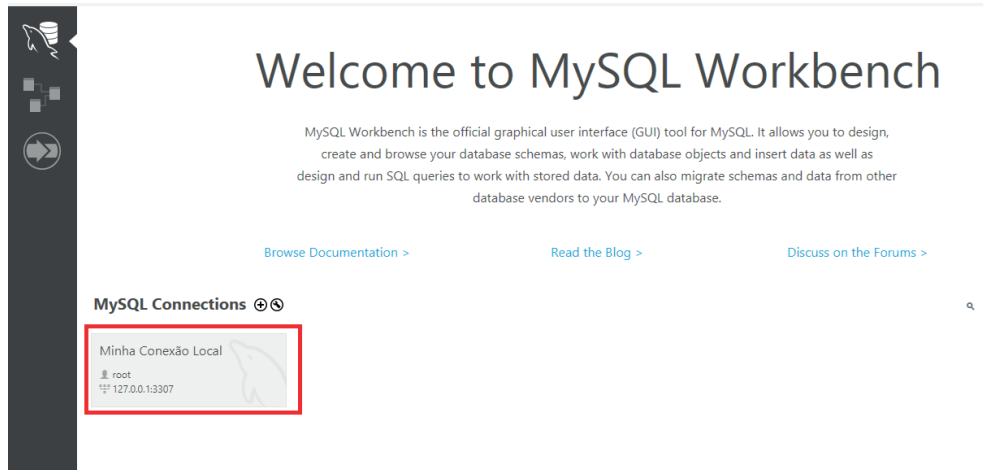
#### Nome do banco de dados

**Para evitar erros de importação, verifique se o nome do banco a receber a importação já existe em seu servidor. Caso contrário, crie-o primeiro.**

Importação de dados com MySQL Workbench

Se você deseja uma opção mais visual e interativa, é possível utilizar o MySQL Workbench para executar o processo de importação de forma muito semelhante ao que fizemos com a exportação, conforme os passos a seguir.

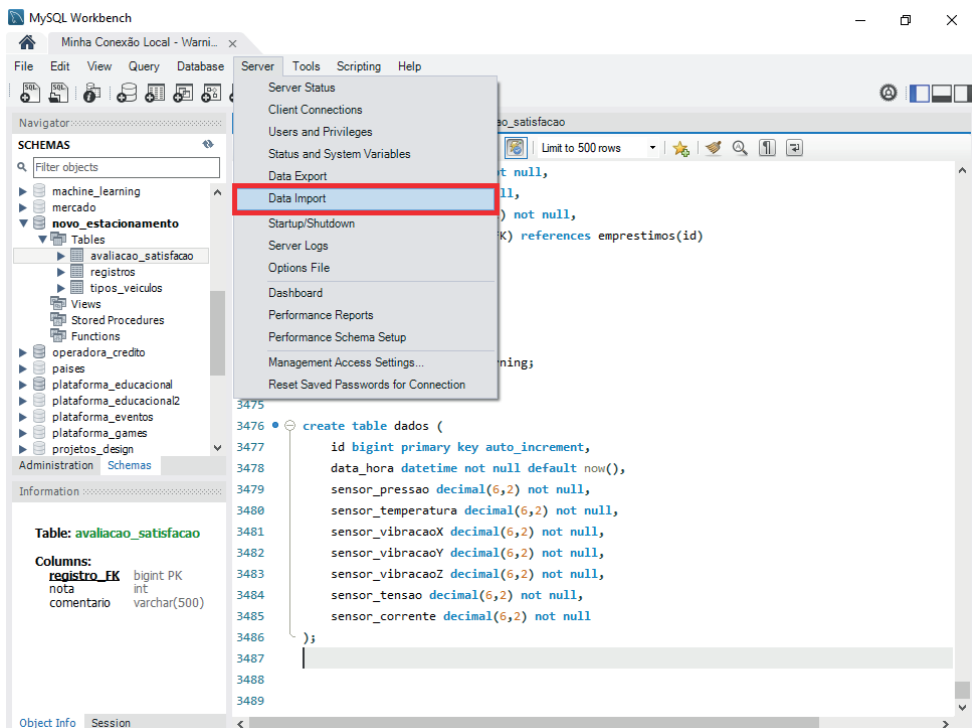
Abra o MySQL Workbench e faça a conexão com o servidor desejado, conforme já abordamos anteriormente neste livro.



**Figura 5.55** – Conexão com o servidor no MySQL Workbench.

Crédito: do elaborador, 2025.

Selecione no menu superior as opções “Server” e “Data Import”.

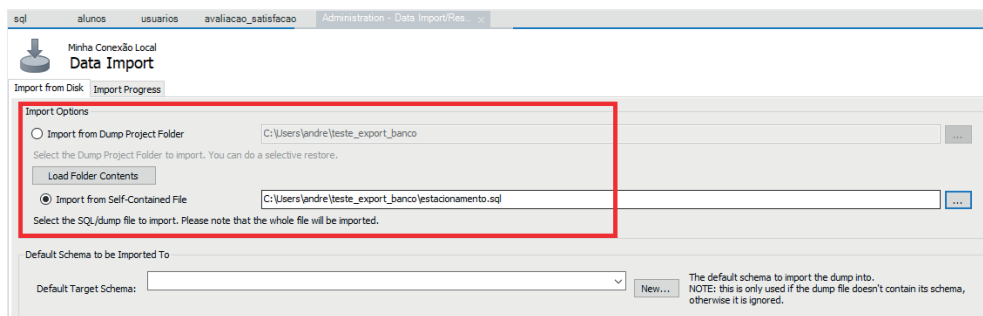


**Figura 5.56** – Seleção da opção de “Data Import” com o MySQL Workbench.

Crédito: do elaborador, 2025.

Em seguida, selecione uma das duas alternativas de importação disponíveis e o caminho no qual está o arquivo ou a pasta a ser lida na migração, considerando as opções seguintes:

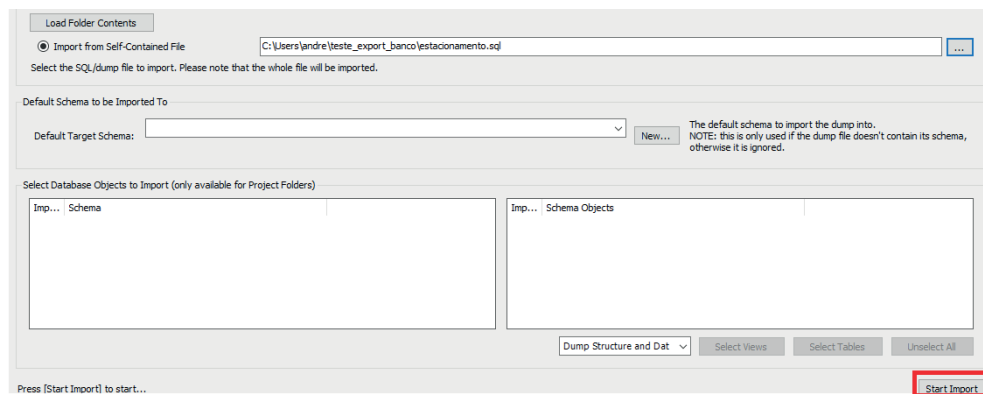
- **“Import from Dump Project Folder”**: serão importados todos os arquivos de uma pasta (para os casos em que a exportação foi segregada dessa maneira);
- **“Import from Self-Contained File”**: será importado um único arquivo que contém toda a migração (para os casos em que a exportação foi criada dessa maneira).



**Figura 5.57** – Seleção da alternativa de importação com o MySQL Workbench.

Crédito: do elaborador, 2025.

Finalmente, clique no botão de importação.

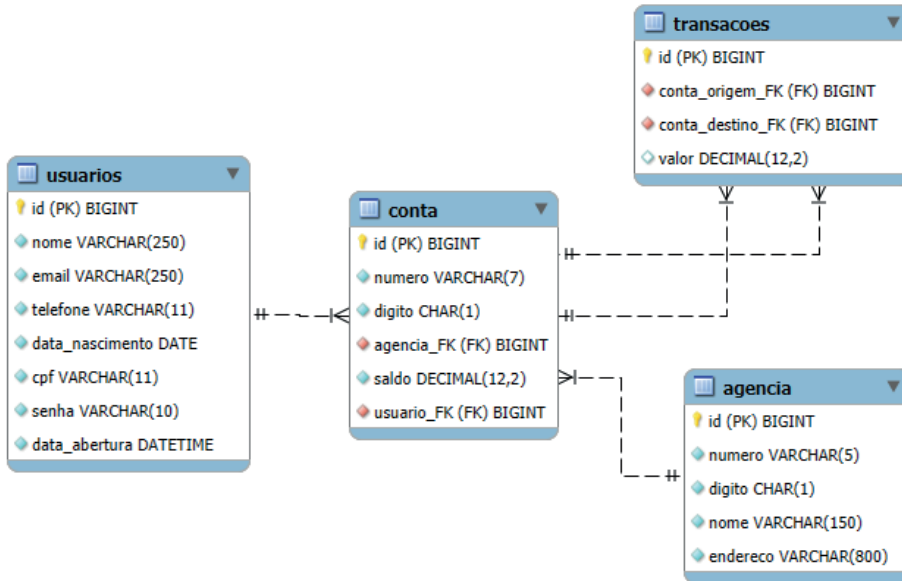


**Figura 5.58** – Seleção dos bancos e das tabelas a serem importados com o MySQL Workbench.

Crédito: do elaborador, 2025.

## Exercícios – Transferências bancárias

Aproveitaremos que você está familiarizado com a aplicação de um sistema bancário, uma vez que a utilizamos em diversos exemplos, para que pratique seus conhecimentos adquiridos na migração de dados.



**Figura 5.59** – Diagrama UML de uma aplicação bancária.

Crédito: do elaborador, 2025.

É muito comum que empresas de *software* possuam servidores com cópias do ambiente produtivo, nos quais são aplicados testes e outros desenvolvimentos, também denominados ambientes QA (*Quality Assurance* ou Garantia de Qualidade) e DEV (*Development* ou Desenvolvimento).

Desse modo, chegam até você as seguintes demandas:

1. Crie uma cópia do banco de dados atual (produtivo), exportando-o com todas as tabelas existentes;
2. Crie um novo banco e faça a importação dos dados anteriormente importados, pois isso será utilizado como ambiente “DEV”;
3. Faça uma nova importação, mas agora importe apenas os usuários com data de abertura de conta equivalente ao ano atual, pois esses



dados serão utilizados pelo departamento de “Data Analytics – Análise de Dados”.

## Resumo

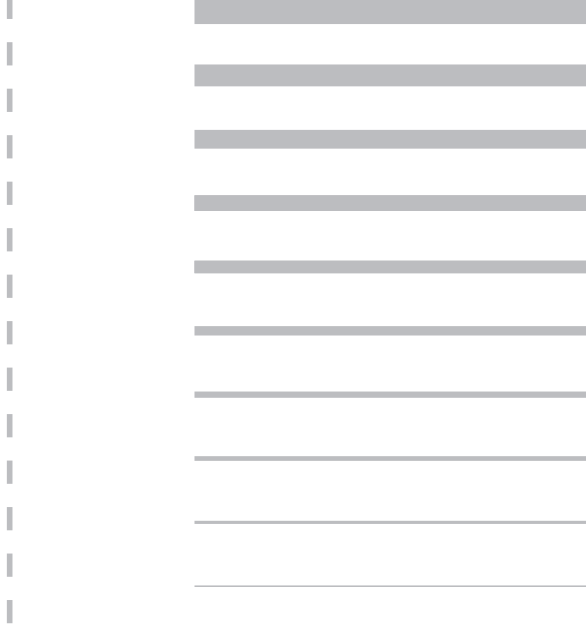
Meus parabéns a você, aluno(a), que chegou ao último capítulo de nossa jornada!

Ao longo deste livro, percorremos um vasto caminho, abordando os mais diversos conceitos e fundamentos, com o objetivo de orientá-lo(a) aos conhecimentos e às vivências práticas, capacitando-o(a) para que se torne um(a) desenvolvedor(a) preparado(a) para as necessidades do mercado.

Sem dúvida, você literalmente vivenciou um arcabouço de “exemplos e casos de uso”, os quais lhe desafiaram a tentar entender os relacionamentos e as regras de negócio envolvidos, assim como certos contextos mais complexos. Temos a certeza de que, ao final de cada um deles, você obteve uma compreensão da utilidade, da aplicação e de determinadas peculiaridades de cada comando ou recurso existente no mundo do MySQL.

É com essa sensação de missão cumprida que nos despedimos de você, parabenizando-o(a) novamente por agora encontrar-se muito mais capacitado para a realidade do mercado, mas alertando-o também que, quando o assunto é Tecnologia da Informação, cada passo dado é apenas mais um dos inúmeros que precisaremos dar para nos manter atualizados e competentes.

Até a próxima!



## SOBRE O ELABORADOR

### **André Felipe Savedra Cruz**

Possui pós-graduação em *Data Science* pela Unicamp, com especializações em Desenvolvimento de *Software Web* utilizando Java (Universidade Estadual) e Engenharia de Controle e Automação (Anhanguera Educacional). Tem ampla experiência em integrações entre Tecnologia da Informação e Tecnologia da Automação, com inúmeros projetos envolvendo Indústria 4.0, dedicando-se mais recentemente aos estudos de Inteligência Artificial.

Trabalhou por mais de sete anos em ambiente de Engenharia de Produtos, com foco principal no desenvolvimento de projetos especiais e adequações às normas de segurança NR 12 em injetoras e sopradoras de termoplástico. Atualmente, trabalha como Desenvolvedor de Sistemas *Web Full Stack* Sênior na Robert Bosch Ltda., onde vem criando soluções para clientes externos e internos da empresa com o uso das mais recentes tecnologias e *frameworks* disponíveis, participando também de projetos nacionais e internacionais, auxiliando em decisões técnicas e em desenvolvimentos para garantir a automação de processos e a digitalização.

Atua há mais de oito anos na área docente da Escola SENAI “Roberto Mange”, em Campinas-SP, onde ministrou aulas na área de Automação (robótica, controladores programáveis, microcontroladores, eletrônica digital, IIoT, Indústria 4.0, comandos elétricos e gestão de projetos) e na área de Informação (banco de dados, linguagem de programação, *backend*, *frontend*, *chatbots* e gestão de projetos ágeis). Atualmente, tem focado sua vida profissional na área de Tecnologia da Informação como professor de Ensino Superior no curso de Análise e Desenvolvimento de Sistemas. No SENAI, além da docência, atuou na criação de diversos projetos, cursos e treinamentos, tendo também sido escalado em 2018 para desenvolver a primeira Escola Móvel de Indústria 4.0 da América Latina, responsável por oferecer treinamentos itinerantes em diversas regiões do estado de São Paulo.

Esta publicação integra uma  
série da SENAI-SP Editora  
especialmente criada para apoiar  
os cursos do SENAI-SP.

O mercado de trabalho em  
permanente mudança exige que  
o profissional se atualize  
continuamente ou, em muitos  
casos, busque qualificações.

É para esse profissional,  
sintonizado com a evolução  
tecnológica e com as inovações  
nos processos produtivos, que o  
SENAI-SP oferece muitas opções  
em cursos, em diferentes níveis,  
nas diversas áreas tecnológicas.

**SENAI**

ISBN 978-85-534-0192-5

